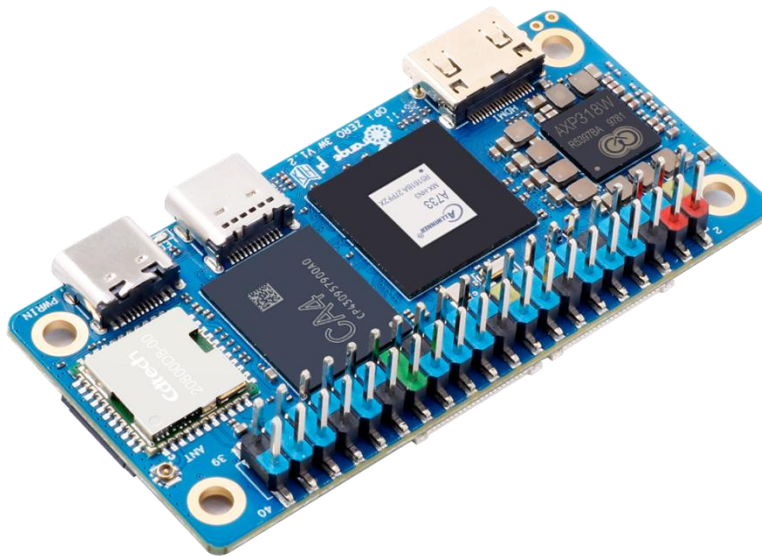




Orange Pi Zero 3w

User Manual





Catalogue

1. Basic characteristics of Orange Pi Zero 3w	1
1. 1. What is Orange Pi Zero 3w	1
1. 2. Use of Orange Pi Zero 3w	1
1. 3. Who was the Orange Pi Zero 3w designed for	2
1. 4. Hardware Features of Orange Pi Zero 3w	2
1. 5. Top and Bottom Views of Range Pi Zero 3w	4
1. 6. Interface Details of Range Pi Zero 3w	5
2. Introduction to using the development board	7
2. 1. Prepare the necessary accessories	7
2. 2. Download the image of the development board and related materials	10
2. 3. Method of burning Linux image to TF card based on Windows PC	10
2. 3. 1. Method of burning Linux images using BalenaEtcher 用 balenaEtcher	10
2. 3. 2. Method of burning Linux images using Win32Diskimager	14
2. 4. Method for burning Linux images to TF cards based on Ubuntu PC	17
2. 5. Method for burning Linux images to eMMC	21
2. 6. Method for burning Linux images to UFS	21
2. 7. Method of burning Android image to TF card	21
2. 8. Method for burning Android images to eMMC	28
2. 9. Method of burning Android images to eMMC or UFS through TypeC port	34
2. 10. Launch the Orange Pie development board	38
2. 11. Instructions for Debugging Serial Ports	39
2. 11. 1. Debugging serial port connection instructions	39
2. 11. 2. Ubuntu platform debugging serial port usage method	40
2. 11. 3. Instructions for Debugging Serial Ports on Windows Platform	43



2. 12. Instructions for powering the 5V pin in the 40 pin interface of the development board	46
3. Instructions for using Debian/Ubuntu Server and Xfce desktop system	47
3. 1. Supported Linux image types and kernel versions	47
3. 2. linux kernel driver adaptation situation	49
3. 3. Explanation of linux Command Format in This Manual	50
3. 4. linux system login instructions	52
3. 4. 1. linux system default login account and password	52
3. 4. 2. Method for setting up automatic login for linux system terminals	52
3. 4. 3. linux Desktop System Automatic Login Instructions	53
3. 4. 4. Linux Desktop System Root User Automatic Login Setting Method	53
3. 5. Test instructions for onboard LED lights	54
3. 6. Operating Instructions for linux System rootfs Partition Capacity in TF Card	54
3. 6. 1. The system will automatically expand the capacity of the rootfs partition on the TF card during the first boot.	54
3. 6. 2. Method to prevent automatic expansion of the rootfs partition size on the TF card	56
3. 6. 3. Method for manually expanding the capacity of the rootfs partition on a TF card	58
3. 6. 4. Methods to reduce the size of the rootfs partition on a TF card	63
3. 7. Network Connectivity Test	68
3. 7. 1. WiFi Connection Test	68
3. 7. 2. Method for setting a static IP address	75
3. 7. 3. How to configure a Linux system to automatically connect to the network on the first boot	76
3. 8. SSH Remote Login to Development Board	79
3. 8. 1. SSH Remote Login to Development Board under Ubuntu	80
3. 8. 2. SSH Remote Login to Development Board under Windows	81
3. 9. How to use ADB	82
3. 9. 1. How to use ADB for networking	82
3. 9. 2. Connect adb using a Type-C data cable	83
3. 10. HDMI Test	85



3. 10. 1. HDMI Display Test	85
3. 10. 2. HDMI to VGA display test	85
3. 11. Bluetooth Usage Instructions	86
3. 11. 1. Testing Methods for Desktop Image	86
3. 11. 2. How to use the server version image	90
3. 12. USB Interface Test	93
3. 12. 1. USB Interface Expansion Description	93
3. 12. 2. Test connecting a USB mouse or keyboard	94
3. 12. 3. Test connecting USB storage devices	94
3. 12. 4. USB Ethernet Card Test	95
3. 12. 5. USB Camera Test	96
3. 13. Temperature Sensor	98
3. 14. 40 Pinout of the Interface	99
3. 15. Method for installing wiringOP	100
3. 16. 40-pin interface GPIO, I2C, UART, SPI and PWM testing	101
3. 16. 1. 40-pin GPIO port test	101
3. 16. 2. 40 Method for setting pull-up/pull-down resistors for GPIO pin	103
3. 16. 3. 40 pin SPI test	103
3. 16. 4. 40 pin I2C test	106
3. 16. 5. 40 pin UART test	109
3. 16. 6. Methods for testing PWM using /sys/class/pwm/	111
3. 17. How to install and use wiringOP-Python	115
3. 17. 1. Installation method of wiringOP-Python	115
3. 17. 2. 40-pin GPIO port test	117
3. 17. 3. 40-pin SPI test	119
3. 17. 4. 40-pin I2C test	122
3. 17. 5. 40-pin UART test	125
3. 18. Hardware Watchdog Test	127
3. 19. View the chipid of the A733 chip	128
3. 20. Python Related Notes	128
3. 20. 1. Methods for compiling and installing Python from source code	128



3. 20. 2. Methods for changing the pip source in Python	129
3. 21. How to install Docker	130
3. 22. How to install Home Assistant	131
3. 22. 1. Installation via Docker	131
3. 22. 2. Install via Python	135
3. 23. How to install OpenCV	137
3. 23. 1. Install OpenCV using apt	137
3. 24. Setting up a Chinese environment and installing a Chinese input method	138
3. 24. 1. Debian system installation method	138
3. 24. 2. How to install Ubuntu 22.04	144
3. 25. Installation Method of Baota Linux Panel	149
3. 26. QT installation method	153
3. 27. ROS Installation Method	160
3. 27. 1. How to install ROS 2 Humble on Ubuntu 22.04	160
3. 28. Methods for installing kernel header files	162
3. 29. Testing of Some Programming Languages Supported by Linux Systems	164
3. 29. 1. Debian Bullseye System	164
3. 29. 2. Debian Bookworm System	165
3. 29. 3. Ubuntu Jammy System	167
3. 30. Method of uploading files to the Linux system of the development board	169
3. 30. 1. Method for uploading files to the Linux system on the development board in Ubuntu PC	169
3. 30. 2. Method for uploading files from Windows PC to development board Linux system	173
3. 31. Instructions for using IMX219 MIPI camera	178
3. 31. 1. Installation method of camera	178
3. 31. 2. Method of opening the camera through Gstreamer	179
3. 31. 3. Method of opening MIPI camera through OpenCV program	180
3. 32. 10.1 inch MIPI LCD screen usage method	183
3. 32. 1. Assembly method for 10.1-inch MIPI screen	183



3. 32. 2. Method for configuring a 10.1-inch MIPI LCD screen	185
3. 32. 3. Method for Rotating Display Direction of Server Version Mirrors ...	188
3. 33. Instructions for Hard Debugging Test of Debian Bulleye System	188
3. 34. GPU Testing Instructions for Debian Bulleye System	190
3. 35. Instructions for using NPU	192
3. 35. 1. PC Development Environment Configuration	192
3. 35. 2. On-board Model Deployment	194
3. 36. Method for burning Linux images to eMMC	200
3. 37. Method for burning Linux images to UFS	202
3. 38. Instructions for using the system backup script opi-bking	205
3. 39. Usage of Linux Overlayoot	206
3. 39. 1. Configure Overlayboot to tmpfs mode	207
3. 39. 2. Configure Overlayboot to block device storage mode	208
3. 39. 3. Methods to disable Overlayboot	211
3. 39. 4. Method of backing up overlayroot image through opi-bking	212
3. 40. Methods for shutting down and restarting the development board	214
4. Linux SDK——orangePi build usage instructions	215
4. 1. Compilation System Requirements	215
4. 2. Obtain the source code of Linux SDK	217
4. 2. 1. Download orangePi-build	217
4. 2. 2. Download the cross compilation toolchain	219
4. 2. 3. Explanation of the complete directory structure of orangePi-build	220
4. 3. Compiling u-boot	221
4. 4. Compiling Linux Kernel	224
4. 5. Compile rootfs	228
4. 6. Compiling Linux Images	232
5. Instructions for using Android 13 system	236
5. 1. Supported Android versions	236
5. 2. Adaptation of Android 13 Features	236



5. 3. Instructions for Using ADB	237
5. 3. 1. Method for switching USB OTG mode	237
5. 3. 2. Connect adb debugging with data cable	240
5. 3. 3. Debugging with network connection adb	241
5. 3. 4. HDMI to VGA Display Test	242
5. 4. Connection method of WI-FI	242
5. 5. Bluetooth connection method	245
5. 6. Instructions for Using MIPI Camera	248
5. 7. 10.1 inch MIPI screen usage method	250
5. 8. Instructions for Using USB Camera	251
5. 9. Android System ROOT Description	253
5. 10. 40 pin interface GPIO, UART, SPI testing	255
5. 10. 1. 40 Pin GPIO Port Testing Method	255
5. 10. 2. 40 pin UART testing method	259
5. 10. 3. 40 pin SPI testing method	261
5. 10. 4. 40 pin I2C testing method	264
5. 10. 5. 40 pin PWM testing method	267
6. Compilation method of Android 13 source code	270
6. 1. Download the source code of Android 13	270
6. 2. Compile the source code for Android 13	271
7. Appendix	273
7. 1. User Manual Update History	273
7. 2. Image update history	273



1. Basic characteristics of Orange Pi Zero 3w

1. 1. What is Orange Pi Zero 3w

The Orange Pi Zero 3w adopts the Quanzhi A733 eight core processor, which integrates an eight core high-performance CPU, a dual core Arm Cortex-A76+six core Arm Cortex-A55 multi-core architecture, a 12nm process, and a maximum clock speed of 2.0GHz; Support 3Tops NPU to meet the needs of edge intelligent AI acceleration applications; Supports up to 16GB LPDDR5, supports up to 8K@24fps H. 265/VP9/AVS2 video format decoding, supports up to 4K@30fps H. 265/H.264 video format encoding; Rich interfaces, including commonly used functional interfaces such as PCIe 3.0, TypeC USB 3.0, UFS, MIPI-CSI, MIPI-DSI, 40Pin expansion interface, etc. Supports operating systems such as Ubuntu, Debian, Android 13, etc.

Orange Pi Zero 3w can provide a solid hardware foundation for the scenario landing of generative AI and artificial intelligence algorithms, and can be widely used in intelligent industrial control, intelligent business display, retail payment, intelligent education, commercial robots, vehicle terminals, visual aids, edge computing, intelligent power distribution terminals, etc.

1. 2. Use of Orange Pi Zero 3w

We can use it to achieve:

- A small Linux desktop computer
- A small Linux network server
- Android tablet
- Android game consoles, etc

Of course, there are many other features as well. With a powerful ecosystem and various expansion accessories, Orange Pi can help users easily achieve delivery from creativity to prototype to mass production. It is an ideal creative platform for makers, dreamers, and hobbyists.



1. 3. Who was the Orange Pi Zero 3w designed for

The Orange Pi development board is not only a consumer product, but also designed for anyone who wants to use technology for creative innovation. It is a simple, fun, and practical tool that you can use to create the world around you.

1. 4. Hardware Features of Orange Pi Zero 3w

Introduction to Hardware Features	
processor	A733, 2 Cortex-A76 cores+6 Cortex-A55 cores, with a maximum frequency of 2.0GHz RISC-V E902 coprocessor (200MHz) GPU: Imagination BXM-4-64 NPU: 3TOPS
memory	Supports up to 16GB LPDDR5
storage	Onboard eMMC: 8GB/16GB/32GB optional (default blank sticker) Onboard UFS: 32GB/64GB/128GB optional (default blank sticker) TF card slot: Supports up to 128GB
Wi Fi+Bluetooth	Wi Fi+Bluetooth two in one module Wi-Fi6 + BT 5.4, BLE
display	1 x HDMI TX 2.0 interface up to 4K@60fps 1 x MIPI DSI 4 LANE 1 x DP1.4 (DisplayPort)
camera	2x 4-lane MIPI-CSI camera interface
USB	1xType-C USB 3.0 HOST 1xType-C USB 2.0 supports HOST and Device
FAN	1x cooling fan interface
button	1 * BOOT (test point), 1 * PWR ON (test point)

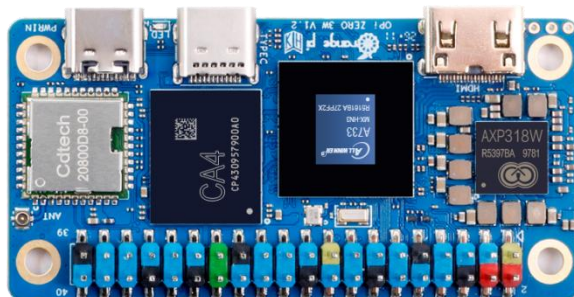


40Pin	40Pin function extension interface, supporting the following interface types: GPIO、UART、I2C、SPI、PWM
DEBUG	3Pin debugging serial port
power supply	Type-C 5V 3A DCIN
Supported OS	Ubuntu、Debian、Android13 etc
Introduction to appearance specifications	
PCB	65mm*30mm
weight	14g

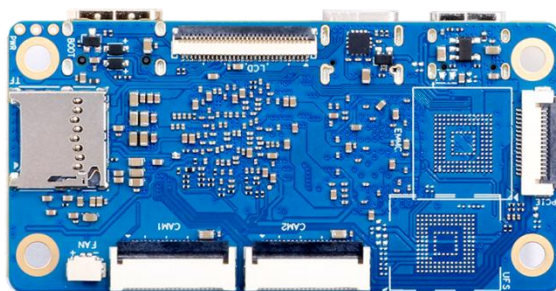


1. 5. Top and Bottom Views of Range Pi Zero 3w

top view:



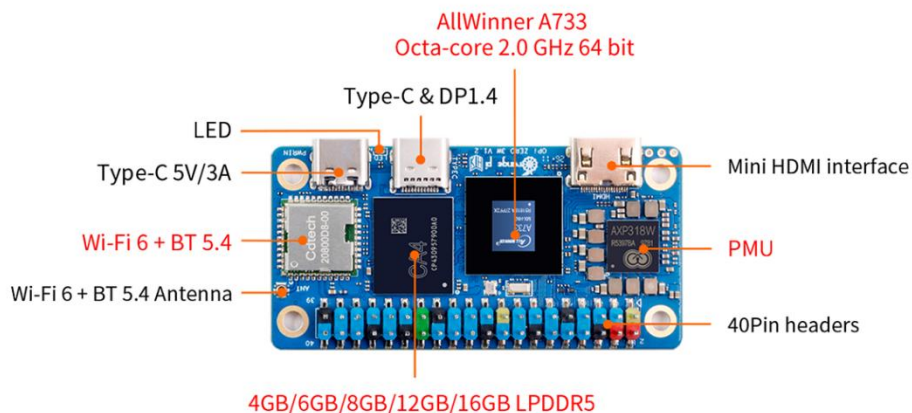
Bottom level view:



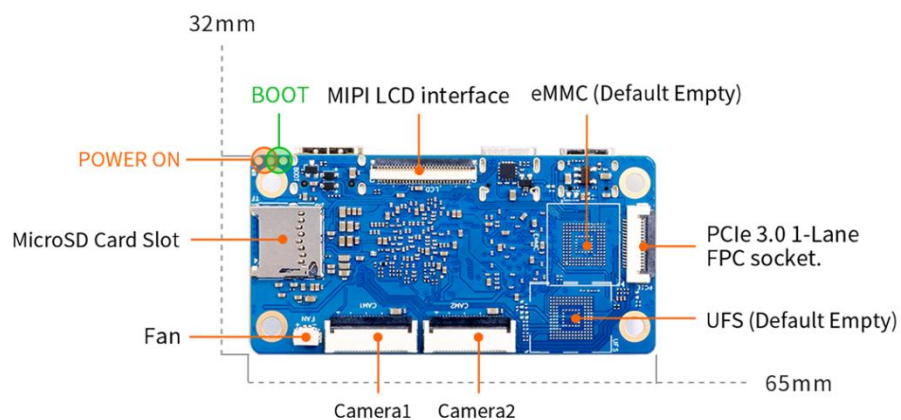


1. 6. Interface Details of Range Pi Zero 3w

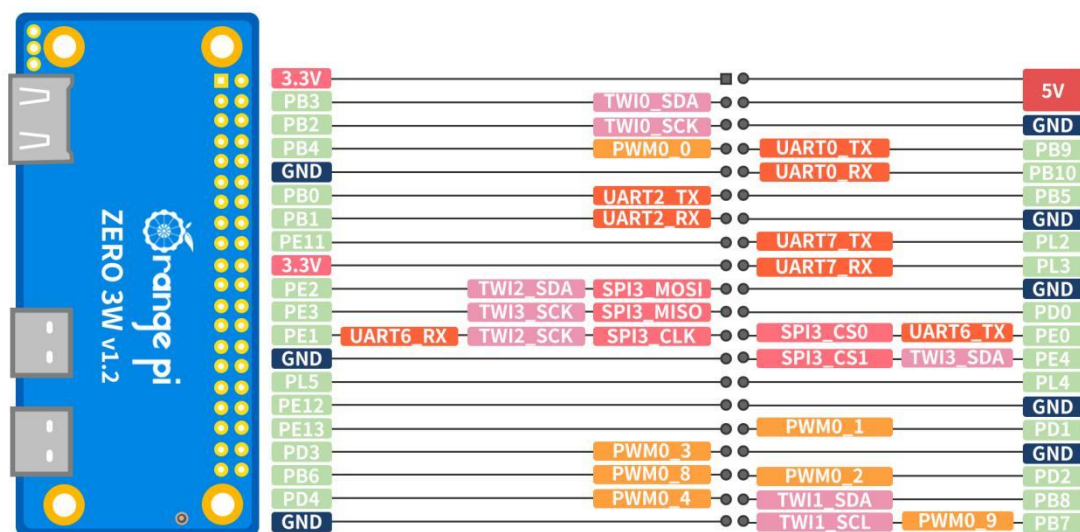
Product display



Top View



Bottom View



The diameter of the four positioning holes is 3.0mm.



2. Introduction to using the development board

2.1. Prepare the necessary accessories

- 1) TF card, high-speed SanDisk card with a minimum capacity of 8GB and **class10** or above



- 2) TF card reader, used for reading and writing TF cards



- 3) HDMI interface display



- 4) Mini HDMI to HDMI cable, used to connect the development board to an HDMI monitor or TV for display



5) Type-C to HDMI cable, connect the development board to an HDMI monitor or TV for display through the Type-C interface



6) Power adapter, Orange Pi Zero 3w. It is recommended to use a 5V/3A Type-C power supply for power supply



The Type-C power interface of the development board does not support PD negotiation function and only supports a fixed 5V voltage input.

7) USB interface mouse and keyboard, any standard USB interface mouse and keyboard can be used to control the Orange Pi development board



8) Type-C to USB cable, used for connecting USB devices



9) When using the serial port debugging function, USB to TTL module and DuPont cable are required to connect the development board and computer



Note that the TTL level used by the development board is 3.3V. In addition to the USB to TTL module shown in the above figure, other similar 3.3V USB to TTL modules are generally acceptable.

10) X64 computer with Ubuntu and Windows operating systems installed

1	Ubuntu22.04 PC	Optional, used for compiling Android and Linux source code
2	Windows PC	Used for burning Android and Linux images



2.2. Download the image of the development board and related materials

1) The download link for the Chinese version of the materials is

<http://www.orange-pi.cn/html/hardware/computerAndMicrocontrollers/service-and-support/Orange-Pi-Zero-3W.html>

2) The download link for the English version of the material is

<http://www.orange-pi.org/html/hardware/computerAndMicrocontrollers/service-and-support/Orange-Pi-Zero-3W.html>

3) The information mainly includes

- a. **Linux source code:** Save on Github
- b. **Android Image:** Save on Baidu Cloud Drive and Google Cloud Drive
- c. **Ubuntu Image:** Save on Baidu Cloud Drive and Google Cloud Drive
- d. **Debian Image:** Save on Baidu Cloud Drive and Google Cloud Drive
- e. **User manual and schematic diagram:** The data manual related to the chip will also be placed here
- f. **Official Tool:** Mainly including the software required during the use of the development board

2.3. Method of burning Linux image to TF card based on Windows PC

Note that the Linux image referred to here specifically refers to Linux distribution images such as Debian or Ubuntu downloaded from the Orange Pi data download page.

2.3.1. Method of burning Linux images using BalenaEtcher 用 balenaEtcher

1) Firstly, prepare a TF card with 8GB or larger capacity. The transfer speed of the TF card must be at or above **class10**. It is recommended to use TF cards from brands such as SanDisk

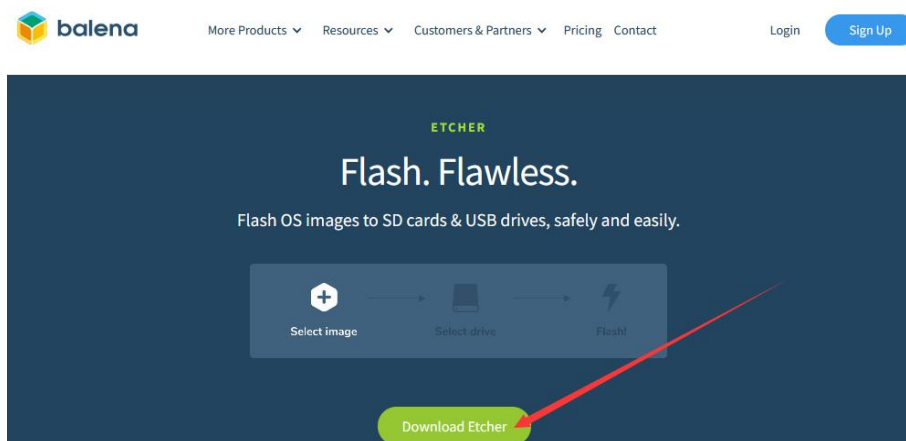


- 2) Then use a card reader to insert the TF card into the computer
- 3) Download the compressed file of the Linux operating system image that you want to burn from the [Orange Pi's download page](#), and then use decompression software to decompress it. In the decompressed file, the file ending with ".img" is the operating system image file, which is usually over 1GB in size

- 4) Then download the Linux image burning software - **balenaEtcher**, from the download link

<https://www.balena.io/etcher/>

- 5) After entering the balenaEtcher download page, clicking the green download button will jump to the software download selection interface



- 6) Then download the installation package for balenaEtcher Windows version.

DOWNLOAD

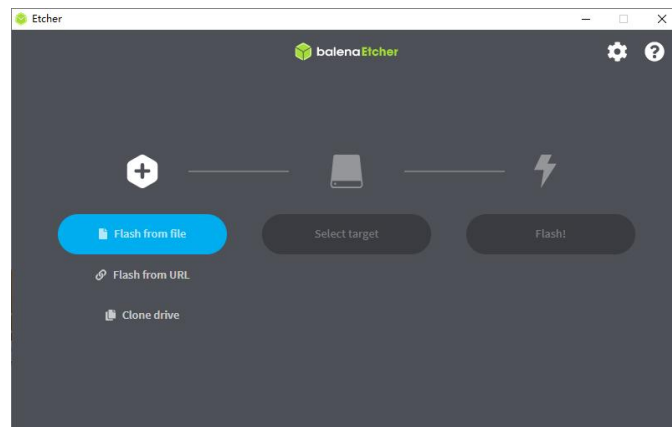
Download Etcher

ASSET	OS	ARCH	
ETCHER FOR WINDOWS (X86 X64) (INSTALLER)	WINDOWS	X86 X64	Download
ETCHER FOR MACOS	MACOS	X64	Download
ETCHER FOR MACOS (ARM64)	MACOS	ARM64	Download
ETCHER FOR LINUX X64 (64-BIT) (ZIP)	LINUX	X64	Download
ETCHER FOR LINUX (LEGACY 32 BIT) (APPIMAGE)	LINUX	X86	Download

- 7) Then install balenaEtcher and open it again. The balenaEtcher interface after opening



is shown in the following figure



When opening balenaEtcher if prompted with the following error:

Attention

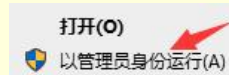
Something went wrong. If it is a compressed image, please check that the archive is not corrupted.

User did not grant permission.

Cancel

Retry

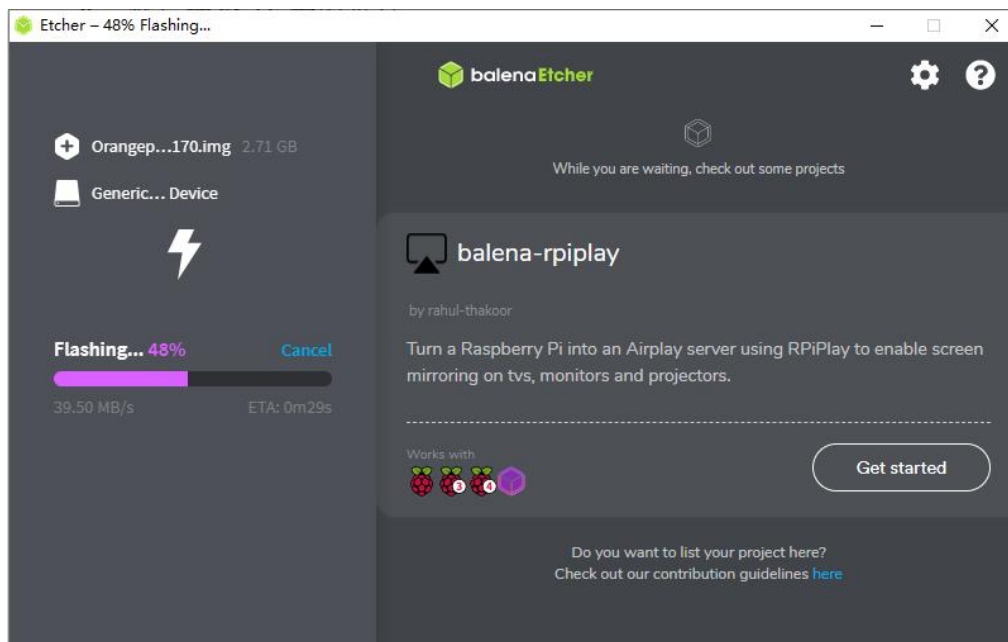
Please select balenaEtcher and right-click, then choose to run as administrator.



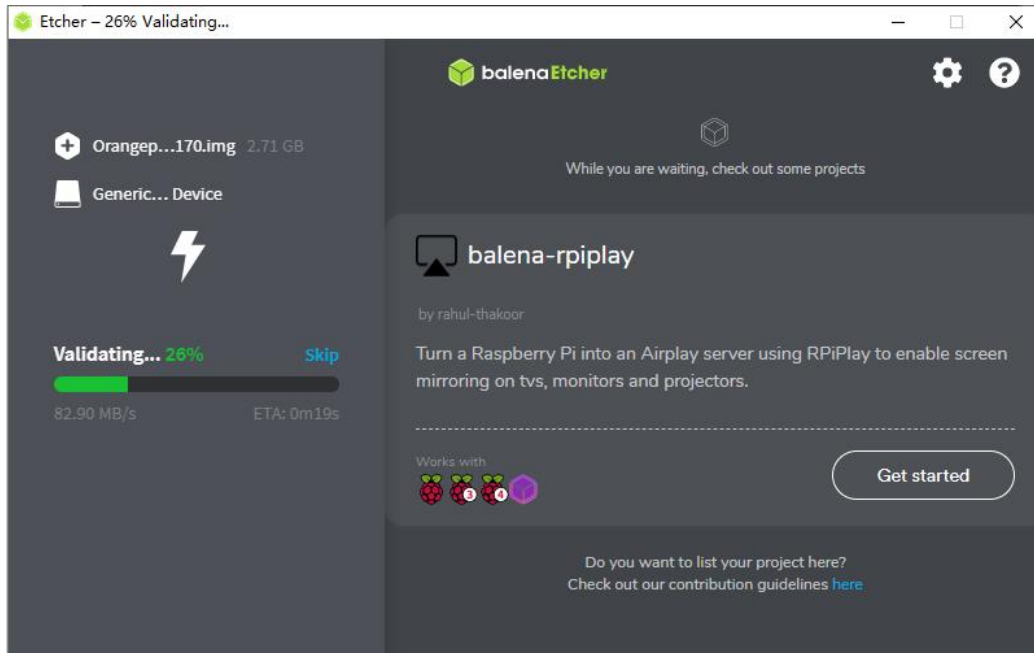
- 8) The specific steps for burning a Linux image using balenaEtcher are as follows
- Firstly, select the path of the Linux image file to be burned
 - Then select the drive letter of the TF card
 - Finally, clicking Flash will start burning the Linux image onto the TF card



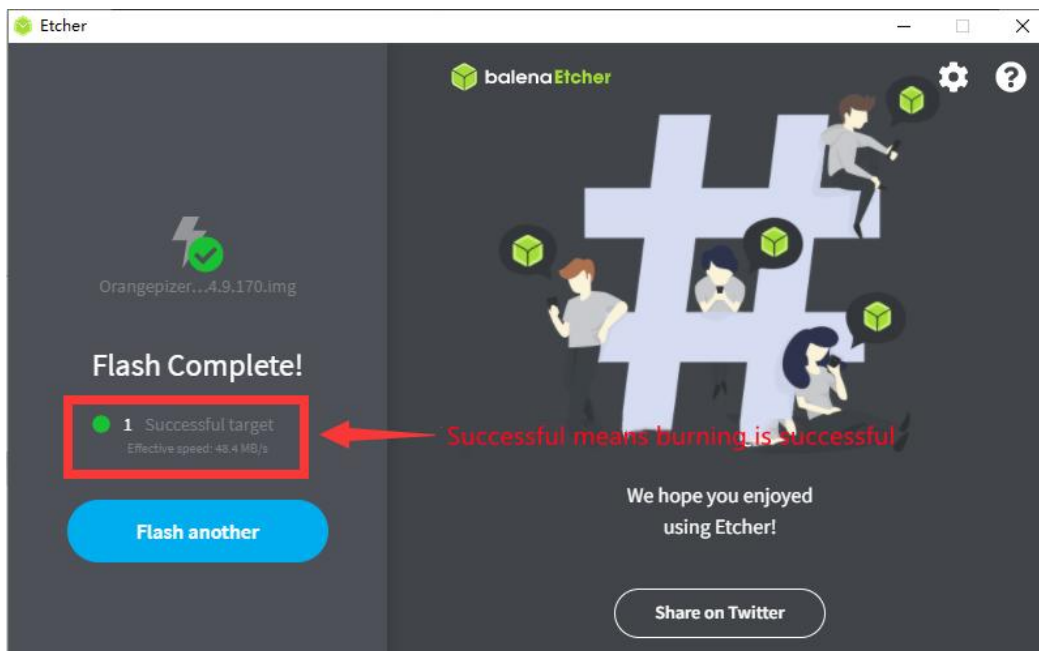
9) The interface displayed during the process of burning a Linux image by balenaEtcher is shown in the following figure. In addition, the progress bar displaying purple indicates that the Linux image is being burned to the TF card



10) After the Linux image is burned, balenaEtcher will also verify the image burned to the TF card by default to ensure that there are no problems during the burning process. As shown in the following figure, a green progress bar indicates that the image has been burned and balenaEtcher is verifying the burned image



11) After successful burning, the display interface of balenaEtcher is shown in the following figure. If a green indicator icon is displayed, it indicates that the image burning is successful. At this time, you can exit balenaEtcher, then unplug the TF card and insert it into the TF card slot of the development board for use



2. 3. 2. Method of burning Linux images using Win32Diskimager

1) Firstly, prepare a TF card with 8GB or larger capacity. The transfer speed of the TF card must be at or above **class10**. It is recommended to use TF cards from brands such as



SanDisk

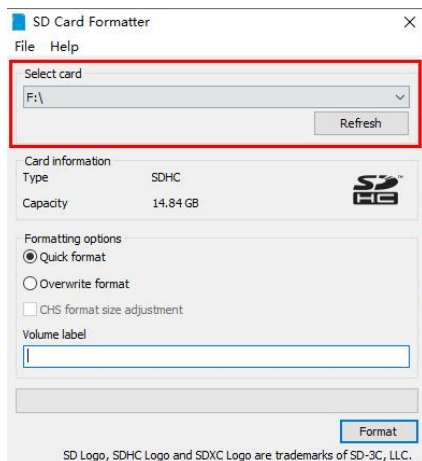
2) Then use a card reader to insert the TF card into the computer

3) Next, format the TF card

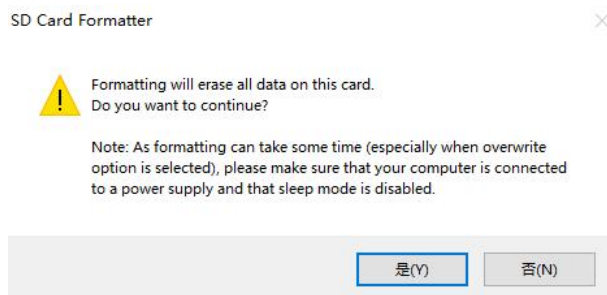
- a. You can use the **SD Card Formatter** software to format TF cards, and its download link is

https://www.sdcard.org/downloads/formatter/eula_windows/SDCardFormatterv5_WinEN.zip

- b. After downloading, simply unzip and install it, and then open the software
- c. If the computer only has a TF card inserted, the "**Select card**" column will display the drive letter of the TF card. If the computer has multiple USB storage devices inserted, you can select the drive letter corresponding to the TF card through the drop-down menu

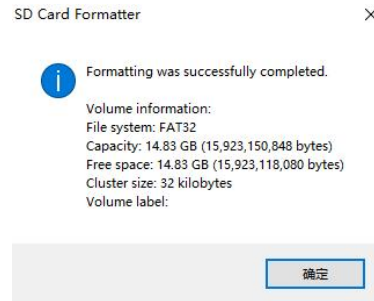


- d. Then click on "**Format**", a warning box will pop up before formatting, and selecting "**Yes (Y)**" will start formatting





- e. After formatting the TF card, a message as shown in the following image will pop up. Click OK to proceed



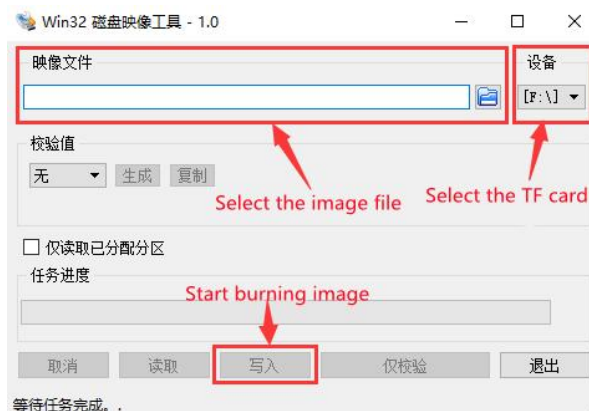
- 4) Download the compressed file of the Linux operating system image that you want to burn from the [Orange Pi's download page](#), and then use decompression software to decompress it. In the decompressed file, the file ending with ".img" is the operating system image file, which is usually over 1GB in size

5) Burn Linux image to TF card using **Win32Diskimager**

- a. The download page for Win32Diskimager is

<http://sourceforge.net/projects/win32diskimager/files/Archive/>

- b. After downloading, simply install it directly. The interface of Win32Diskimager is shown below
- First, select the path of the image file
 - Then confirm that the drive letter of the TF card matches the one displayed in the "Device" column
 - Finally, click "Write" to start burning





- c. After the image writing is completed, click the "Exit" button to exit, and then you can unplug the TF card and insert it into the development board to start

2.4. Method for burning Linux images to TF cards based on Ubuntu PC

Note that the Linux image referred to here specifically refers to Linux distribution images such as Debian or Ubuntu downloaded from the Orange Pi data download page, while Ubuntu PC refers to a personal computer with the Ubuntu system installed.

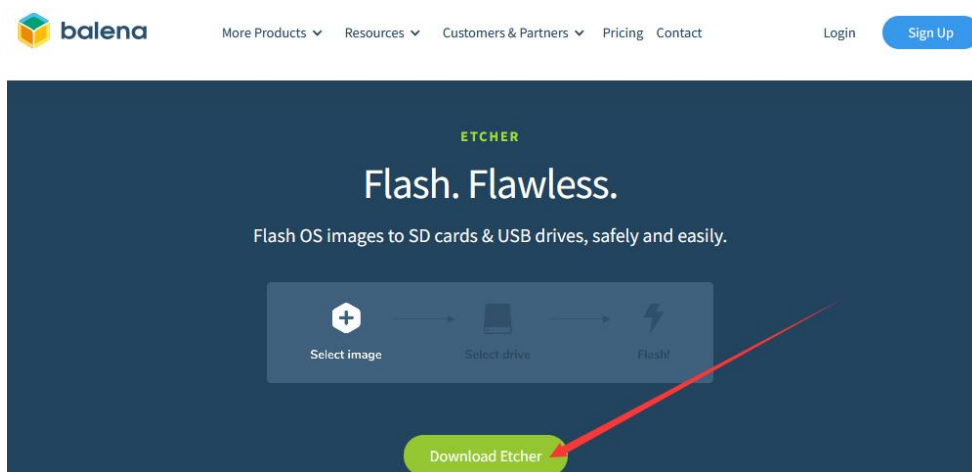
1) Firstly, prepare a TF card with 8GB or larger capacity. The transfer speed of the TF card must be at or above **class10**. It is recommended to use TF cards from brands such as SanDisk

2) Then use a card reader to insert the TF card into the computer

3) Download the balenaEtcher software from the following link:

<https://www.balena.io/etcher/>

4) After entering the balenaEtcher download page, clicking the green download button will jump to the software download location



5) Then choose to download the Linux version of the software



DOWNLOAD

Download Etcher

ASSET	OS	ARCH	
ETCHER FOR WINDOWS (X86 X64) (INSTALLER)	WINDOWS	X86 X64	Download
ETCHER FOR WINDOWS (X86 X64) (PORTABLE)	WINDOWS	X86 X64	Download
ETCHER FOR WINDOWS (LEGACY 32 BIT) (X86 X64) (PORTABLE)	WINDOWS	X86 X64	Download
ETCHER FOR MACOS	MACOS	X64	Download
ETCHER FOR LINUX X64 (64-BIT) (APPIMAGE)	LINUX	X64	Download
ETCHER FOR LINUX (LEGACY 32 BIT) (APPIMAGE)	LINUX	X86	Download

Looking for [Debian \(.deb\)](#) packages or [Red Hat \(.rpm\)](#) packages?OSS hosting by [cloudsmith](#)

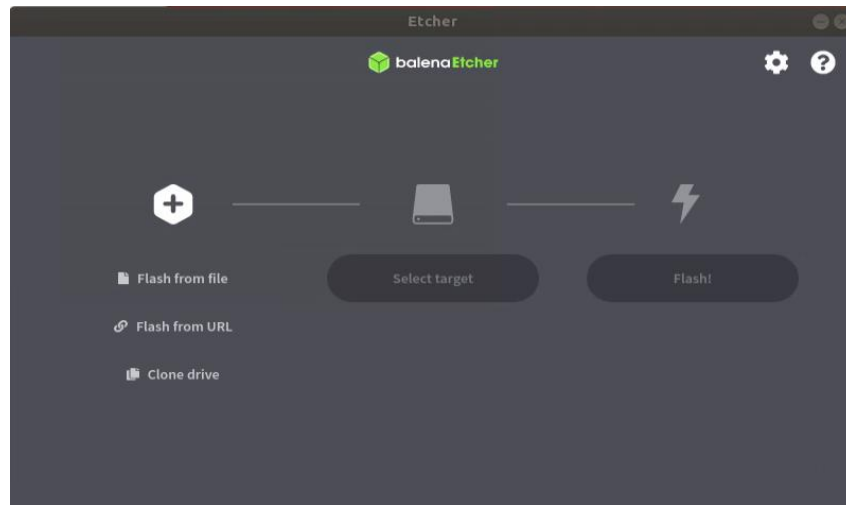
6) Download the compressed file of the Linux operating system image that you want to burn from the [Orange Pi download page](#), and then use decompression software to decompress it. In the decompressed file, the file ending with ".img" is the operating system image file, which is usually over 1GB in size. The decompression command for the compressed file ending in 7z is as follows:

```
test@test:~$ 7z x OrangepiZero3w_1.0.0_ubuntu_jammy_desktop_linux5.15.147.7z
test@test:~$ ls OrangepiZero3w_1.0.0_ubuntu_jammy_desktop_linux5.15.147.*
OrangepiZero3w_1.0.0_ubuntu_jammy_desktop_linux5.15.147.7z
OrangepiZero3w_1.0.0_ubuntu_jammy_desktop_linux5.15.147.sha      #Verification
and file
OrangepiZero3w_1.0.0_ubuntu_jammy_desktop_linux5.15.147.img     #image file
```

7) After decompressing the image, you can first use the `sha256sum -c *.sha` command to calculate if the checksum is correct. If the prompt is **successful**, it means that the downloaded image is correct and can be safely burned to the TF card. If the prompt is that the **checksum does not match**, it means that the downloaded image has a problem. Please try downloading it again

```
test@test:~$ sha256sum -c *.sha
OrangepiZero3w_1.0.0_ubuntu_jammy_desktop_linux5.15.147.img: success
```

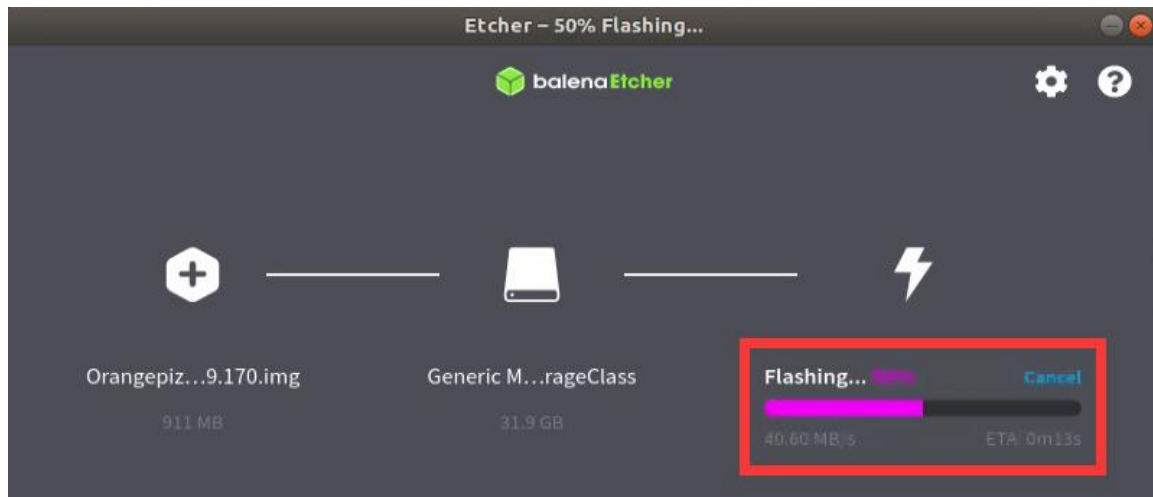
8) Then double-click **balenaEtcher-1.14.3-x64.AppImage** on the graphical interface of Ubuntu PC to open BalenaEtcher (**no installation required**). The interface displayed after opening BalenaEtcher is shown in the following figure



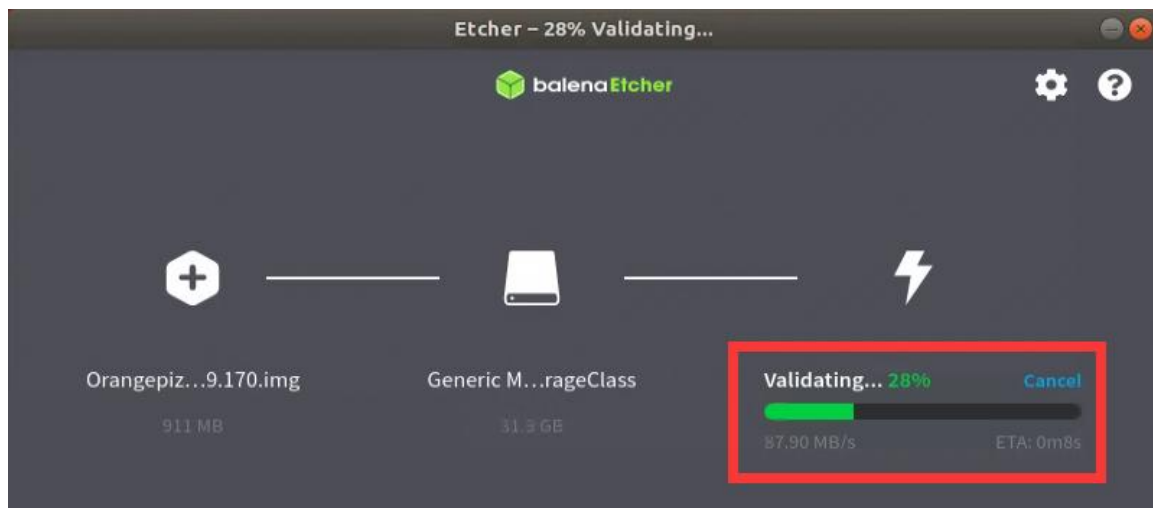
- 9) The specific steps for burning a Linux image using balenaEtcher are as follows
- Firstly, select the path of the Linux image file to be burned
 - Then select the drive letter of the TF card
 - Finally, clicking Flash will start burning the Linux image onto the TF card



- 10) The interface displayed during the process of burning a Linux image by balenaEtcher is shown in the following figure. In addition, the progress bar displaying purple indicates that the Linux image is being burned to the TF card



11) After the Linux image is burned, balenaEtcher will also verify the image burned to the TF card by default to ensure that there are no problems during the burning process. As shown in the following figure, a green progress bar indicates that the image has been burned and balenaEtcher is verifying the burned image



12) After successful burning, the display interface of balenaEtcher is shown in the following figure. If a green indicator icon is displayed, it indicates that the image burning is successful. At this time, you can exit balenaEtcher, then unplug the TF card and insert it into the TF card slot of the development board for use



2. 5. Method for burning Linux images to eMMC

See [the method of burning Linux images to EMMC](#)

2. 6. Method for burning Linux images to UFS

See [the method of burning Linux images to UFS](#)

2. 7. Method of burning Android image to TF card

The Android image of the development board can only be burned to a TF card using **PhoenixCard** software on the Windows platform, and the version of PhoenixCard software must be **PhonixCard-4.2.8**.

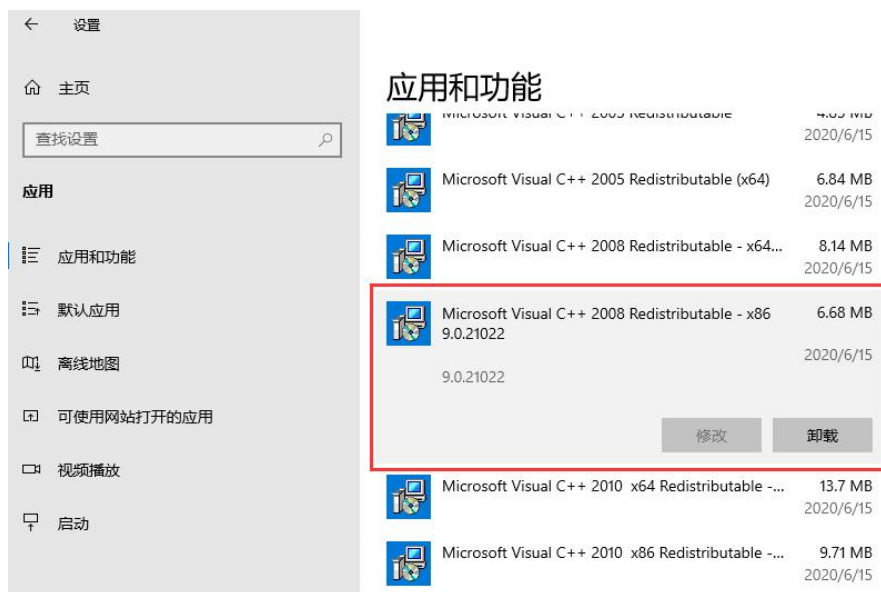
Please do not use software that burns Linux images, such as Win32Diskimager or balenaEtcher, to burn Android images.

In addition, PhoenixCard software does not have versions for Linux and Mac platforms, so it is not possible to burn Android images to TF cards on Linux and

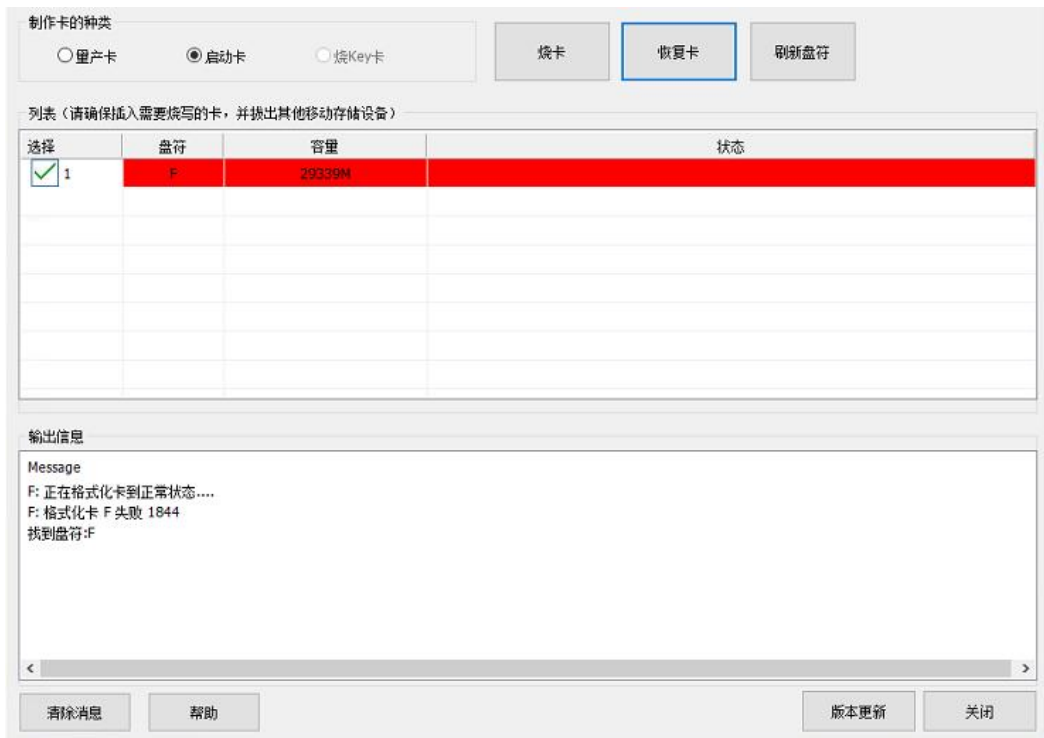


Mac platforms.

1) Firstly, please ensure that the Windows system has installed **Microsoft Visual C++ 2008 Redistributable - x86**

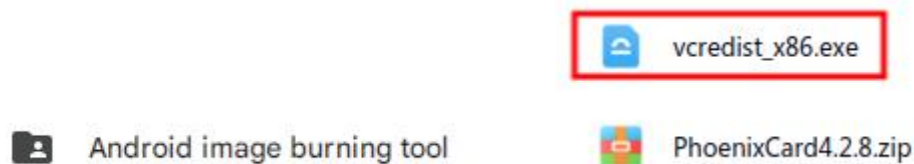


2) If **Microsoft Visual C++ 2008 Redistributable - x86** is not installed, formatting the TF card with **PhoenixCard** or burning the Android image will prompt the following error





3) The installation package for **Microsoft Visual C++ 2008 Redistributable - x86** can be downloaded from the **official tool** of Orange Pi Zero 3w or from the **Microsoft official website**



4) Then prepare a TF card with 8GB or larger capacity, and the transfer speed of the TF card must be at or above **class10**. It is recommended to use TF cards from brands such as SanDisk

5) Then use a card reader to insert the TF card into the computer

6) Download the Android image and PhoenixCard burning tool from the **Orange Pi's download page**. Please ensure that the version of the PhonenixCrad tool is **PhonixCard-4.2.8. Do not use PhonixCard software below version 4.2.8 to burn the Android image**, as Android images burned by PhonixCard tools below this version may have problems



7) Then use decompression software to decompress the downloaded Android image compressed file. In the decompressed file, the file ending with **".img"** is the Android image file, with a size of 1GB or more. If you don't know how to decompress the compressed file of an Android image, you can install a **360 compression software** to decompress the image.



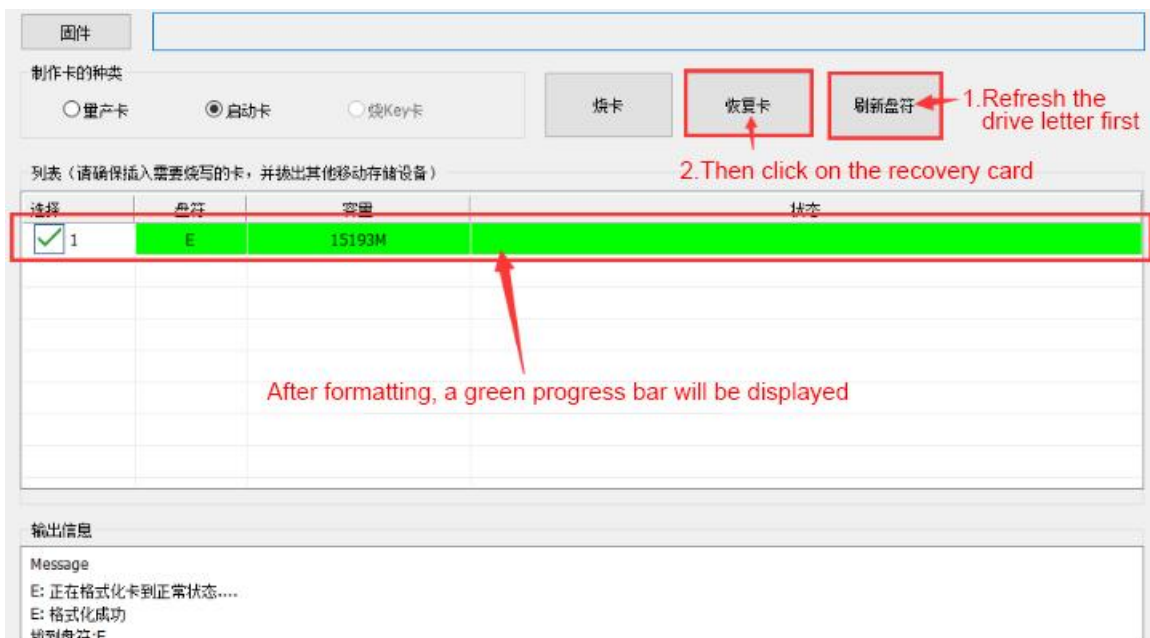
8) Then use decompression software to extract **PhonixCard4.2.8.zip**. This software does not require installation, just find PhoenixCard in the extracted folder and open it

readme.txt	2019/12/31 11:33	应用程序扩展	24 KB
Mbr2Gpt.dll	2019/2/27 13:34	应用程序扩展	9 KB
option.cfg	2019/4/22 15:57	CFG 文件	1 KB
ParserManager.dll	2019/1/10 14:51	应用程序扩展	81 KB
PhoenixCard	2019/12/31 11:29	应用程序	1,748 KB
PhoenixCard.exe	2019/12/31 10:42	LAN 文件	3 KB

9) After opening PhoenixCard, if the TF card is recognized normally, the drive letter and capacity of the TF card will be displayed in the middle list. **Please make sure that the displayed drive letter is consistent with the drive letter of the TF card you want to burn.** If it is not displayed, you can try unplugging the TF card or clicking the "**Refresh Drive Letter**" button in PhoenixCard



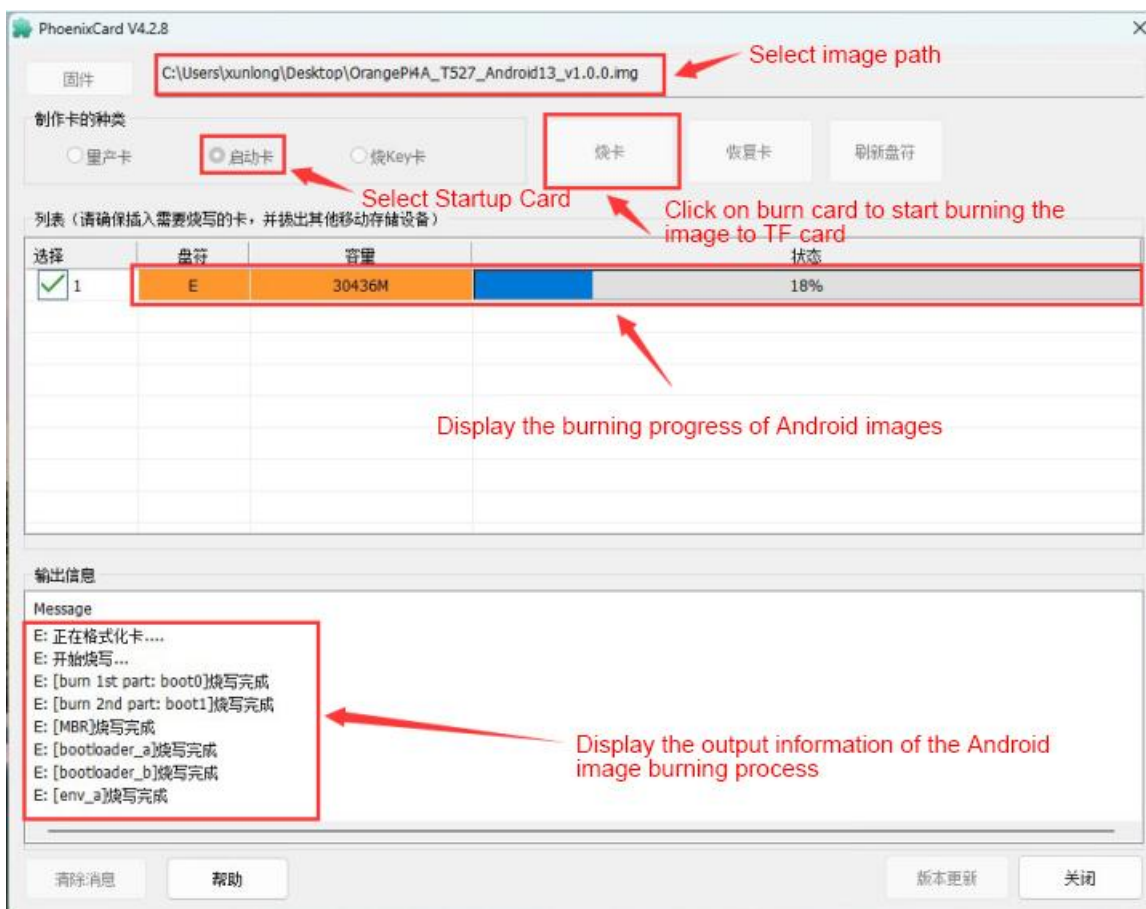
10) After confirming the drive letter, format the TF card first and click the "**Restore Card**" button in PhoenixCard (if the "**Restore Card**" button is gray and cannot be pressed, you can click the "**Refresh Drive Letter**" button first)



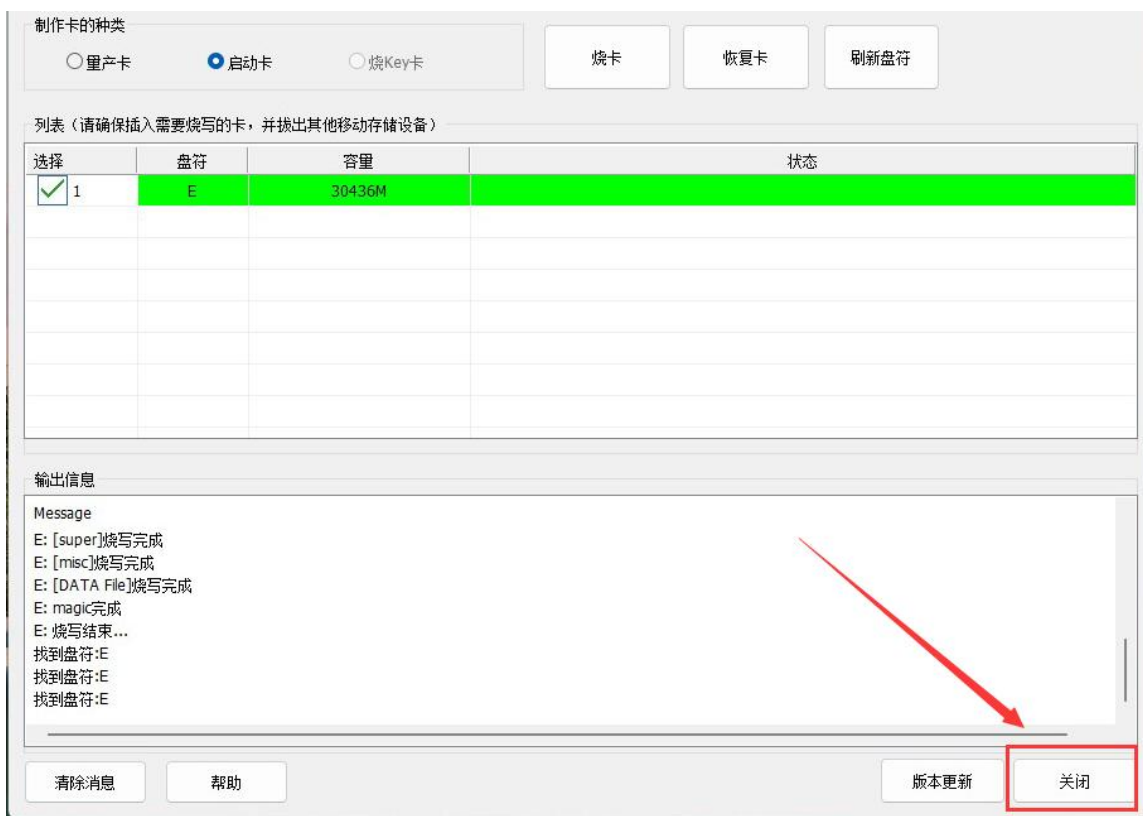
If there is a problem with formatting, please try unplugging and unplugging the TF card before testing again. If the problem persists after unplugging and unplugging the TF card again, you can restart your Windows computer or switch to another computer and try again.

11) Then start writing the Android image to the TF card

- Firstly, select the path of the Android image in the "**Firmware**" column
- Select "**Startup Card**" in the "**Types of Cards to Make**" section
- Then click the "**Burn Card**" button to start burning



12) After burning, the display of PhoenixCard is shown in the following figure. Click the "Close" button to exit PhoenixCard, and then you can unplug the TF card from the computer and insert it into the development board to start it



After burning the Android system, only one 128 MB partition can be seen on the TF card in Windows, as shown in the following figure (some computers may pop up more than twenty disk partitions, but can only open the 128 MB partition). Please note that this is normal and do not burn out the TF card. The reason for this is that the Android system has over twenty partitions, but most of them cannot be recognized properly in the Windows system. At this point, please feel free to unplug the TF card and insert it into the development board to start.



After starting the Android system, use the following command to see these twenty partitions in the TF card:

```
console:/ # ls /dev/block/mmcblk0*
/dev/block/mmcblk0      /dev/block/mmcblk0p18  /dev/block/mmcblk0p27
/dev/block/mmcblk0p1   /dev/block/mmcblk0p19  /dev/block/mmcblk0p28
/dev/block/mmcblk0p10  /dev/block/mmcblk0p2   /dev/block/mmcblk0p3
/dev/block/mmcblk0p11  /dev/block/mmcblk0p20  /dev/block/mmcblk0p4
/dev/block/mmcblk0p12  /dev/block/mmcblk0p21  /dev/block/mmcblk0p5
/dev/block/mmcblk0p13  /dev/block/mmcblk0p22  /dev/block/mmcblk0p6
/dev/block/mmcblk0p14  /dev/block/mmcblk0p23  /dev/block/mmcblk0p7
/dev/block/mmcblk0p15  /dev/block/mmcblk0p24  /dev/block/mmcblk0p8
/dev/block/mmcblk0p16  /dev/block/mmcblk0p25  /dev/block/mmcblk0p9
/dev/block/mmcblk0p17  /dev/block/mmcblk0p26
```



Using the `df -h` command, you can see that after burning the 32GB TF card to the Android system, there is still approximately 24GB of space available for use (not all of the twenty partitions will be mounted to the Android system, so focus on the visible partitions).

```
console:/ # df -h
Filesystem                Size      Used Avail Use% Mounted on
tmpfs                     963M    1.2M    961M   1% /dev
tmpfs                     963M         0 963M   0% /mnt
/dev/block/dm-0            803M    803M         0 100% /
/dev/block/dm-4            232K     36K    196K  16% /system_dlxm
/dev/block/dm-1             88M     88M         0 100% /vendor
/dev/block/dm-3             11M     11M         0 100% /vendor_dlxm
/dev/block/dm-2           106M    106M         0 100% /product
tmpfs                     963M    8.0K    963M   1% /apex
tmpfs                     963M   488K    962M   1% /linkerconfig
/dev/block/mmcblk0p21       10M    120K     10M   2% /metadata
/dev/block/mmcblk0p22       80M     54M     26M  67% /treadahead
/dev/block/mmcblk0p26       16M         0     16M   0% /oem
/dev/block/mmcblk0p27       64M    4.0K     64M   1% /Reserve0
/dev/block/dm-5           26G    1.8G    24G   8% /data
tmpfs                     963M         0 963M   0% /data_mirror
/dev/fuse                  26G    1.8G    24G   8% /mnt/user/0/emulated
/dev/block/vold/public:179,1 128M    18M    109M  15% /mnt/media_rw/extsd
/dev/fuse                  128M    18M    109M  15% /mnt/user/0/extsd
```

2.8. Method for burning Android images to eMMC

The Android image of the development board can only be burned to eMMC using **PhoenixCard** software on the Windows platform, and the version of PhoenixCard software must be **PhonixCard-4.2.8**.

Please do not use software that burns Linux images, such as Win32Diskimager or balenaEtcher, to burn Android images.

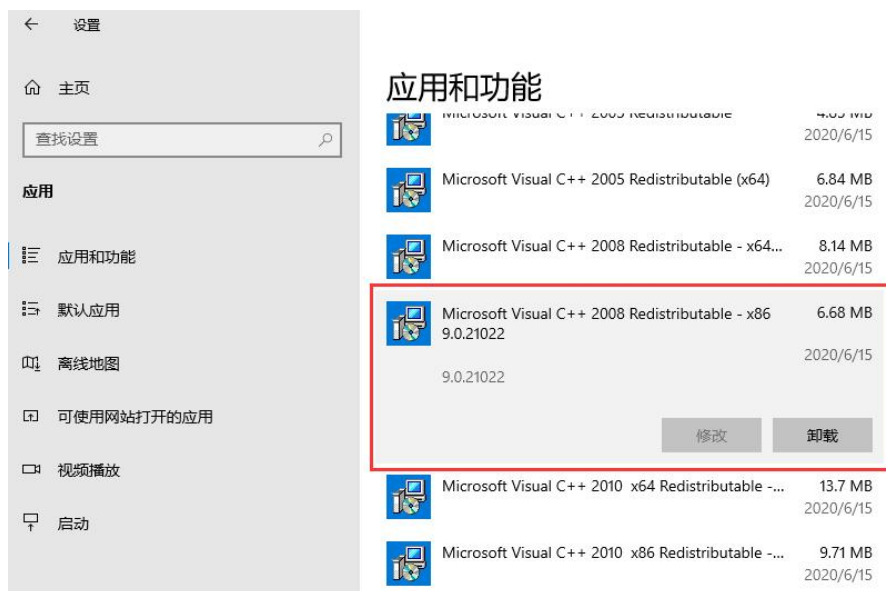
In addition, PhoenixCard software does not have versions for Linux and Mac platforms, so it is not possible to burn Android images to eMMC on Linux and Mac platforms.

Before performing the following operations, please ensure that the development board is labeled with eMMC.

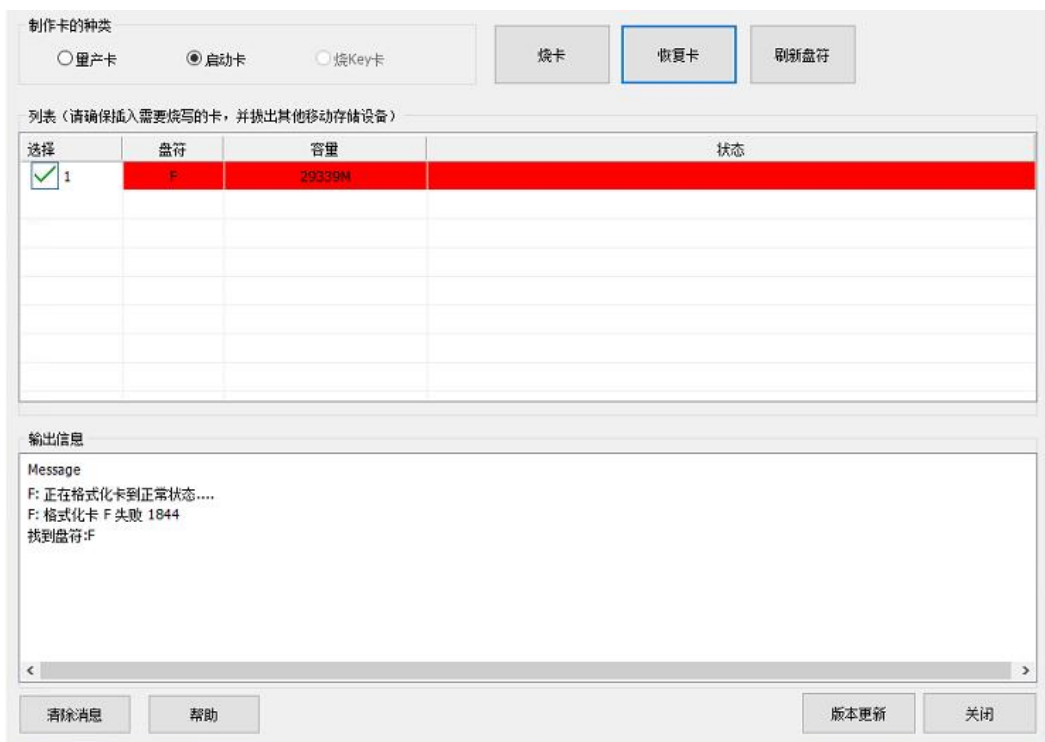
- 1) Firstly, please note that this method requires the use of a TF card and is mainly divided into the following two steps
 - a. First, use PhoenixCard to burn the Android firmware onto the TF card as a production card
 - b. Then use a TF card to burn the Android firmware into eMMC



2) Please ensure that the Windows system has installed **Microsoft Visual C++ 2008 Redistributable - x86**

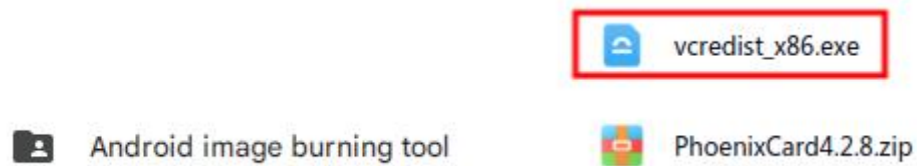


3) If **Microsoft Visual C++ 2008 Redistributable - x86** is not installed, formatting the TF card with PhoenixCard or burning the Android image will prompt the following error





- 4) **Microsoft Visual C++ 2008 Redistributable - x86** 的 The installation package for **Microsoft Visual C++ 2008 Redistributable - x86** can be downloaded from the **official tool** of Orange Pi Zero 3w or from the **Microsoft official website**



- 5) Then prepare a TF card with 8GB or larger capacity, and the transfer speed of the TF card must be at or above **class10**. It is recommended to use TF cards from brands such as SanDisk

- 6) Then use a card reader to insert the TF card into the computer

- 7) Download the Android image and PhoenixCard burning tool from the **Orange Pi's download page**. **Please ensure that the version of the PhonenixCrad tool is PhonexCard-4.2.8. Do not use PhonixCard software below version 4.2.8 to burn the Android image**, as Android images burned by PhonixCard tools below this version may have problems



- 8) Then use decompression software to decompress the downloaded Android image compressed file. In the decompressed file, the file ending with **".img"** is the Android image file, with a size of 1GB or more. If you don't know how to decompress the compressed file of an Android image, you can install a **360 compression software** to decompress the image.



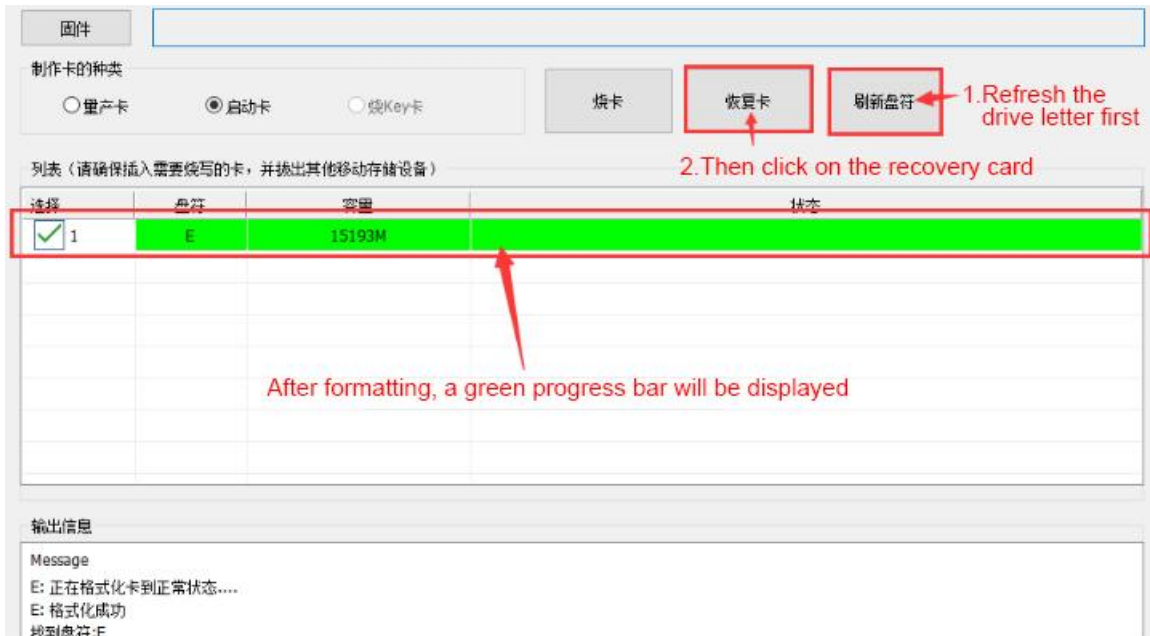
9) Then use decompression software to extract **PhonixCard4.2.8.zip**. This software does not require installation, just find PhoenixCard in the extracted folder and open it

mssock.dll	2019/2/27 11:33	应用程序扩展	9 KB
Mbr2Gpt.dll	2019/2/27 13:34	应用程序扩展	9 KB
option.cfg	2019/4/22 15:57	CFG 文件	1 KB
ParserManager.dll	2019/1/10 14:51	应用程序扩展	81 KB
PhoenixCard	2019/12/31 11:29	应用程序	1,748 KB
PhoenixCard.dll	2019/12/31 10:42	LAN 文件	3 KB

10) After opening PhoenixCard, if the TF card is recognized normally, the drive letter and capacity of the TF card will be displayed in the middle list. **Please make sure that the displayed drive letter is consistent with the drive letter of the TF card you want to burn.** If it is not displayed, you can try unplugging the TF card or clicking the "Refresh Drive Letter" button in PhoenixCard



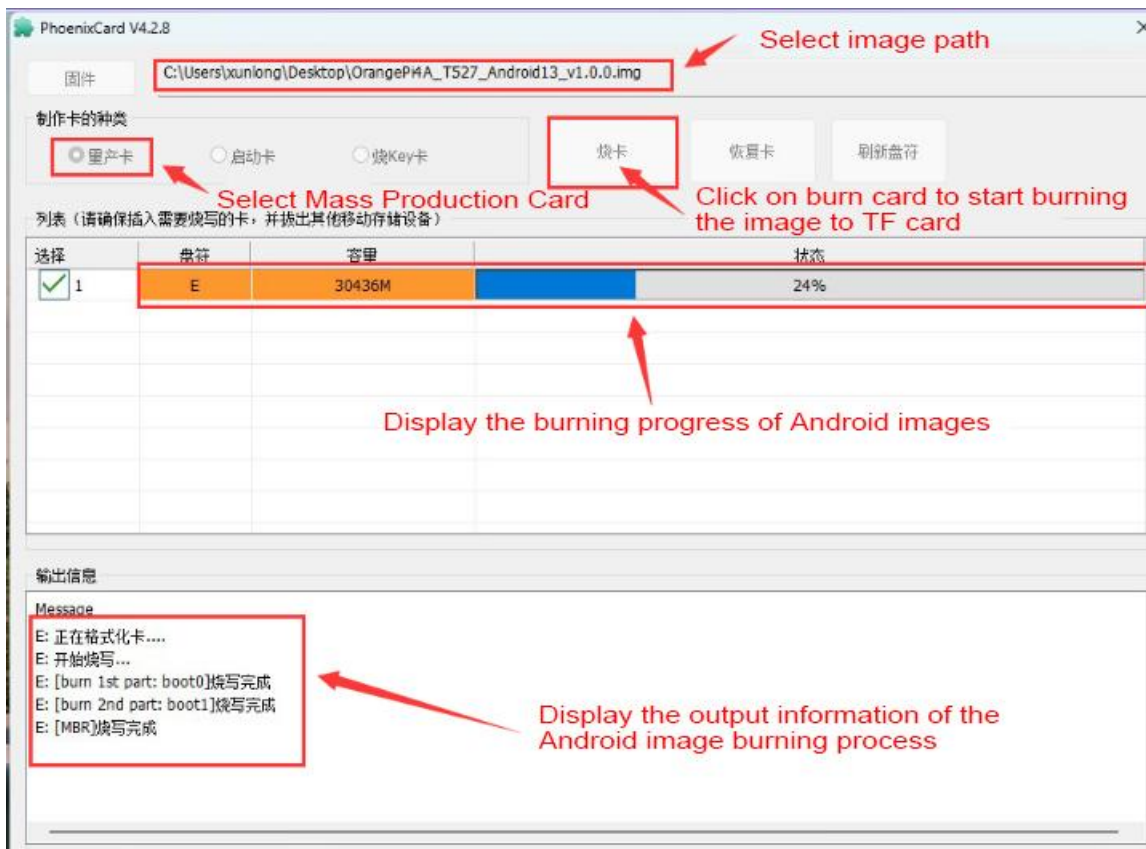
11) After confirming the drive letter, format the TF card first and click the "Restore Card" button in PhoenixCard (if the "Restore Card" button is gray and cannot be pressed, you can click the "Refresh Drive Letter" button first)



If there is a problem with formatting, please try unplugging and unplugging the TF card before testing again. If the problem persists after unplugging and unplugging the TF card again, you can restart your Windows computer or switch to another computer and try again.

12) Then start writing the Android image to the TF card

- Firstly, select the path of the Android image in the "**Firmware**" column
- Select "**Mass Production Card**" in the "**Types of Card Production**" section
- Then click the "**Burn Card**" button to start burning



13) After burning, the display of PhoenixCard is shown in the following figure. Click the "Close" button to exit PhoenixCard





14) Then insert the TF card into the development board. After powering on and starting the development board, the Android firmware in the TF card will be automatically burned to the eMMC of the development board. After the burning is completed, it will automatically shut down and the LED light on the development board will turn off

15) At this point, you can unplug the TF card and then power it back on, which will start the Android system in eMMC

2.9. Method of burning Android images to eMMC or UFS through TypeC port

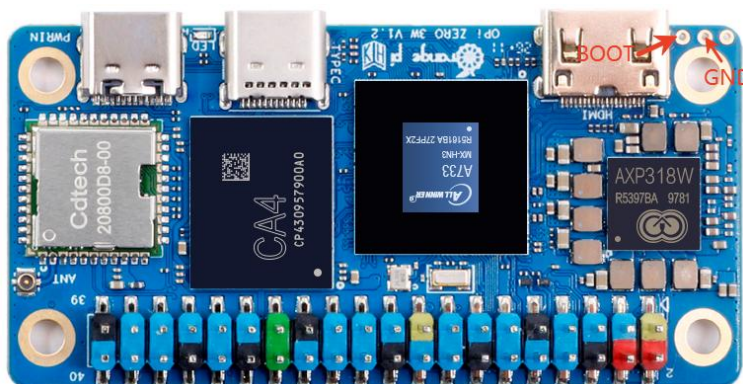
Note that all the following operations are performed on a Windows computer, and it is also necessary to ensure that eMMC or UFS is already affixed to the board.

1) Firstly, it is necessary to prepare a high-quality Type-C interface data cable

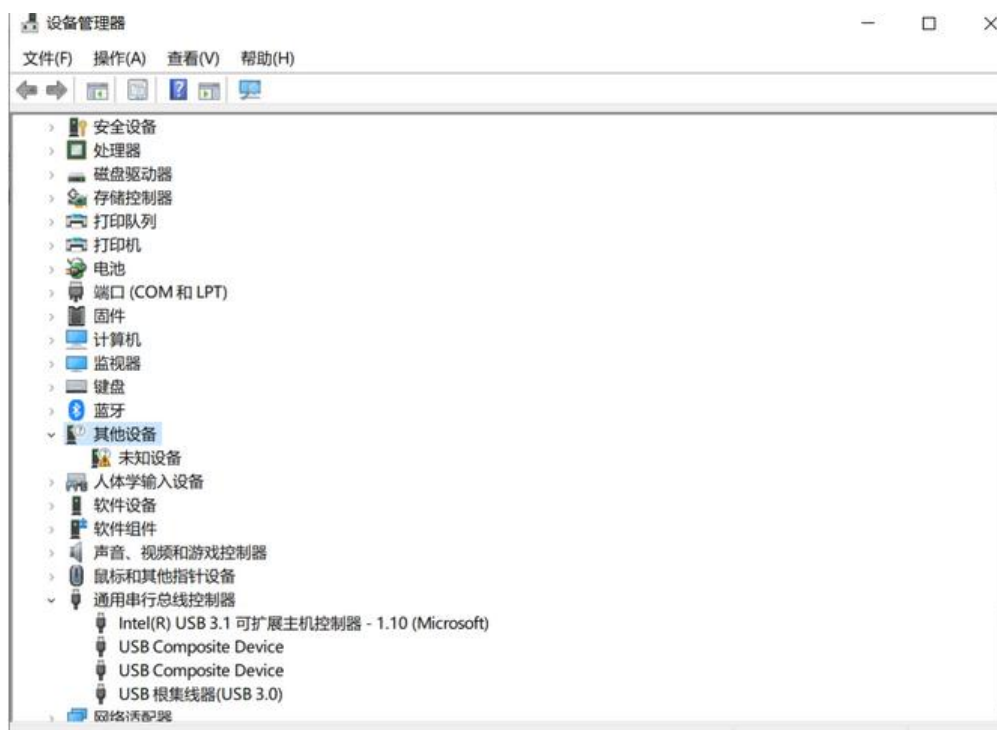


2) Then download the **AllwinnerUSBFlashDeviceDriver.zip** and burning tool **AllwinnertechPhoeniSuit.zip** from the [Orange Pi's download page](#)

3) Then short-circuit the two test points pointed by the arrows in the figure, with the purpose of short circuiting BOOT and GND to enter FEL mode (**if there is no system on the board, it can also be short circuited**)



4) Then connect the development board and computer through a TypeC data cable (**note that BOOT and GND need to be short circuited before connecting the development board and computer to power on**). At this point, the device manager of the computer should be able to detect an unknown device



5) Then unzip **AllwinnerUSBFlashDeviceDriver.zip**. If it is a Win7 system, find the **install** script executable file in the decompressed folder and open it to start the installation



UsbDriver	3/25/2026 4:17 PM	File folder	
drvinstaller_IA64	4/6/2016 11:27 AM	Application	51 KB
drvinstaller_X64	4/6/2016 11:27 AM	Application	27 KB
drvinstaller_X86	4/6/2016 11:27 AM	Application	34 KB
install	4/6/2016 11:27 AM	Windows Batch File	1 KB
InstallUSBDrv	3/24/2020 4:41 PM	Application	256 KB

6) If it is a Windows 10 system, you need to right-click on the unknown device, **click on "Browse My Computer" in the pop-up dialog box to find the driver software**, and select the driver path above for installation

← 更新驱动程序 - 未知设备

你要如何搜索驱动程序?

→ 自动搜索更新的驱动程序软件(S)

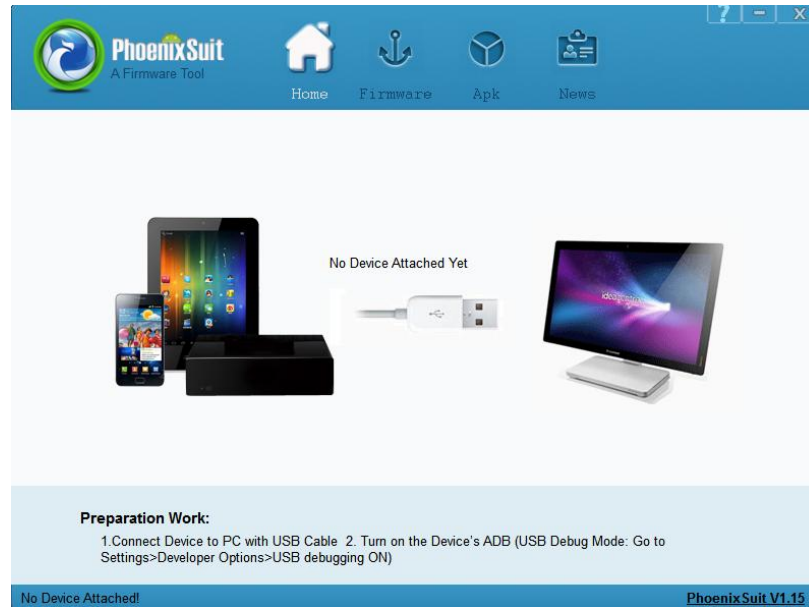
Windows 将搜索你的计算机和 Internet 以获取适合你设备的最新驱动程序软件, 除非你已在设备安装设置中禁用此功能。

→ 浏览我的计算机以查找驱动程序软件(R)

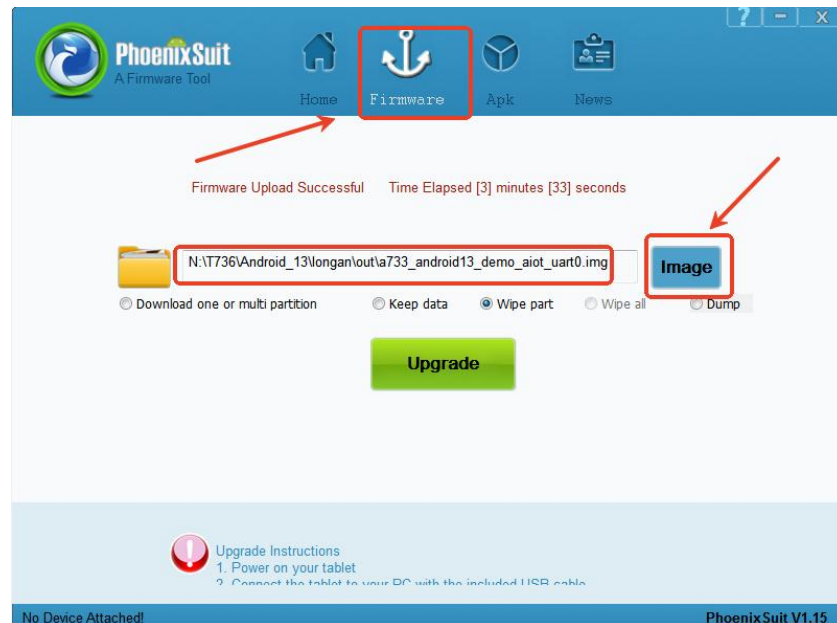
手动查找并安装驱动程序软件。

7) After the installation of the Quanzhi USB driver is completed, we will unzip **AllwinnertechPhoeniSuit.zip** and open the **PhoenixSuit** burning program

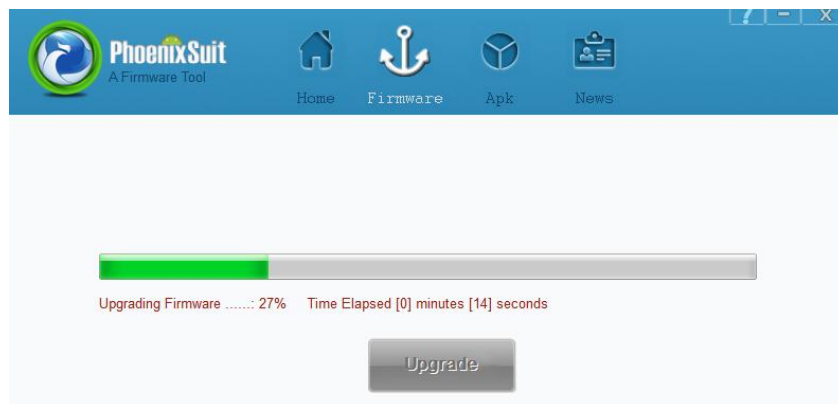
PhoenixDaemon	6/15/2020 10:29 AM	Application	14,822 KB
PhoenixInstall	10/14/2020 7:04 PM	Application	1,705 KB
PhoenixSuit.cfg	3/25/2026 10:29 AM	CFG File	1 KB
PhoenixSuit	10/14/2020 7:04 PM	Application	2,218 KB
PhoenixSuit.lan	10/14/2020 7:04 PM	LAN File	30 KB
PhoenixSuit	10/14/2020 7:04 PM	WPS PDF 文档	1,403 KB
PhoenixSuit_ChangeLog	10/14/2020 7:04 PM	WPS PDF 文档	189 KB
PlgMan.dll	6/15/2020 10:29 AM	Application extens...	24 KB



8) Then click on **Firmware** to switch to the flashing window, and then click on Image to select the Android **Image** path



9) Then we first disconnect the TypeC data cable from the computer, short-circuit the FEL and GND, and reconnect the development board to the computer. At this point, the tool will automatically burn the image to eMMC or UFS. After burning is complete, the board will automatically start the Android system



2. 10. Launch the Orange Pie development board

- 1) Insert the TF card with the burned image into the TF card slot of the Orange Pi development board
- 2) The development board has a Mini HDMI interface, which can be connected to a TV or HDMI monitor through a Mini HDMI to HDMI cable. If there is a Type-C to HDMI cable, the system interface of the development board can also be displayed through the Type-C interface.
- 3) Connect a USB mouse and keyboard to control the Orange Pie development board
- 4) Connect a **high-quality** power adapter with a 5V/3A (5V/4A is also acceptable) USB Type C interface

Remember not to insert a power adapter with a voltage output greater than 5V, as it may burn out the development board.

Many unstable phenomena during the power on startup process of the system are basically caused by power supply problems, so a reliable power adapter is very important. If you notice continuous restarts during the startup process, please replace the power supply or Type C data cable and try again.

- 5) Then turn on the power adapter switch. If everything is normal, the HDMI monitor will be able to see the system startup screen
- 6) If you want to view the system's output information by debugging the serial port,



please connect the development board to the computer using a serial port cable. For the method of connecting the serial port, please refer to the section on [debugging the serial port usage](#)

2. 11. Instructions for Debugging Serial Ports

2. 11. 1. Debugging serial port connection instructions

1) First, you need to prepare a 3.3v USB to TTL module, and then plug one end of the USB interface of the USB to TTL module into the USB interface of the computer

香橙派

USB to TTL module



The 3.3V USB to TTL module does not require connection

TXD of USB to TTL module connected to development board for debugging RXD of serial port

RXD of USB to TTL module connected to development board for debugging TXD of serial port

Connect the GND of the USB to TTL module to the development board and debug the GND of the serial port

The 5V conversion from USB to TTL module does not require connection

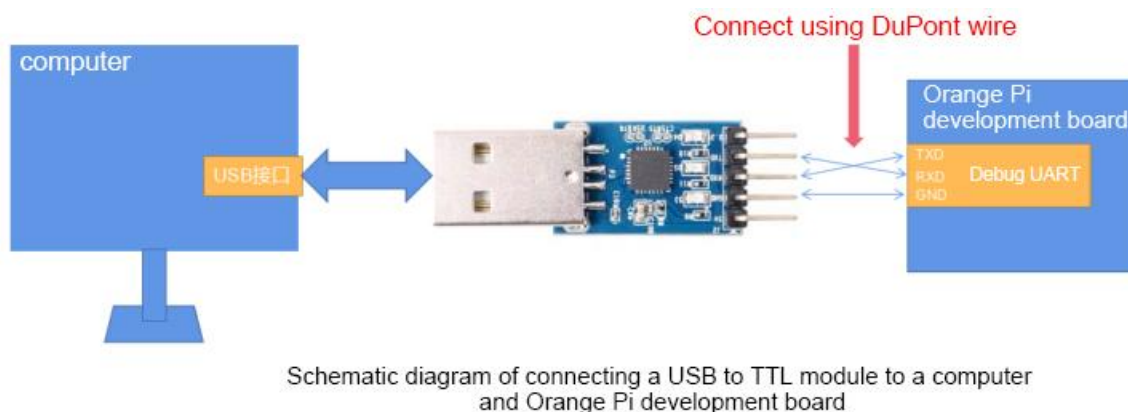
2) The corresponding relationship between the GND, TX, and RX pins of the debugging serial port of the development board is shown in the following figure



3) The GND, TX, and RX pins of the USB to TTL module need to be connected to the debugging serial port of the development board through DuPont wires

- a. Connect the GND of the USB to TTL module to the GND of the development board
- b. Connect the **RX of the USB to TTL module to the TX** of the development board
- c. Connect the **TX of the USB to TTL module to the RX** of the development board

4) The schematic diagram of connecting a USB to TTL module to a computer and Orange Pi development board is shown below



The TX and RX of the serial port need to be cross connected. If you don't want to carefully distinguish the order of TX and RX, you can randomly connect the TX and RX of the serial port first. If there is no output from the test serial port, then switch the order of TX and RX. This way, there will always be a correct order.

2. 11. 2. Ubuntu platform debugging serial port usage method

There are many serial debugging software that can be used under Linux, such as putty, minicom, etc. Below is a demonstration of how to use putty.

1) Firstly, insert the USB to TTL module into the USB interface of the Ubuntu computer. If the USB to TTL module is recognized as connected properly, the corresponding device node name can be seen in the `/dev` section of the Ubuntu PC. Remember this node name, which will be used later when setting up the serial port software

```
test@test:~$ ls /dev/ttyUSB*
/dev/ttyUSB0
```

2) Then use the following command to install putty on Ubuntu PC

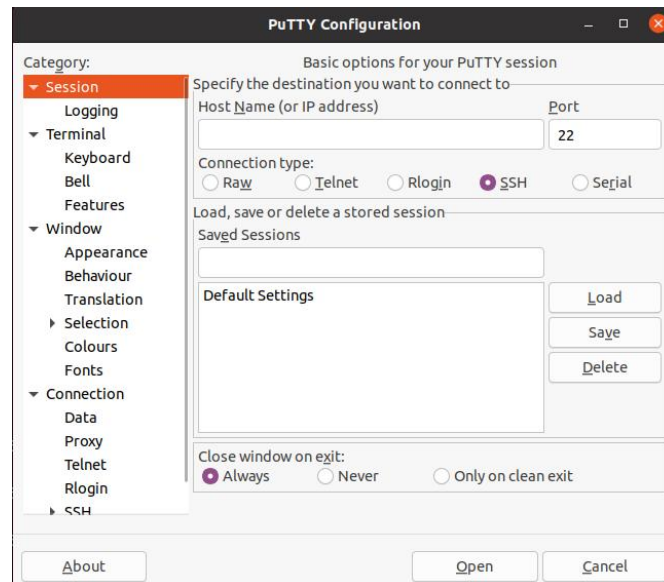
```
test@test:~$ sudo apt update
test@test:~$ sudo apt install -y putty
```

3) Then run putty, **remember to add sudo privileges**

```
test@test:~$ sudo putty
```



4) After executing the putty command, the following interface will pop up

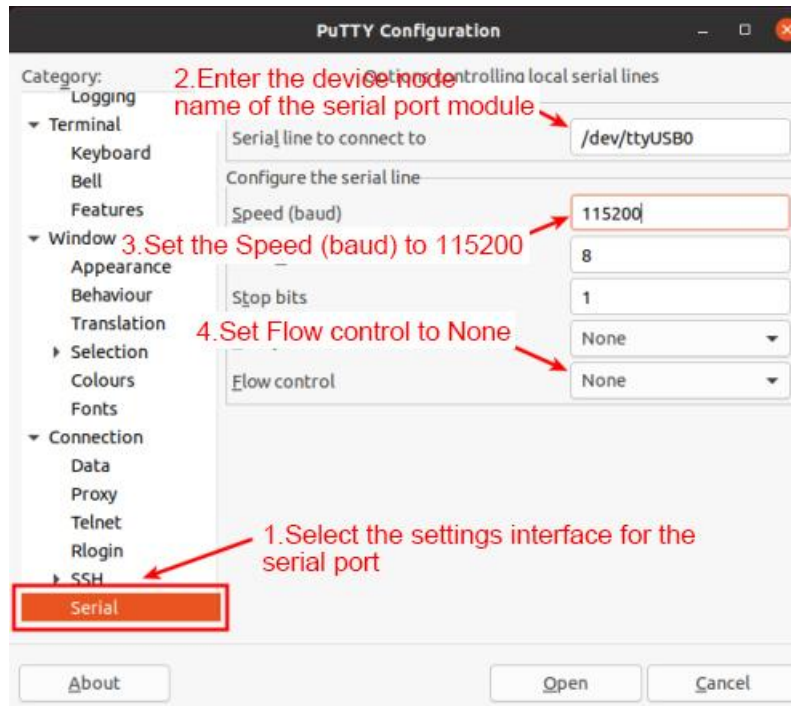


5) Firstly, select the settings interface for the serial port



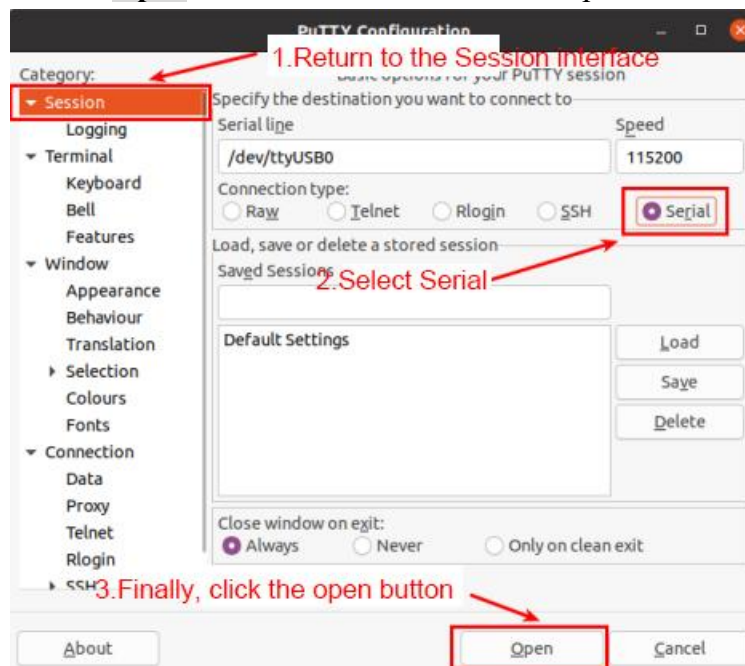
6) Then set the parameters of the serial port

- Set **Serial line to connect to** to **/dev/ttyUSB0** (modify to the corresponding node name, usually **/dev/ttyUSB0**)
- Set **Speed(baud)** to **115200** (baud rate of serial port)
- Set **Flow control** to **None**

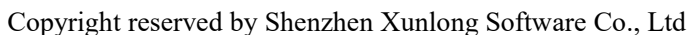


7) After completing the settings on the serial port interface, return to the Session interface

- First, select **Connection type** as **Serial**
- Then click the **Open** button to connect to the serial port



8) Then start the development board and you can see the log information output by the



```

/dev/ttyUSB0 - PuTTY
[198]HELLO! BOOT0 is starting May 13 2020 14:10:41
[163]BOOT0 commit : 583cfe83
[166]set pll start
[168]seripad has been enabled
[172]set pll end
[173]unknown PDI
[175]FWID 0x0005
[182]valid para:1 select dram para0
[186]board init ok
[188]DRAM NOT BURN INFO: V0.52
[191]the chip id is 0-5000
[194]chip id check OK
[196]IRWF_VCC set to 1500 mv
[200]read_calibration error
[204]read_calibration error
[206]read_calibration error
[212]read_calibration error
[216]read_calibration error
[219]read_calibration error
[223]read_calibration error
[227]read_calibration error
[231]read_calibration error
[235]read_calibration error
[238]retraining final error
[242][LUT0 IEIUG]32bit.1 ranks training success!
[250]IPERL CLK #720 MHz
[252]DRAM Type =3 (DDR3.4:DDR4.7;LPDDR3.8;LPDDR4)
[259]actual DRAM SIZE =1024 M
[261]DRAM SIZE =1024 Mbytes, para1 = 30fa, para2 = 4000000, dram_tpr13 = 6041
[274]DRAM single test OK
[277]rtc standby flag is 0x0, super standby flag is 0x0
[282]dram size=1024
[284]***dram handle ok***
[288]card no is 0
[290]sdcard 0 line count 4
[293][mmc]: mmc driver ver 2019-12-19 10:41
[297][mmc]: ccs0 spd mode error, 2
[300][mmc]: set f_max to 50M, set f_max_ddr to 25M
[305][mmc]: mmc 0 bit 0
[313][mmc]: wrong media type 0x0
[316][mmc]: ***Try SD card p***
[324][mmc]: HS200SD/SICR25 4 bit
[325][mmc]: 5000000 Hz
```

There are many serial debugging software that can be used under Windows, such as SecureCRT, MobaXterm, etc. Below is a demonstration of how to use MobaXterm. This software has a free version and can be used without purchasing a serial number.

- The download link for MobaXterm is as follows

b. After entering the MobaXterm download webpage, click **GET XOBATERM NOW!**





Home Edition	Professional Edition
Free Full X server and SSH support Remote desktop (RDP, VNC, Xdmcp) Remote terminal (SSH, telnet, rlogin, Mosh) X11-Forwarding Automatic SFTP browser Master password protection Plugins support Portable and installer versions Full documentation Max. 12 sessions Max. 2 SSH tunnels Max. 4 macros Max. 360 seconds for Tftp, Nfs and Cron Download now	\$69 / 49€ per user* * Excluding tax. Volume discounts available Every feature from Home Edition + Customize your startup message and logo Modify your profile script Remove unwanted games, screensaver or tools Unlimited number of sessions Unlimited number of tunnels and macros Unlimited run time for network daemons Enhanced security settings 12-months updates included Deployment inside company Lifetime right to use Subscribe online / Get a quote

- d. Then select the Portable version, and after downloading, there is no need to install it. Simply open it and you can use it

MobaXterm Home Edition

Download MobaXterm Home Edition (current version):

[MobaXterm Home Edition v20.3 \(Portable edition\)](#)
[MobaXterm Home Edition v20.3 \(Installer edition\)](#)

Download previous stable version: [MobaXterm Portable v20.2](#) [MobaXterm Installer v20.2](#)

You can also get early access to the latest features and improvements by downloading MobaXterm Preview version: [MobaXterm Preview Version](#)

By downloading MobaXterm software, you accept [MobaXterm terms and conditions](#)

You can download MobaXterm and plugins sources [here](#)

If you use MobaXterm inside your company, you should consider subscribing to [MobaXterm Professional Edition](#): your subscription will give you access to professional support and to the "Customizer" software. This customizer will allow you to generate personalized versions of MobaXterm including your own logo, your default settings and your welcome message. Please [contact us](#) for more information.

2) After downloading, use decompression software to extract the downloaded compressed file and obtain the executable software of MobaXterm. Then double-click to open it

名称	修改日期	类型	大小
CygUtils.plugin	2020/5/21 4:06	PLUGIN 文件	15,570 KB
MobaXterm_Personal_20.3	2020/6/5 4:30	应用程序	14,104 KB

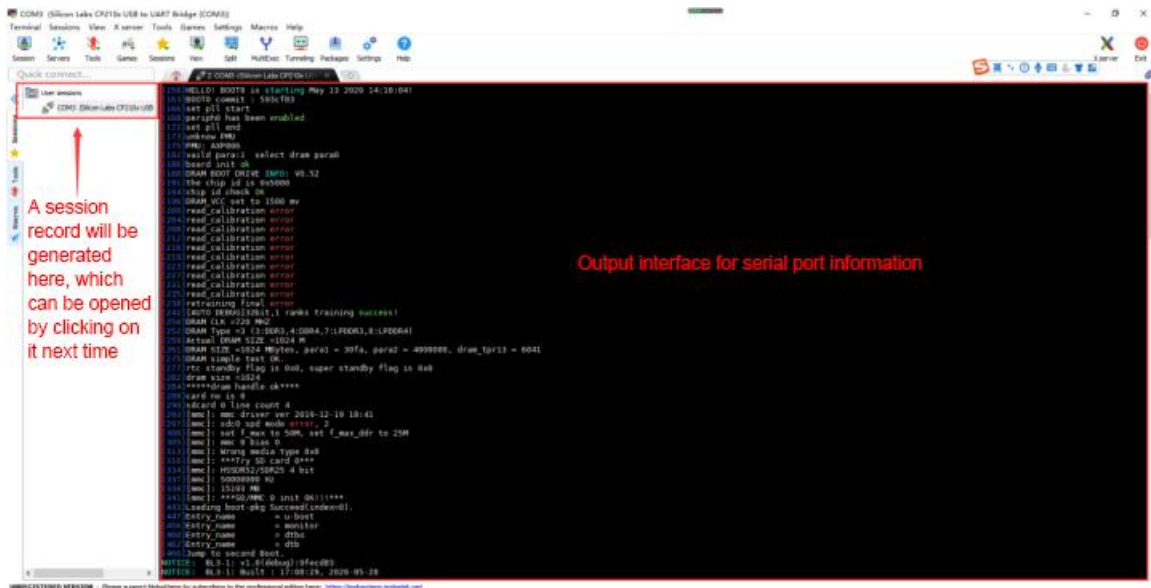
- 3) After opening the software, the steps to set up a serial port connection are as follows
- Open the session settings interface
 - Select serial port type
 - Select the port number for the serial port (choose the corresponding port number according to the actual situation). If you cannot see the port number, please use **360 Driver Master** to scan and install the driver for the USB to TTL serial port chip



- d. Choose a baud rate of **115200** for the serial port
- e. Finally, click the **"OK"** button to complete the setup



4) After clicking the **"OK"** button, you will enter the interface below. At this time, you can start the development board and see the output information of the serial port





2. 12. Instructions for powering the 5V pin in the 40 pin interface of the development board

The recommended power supply method for the development board is to use a 5V/3A Type C interface power cord plugged into the Type C power interface of the development board to provide power. If you need to use the 5V pin in the 40 pin interface to power the development board, please ensure that the power cord used can meet the power supply requirements of the development board. If there is unstable usage, please switch back to the Type C power supply.

1) Firstly, it is necessary to prepare a power cord as shown in the following diagram



The power cord shown in the picture can be purchased on Taobao, please search and purchase it yourself.

2) Use the 5V pin in the 40 pin interface to power the development board, and connect the power cord as follows

- a. The USB A port of the power cord shown in the above figure needs to be plugged into a 5V/3A power adapter connector **(it is not recommended to plug it into the USB port of the computer for power supply, as it may be unstable to use if there are too many peripherals connected to the development board)**
- b. The red DuPont wire needs to be plugged into the 5V pin of the 40 pin interface on the development board
- c. The positions of the 5V pin and GND pin of the 40 pin interface in the



development board are shown in the following figure. **Remember not to connect them in reverse**



3. Instructions for using Debian/Ubuntu Server and Xfce desktop system

3. 1. Supported Linux image types and kernel versions

Linux image type	kernel version	desktop version
Ubuntu 22.04 - Jammy	Linux5.15	support
Debian 12 - Bookworm	Linux5.15	support
Debian 11 - Bullseye	Linux5.15	support
Ubuntu 22.04 - Jammy	Linux6.6	support
Debian 12 - Bookworm	Linux6.6	support
Debian 11 - Bullseye	Linux6.6	support

After entering the download page of the corresponding development board on the



[Orange Pi data download page](#), you can see the following download options. In the following description, **Ubuntu images and Debian images are generally referred to as Linux images.**



Ubuntu Image

[Downloads](#)

Debian Image

[Downloads](#)

The naming convention for Linux images is:

Development board model _ version number _ Linux release type _ release code _ server or desktop _ kernel version

- Development board model:** All are **Orangezero3w**. The model names of different development boards are generally different. Before burning the image, please ensure that the model name of the selected image matches the development board.
- Version number:** For example, **1.x.x**, this version number will increase with the update of the image function, and the last digit of the version number of the Linux image on the development board is even.
- Types of Linux distributions:** Currently supports **Ubuntu** and **Debian**. Due to Ubuntu's origin from Debian, there is generally not much difference in usage between the two systems. But there are still some differences in the default configuration and command usage of some software. In addition, Ubuntu and Debian each maintain their own supported software repositories, and there are also some differences in the supported installable software packages. These require personal experience to gain a deeper understanding. For more details, you can refer to the official documentation provided by Ubuntu and Debian.
- Release code:** Used to distinguish between different versions of specific Linux distributions such as Ubuntu or Debian. Among them, **jammy** is the Ubuntu distribution, referring to Ubuntu 22.04. The biggest difference between different versions is that many of the software in the software repository maintained by the new version of Ubuntu system are newer than those in the old version, such as Python and GCC compilation toolchains. **bookworm** is the specific version



code for Debian, with **bookworm** representing Debian12.

- e. **Server or Desktop** : Used to indicate whether the system has a desktop environment. If it is a **server**, it means that the system does not have a desktop environment installed. The storage space and resources occupied by the image are relatively small, and the system is mainly operated and controlled using the command line. If it is **desktop_gnome**, it means that the system has the GNOME desktop environment installed by default. The image occupies a relatively large amount of storage space and resources, and can be operated through the interface with a monitor, mouse, and keyboard. Of course, the desktop version of the system can also be operated through the command line like the server version.
- f. **Kernel version** : Used to represent the version number of the Linux kernel, currently supporting **linux5.15**.

3. 2. linux kernel driver adaptation situation

Function	Linux5.15	Linux6.6
HDMI Video	OK	OK
HDMI Audio	OK	OK
TypeC USB2.0	OK	OK
TypeC USB3.0	OK	OK
TypeC DP display	OK	OK
TF card startup	OK	OK
eMMC	OK	OK
UFS	OK	OK
WIFI	OK	OK
Bluetooth	OK	OK
CAM1	OK	OK
CAM2	OK	OK
MIPI LCD	OK	OK
LED lights	OK	OK
FAN cooling fan	OK	OK
40 pin GPIO	OK	OK
40 pin I2C	OK	OK



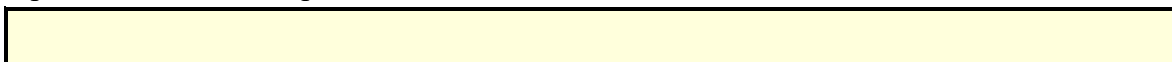
40 pin SPI	OK	OK
40 pin UART	OK	OK
40 pin PWM	OK	OK
temperature sensor	OK	OK
hardware watchdog	OK	OK
GPU	Only supported by Debian bullseye	Only supported by Debian bullseye
NPU	OK	OK
Video Encoding and Decoding	Only supported by Debian bullseye	Only supported by Debian bullseye

3.3. Explanation of linux Command Format in This Manual

1) All commands that need to be entered in the Linux system in this manual will be enclosed in the boxes below



As shown below, the content in the yellow box represents the content that needs special attention, except for the commands inside.



2) Description of prompt types before commands

- a. The prompt in front of the command refers to the content in the red part of the box below, which is not part of linux commands. Therefore, when entering commands in the Linux system, please do not also enter the content in the red font.

```
orangepi@orangepi:~$ sudo apt update
root@orangepi:~# vim /boot/boot.cmd
test@test:~$ ssh root@192.168.1.xxx
root@test:~# ls
```

- b. **orangepi@orangepi:~\$** The prompt indicates that this command was entered in the linux system of the development board, and the **\$** at the end of the prompt indicates that the current user of the system is a regular user. When executing privileged commands, **sudo** needs to be added



- c. **root@orangepi:~#** The prompt indicates that this command was entered in **the linux system of the development board**, and the **#** at the end of the prompt indicates that the current user of the system is the **root** user, who can execute any command they want
- d. **test@test:~\$** The prompt indicates that this command was entered on an Ubuntu PC or Ubuntu virtual machine, not on the Linux system of the development board. The **\$** at the end of the prompt indicates that the current user of the system is a regular user. When executing privileged commands, **sudo** needs to be added
- e. **root@test:~#** The prompt indicates that this command was entered on an Ubuntu PC or Ubuntu virtual machine, not on the linux system of the development board. The **#** at the end of the prompt indicates that the current system user is the root user and can execute any command they want

3) What are the commands that need to be entered?

- a. As shown below, **the bold black part represents** the commands that need to be inputted, and the content below the commands is the output (some commands have output, while others may not). This part of the content does not need to be inputted

```
root@orangepi:~# cat /boot/orangepiEnv.txt
verbosity=7
bootlogo=false
console=serial
```

- b. As shown below, some commands that cannot be written on one line will be moved to the next line, and any black and bold parts are commands that need to be entered. When these commands are inputted on a single line, the '\ ' at the end of each line needs to be removed, which is not part of the command. Also, different parts of the command have spaces, please don't miss them

```
orangepi@orangepi:~$ echo \
"deb [arch=$(dpkg --print-architecture) \
signed-by=/usr/share/keyrings/docker-archive-keyring.gpg] \
https://download.docker.com/linux/debian \
$(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

3. 4. linux system login instructions

3. 4. 1. linux system default login account and password

account	password
root	orangepi
orangepi	orangepi

Note that when entering the password, the specific content of the entered password will not be displayed on the screen. Please do not assume that there is any malfunction. After entering, simply press Enter.

When the password prompt is incorrect or there is a problem with the SSH connection, please note that as long as you are using the Linux image provided by Orange Pi, **do not suspect that the password is incorrect, but instead look for other reasons.**

3. 4. 2. Method for setting up automatic login for linux system terminals

1) The linux system defaults to automatically logging into the terminal, and the default login username is **orange**

```
orange pi login: orangepi (automatic login)
```

```
O  
O  
| |  
| | O  
| |  
| |  
| |  
| |
```

```
Welcome to Orange Pi 1.0.0 Bullseye with Linux 6.1.31-sun50iw9
```

```
System load:   43%                Up time:       0 min  
Memory usage:  18% of 1.45G      IP:           192.168.1.121  
CPU temp:     53°C              Usage of /:    13% of 29G
```

```
Last login: Thu Jun  8 06:37:02 UTC 2023 on tty1  
orange pi@orange pi:~$
```

2) The following command can be used to set the root user to automatically log in to the terminal

```
orangepi@orangepi:~$ sudo auto login cli.sh root
```

3) Use the following command to disable automatic login to the terminal

```
orangepi@orangepi:~$ sudo auto login cli.sh -d
```



4) The following command can be used to reset the automatic login of orangepi users to the terminal

```
orangepi@orangepi:~$ sudo auto_login_cli.sh orangepi
```

3. 4. 3. linux Desktop System Automatic Login Instructions

1) After the desktop version system starts, it will automatically log in to the desktop without entering a password



3. 4. 4. Linux Desktop System Root User Automatic Login Setting Method

1) Execute the following command to set the desktop system to automatically log in as the root user

```
orangepi@orangepi:~$ sudo desktop_login.sh root
```

2) Then restart the system, and it will automatically log in to the desktop as the root user

3) Execute the following command to reset the desktop system to use orangepi for automatic user login

```
orangepi@orangepi:~$ sudo desktop_login.sh orangepi
```




3. 5. Test instructions for onboard LED lights

1) There is only one red LED light on the development board, located as shown in the following figure:



2) The red LED light will flash continuously after the kernel is started, which is controlled by software.

3) The method for setting the red light on/off and flashing is as follows:

Please note that the following operations should be performed under the root user.

a. First, enter the directory for setting red lights

```
root@orangepi:~# cd /sys/class/leds/status_led
```

b. The command to set the red light to stop flashing is as follows

```
root@orangepi:/sys/class/leds/status_led# echo none > trigger
```

c. The command to set the red light to always on is as follows

```
root@orangepi:/sys/class/leds/status_led# echo default-on > trigger
```

d. The command to set the red light flashing is as follows

```
root@orangepi:/sys/class/leds/status_led# echo heartbeat > trigger
```

3. 6. Operating Instructions for linux System rootfs Partition Capacity in TF Card

3. 6. 1. The system will automatically expand the capacity of the rootfs partition on the TF card during the first boot..

1) After burning the Linux image of the development board to the TF card, you can



check the usage of the TF card's capacity on your **Ubuntu computer**. The steps are as follows:

Note that skipping this step will not affect the automatic expansion of the Linux system on the development board. This is simply to illustrate how to check the TF card capacity after burning the Linux image to the TF card.

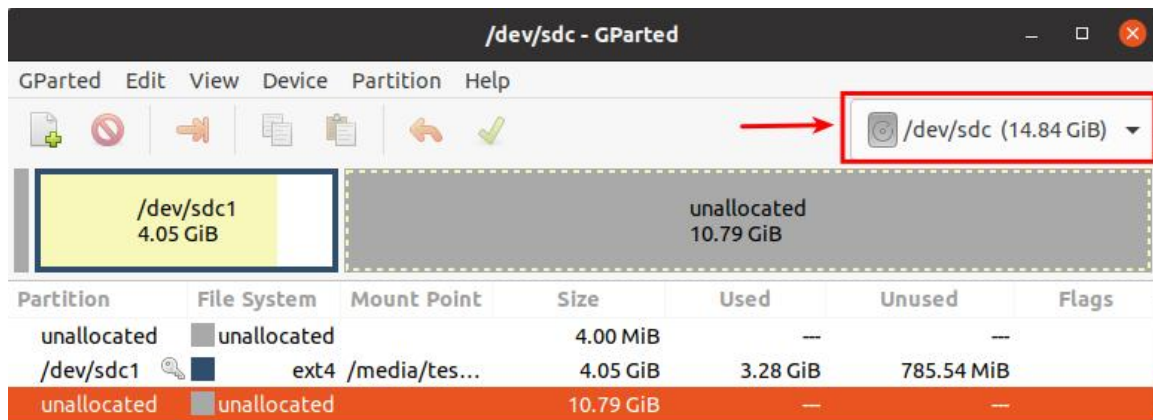
- a. Note that skipping this step will not affect the automatic expansion of the Linux system on the development board. This is simply to illustrate how to check the TF card capacity after burning the Linux image to the TF card.

```
test@test:~$ sudo apt install -y gparted
```

- b. Then open gparted

```
test@test:~$ sudo gparted
```

- c. After opening gparted, you can select the TF card in the upper right corner, and then you can see the usage of the TF card capacity.



- d. The image above shows the status of the TF card after burning the Linux desktop system. As you can see, although the total capacity of the TF card is 16GB (displayed as 14.84GiB in GParted), the rootfs partition (/dev/sdc1) has only allocated 4.05GiB, leaving 10.79GiB unallocated.

2) Then you can insert the TF card with the Linux system burned into the development board to boot. When the TF card boots the Linux system for the first time, it will automatically expand the rootfs partition by calling the

orange-pi-resize-file-system.service script through the systemd service

orange-pi-resize-file-system.service, **so there is no need to manually expand it**

3) After logging into the system, you can use the **df -h** command to check the size of the rootfs. If it matches the actual capacity of the TF card, it means that the automatic

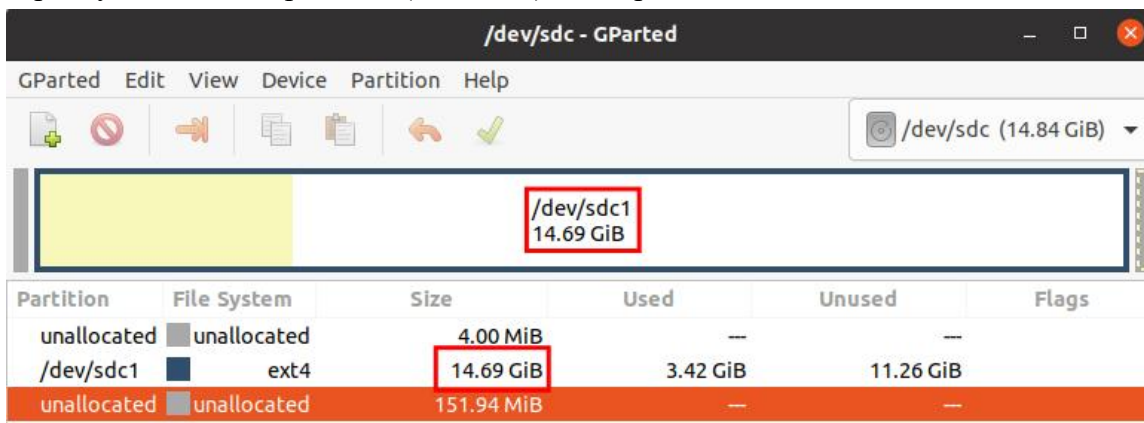


expansion is running correctly.

```
orange@orange:~$ df -h
```

Filesystem	Size	Used	Avail	Use%	Mounted on
udev	430M	0	430M	0%	/dev
tmpfs	100M	5.6M	95M	6%	/run
/dev/mmcblk0p1	15G	915M	14G	7%	/
tmpfs	500M	0	500M	0%	/dev/shm

4) After the Linux system boots up for the first time, we can remove the TF card from the development board and reinsert it into the **Ubuntu computer**. Then, we can use gparted again to check the status of the TF card. As shown in the image below, the capacity of the rootfs partition (/dev/sdc1) has expanded to 14.69 GiB.



It's important to note that the Linux system has only one ext4 partition and does not use a separate BOOT partition to store files such as the kernel image, so there is no issue of expanding the BOOT partition.

3. 6. 2. Method to prevent automatic expansion of the rootfs partition size on the TF card

1) First, burn the Linux image of the development board to the TF card on an **Ubuntu computer** (Windows will not work), **then reinsert the TF card**.

2) Then, Ubuntu computers usually automatically mount the TF card partition. If the automatic mounting is successful, you can see the following output using the ls command.

```
test@test:~$ ls /media/test/opi_root/
```

```
bin boot dev etc home lib lost+found media mnt opt proc root run
```



```
sbin  selinux  srv  sys  tmp  usr  var
```

3) Then switch the current user to the root user on the Ubuntu computer.

```
test@test:~$ sudo -i
[sudo] test Password:
root@test:~#
```

4) Then, go to the root directory of the Linux system on the TF card and create a file named **.no_rootfs_resize**.

```
root@test:~# cd /media/test/opi_root/
root@test:/media/test/opi_root/# cd root
root@test:/media/test/opi_root/root# touch .no_rootfs_resize
root@test:/media/test/opi_root/root# ls .no_rootfs*
.no_rootfs_resize
```

5) Then you can uninstall the TF card, remove it, insert it into the development board, and boot. When the Linux system boots, if it detects the existence of the file **.no_rootfs_resize** in the **/root** directory, it will no longer automatically expand the rootfs.

6) After disabling automatic expansion of the rootfs partition and entering the Linux system, you can see that the total capacity of the rootfs partition is only 4GB (this test uses a desktop version image), which is much smaller than the actual capacity of the TF card, indicating that disabling automatic expansion of the rootfs partition was successful.

```
orangeypi@orangeypi:~$ df -h
```

Filesystem	Size	Used	Avail	Use%	Mounted on
udev	925M	0	925M	0%	/dev
tmpfs	199M	3.2M	196M	2%	/run
/dev/mmcblk0p1	4.0G	3.2G	686M	83%	/

7) If you need to expand the capacity of the rootfs partition on the TF card, simply execute the following command and then restart the Linux system on the development board.

Note that you should execute the following command as the **root user.**

```
root@orangeypi:~# rm /root/.no_rootfs_resize
```



```
root@orangepi:~# systemctl enable orangepi-resize-filesystem.service
root@orangepi:~# sudo reboot
```

After rebooting and entering the Linux system on the development board again, you can see that the rootfs partition has expanded to the actual capacity of the TF card.

```
root@orangepi:~# df -h
Filesystem      Size  Used Avail Use% Mounted on
udev            925M    0  925M   0% /dev
tmpfs           199M  3.2M  196M   2% /run
/dev/mmcblk0p1  15G   3.2G   12G  23% /
```

3. 6. 3. Method for manually expanding the capacity of the rootfs partition on a TF card

If your TF card has a large total capacity, such as 128GB, and you don't want the Linux system's rootfs partition to use all of the TF card's capacity, but only want to allocate a portion, such as 16GB, for the Linux system, then the remaining capacity of the TF card can be used for other purposes. You can use the methods described in this section to manually expand the capacity of the rootfs partition on your TF card.

1) First, burn the Linux image of the development board to the TF card on an **Ubuntu computer** (Windows will not work), **then reinsert the TF card**.

2) Then, Ubuntu computers usually automatically mount the TF card partition. If the automatic mounting is successful, you can see the following output using the ls command.

```
test@test:~$ ls /media/test/opi_root/
bin  boot  dev  etc  home  lib  lost+found  media  mnt  opt  proc  root  run
sbin  selinux  srv  sys  tmp  usr  var
```

3) Then switch the current user to the root user on the Ubuntu computer.

```
test@test:~$ sudo -i
[sudo] test Password:
root@test:~#
```

4) Then, go to the root directory of the Linux system on the TF card and create a file



named **.no_rootfs_resize**.

```
root@test:~# cd /media/test/opi_root/
root@test:/media/test/opi_root# cd root
root@test:/media/test/opi_root/root# touch .no_rootfs_resize
root@test:/media/test/opi_root/root# ls .no_rootfs*
.no_rootfs_resize
```

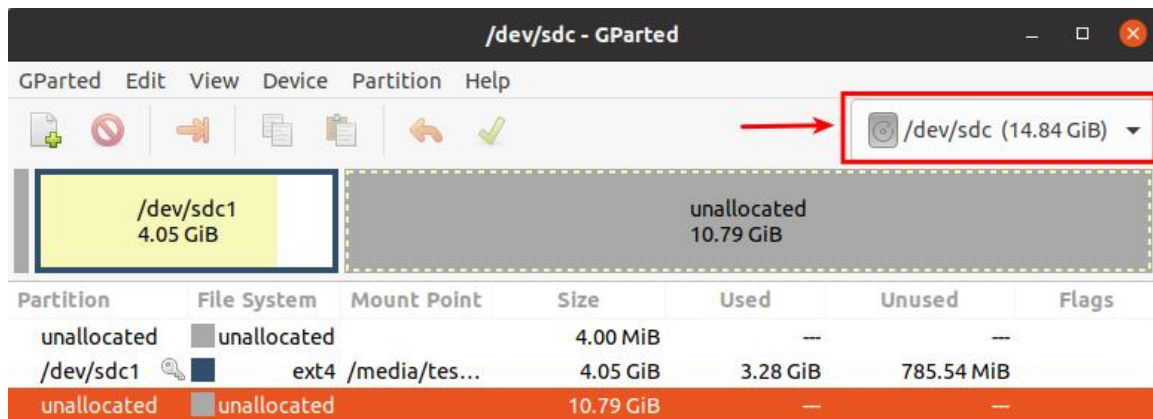
5) Then install the gparted software on your Ubuntu computer.

```
test@test:~$ sudo apt install -y gparted
```

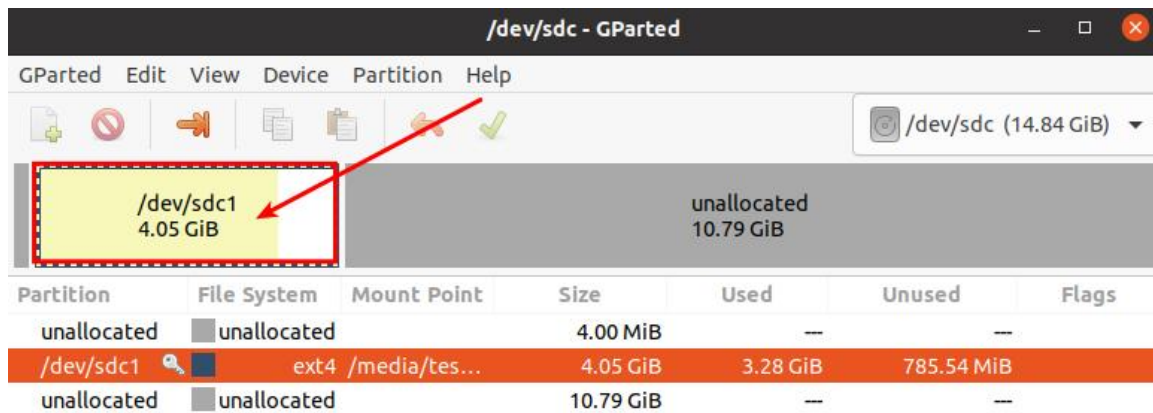
6) Then open gparted

```
test@test:~$ sudo gparted
```

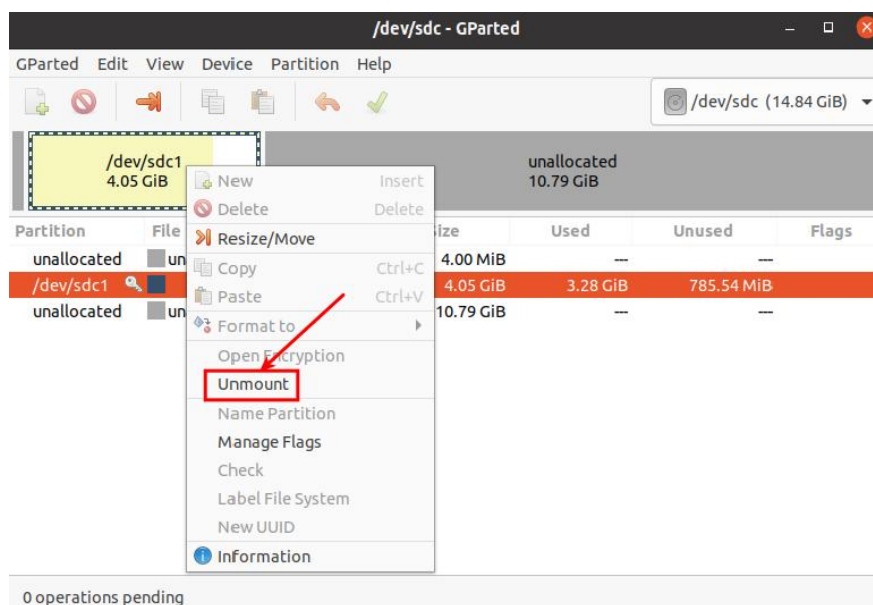
7) After opening gparted, you can select the TF card in the upper right corner to see the TF card capacity usage. The image below shows the TF card status after burning the Linux desktop system. You can see that although the total capacity of the TF card is 16GB (displayed as 14.84GiB in GParted), the rootfs partition (/dev/sdc1) has only allocated 4.05GiB, leaving 10.79GiB unallocated.



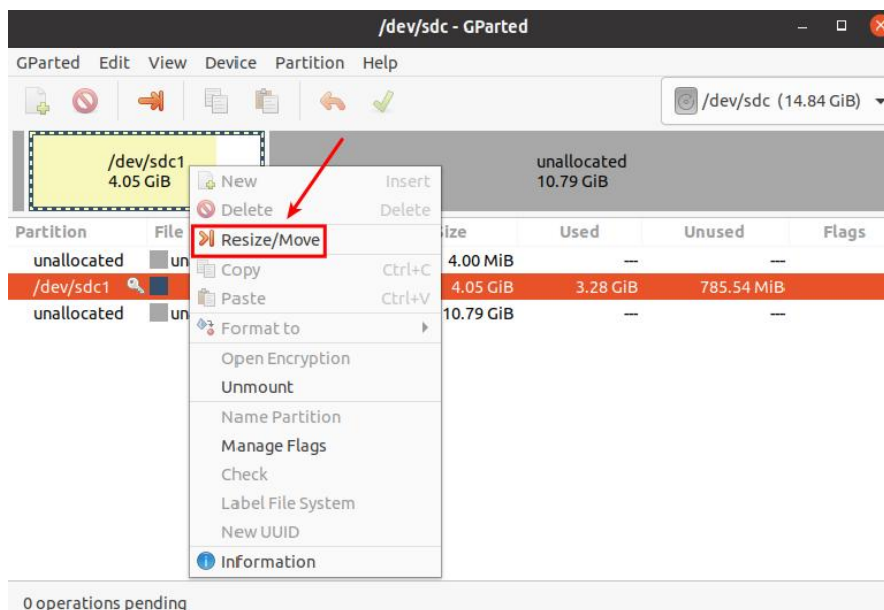
8) Then select the rootfs partition (/dev/sdc1)



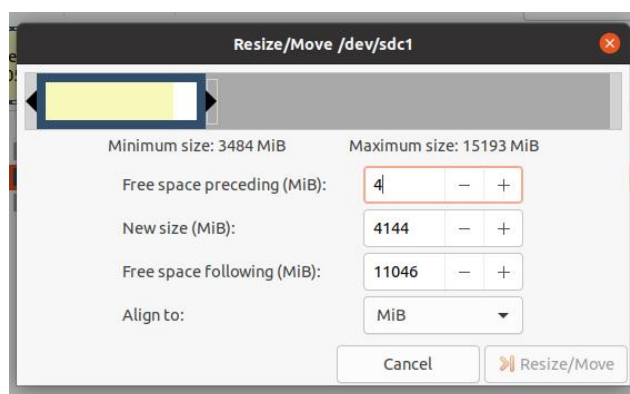
9) Right-clicking again will show the operation options as shown in the image below. If the TF card is already mounted, you first need to unmount the TF card's rootfs partition.



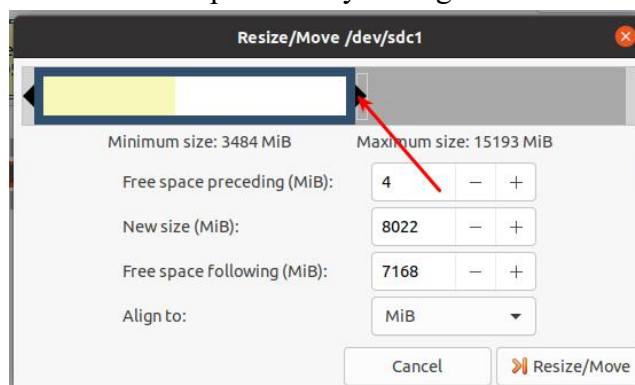
10) Then select the rootfs partition again, right-click, and select **Resize/Move** to start expanding the size of the rootfs partition.



11) After enabling the **Resize/Move** option, the following settings interface will pop up.

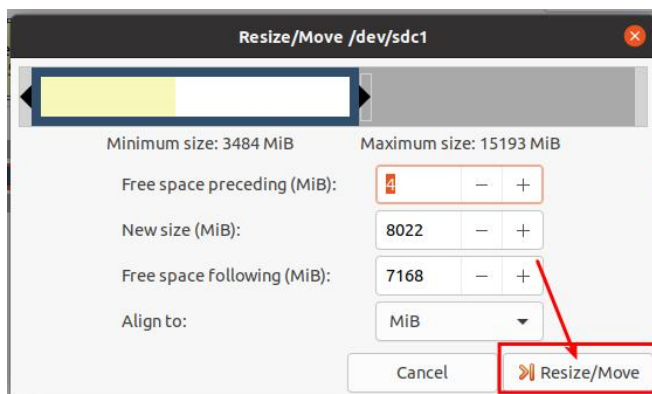


12) You can then directly drag the position shown in the image below to set the capacity, or you can set the size of the rootfs partition by setting the number in **New size(MiB)**.

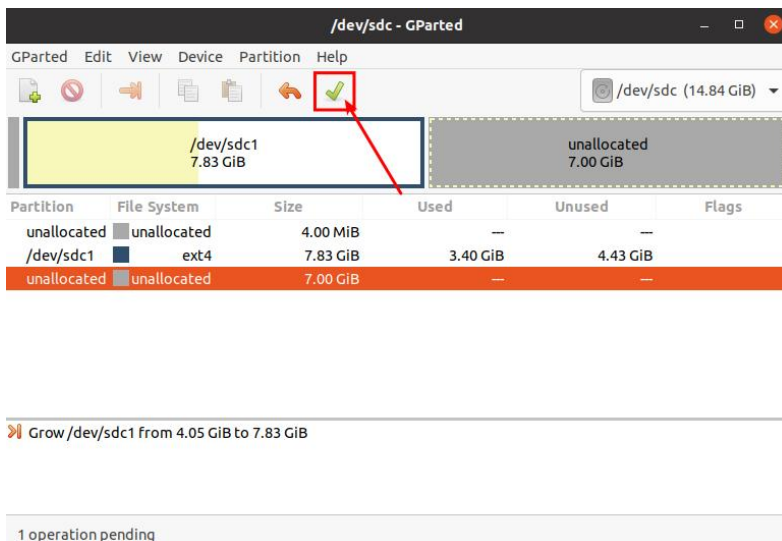




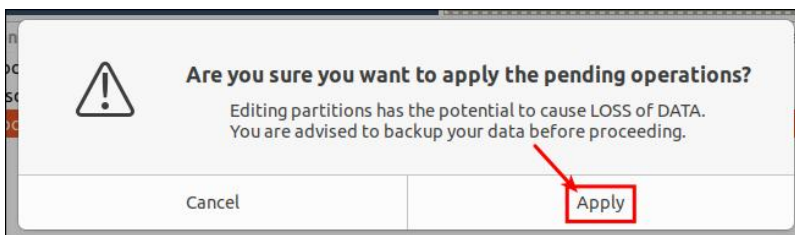
13) After setting the capacity, click **Resize/Move** in the lower right corner.



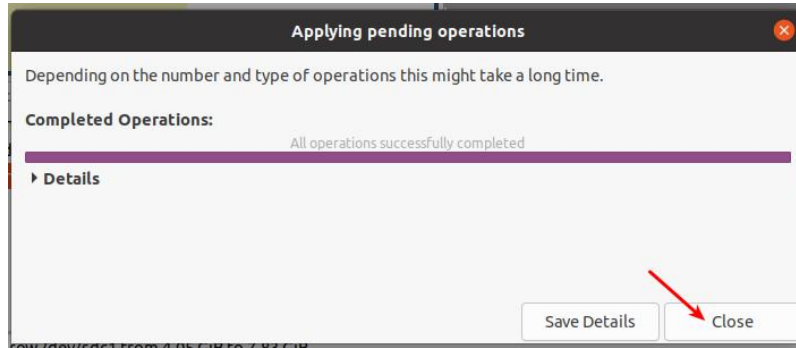
14) After confirming that everything is correct, click the green  shown in the image below.



15) Then select **Apply**, and the process of expanding the rootfs partition will begin.



16) After the expansion is complete, click **Close** to close the window.



17) Then you can remove the TF card, insert it back into the development board, and boot from there. Once in the Linux system on the development board, if you can see the rootfs partition size using the **df -h** command and it matches the size you set earlier, then the manual expansion was successful.

```
root@orangepi:~# df -h
Filesystem      Size  Used Avail Use% Mounted on
udev            925M    0  925M   0% /dev
tmpfs           199M  3.2M  196M   2% /run
/dev/mmcblk0p1  7.7G  3.2G  4.4G  42% /
```

3. 6. 4. Methods to reduce the size of the rootfs partition on a TF card

After configuring applications or other development environments in the Linux system on the TF card, if you want to back up the Linux system on the TF card, you can use the method in this section to first reduce the size of the rootfs partition before starting the backup.

1) First, insert the TF card you want to operate into your **Ubuntu computer** (Windows will not work).

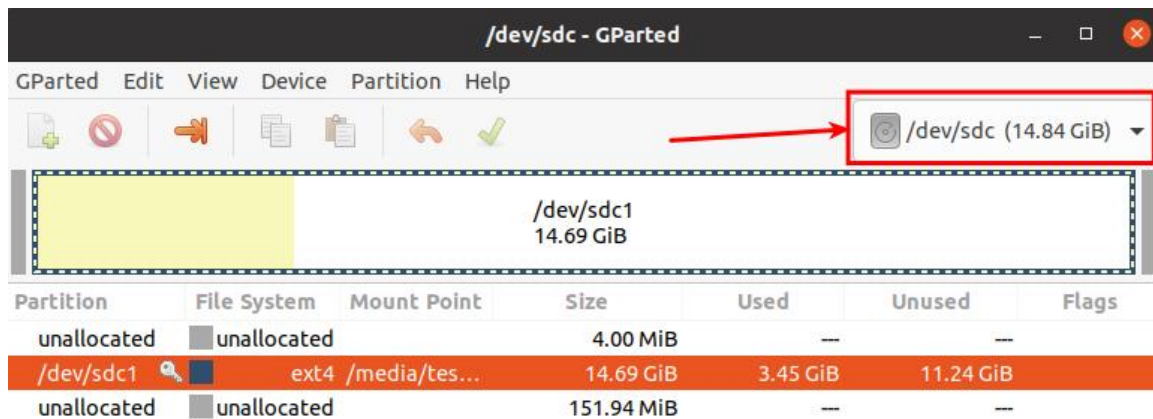
2) Then install the gparted software on your Ubuntu computer.

```
test@test:~$ sudo apt install -y gparted
```

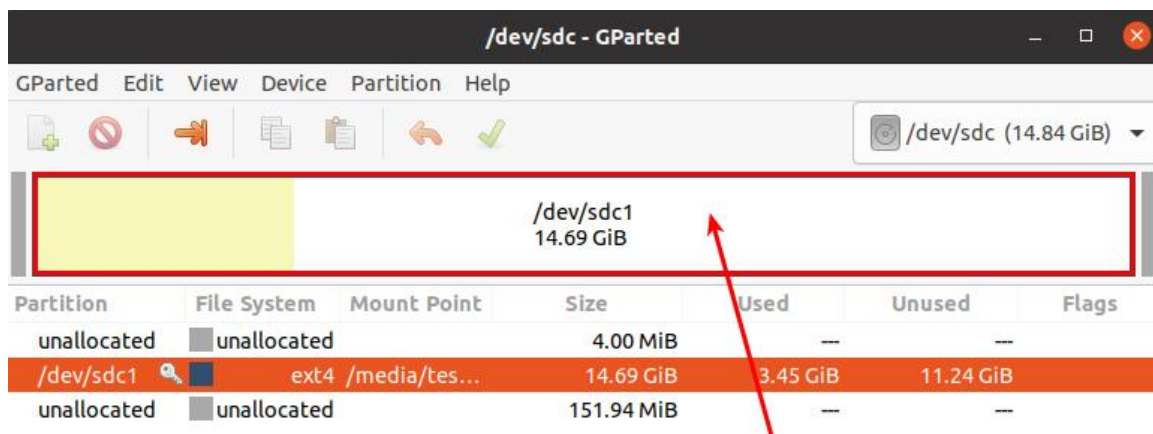
3) Then open gparted

```
test@test:~$ sudo gparted
```

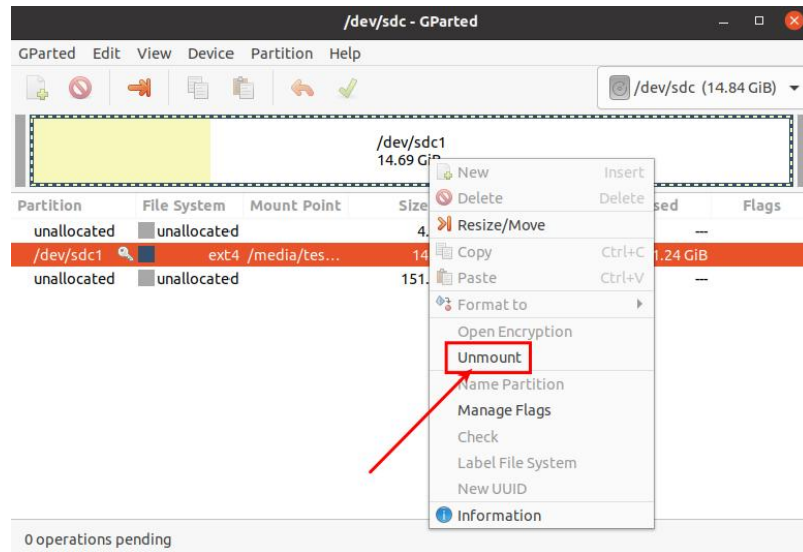
4) After opening gparted, you can select the TF card in the upper right corner, and then you can see the usage of the TF card capacity.



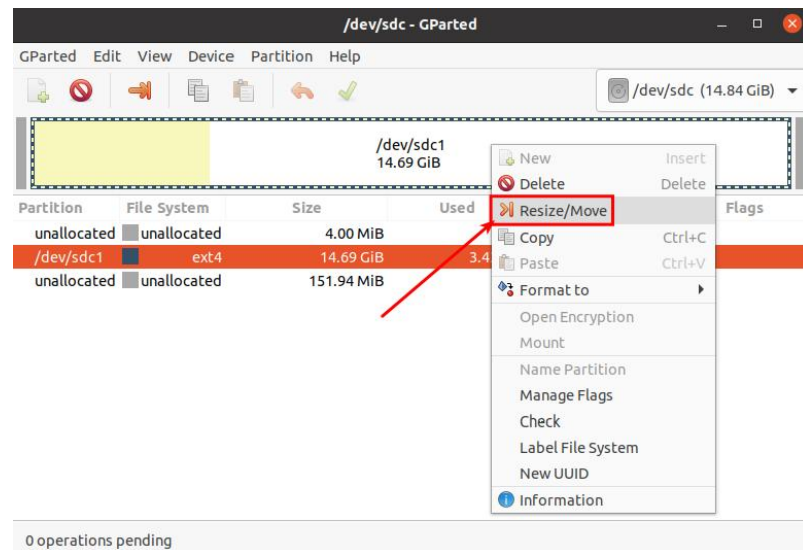
5) Then select the rootfs partition (/dev/sdc1)



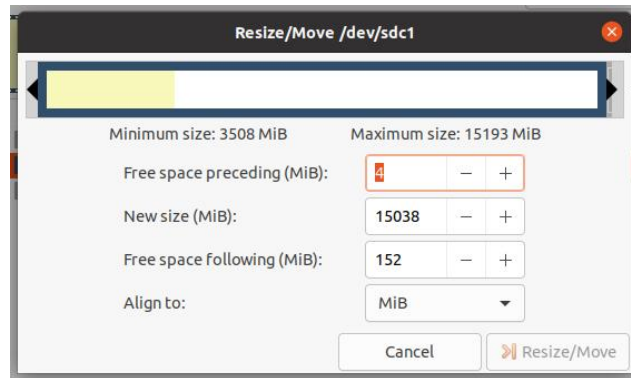
6) Right-clicking again will show the operation options as shown in the image below. If the TF card is already mounted, you first need to unmount the TF card's rootfs partition.



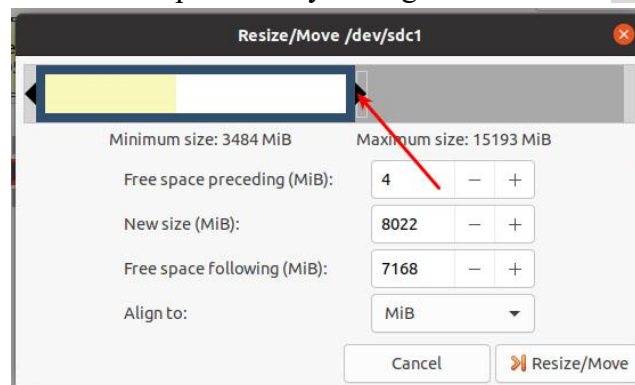
7) Then select the rootfs partition again, right-click, and select **Resize/Move** to start setting the size of the rootfs partition.



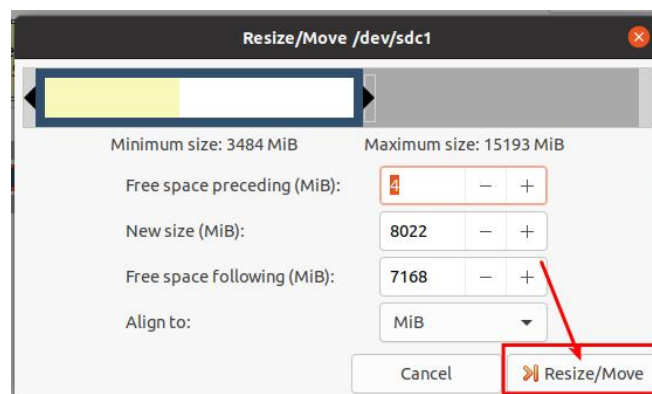
8) After enabling the **Resize/Move** option, the following settings interface will pop up.



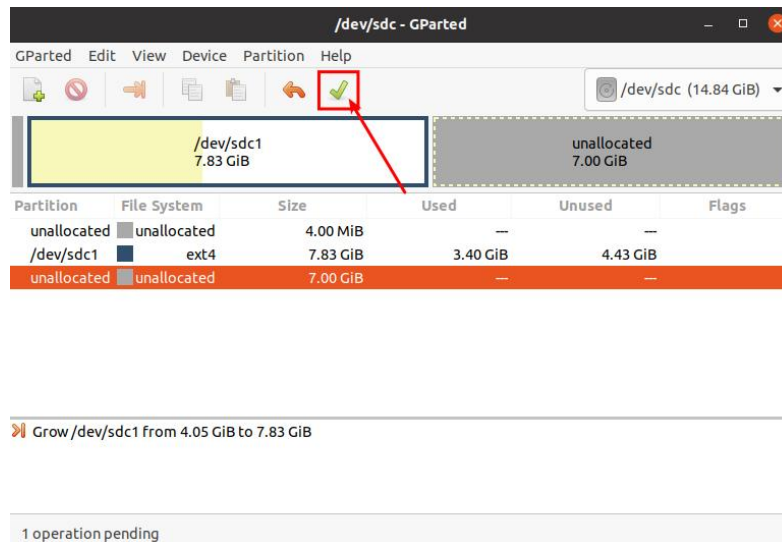
9) You can then directly drag the slider shown in the image below to set the capacity, or you can set the size of the rootfs partition by setting the number in **New size(MiB)**.



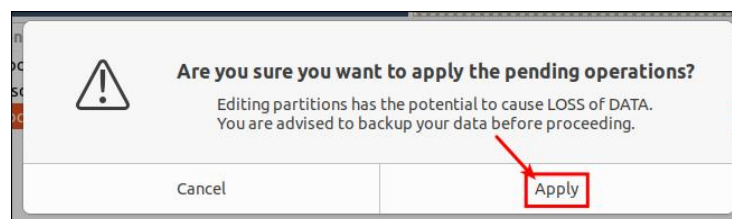
10) After setting the capacity, click **Resize/Move** in the lower right corner.



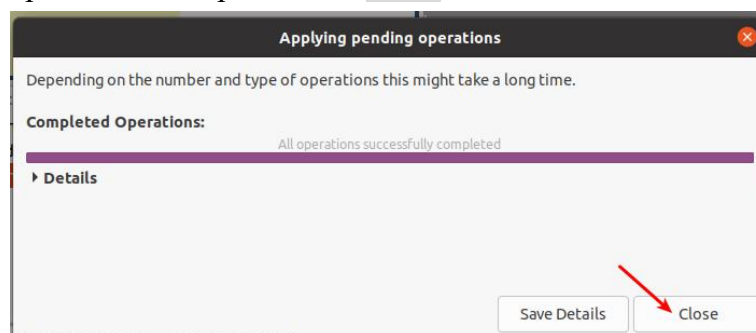
11) After confirming that everything is correct, click the green ✓ shown in the image below.



12) Then select **Apply**, and the process of expanding the rootfs partition will begin.



13) After the expansion is complete, click **Close** to close the window.



14) Then you can remove the TF card, insert it back into the development board, and boot from there. Once in the Linux system on the development board, if you use the **df -h** command and see that the size of the rootfs partition is the same as the size you set earlier, then the reduction in size was successful.

```
root@orangePi:~# df -h
Filesystem      Size  Used Avail Use% Mounted on
udev            925M    0  925M   0% /dev
```



```
tmpfs          199M  3.2M  196M   2% /run
/dev/mmcblk0p1  7.7G  3.2G  4.4G  42% /
```

3. 7. Network Connectivity Test

3. 7. 1. WiFi Connection Test

Please do not connect to Wi-Fi by modifying the `/etc/network/interfaces` configuration file, as this will cause problems.

3. 7. 1. 1. Server image connects to WIFI via command line

When the development board is not connected to an Ethernet or HDMI display, but only to a serial port, it is recommended to use the commands demonstrated in this section to connect to a Wi-Fi network. This is because `nmtui` can only display characters in some serial port software (such as `minicom`) and cannot display a graphical interface properly. Of course, if the development board is connected to an Ethernet or HDMI display, you can also use the commands demonstrated in this section to connect to a Wi-Fi network.

- 1) First, log in to the Linux system. There are two methods:
 - a. If the development board is connected to the debugging serial port, you can use the serial terminal to log in to the Linux system.
 - b. If the development board is connected to an HDMI monitor, you can log in to the Linux system through the terminal displayed on the HDMI screen.

- 2) First, use the `nmcli dev wifi` command to scan for nearby Wi-Fi hotspots.

```
orangepi@orangepi:~$ nmcli dev wifi
```




```

root@orangepi:~# nmcli dev wifi
IN-USE  BSSID          SSID          MODE  CHAN  RATE        SIGNAL  BARS  SECURITY
28:6C:07:6E:87:2E  orangepi      Infra    9    260 Mbit/s  97      █████ WPA1 WPA2
D8:D8:66:A5:BD:D1  orangepi      Infra   10    270 Mbit/s  90      █████ WPA1 WPA2
A0:40:A0:A1:72:20  orangepi      Infra    4    405 Mbit/s  82      █████ WPA2
28:6C:07:6E:87:2F  orangepi_5G   Infra  149    540 Mbit/s  80      █████ WPA1 WPA2
CA:50:E9:89:E2:44  ChinaNet_5G15 Infra    1    130 Mbit/s  79      █████ WPA1 WPA2
A0:40:A0:A1:72:31  orangepi      Infra   100    405 Mbit/s  67      █████ WPA2
D4:EE:07:08:A9:E0  orangepi      Infra    4    130 Mbit/s  55      █████ WPA1 WPA2
88:C3:97:49:25:13  orangepi      Infra    6    130 Mbit/s  52      █████ WPA1 WPA2
00:BD:82:51:53:C2  orangepi      Infra   12    130 Mbit/s  49      █████ WPA1 WPA2
C0:61:18:FA:49:37  orangepi      Infra  149    270 Mbit/s  47      █████ WPA1 WPA2
04:79:70:8D:0C:B8  orangepi      Infra  153    270 Mbit/s  47      █████ WPA2
04:79:70:FD:0C:B8  orangepi      Infra  153    270 Mbit/s  47      █████ WPA2
9C:A6:15:DD:E6:0C  orangepi      Infra   10    270 Mbit/s  45      █████ WPA1 WPA2
B4:0F:3B:45:D1:F5  orangepi      Infra   48    270 Mbit/s  45      █████ WPA1 WPA2
E8:CC:18:4F:7B:44  orangepi      Infra  157    135 Mbit/s  45      █████ WPA1 WPA2
B0:95:8E:D8:2F:ED  orangepi      Infra   11    405 Mbit/s  39      █████ WPA1 WPA2
C0:61:18:FA:49:36  orangepi      Infra   11    270 Mbit/s  24      █████ WPA1 WPA2
root@orangepi:~#

```

3) Then use the **nmcli** command to connect to the scanned Wi-Fi hotspot, where:

- wifi_name** You need to change it to the name of the Wi-Fi hotspot you want to connect to.
- wifi_passwd** You need to change it to the password of the Wi-Fi hotspot you want to connect to.

```

orangepi@orangepi:~$ sudo nmcli dev wifi connect wifi_name password wifi_passwd
Device 'wlan0' successfully activated with 'cf937f88-ca1e-4411-bb50-61f402eef293'.

```

4) You can view the Wi-Fi IP address using the command **ip addr show wlan0**.

```

orangepi@orangepi:~$ ip a s wlan0
11: wlan0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast
state UP group default qlen 1000
    link/ether 23:8c:d6:ae:76:bb brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.11/24 brd 192.168.1.255 scope global dynamic noprefixroute wlan0
        valid_lft 259192sec preferred_lft 259192sec
    inet6 240e:3b7:3240:c3a0:c401:a445:5002:ccdd/64 scope global dynamic
noprefixroute
        valid_lft 259192sec preferred_lft 172792sec
    inet6 fe80::42f1:6019:a80e:4c31/64 scope link noprefixroute
        valid_lft forever preferred_lft forever

```

5) The **ping** command can be used to test the connectivity of a Wi-Fi network. The **ping** command can be interrupted using the shortcut **Ctrl+C**.



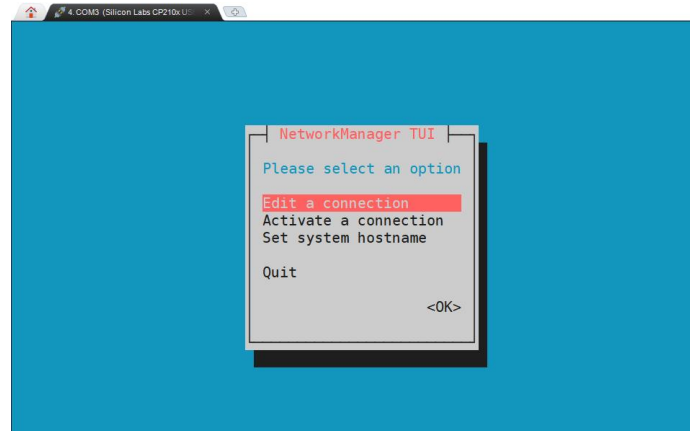
```
orangepi@orangepi:~$ ping www.orangepi.org -I wlan0
PING www.orangepi.org (182.92.236.130) from 192.168.1.49 wlan0: 56(84) bytes of
data.
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=1 ttl=52 time=43.5 ms
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=2 ttl=52 time=41.3 ms
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=3 ttl=52 time=44.9 ms
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=4 ttl=52 time=45.6 ms
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=5 ttl=52 time=48.8 ms
^C
--- www.orangepi.org ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4006ms
rtt min/avg/max/mdev = 41.321/44.864/48.834/2.484 ms
```

3.7.1.2. The server version image connects to Wi-Fi via a graphical interface.

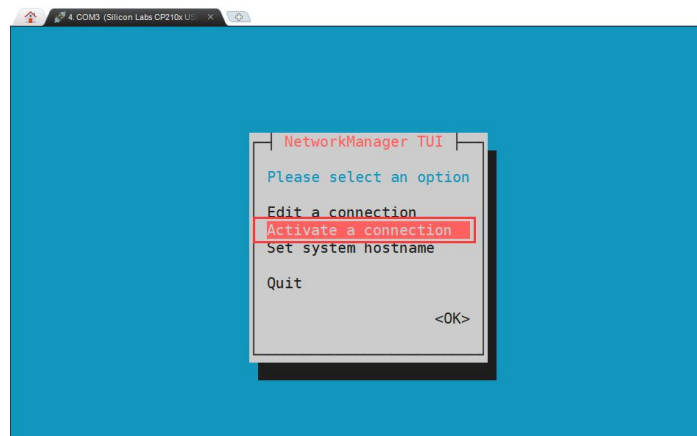
- 1) First, log in to the Linux system. There are two methods:
 - a. If the development board is connected to the debugging serial port, you can log in to the Linux system using the serial terminal (please use MobaXterm for the serial port software; minicom cannot display a graphical interface).
 - b. If the development board is connected to an HDMI monitor, you can log in to the Linux system through the terminal displayed on the HDMI screen.
- 2) Then, enter the command `nmtui` in the command line to open the Wi-Fi connection interface.

```
orangepi@orangepi:~$ sudo nmtui
```

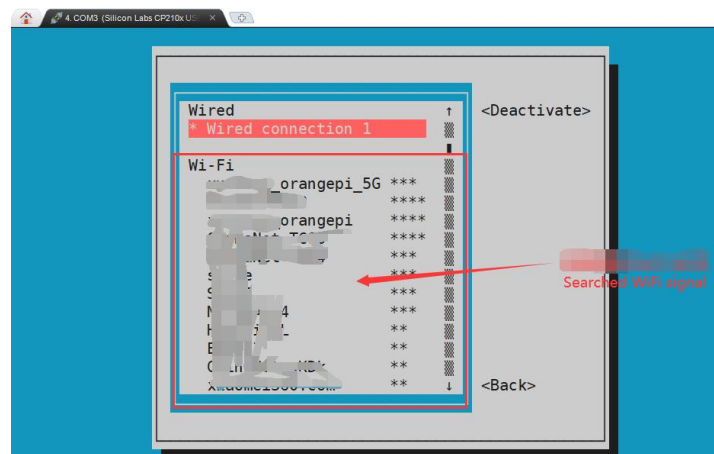
- 3) The interface that opens when you enter the nmtui command is shown below.



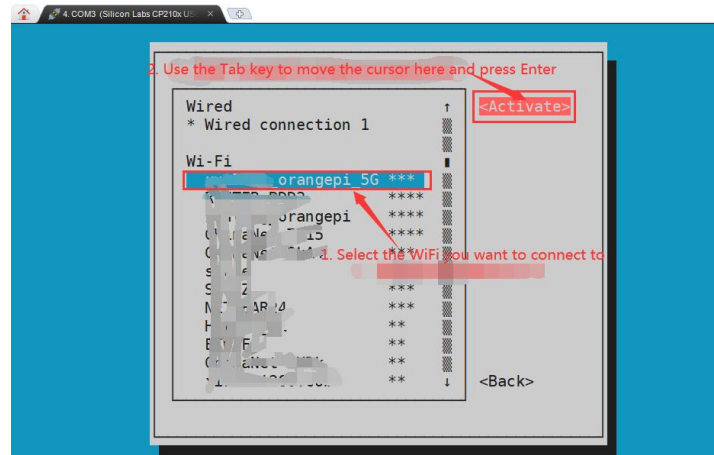
4) Select "**Activate a connect**" and press Enter.



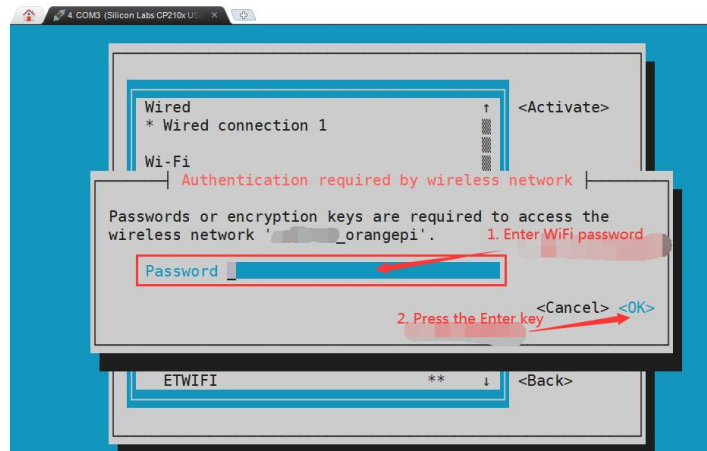
5) Then you can see all the searched Wi-Fi hotspots.



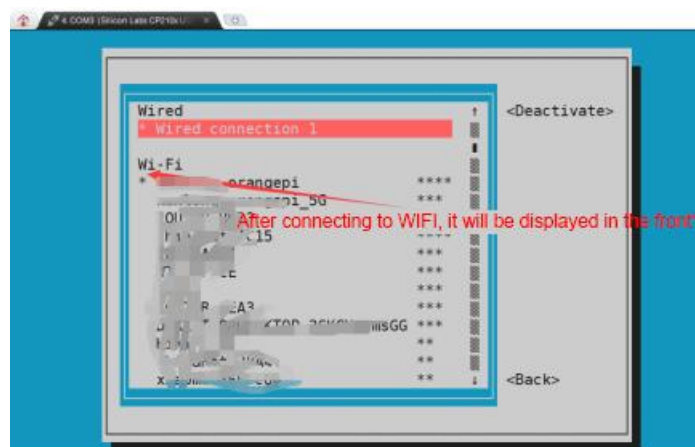
6) Select the Wi-Fi hotspot you want to connect to, then use the Tab key to position the cursor on **Activate** and press Enter.



7) A dialog box will then pop up asking for a password. Enter the corresponding password in **Password** and press Enter to start connecting to Wi-Fi.



8) Once a Wi-Fi connection is successfully established, an asterisk (*) will be displayed before the name of the connected Wi-Fi network.





9) You can view the IP address of the Wi-Fi network using the command **ip a s wlan0**

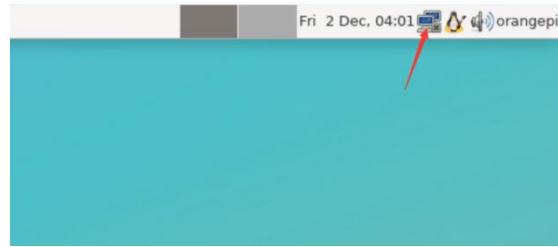
```
orangepi@orangepi:~$ ip a s wlan0
11: wlan0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast
state UP group default qlen 1000
    link/ether 24:8c:d3:aa:76:bb brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.11/24 brd 192.168.1.255 scope global dynamic noprefixroute wlan0
        valid_lft 259069sec preferred_lft 259069sec
    inet6 240e:3b7:3240:c4a0:c401:a445:5002:ccdd/64 scope global dynamic
noprefixroute
        valid_lft 259071sec preferred_lft 172671sec
    inet6 fe80::42f1:6019:a80e:4c31/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

10) The **ping** command can be used to test the connectivity of a Wi-Fi network. The **ping** command can be interrupted using the shortcut **Ctrl+C**.

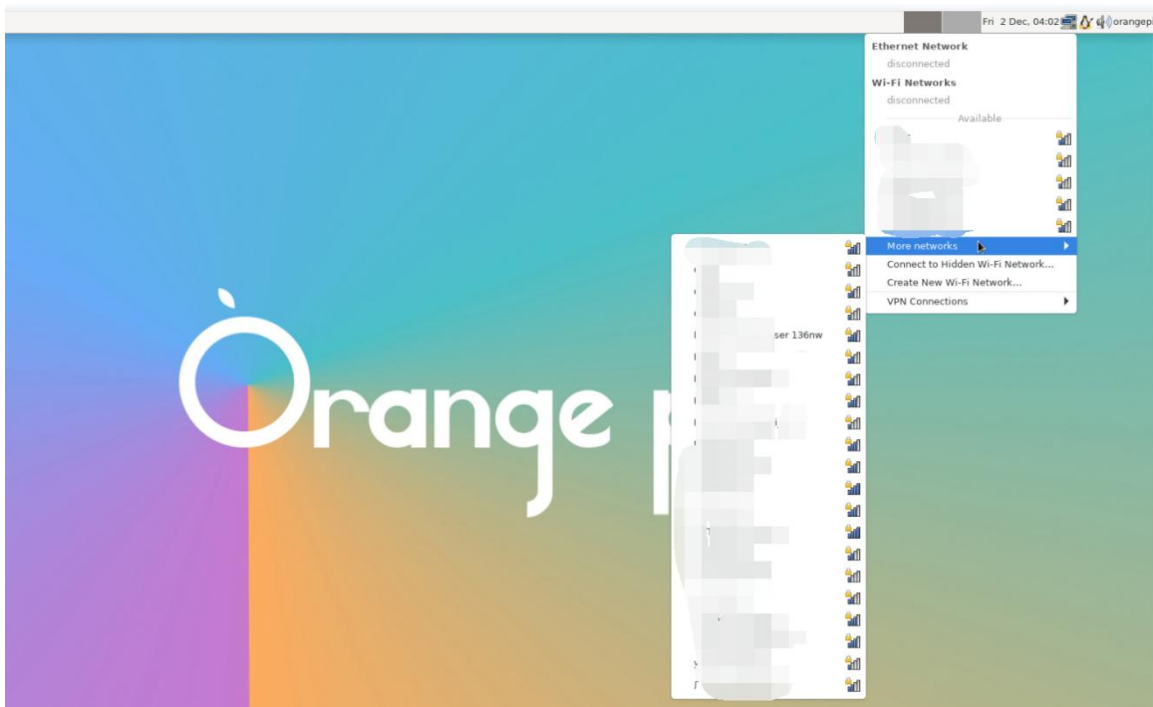
```
orangepi@orangepi:~$ ping www.orangepi.org -I wlan0
PING www.orangepi.org (182.92.236.130) from 192.168.1.49 wlan0: 56(84) bytes of
data.
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=1 ttl=52 time=43.5 ms
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=2 ttl=52 time=41.3 ms
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=3 ttl=52 time=44.9 ms
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=4 ttl=52 time=45.6 ms
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=5 ttl=52 time=48.8 ms
^C
--- www.orangepi.org ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4006ms
rtt min/avg/max/mdev = 41.321/44.864/48.834/2.484 ms
```

3. 7. 1. 3. Testing Methods for Desktop Images

1) Click the network configuration icon in the upper right corner of the desktop.



2) In the pop-up drop-down menu, click **More networks** to see all the scanned Wi-Fi hotspots, then select the Wi-Fi hotspot you want to connect to.



3) Then enter the Wi-Fi hotspot password and click **Connect** to start connecting to Wi-Fi.





3. 7. 2. Method for setting a static IP address

Please do not set a static IP address by modifying the `/etc/network/interfaces` configuration file.

3. 7. 2. 1. Use the nmcli command to set a static IP address

- 1) Please connect to Wi-Fi first, and then start setting a static IP address.
- 2) Then you can use the `nmcli con show` command to view the names of network devices, as shown below.
 - a. `orangepi` is the name of the Wi-Fi network interface (the name may vary).

```
orangepi@orangepi:~$ nmcli con show
```

NAME	UUID	TYPE	DEVICE
orangepi	cfc4f922-ae48-46f1-84e1-2f19e9ec5e2a	wifi	wlan0

- 3) Then enter the following command, where
 - a. `"orangepi"` indicates setting a static IP address for the Wi-Fi network (this can be obtained using the `nmcli con show` command).
 - b. `ipv4.addresses` is followed by the static IP address to be set; you can modify it to your desired value.
 - c. `ipv4.gateway` represents the gateway address.

```
orangepi@orangepi:~$ sudo nmcli con mod "orangepi" \
  ipv4.addresses "192.168.1.110/24" \
  ipv4.gateway "192.168.1.1" \
  ipv4.dns "8.8.8.8" \
  ipv4.method "manual"
```

- 4) Then restart the Linux system.

```
orangepi@orangepi:~$ sudo reboot
```

- 5) Then, re-enter the Linux system and use the command `ip addr show wlan0` to see that the IP address has been set to the desired value.

```
orangepi@orangepi:~$ ip addr show wlan0
```

```
6: wlan0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP
group default qlen 1000
```



```
link/ether 18:4e:f2:9c:1b:49 brd ff:ff:ff:ff:ff:ff
inet 192.168.1.110/32 brd 192.168.1.110 scope global dynamic noprefixroute wlan0
    valid_lft 3570sec preferred_lft 3570sec
inet6 fe80::be1a:8847:e5a6:6d52/64 scope link noprefixroute
    valid_lft forever preferred_lft forever
```

3. 7. 3. How to configure a Linux system to automatically connect to the network on the first boot

The development board has Wi-Fi. To remotely log in to the Linux system on the development board via Wi-Fi, you need to connect to Wi-Fi using commands through the HDMI screen or serial port.

However, if neither the HDMI screen nor the serial port module is available, and only Wi-Fi is accessible, you can use the method described in this section to automatically connect to Wi-Fi and configure a static IP address for either the Wi-Fi or the Ethernet port.

To use the methods described in this section, you first need a Linux machine. For example, a computer or virtual machine running Ubuntu.

Why a Linux machine? Because the root filesystem of the Linux system burned into the development board on the TF card is in ext4 format. A Linux machine can mount it correctly, and then modify the configuration files within it.

If you want to modify it in Windows, you can use **Paragon ExtFS for Windows**. Since this software is paid and there aren't many good free alternatives available, we won't demonstrate it here.

Also, if you encounter any problems using **Paragon ExtFS for Windows**, please troubleshoot them yourself; we will not provide support.

1) First, burn the Linux image of the development board you want to use to the TF card. Then, use a card reader to insert the TF card with the Linux image of the development board into the machine with Linux installed (such as a computer with Ubuntu installed; the following demonstration will use an Ubuntu computer as an example).

2) When the TF card is inserted into the Ubuntu computer, the Ubuntu computer will generally automatically mount the Linux root file system partition on the TF card. The



following command shows that **/media/test/opi_root** is the mount path of the Linux root file system on the TF card.

```
test@test:~$ df -h | grep "media"
/dev/sdd1 1.4G 1.2G 167M 88% /media/test/opi_root
test@test:~$ ls /media/test/opi_root
bin boot dev etc home lib lost+found media mnt opt proc root run
sbin selinux srv sys tmp usr var
```

3) Then go to the **/boot** directory of the Linux system burned into the TF card.

```
test@test:~$ cd /media/test/opi_root/boot/
```

4) Then copy **orange_pi_first_run.txt.template** to **orange_pi_first_run.txt**. The **orange_pi_first_run.txt** configuration file allows you to configure the development board's Linux system to automatically connect to a specific Wi-Fi hotspot upon first boot, or to set a static IP address for the Wi-Fi or Ethernet port.

```
test@test:/media/test/opi_root/boot$ sudo cp orange_pi_first_run.txt.template orange_pi_first_run.txt
```

5) You can open the **orange_pi_first_run.txt** file using the following command, and then view and modify its contents.

```
test@test:/media/test/opi_root/boot$ sudo vim orange_pi_first_run.txt
```

6) Instructions for using variables in the **orange_pi_first_run.txt** file

- a. The variable **FR_general_delete_this_file_after_completion** sets whether the file **'orange_pi_first_run.txt'** is deleted after the first startup. The default value is 1, meaning it is deleted. If set to 0, **'orange_pi_first_run.txt'** will be renamed to **'orange_pi_first_run.txt.old'** after the first startup. Generally, the default value is sufficient.
- b. The variable **FR_net_change_defaults** is used to set whether to change the default network settings. This must be set to 1; otherwise, all network settings will not take effect.
- c. The variable **FR_net_ethernet_enabled** controls whether the Ethernet port configuration is enabled. If you need to set a static IP address for the Ethernet port, set it to 1.
- d. The **FR_net_wifi_enabled** controls whether Wi-Fi is enabled. If you need to configure the development board to automatically connect to a Wi-Fi hotspot,



you must set it to 1. Also note that if this variable is set to 1, the Ethernet port settings will be disabled. In other words, Wi-Fi and Ethernet cannot be configured simultaneously (why? Because it's unnecessary...).

- e. The variable **FR_net_wifi_ssid** is used to set the name of the Wi-Fi hotspot you want to connect to.
- f. The variable **FR_net_wifi_key** is used to set the password for the Wi-Fi hotspot you want to connect to.
- g. The **FR_net_use_static** variable is used to set whether a static IP address should be configured for the Wi-Fi or Ethernet port.
- h. The **FR_net_static_ip** variable is used to set the static IP address. Please set it according to your actual situation.
- i. The **FR_net_static_gateway** variable is used to configure the gateway; please set it according to your specific needs.

7) Below are some specific setup examples:

- a. For example, if you want the Linux system on the development board to automatically connect to a Wi-Fi hotspot after its first boot, you can set it up like this:
 - a) Set **FR_net_change_defaults** to **1**
 - b) Set **FR_net_wifi_enabled** to **1**
 - c) Set **FR_net_wifi_ssid** to the name of the Wi-Fi hotspot you want to connect to
 - d) Set **FR_net_wifi_key** to the password of the Wi-Fi hotspot you want to connect to
- b. For example, if you want the Linux system on the development board to automatically connect to a Wi-Fi hotspot upon its first boot, and you want to set the Wi-Fi IP address to a specific static IP address (so that after the Linux system boots, you can directly use the set static IP address to remotely log in to the development board via SSH without having to check the development board's IP address through the router's backend), you can set it up like this:
 - a) Set **FR_net_change_defaults** to **1**
 - b) Set **FR_net_wifi_enabled** to **1**
 - c) Set **FR_net_wifi_ssid** to the name of the Wi-Fi hotspot you want to connect to
 - d) Set **FR_net_wifi_key** to the password of the Wi-Fi hotspot you want to connect to



- e) Set **FR_net_use_static** to 1
 - f) Set **FR_net_static_ip** to the desired IP address
 - g) Set **FR_net_static_gateway** to the corresponding gateway address
- c. For example, if you want the Linux system on the development board to automatically set the Ethernet port's IP address to the desired static IP address upon its first boot, you can configure it as follows:
- a) Set **FR_net_change_defaults** to 1
 - b) Set **FR_net_ethernet_enabled** to 1
 - c) Set **FR_net_use_static** to 1
 - d) Set **FR_net_static_ip** to the desired IP address
 - e) Set **FR_net_static_gateway** to the corresponding gateway address
- 8) After modifying the `orange_pi_first_run.txt` file, you can exit the `/boot` directory of the development board's Linux system on the TF card, then unmount the TF card, and then insert the TF card into the development board to boot.
- 9) If a static IP address is not configured, you will still need to check the IP address through the router's admin panel. If a static IP address is configured, you can ping the configured static IP address from your computer. If the ping is successful, it means the system has started normally and the network is configured correctly. Then you can use the configured IP address to remotely log in to the development board's Linux system via SSH.

After the Linux system on the development board boots up for the first time, `orange_pi_first_run.txt` will be deleted or renamed to `orange_pi_first_run.txt.old`. Even if you reset the `orange_pi_first_run.txt` configuration file and then restart the Linux system on the development board, the configuration in `orange_pi_first_run.txt` will not take effect again, because this configuration only takes effect on the first boot after the Linux system is flashed. Please pay special attention to this point.

3. 8. SSH Remote Login to Development Board

Linux systems typically enable SSH remote login by default and allow root user access. Before logging in via SSH, ensure an Ethernet or Wi-Fi connection is



established. Then, use the `ip addr` command or check your router to obtain the development board's IP address.

3. 8. 1. SSH Remote Login to Development Board under Ubuntu

1) Obtain the IP address of the development board

2) Then you can remotely log in to the Linux system using the ssh command.

```
test@test:~$ ssh orangepi@192.168.1.xxx
```

(It needs to be replaced with the development board's IP address.)

```
orangepi@192.168.1.xx's password:
```

(Enter your password here. The default password is orangepi.)

Note that the password will not be displayed on the screen while you are entering it. **Please do not assume there is a problem; simply press Enter after entering the password.**

If you receive a connection refused message, as long as you are using an Orange Pi mirror, **please do not suspect that the Orange Pi password is incorrect; instead, look for other possible causes.**

3) The screen after successfully logging into the system is shown in the image below.

```
orangepi@orangepi:~$ ssh orangepi@10.31.0.166
orangepi@10.31.0.166's password:
OPINION SW
Welcome to Orange Pi 1.0.0 Bullseye with Linux 5.15.147-sun60iw2
System load:  2%           Up time:      4 min    Local users:  3
Memory usage: 6% of 7.68G  IP:        10.31.0.166
CPU temp:    63°C         Usage of /: 12% of 29G
[ 0 security updates available, 5 updates total: apt upgrade ]
Last check: 2025-10-27 01:37
Last login: Wed Nov  5 07:46:16 2025 from 10.31.0.92
orangepi@orangepizero3w:~$
```

If SSH fails to log in to the Linux system, first check if the development board's IP address can be pinged. If the ping is successful, you can log in to the Linux system via serial port or HDMI monitor. Then, enter the following command on the development board and try connecting again:

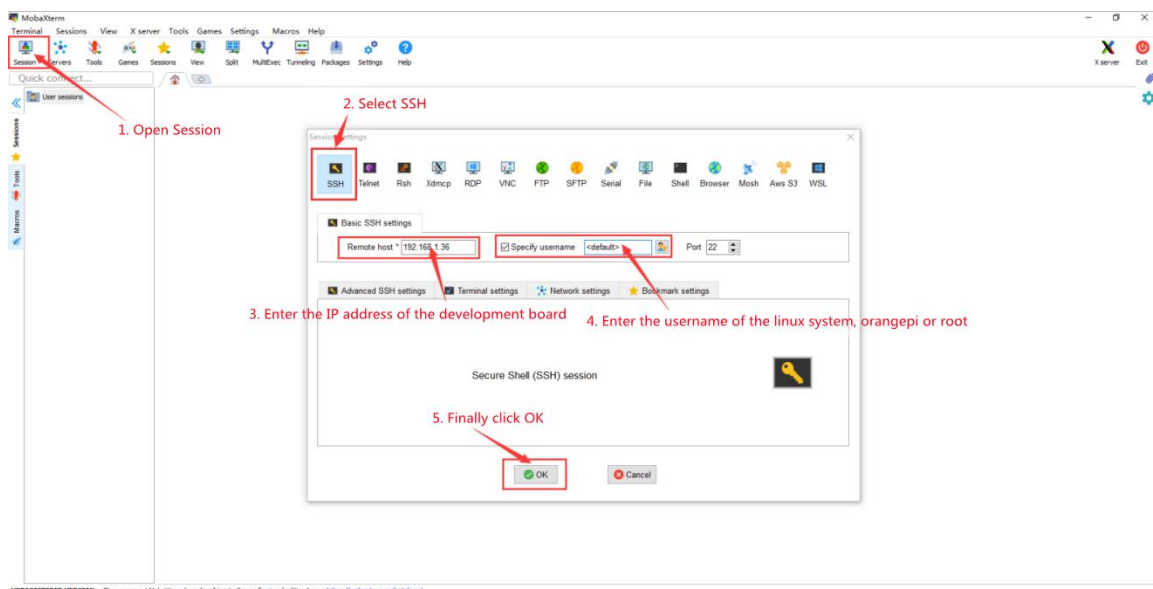


```
root@orangepi:~# reset_ssh.sh
```

If that still doesn't work, please try re-flashing the system.

3. 8. 2. SSH Remote Login to Development Board under Windows

- 1) First, obtain the IP address of the development board.
- 2) On Windows, you can use MobaXterm to remotely log in to the development board. First, create a new SSH session.
 - a. **Open Session**
 - b. Then select **SSH** in **Session Settings**
 - c. Then enter the development board's IP address in **Remote host**
 - d. Then enter the Linux system username root or **orangepi** in **Specify username**
 - e. Finally, click **OK**.

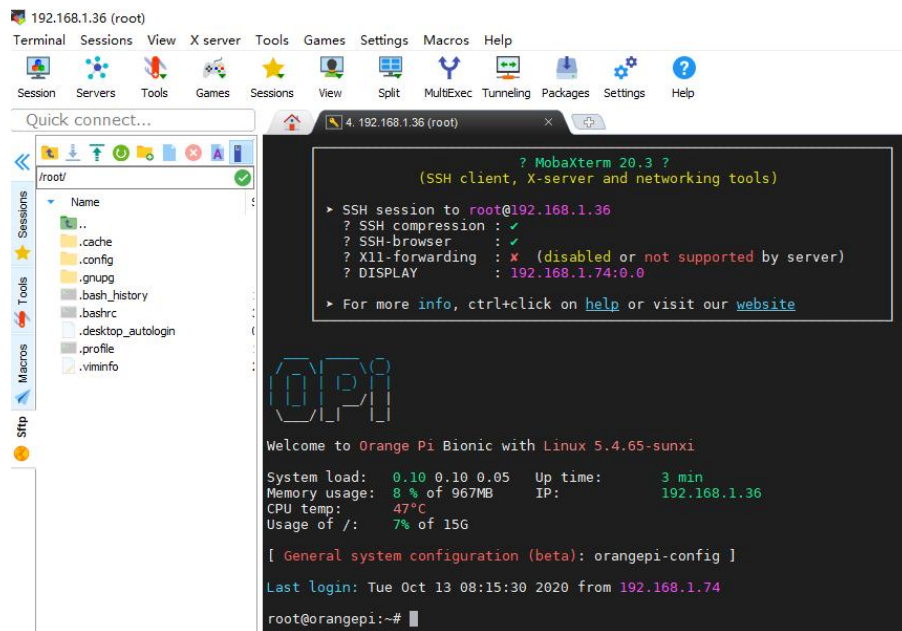


- 3) Then you will be prompted to enter a password. The default password for both root and orangepi users is orangepi.

Note that the password will not be displayed on the screen while you are entering it. Please do not assume there is a malfunction. Just press Enter after you finish entering it.



4) The screen after successfully logging into the system is shown in the image below.



3.9. How to use ADB

3.9.1. How to use ADB for networking

1) After the system starts, please execute the following command first.

```
orangepi@orangepi:~$ sudo /etc/adb_conf.sh
```

2) Then please confirm that adbd has started.

```
orangepi@orangepi:~$ ps -ax | grep "adbd"
808 ?        Sl      0:00 /usr/bin/adbd
3707 ttyFIQ0 S+      0:00 grep --color=auto adbd
```



3) Then check the IP address of the development board and write it down.

4) Then install the adb tool on your Ubuntu PC.

```
test@test:~$ sudo apt-get update
test@test:~$ sudo apt-get install -y adb
```

5) Then use the following command to connect to the network using adb.

```
test@test:~$ adb connect 192.168.1.xx:5555      ##Please replace the IP address
with the development board's IP address.
* daemon not running; starting now at tcp:5037
* daemon started successfully
connected to 192.168.1.xx:5555
test@test:~$ adb devices
List of devices attached
192.168.1.xx:5555      device
```

6) Then use the following command to log in to the Linux system on the development board.

```
test@test:~$ adb shell
root@orangepi3w:/#      <--- Seeing this prompt indicates that you have
successfully logged into the development board.
```

7) The command to upload files to the development board using adb is as follows:

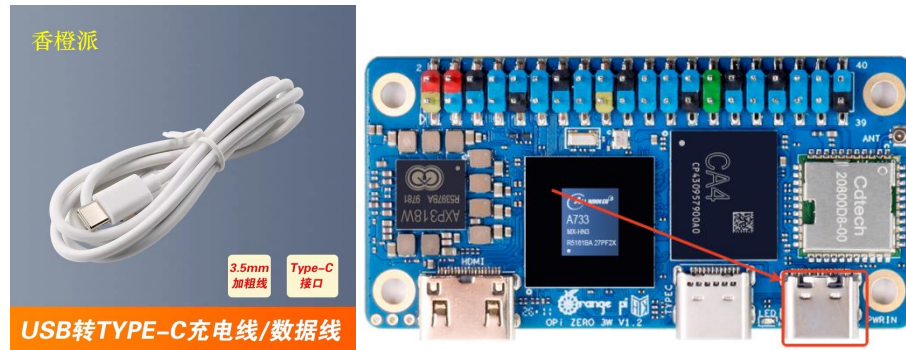
```
test@test:~$ adb push filename /root
filename: 1 file pushed. 3.7 MB/s (1075091 bytes in 0.277s)
```

8) The command to restart the development board using adb is as follows:

```
test@test:~$ adb reboot
```

3. 9. 2. Connect adb using a Type-C data cable

1) Prepare a USB Type-C data cable. Plug the USB end into the computer's USB port and the USB Type-C end into the development board's Type-C power connector (see the image on the right below for the connector location). In this case, the development board is powered by the computer's USB port, so please ensure that the computer's USB port can provide sufficient power to drive the development board.



2) Then run the following command to set the Type-C interface to device mode.

```
orangepi@orangepi:~$ sudo /etc/adb_conf.sh
```

3) Then please confirm that adbd has started.

```
orangepi@orangepi:~$ ps -ax | grep "adbd"
808 ?          Sl      0:00 /usr/bin/adbd
3707 ttyFIQ0  S+      0:00 grep --color=auto adbd
```

4) Then install the adb tool on your Ubuntu PC.

```
test@test:~$ sudo apt-get update
test@test:~$ sudo apt-get install -y adb
```

5) Then use the following command to check if the adb device has been recognized.

```
test@test:~$ adb devices
List of devices attached
e0f9f71bc343c305    device
```

6) Then use the following command to log in to the Linux system on the development board.

```
test@test:~$ adb shell
root@orangepi3w:/# <--- Seeing this prompt indicates that you have
successfully logged into the development board.
```

7) The command to upload files to the development board using adb is as follows:

```
test@test:~$ adb push filename /root
filename: 1 file pushed. 3.7 MB/s (1075091 bytes in 0.277s)
```


3. 10. HDMI Test

3. 10. 1. HDMI Display Test

1) Use a Mini HDMI to HDMI cable to connect the Orange Pi development board and the HDMI monitor.



2) If the HDMI monitor outputs an image after booting the Linux system, it means the Mini HDMI interface is working properly.

Note that many laptops, while equipped with HDMI ports, typically only have output functionality and lack HDMI input capability. This means that you cannot display the HDMI output from other devices on the laptop screen.

When connecting the development board's HDMI port to a laptop's HDMI port, please first confirm that your laptop supports HDMI input.

If there is no display via HDMI, please first check if the HDMI cable is plugged in securely. After confirming that the connection is correct, you can try a different screen to see if there is a display.

3. 10. 2. HDMI to VGA display test

1) First, you need to prepare the following accessories.

- a. HDMI to VGA converter



- b. A VGA cable



- c. A monitor or television that supports a VGA interface.

2) The HDMI to VGA display test is shown below.

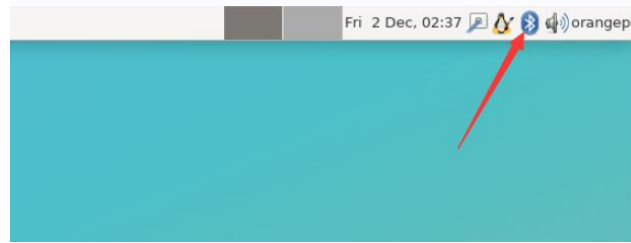


When using an HDMI to VGA adapter, no configuration is required for the development board or its Linux system; only the HDMI port on the development board needs to display correctly. Therefore, if you encounter problems during testing, please check the HDMI to VGA adapter, VGA cable, and monitor for issues.

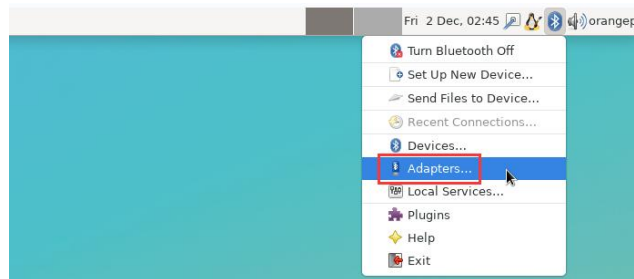
3. 11. Bluetooth Usage Instructions

3. 11. 1. Testing Methods for Desktop Image

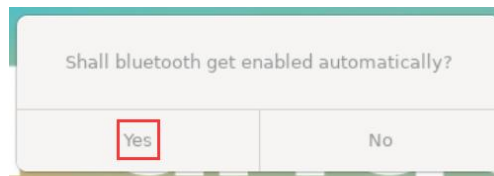
- 1) First, click the Bluetooth icon in the upper right corner of the desktop.



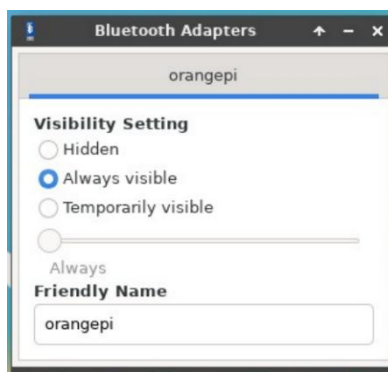
2) Then select the adapter



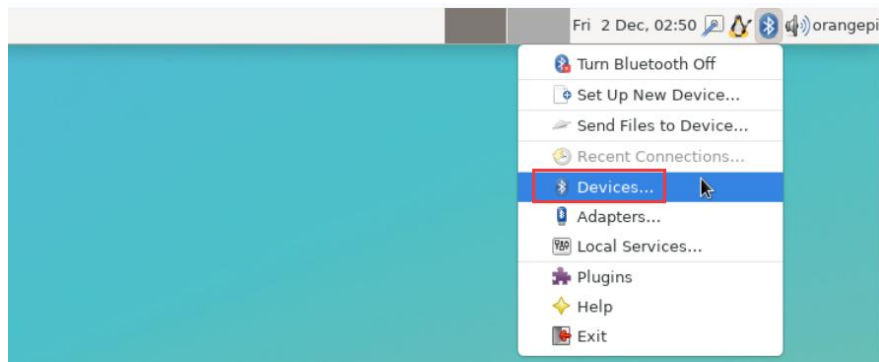
3) If the following prompt appears, please select **Yes**



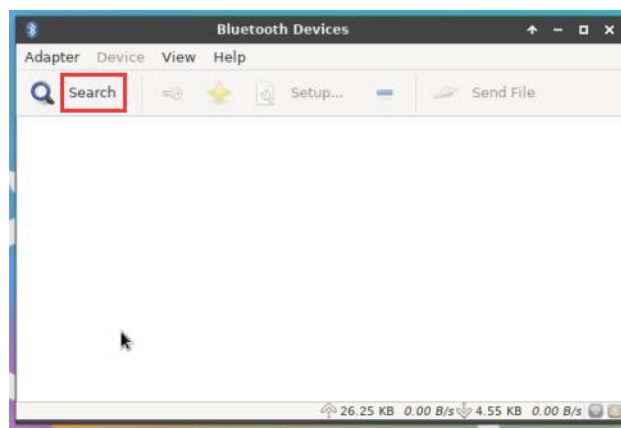
4) Then, in the Bluetooth adapter settings interface, set the **Visibility Setting** to **Always visible**, and then turn it off.



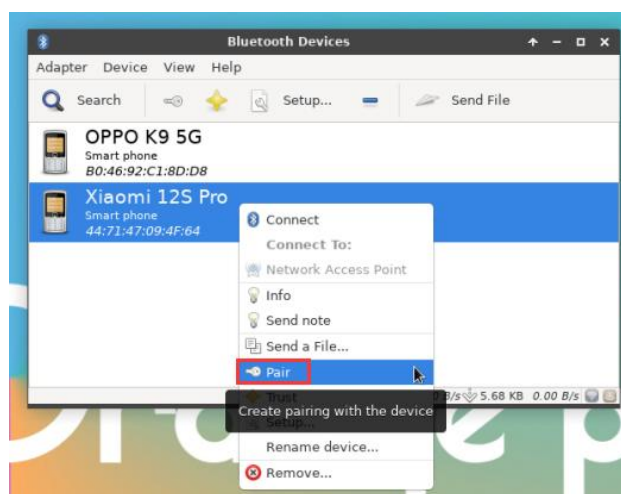
5) Then open the Bluetooth device's configuration interface.



6) Click **Search** to start scanning for nearby Bluetooth devices.



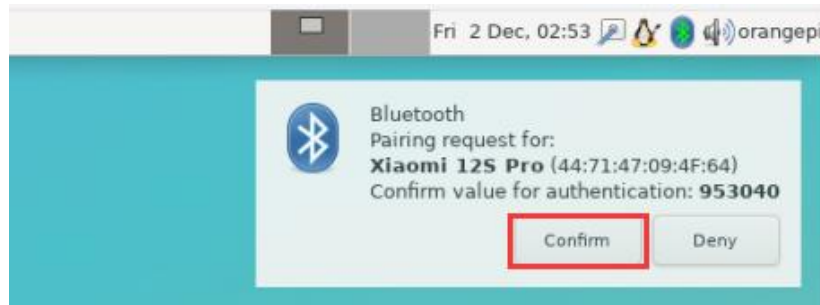
7) Then select the Bluetooth device you want to connect to, and right-click to bring up the operation interface for that Bluetooth device. Select **Pair** to start pairing. This demonstration shows pairing with an Android phone.



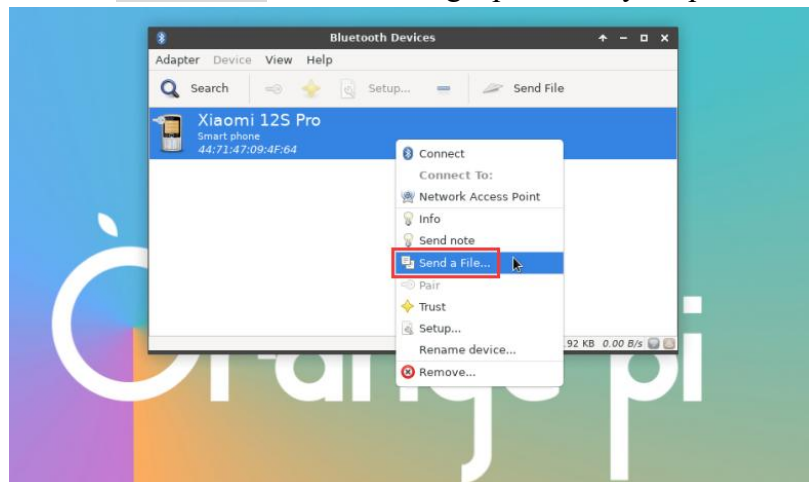
8) During pairing, a pairing confirmation box will pop up in the upper right corner of the



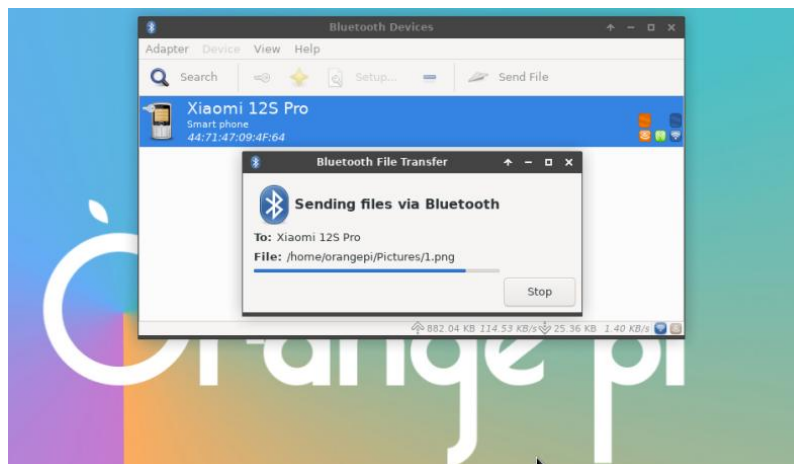
desktop. Select **Confirm** to confirm. You will also need to confirm on your phone at this time.



9) After pairing with your phone, you can select the paired Bluetooth device, then right-click and select **Send a File** to start sending a picture to your phone.



10) The interface for sending pictures is shown below.





3. 11. 2. How to use the server version image

1) After entering the system, you can first use the **hciconfig** command to check if there is a Bluetooth device node. If it exists, it means that Bluetooth initialization is normal.

```
orangepi@orangepi:~$ sudo apt update && sudo apt install -y bluez
orangepi@orangepi:~$ hciconfig -a
hci0:   Type: Primary   Bus: UART
        BD Address: 3E:61:3D:19:0E:52   ACL MTU: 1021:8   SCO MTU: 240:3
        UP RUNNING
        RX bytes:925 acl:0 sco:0 events:72 errors:0
        TX bytes:5498 acl:0 sco:0 commands:72 errors:0
        Features: 0xbf 0xff 0x8d 0xfe 0xdb 0x3d 0x7b 0xc7
        Packet type: DM1 DM3 DM5 DH1 DH3 DH5 HV1 HV2 HV3
        Link policy: RSWITCH SNIFF
        Link mode: SLAVE ACCEPT
        Name: 'orangepi'
        Class: 0x3c0000
        Service Classes: Rendering, Capturing, Object Transfer, Audio
        Device Class: Miscellaneous,
        HCI Version: 5.0 (0x9)   Revision: 0x400
        LMP Version: 5.0 (0x9)   Subversion: 0x400
        Manufacturer: Spreadtrum Communications Shanghai Ltd (492)
```

2) Use **bluetoothctl** to scan for Bluetooth devices.

```
orangepi@orangepi:~$ sudo bluetoothctl
[NEW] Controller 10:11:12:13:14:15 orangepizero3w [default]
Agent registered
[bluetooth]# power on           #Enable controller
Changing power on succeeded
[bluetooth]# discoverable on    #Set the controller to be discoverable.
Changing discoverable on succeeded
[CHG] Controller 10:11:12:13:14:15 Discoverable: yes
[bluetooth]# pairable on       #Configure the controller to be pairable.
Changing pairable on succeeded
[bluetooth]# scan on           #Start scanning for nearby Bluetooth devices
Discovery started
[CHG] Controller 10:11:12:13:14:15 Discovering: yes
```

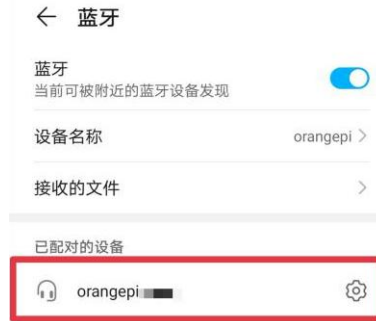


```
[NEW] Device 76:60:79:29:B9:31 76-60-79-29-B9-31
[NEW] Device 9C:2E:A1:42:71:11 小米手机
[NEW] Device DC:72:9B:4C:F4:CF orangepi
[bluetooth]# scan off      #Once you've scanned for the Bluetooth device you want
to connect to, you can stop scanning. Then, note down the Bluetooth device's MAC
address. In this test, the Bluetooth device is an Android phone with the Bluetooth
name "orangepi" and its corresponding MAC address is DC:72:9B:4C:F4:CF.
Discovery stopped
[CHG] Controller 10:11:12:13:14:15 Discovering: no
[CHG] Device DC:72:9B:4C:F4:CF RSSI is nil
```

3) Once the desired device is detected, pairing can proceed. Pairing requires the device's MAC address.

```
[bluetooth]# pair DC:72:9B:4C:F4:CF      #Pair using the MAC address of the
scanned Bluetooth device.
Attempting to pair with DC:72:9B:4C:F4:CF
[CHG] Device DC:72:9B:4C:F4:CF Connected: yes
Request confirmation
[leeb1m[agent] Confirm passkey 764475 (yes/no): yes  #Enter "yes" here; you'll also
need to confirm on your phone.
[CHG] Device DC:72:9B:4C:F4:CF Modalias: bluetooth:v010Fp107Ed1436
[CHG] Device DC:72:9B:4C:F4:CF UUIDs: 0000046a-0000-1000-8000-00805f9b34fb
[CHG] Device DC:72:9B:4C:F4:CF ServicesResolved: yes
[CHG] Device DC:72:9B:4C:F4:CF Paired: yes
Pairing successful    #Pairing successful
[CHG] Device DC:72:9B:4C:F4:CF ServicesResolved: no
[CHG] Device DC:72:9B:4C:F4:CF Connected: no
```

4) After successful pairing, the phone's Bluetooth interface will display as shown below.



5) Connecting Bluetooth devices requires installing the **pulseaudio-module-bluetooth** package and then starting the **pulseaudioservice**.

```
orange_pi@orange_pi:~$ sudo apt update
orange_pi@orange_pi:~$ sudo apt -y install pulseaudio-module-bluetooth
orange_pi@orange_pi:~$ pulseaudio --start
```

6) Methods for connecting Bluetooth devices

```
orange_pi@orange_pi:~$ sudo bluetoothctl
Agent registered
[bluetooth]# paired-devices      #View the MAC address of paired Bluetooth
devices

Device DC:72:9B:4C:F4:CF orange_pi
[bluetooth]# connect DC:72:9B:4C:F4:CF  #Connect Bluetooth devices using
MAC address
Attempting to connect to DC:72:9B:4C:F4:CF
[CHG] Device DC:72:9B:4C:F4:CF Connected: yes
Connection successful
[CHG] Device DC:72:9B:4C:F4:CF ServicesResolved: yes
[CHG] Controller 10:11:12:13:14:15 Discoverable: no
[orange_pi]#                    #The appearance of this prompt indicates a successful
connection.
```

7) After connecting the Bluetooth device, the Android phone's Bluetooth configuration interface will show a notification that audio for calls and media has been connected.

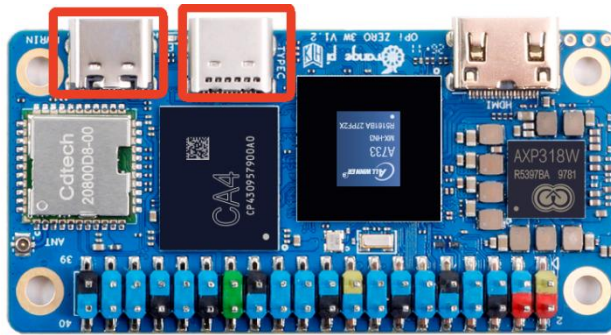


3. 12. USB Interface Test

The number of USB ports can be expanded by connecting a USB hub.

3. 12. 1. USB Interface Expansion Description

As shown in the image below, the development board has two Type-C USB ports, which cannot be directly connected to USB Type-A devices such as mice and keyboards. Note that only the left Type-C port supports power supply.



You can prepare a Type-C to USB cable as shown in the picture below, insert one end of the Type-C interface into the Type-C interface of the motherboard, and then connect the other end to USB devices such as mouse and keyboard. If you feel that one USB interface is not enough, you can also use a USB hub to expand multiple USB interfaces.



3. 12. 2. Test connecting a USB mouse or keyboard

- 1) Plug the USB keyboard into the USB port of the Orange Pi development board.
- 2) Connect the Orange Pi development board to an HDMI monitor.
- 3) If the mouse or keyboard functions normally in the operating system, it means the USB port is working properly (the mouse can only be used in desktop versions of the system).

3. 12. 3. Test connecting USB storage devices

- 1) First, insert the USB flash drive or USB external hard drive into the USB port of the Orange Pi development board.
- 2) Execute the following command. If you see the output of sdX, the USB drive has been successfully recognized.

```
orange@orange:~$ cat /proc/partitions | grep "sd*"
major minor  #blocks  name
   8         0   30044160 sda
   8         1   30043119 sda1
```

- 3) You can use the mount command to mount the USB drive to **/mnt**, and then you can view the files on the USB drive.

```
orange@orange:~$ sudo mount /dev/sda1 /mnt/
orange@orange:~$ ls /mnt/
test.txt
```

- 4) After mounting, you can use the **df -h** command to check the USB drive's capacity usage and mount point.



```
orange@orange:~$ df -h | grep "sd"
/dev/sda1      29G  208K  29G   1% /mnt
```

3. 12. 4. USB Ethernet Card Test

1) The following are the USB Ethernet adapters that have been tested and are currently working. The RTL8153 USB Gigabit Ethernet adapter works correctly when plugged into the USB 2.0 Host port on the development board, but the speed does not reach gigabit speeds. Please note this.

Serial Number	Model
1	RTL8152B USB 100Mbps network card
2	RTL8153 USB Gigabit network card

2) First, insert the USB network adapter into the USB port of the development board. Then, insert a network cable into the USB network adapter, ensuring that the network cable can access the internet. If you can see the following log information using the **dmesg** command, it means that the USB network adapter is being recognized normally.

```
orange@orange:~$ dmesg | tail
[ 121.985016] usb 3-1: USB disconnect, device number 2
[ 126.873772] sunxi-ehci 5311000.ehci3-controller: ehci_irq: highspeed device connect
[ 127.094054] usb 3-1: new high-speed USB device number 3 using sunxi-ehci
[ 127.357472] usb 3-1: reset high-speed USB device number 3 using sunxi-ehci
[ 127.557960] r8152 3-1:1.0 eth1: v1.08.9
[ 127.602642] r8152 3-1:1.0 enx00e04c362017: renamed from eth1
[ 127.731874] IPv6: ADDRCONF(NETDEV_UP): enx00e04c362017: link is not ready
[ 127.763031] IPv6: ADDRCONF(NETDEV_UP): enx00e04c362017: link is not ready
[ 129.892465] r8152 3-1:1.0 enx00e04c362017: carrier on
[ 129.892583] IPv6: ADDRCONF(NETDEV_CHANGE): enx00e04c362017: link becomes ready
```

3) Then, you can use the **ifconfig** command to see the USB network adapter's device node and the automatically assigned IP address.

```
orange@orange:~$ sudo ifconfig
eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
    inet 192.168.1.177  netmask 255.255.255.0  broadcast 192.168.1.255
    inet6 fe80::681f:d293:4bc5:e9fd  prefixlen 64  scopeid 0x20<link>
    ether 00:e0:4c:36:20:17  txqueuelen 1000  (Ethernet)
```



```

RX packets 1849  bytes 134590 (134.5 KB)
RX errors 0   dropped 125  overruns 0  frame 0
TX packets 33   bytes 2834 (2.8 KB)
TX errors 0   dropped 0 overruns 0  carrier 0  collisions 0

```

4) The command to test network connectivity is as follows (note that eth1 needs to be replaced with the actual network device).

```

orangepi@orangepi:~$ ping www.baidu.com -I eth1
PING www.a.shifen.com (14.215.177.38) from 192.168.1.12 eth0: 56(84) bytes of data.
64 bytes from 14.215.177.38 (14.215.177.38): icmp_seq=1 ttl=56 time=6.74 ms
64 bytes from 14.215.177.38 (14.215.177.38): icmp_seq=2 ttl=56 time=6.80 ms
64 bytes from 14.215.177.38 (14.215.177.38): icmp_seq=3 ttl=56 time=6.26 ms
64 bytes from 14.215.177.38 (14.215.177.38): icmp_seq=4 ttl=56 time=7.27 ms
^C
--- www.a.shifen.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3002ms
rtt min/avg/max/mdev = 6.260/6.770/7.275/0.373 ms

```

3. 12. 5. USB Camera Test

- 1) First, plug the USB camera into the USB port of the Orange Pi development board.
- 2) Then, using the lsmod command, you can see that the kernel has automatically loaded the following modules.

```

orangepi@orangepi:~$ lsmod
Module                Size  Used by
uvcvideo              106496  0

```

- 3) The device node information for the USB camera can be seen using the v4l2-ctl command as `/dev/video0`

```

orangepi@orangepi:~$ sudo apt update
orangepi@orangepi:~$ sudo apt install -y v4l-utils
orangepi@orangepi:~$ v4l2-ctl --list-devices
USB 2.0 Camera (usb-sunxi-ehci-1):
    /dev/video0

```

Note that the 'l' in v4l2 is a lowercase letter 'l', not the number '1'.



Also, the video number is not always video0; please refer to what you actually see.

4) Use fswebcam to test the USB camera

a. Install fswebcam

```
orangepi@orangepi:~$ sudo apt update
orangepi@orangepi:~$ sudo apt-get install -y fswebcam
```

b. After installing fswebcam, you can use the following command to take a picture.

- a) The -d option specifies the device node for the USB camera.
- b) --no-banner removes the watermark from the photo.
- c) The -r option specifies the photo resolution.
- d) The -S option sets the number of preceding frames to skip.
- e) ./image.jpg sets the name and path of the generated photo.

```
orangepi@orangepi:~$ sudo fswebcam -d /dev/video0 \
--no-banner -r 1280x720 -S 5 ./image.jpg
```

c. On a server-side Linux system, after taking photos, you can use the 'scp' command to transfer the photos to an Ubuntu PC for mirroring and viewing.

```
orangepi@orangepi:~$ scp image.jpg test@192.168.1.55:/home/test (Modify the IP
address and path according to the actual situation.)
```

d. In desktop Linux systems, captured images can be viewed directly via an HDMI monitor.

5) Use mjpg-streamer to test the USB camera

a. Download mjpg-streamer

a) Download link from Github:

```
orangepi@orangepi:~$ git clone https://github.com/jacksonliam/mjpg-streamer
```

b) The download address for the Gitee mirror is:

```
orangepi@orangepi:~$ git clone https://gitee.com/leeboby/mjpg-streamer
```

b. Install dependent software packages

a) Ubuntu System

```
orangepi@orangepi:~$ sudo apt-get install -y cmake libjpeg8-dev
```

b) Debian System

```
orangepi@orangepi:~$ sudo apt-get install -y cmake libjpeg62-turbo-dev
```

c. Compile and install mjpg-streamer

```
orangepi@orangepi:~$ cd mjpg-streamer/mjpg-streamer-experimental
```



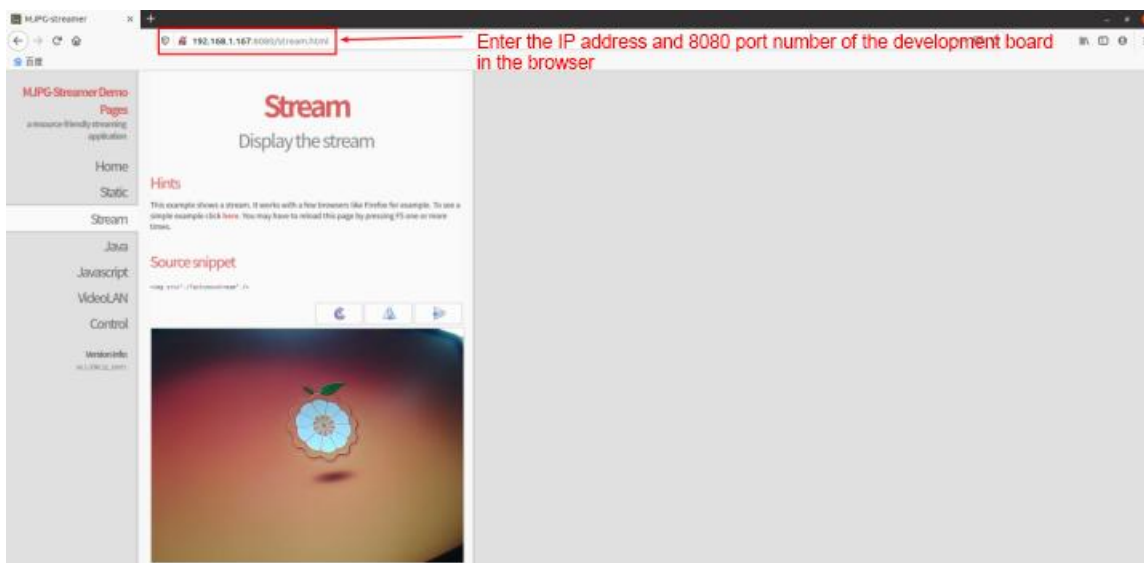
```
orange@orange:~/mjpg-streamer/mjpg-streamer-experimental$ make -j4
orange@orange:~/mjpg-streamer/mjpg-streamer-experimental$ sudo make install
```

d. Then enter the following command to start mjpg_streamer

Note that the video number is not necessarily video0; please refer to what you actually see.

```
orange@orange:~/mjpg-streamer/mjpg-streamer-experimental$ export LD_LIBRARY_PATH=.
orange@orange:~/mjpg-streamer/mjpg-streamer-experimental$ sudo ./mjpg_streamer -i "/input_uvc.so -d \
/dev/video0 -u -f 30" -o "/output_http.so -w ./www"
```

e. Then, in the browser of an Ubuntu PC, Windows PC, or mobile phone on the same local network as the development board, enter **[the development board's IP address: 8080]** to see the video output from the camera.



3. 13. Temperature Sensor

1) The A733 contains multiple temperature sensors, which can be accessed through the system-provided /sys/class/thermal/thermal_zoneX interface, among which:

- a. type: Indicates the sensor type (e.g., CPU, GPU, DDR, etc.)
- b. temp: Represents the temperature value (in millidegrees Celsius, which needs to be divided by 1000).

2) The system also provides the **check_temp.sh** script, which can list all temperature sensors and their corresponding temperatures at once:



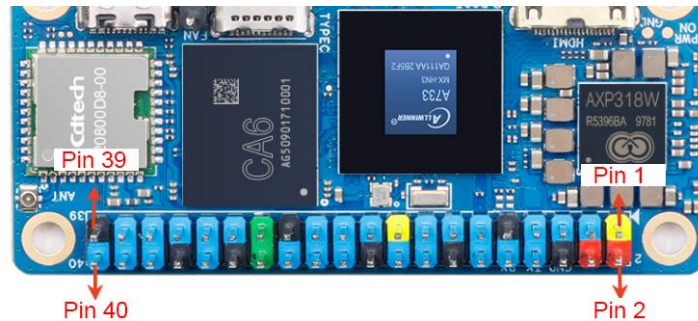
```

orange@orange:~$ check_temp.sh
ZONE          TYPE          TEMP(°C)
thermal_zone5 cpul_idle_zone 48.7
thermal_zone3 cpul_thermal_zone 48.7
thermal_zone1 ddr_thermal_zone 48.7
thermal_zone6 cpub_idle_zone 48.4
thermal_zone4 gpu_thermal_zone 48.0
thermal_zone0 cpub_thermal_zone 47.9
thermal_zone2 npu_thermal_zone 47.5
thermal_zone7 skin_zone 33.7
orange@orange:~$

```

3. 14. 40 Pinout of the Interface

1) Please refer to the silkscreen diagram on the Orange Pi Zero 3w development board for the pin order of the 40-pin interface.



2) The functions of the 40-pin interface pins on the development board are shown in the table below.

复用功能	复用功能	复用功能	GPIO	GPIO序号	引脚序号	引脚序号	GPIO序号	GPIO	复用功能	复用功能
			3.3V		1	2		5V		
		TV10-SDA	PE3	35	3	4		5V		
		TV10-SCK	PE2	34	5	6		GND		
		PWM0-0	PE4	36	7	8	41	PB9	UART0-TX	
			GND		9	10	42	PB10	UART0-RX	
		UART2-TX	PB0	32	11	12	37	PB5		
		UART2-RX	PE1	33	13	14		GND		
			PE11	139	15	16	354	PL2	UART7-TX	
			3.3V		17	18	355	PL3	UART7-RX	
		TV12-SDA	PE2	130	19	20		GND		
		TV13-SCK	PE3	131	21	22	96	PD0		
UART6-RX		TV12-SCK	PE1	129	23	24	128	PE0	SPI3-CS0	UART6-TX
			GND		25	26	132	PE4	SPI3-CS1	TV13-SDA
			PL5	357	27	28	356	PL4		
			PE12	140	29	30		GND		
			PE13	141	31	32	97	PD1	PWM0-1	
		PWM0-3	PD3	99	33	34		GND		
		PWM0-8	PE6	38	35	36	98	PD2	PWM0-2	
		PWM0-4	PD4	100	37	38	40	PE8	TV11-SDA	
			GND		39	40	39	PE7	TV11-SCL	PWM0-9

3) The 40-pin interface has a total of 28 GPIO ports, and the voltage of all GPIO ports is **3.3V**.



3. 15. Method for installing wiringOP

Note that wiringOP is pre-installed in the Linux image released for Orange Pi. Unless the wiringOP code is updated, there is no need to download, compile, and install it again; you can use it directly.

The compiled wiringOP deb package is located in orangepi-build at the following path:

[orangepi-build/external/cache/debs/arm64/wiringpi_x.xx.deb](#)

After entering the system, you can run the gpio readall command. If you see the following output, it means that wiringOP has been pre-installed and can be used normally.

```
orangepi@orangepi3w:~$ gpio readall
```

ZER03W						ZER03W					
GPIO	wPi	Name	Mode	V	Physical	GPIO	wPi	Name	Mode	V	Physical
35	0	3.3V			1	2		5V			
34	1	SDA.0	OFF	0	3	4		5V			
36	2	SCL.0	OFF	0	5	6		GND			
		PWM0-0	OFF	0	7	8	0	ALT2	TXD.0	3	41
		GND			9	10	0	ALT2	RXD.0	4	42
32	5	PB0	ALT14	1	11	12	0	OFF	PB5	6	37
33	7	PB1	ALT14	1	13	14			GND		
137	8	PE9	OFF	0	15	16	0	OFF	PL2	9	354
		3.3V			17	18	0	OFF	PL3	10	355
130	11	MOSI.3	OFF	0	19	20			GND		
131	12	MISO.3	OFF	0	21	22	0	OFF	PD0	13	96
129	14	SCLK.3	OFF	0	23	24	0	OFF	CE.0	15	128
		GND			25	26	0	OFF	PE4	16	132
357	17	PL5	OFF	0	27	28	0	OFF	PL4	18	356
140	19	PE12	OFF	0	29	30			GND		
141	20	PE13	OFF	0	31	32	0	OFF	PD1	21	97
99	22	PD3	OFF	0	33	34			GND		
38	23	PB6	OFF	0	35	36	0	OFF	PD2	24	98
100	25	PD4	OFF	0	37	38	0	OFF	PB8	26	40
		GND			39	40	0	OFF	PB7	27	39

1) Download the wiringOP code

```
orangepi@orangepi:~$ sudo apt update
```

```
orangepi@orangepi:~$ sudo apt install -y git
```

```
orangepi@orangepi:~$ git clone https://github.com/orangepi-xunlong/wiringOP.git -b next
```



Note that you need to download the wiringOP next branch of the source code; please don't forget the `-b next` parameter.

If you encounter problems downloading the code from GitHub, you can directly use the `wiringOP` source code included in the Linux mirror, located at: `/usr/src/wiringOP`.

2) Compile and install wiringOP

```
orangePi@orangePi:~$ cd wiringOP
orangePi@orangePi:~/wiringOP$ sudo ./build clean
orangePi@orangePi:~/wiringOP$ sudo ./build
```

3) The output of the gpio readall command is as follows

```
orangePi@orangePizero3w:~$ gpio readall
```

GPIO	wPi	Name	Mode	V	Physical	V	Mode	Name	wPi	GPIO
		3.3V			1	2		5V		
35	0	SDA.0	OFF	0	3	4		5V		
34	1	SCL.0	OFF	0	5	6		GND		
36	2	PWM0-0	OFF	0	7	8	0	ALT2	3	41
		GND			9	10	0	ALT2	4	42
32	5	PB0	ALT14	1	11	12	0	OFF	6	37
33	7	PB1	ALT14	1	13	14		GND		
137	8	PE9	OFF	0	15	16	0	OFF	9	354
		3.3V			17	18	0	OFF	10	355
130	11	MOSI.3	OFF	0	19	20		GND		
131	12	MISO.3	OFF	0	21	22	0	OFF	13	96
129	14	SCLK.3	OFF	0	23	24	0	OFF	15	128
		GND			25	26	0	OFF	16	132
357	17	PL5	OFF	0	27	28	0	OFF	18	356
140	19	PE12	OFF	0	29	30		GND		
141	20	PE13	OFF	0	31	32	0	OFF	21	97
99	22	PD3	OFF	0	33	34		GND		
38	23	PB6	OFF	0	35	36	0	OFF	24	98
100	25	PD4	OFF	0	37	38	0	OFF	26	40
		GND			39	40	0	OFF	27	39
GPIO	wPi	Name	Mode	V	Physical	V	Mode	Name	wPi	GPIO
					ZER03W					

3. 16. 40-pin interface GPIO, I2C, UART, SPI and PWM testing

3. 16. 1. 40-pin GPIO port test

1) The following example demonstrates how to set the high and low levels of a GPIO port, using pin 7 (corresponding to GPIO PB4, wPi number 2).



```
orangepi@orangezipzero3w:~$ gpio readall
```

ZER03W											
GPIO	wPi	Name	Mode	V	Physical	V	Mode	Name	wPi	GPIO	
		3.3V			1	2		5V			
35	0	SDA.0	OFF	0	3	4		5V			
34	1	SCL.0	OFF	0	5	6		GND			
36	2	PWM0-0	OFF	0	7	8	0	ALT2	TXD.0	3	41
		GND			9	10	0	ALT2	RXD.0	4	42

2) First, set the GPIO port to output mode. The third parameter requires the input wPi serial number corresponding to the pin.

```
orangepi@orangepi:~/wiringOP$ gpio mode 2 out
```

3) Then set the GPIO port output to low level. After setting, you can use a multimeter to measure the voltage value of the pin. If it is 0V, it means that the low level setting was successful.

```
orangepi@orangepi:~/wiringOP$ gpio write 2 0
```

Using GPIO ReadAll, we can see that the value (V) of pin 7 has changed to 0.

```
orangepi@orangezipzero3w:~$ gpio readall
```

ZER03W											
GPIO	wPi	Name	Mode	V	Physical	V	Mode	Name	wPi	GPIO	
		3.3V			1	2		5V			
35	0	SDA.0	OFF	0	3	4		5V			
34	1	SCL.0	OFF	0	5	6		GND			
36	2	PWM0-0	OUT	0	7	8	0	ALT2	TXD.0	3	41
		GND			9	10	0	ALT2	RXD.0	4	42

4) Then set the GPIO port to output a high level. After setting it, you can use a multimeter to measure the voltage value of the pin. If it is 3.3V, it means that the high level has been set successfully.

```
orangepi@orangepi:~/wiringOP$ gpio write 2 1
```

Using GPIO ReadAll, we can see that the value (V) of pin 7 has changed to 1.

```
orangepi@orangezipzero3w:~$ gpio readall
```

ZER03W											
GPIO	wPi	Name	Mode	V	Physical	V	Mode	Name	wPi	GPIO	
		3.3V			1	2		5V			
35	0	SDA.0	OFF	0	3	4		5V			
34	1	SCL.0	OFF	0	5	6		GND			
36	2	PWM0-0	OUT	1	7	8	0	ALT2	TXD.0	3	41
		GND			9	10	0	ALT2	RXD.0	4	42

5) The configuration method for other pins is similar; simply modify the wPi serial



number to match the corresponding pin serial number.

3. 16. 2. 40 Method for setting pull-up/pull-down resistors for GPIO pin

1) The following example demonstrates how to set the pull-up and pull-down resistors of a GPIO port, using pin 7 (corresponding to GPIO PB4, wPi serial number 2) as an example.

```
orangeypi@orangezipero3w:~$ gpio readall
```

ZER03W											
GPIO	wPi	Name	Mode	V	Physical	V	Mode	Name	wPi	GPIO	
35	0	3.3V			1	2		5V			
		SDA.0	OFF	0	3	4		5V			
34	1	SCL.0	OFF	0	5	6		GND			
36	2	PWM0-0	OFF	0	7	8	0	ALT2	TXD.0	3	41
		GND			9	10	0	ALT2	RXD.0	4	42

2) First, you need to set the GPIO port to input mode. The third parameter requires the wPi serial number corresponding to the pin.

```
orangeypi@orangeypi:~/wiringOP$ gpio mode 2 in
```

3) After setting it to input mode, execute the following command to set the GPIO port to pull-up mode.

```
orangeypi@orangeypi:~/wiringOP$ gpio mode 2 up
```

4) Then enter the following command to read the level of the GPIO port. If the level is 1, it means that the pull-up mode is set successfully.

```
orangeypi@orangeypi:~/wiringOP$ gpio read 2
```

```
1
```

5) Then execute the following command to set the GPIO port to pull-down mode.

```
orangeypi@orangeypi:~/wiringOP$ gpio mode 2 down
```

6) Then enter the following command to read the level of the GPIO port. If the level is 0, it means that the pull-down mode is set successfully.

```
orangeypi@orangeypi:~/wiringOP$ gpio read 2
```

```
0
```

3. 16. 3. 40 pin SPI test

1) As shown in the figure below, the SPI available for Orange Pi Zero 3w is SPI3.



复用功能	复用功能	复用功能	GPIO	GPIO序号	引脚序号	引脚序号	GPIO序号	GPIO	复用功能	复用功能
			3.3V		1	2		5V		
		TVIO-SDA	PE3	35	3	4		5V		
		TVIO-SCK	PE2	34	5	6		GND		
		PWM0-0	PE4	36	7	8	41	PE9	UART0-TX	
			GND		9	10	42	PE10	UART0-RX	
		UART2-TX	PE0	32	11	12	37	PE5		
		UART2-RX	PE1	33	13	14		GND		
			PE11	139	15	16	354	PL2	UART7-TX	
			3.3V		17	18	355	PL3	UART7-RX	
	TVI2-SDA	SPI3-MOSI	PE2	130	19	20		GND		
	TVI3-SCK	SPI3-MISO	PE3	131	21	22	96	PD0		
UART6-RX	TVI2-SCK	SPI3-CLK	PE1	129	23	24	128	PE0	SPI3-CS0	UART6-TX
			GND		25	26	132	PE4	SPI3-CS1	TVI3-SDA
			PL5	357	27	28	356	PL4		
			PE12	140	29	30		GND		
			PE13	141	31	32	97	PD1	PWM0-1	
		PWM0-3	PD3	99	33	34		GND		
		PWM0-8	PE6	38	35	36	98	PD2	PWM0-2	
		PWM0-4	PD4	100	37	38	40	PE8	TVI1-SDA	
			GND		39	40	39	PE7	TVI1-SCL	PWM0-9

2) The pins corresponding to SPI3 in the 40-pin configuration are shown in the table below.

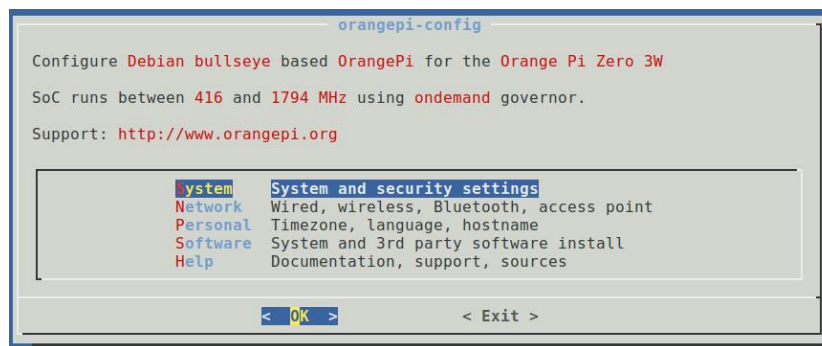
	SPI3 corresponds to 40 pins
MOSI	PIN 19
MISO	PIN 21
CLK	PIN 23
CS0	PIN 24
CS1	PIN 26

3) In Linux systems, the SPI interface on the 40-pin connector is disabled by default and needs to be manually enabled to be used. Detailed steps are as follows:

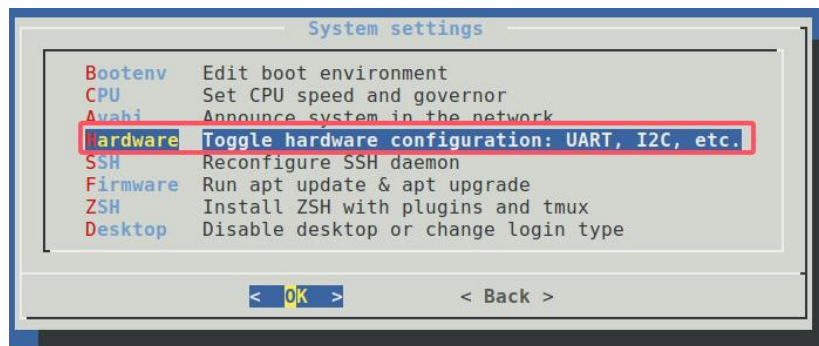
- First, run **orange-pi-config**. Regular users should remember to grant **sudo** privileges.

```
orange-pi@orange-pi:~$ sudo orange-pi-config
```

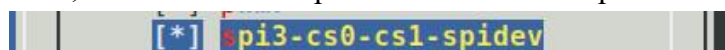
- Then Select **System**



- Then Select **Hardware**



- d. Then use the arrow keys on the keyboard to navigate to the location shown in the image below, and then use the spacebar to select the open SPI configuration.



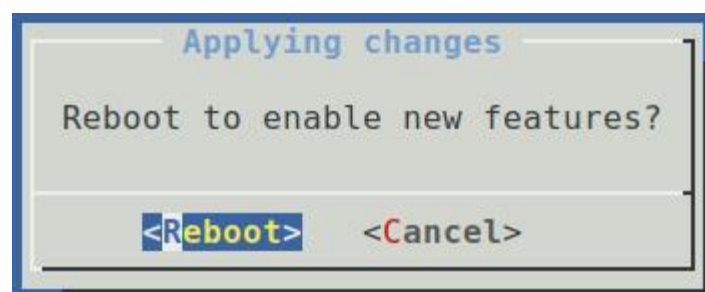
- e. Then select **<Save>** to save.



- f. Then select **<Back>**



- g. Then select **<Reboot>** to restart the system and make the configuration take effect.



- 4) Next, check if the **spidevx.x** device node exists in the Linux system. If it does, it means the SPI configuration has taken effect.

```
orange@orange:~$ ls /dev/spidev*
/dev/spidev3.0
```

- 5) Without shorting the mosi and miso pins of SPI3, the output of `spidev_test` is shown below. It can be seen that the TX and RX data are inconsistent.



```

orange@orange:~$ sudo spidev_test -v -D /dev/spidev3.0
spi mode: 0x0
bits per word: 8
max speed: 500000 Hz (500 KHz)
TX | FF FF FF FF FF FF FF 40 00 00 00 00 95 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
FF FF FF FF FF FF F0 0D | .....@.....
RX | FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
FF FF FF FF FF FF FF FF | .....

```

6) Then short the mosi and miso pins of SPI3 and run spidev_test. The output is as follows. You can see that the sent and received data are the same.

```

orange@orange:~$ sudo spidev_test -v -D /dev/spidev3.0
spi mode: 0x0
bits per word: 8
max speed: 500000 Hz (500 KHz)
TX | FF FF FF FF FF FF FF 40 00 00 00 00 95 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
FF FF FF FF FF FF F0 0D | .....@.....
RX | FF FF FF FF FF FF FF 40 00 00 00 00 95 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
FF FF FF FF FF FF F0 0D | .....@.....

```

3. 16. 4. 40 pin I2C test

1) As shown in the figure below, the Orange Pi Zero 3w has four i2c buses available: i2c0, i2c1, i2c2 and i2c3.

复用功能	复用功能	复用功能	GPIO	GPIO序号	引脚序号	引脚序号	GPIO序号	GPIO	复用功能	复用功能
			3.3V		1	2		5V		
					3	4		5V		
					5	6		GND		
					7	8	41	PB9	UART0-TX	
					9	10	42	PB10	UART0-RX	
					11	12	37	PB5		
					13	14		GND		
					15	16	354	PL2	UART7-TX	
					17	18	355	PL3	UART7-RX	
					19	20		GND		
					21	22	96	PD0		
					23	24	128	PE0	SPI3-CS0	UART6-TX
					25	26	132	PE4	SPI3-CS1	UART6-TX
					27	28	356	PL4		
					29	30		GND		
					31	32	97	PD1	PWM0-1	
					33	34		GND		
					35	36	98	PD2	PWM0-2	
					37	38	40	PB8	UART1-SDA	
					39	40	39	PB7	UART1-SCL	PWM0-9

2) The pins corresponding to the four I2C bus groups in the 40-pin configuration are shown in the table below.

I2C bus	SDA corresponds to 40 pins	SCL corresponds to 40 pins	dtbo corresponding configuration
---------	-------------------------------	-------------------------------	-------------------------------------



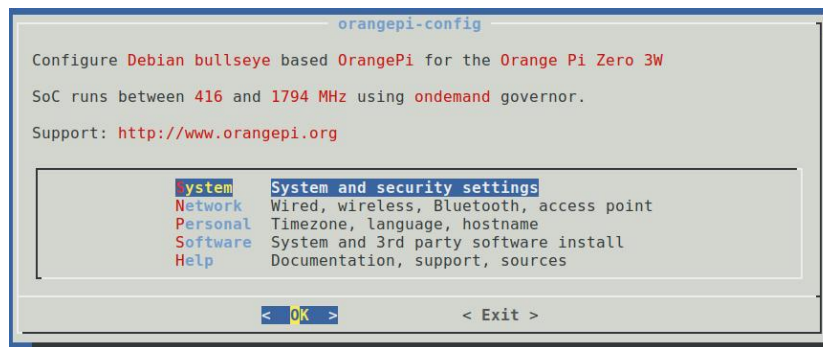
I2C0	PIN 3	PIN 5	i2c0
I2C1	PIN 38	PIN 40	i2c1
I2C2	PIN 19	PIN 23	i2c2
I2C3	PIN 26	PIN 21	i2c3

3) In Linux systems, the I2C port on pin 40 is disabled by default and needs to be manually enabled to function. Detailed steps are as follows:

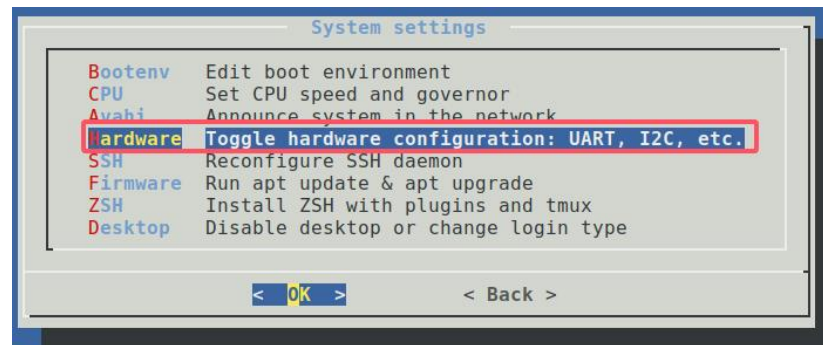
- a. First, run **orange-pi-config**. Regular users should remember to grant **sudo** privileges.

```
orange-pi@orange-pi:~$ sudo orange-pi-config
```

- b. Then select **System**



- c. Then select **Hardware**



- d. Then use the arrow keys on your keyboard to navigate to the location shown in the image below, and then use the spacebar to select the open I2C configuration.



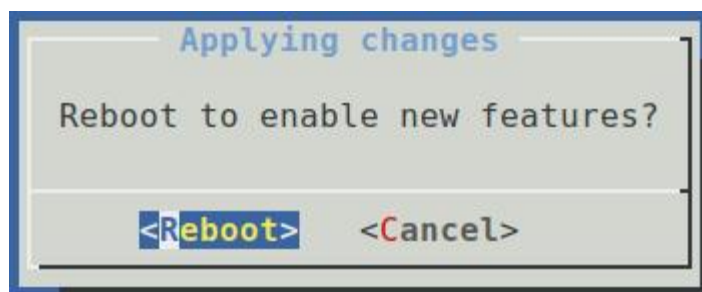
- e. Then select **<Save>** to save.



- f. Then select **<Back>**



- g. Then select **<Reboot>** to restart the system and make the configuration take effect.



- 4) After booting the Linux system, first confirm that there are I2C device nodes under /dev.

```
orangePi@orangePi:~$ ls /dev/i2c-*
/dev/i2c-0 /dev/i2c-1 /dev/i2c-13 /dev/i2c-15 /dev/i2c-2 /dev/i2c-20
/dev/i2c-3 /dev/i2c-5 /dev/i2c-7 /dev/i2c-8 /dev/i2c-9
```

- 5) Then start testing I2C. First, install i2c-tools.

```
orangePi@orangePi:~$ sudo apt-get update
orangePi@orangePi:~$ sudo apt-get install -y i2c-tools
```

- 6) Then connect an I2C device to the I2C pin of the 40-pin connector.

Please select the 5V and 3.3V pins according to the specific I2C device, as different I2C devices may require different voltage values.

- 7) Then use the **i2cdetect -y** command. If the address of the connected I2C device can be detected, it means that I2C is working normally.

```
orangePi@orangePi:~$ sudo i2cdetect -y -r 0 #i2c0 commands
```




```

orange@orange:~$ sudo i2cdetect -y -r 1      #i2c1 commands
orange@orange:~$ sudo i2cdetect -y -r 2      #i2c2 commands
orange@orange:~$ sudo i2cdetect -y -r 3      #i2c3 commands

```

3. 16. 5. 40 pin UART test

1) As shown in the table below, the Orange Pi Zero 3w can use three UART buses: UART2, UART6, and UART7.

复用功能	复用功能	复用功能	GPIO	GPIO序号	引脚序号	引脚序号	GPIO序号	GPIO	复用功能	复用功能
			3.3V		1	2		5V		
		TWI0-SDA	PE3	35	3	4		5V		
		TWI0-SCK	PE2	34	5	6		GND		
		PWM0-0	PE4	36	7	8	41	PE9	UART0-TX	
			GND		9	10	42	PE10	UART0-RX	
		UART2-TX	PE0	32	11	12	37	PE5		
		UART2-RX	PE1	33	13	14		GND		
			PE11	139	15	16	354	PL2	UART7-TX	
			3.3V		17	18	355	PL3	UART7-RX	
	TWI2-SDA	SPI3-MOSI	PE2	130	19	20		GND		
	TWI3-SCK	SPI3-MISO	PE3	131	21	22	96	PD0		
UART6-RX	TWI2-SCK	SPI3-CLK	PE1	129	23	24	128	PE0	SPI3-CS0	UART6-TX
			GND		25	26	132	PE4	SPI3-CS1	TWI3-SDA
			PL5	357	27	28	356	PL4		
			PE12	140	29	30		GND		
			PE13	141	31	32	97	PD1	PWM0-1	
		PWM0-3	PD3	99	33	34		GND		
		PWM0-8	PE6	38	35	36	98	PD2	PWM0-2	
		PWM0-4	PD4	100	37	38	40	PE8	TWI1-SDA	
			GND		39	40	39	PE7	TWI1-SCL	PWM0-9

2) The pins corresponding to the three UART bus groups in the 40-pin configuration are shown in the table below.

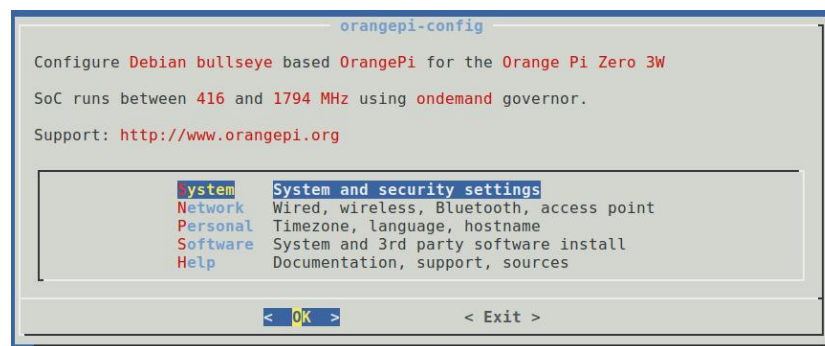
UART BUS	RX corresponds to 40 pin	TX corresponds to 40 pins	dtbo corresponding configuration
UART2	PIN 13	PIN 11	uart2
UART6	PIN 23	PIN 24	uart6
UART7	PIN 18	PIN 16	uart7

3) In Linux systems, the UART pin 40 is disabled by default and needs to be manually enabled to be used. Detailed steps are as follows:

- First, run **orange-pi-config**. Regular users should remember to grant **sudo** privileges.

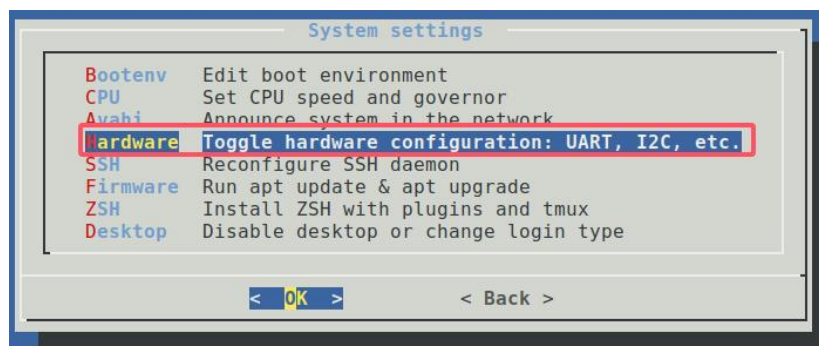
```
orange@orange:~$ sudo orange-pi-config
```

- Then Select **System**





- c. Then Select **Hardware**



- d. Then use the arrow keys on your keyboard to navigate to the location shown in the image below, and then use the spacebar to select the open UART configuration.



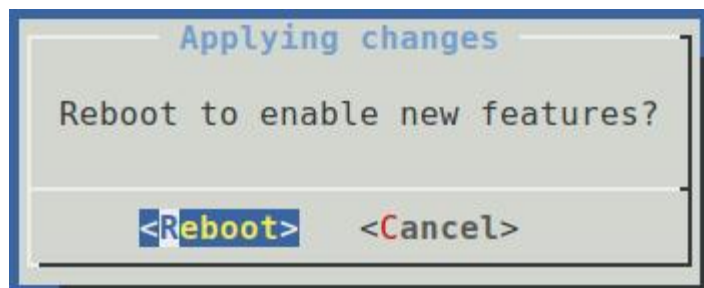
- e. Then select **<Save>** to save.



- f. Then Select **<Back>**



- g. Then select **<Reboot>** to restart the system and make the configuration take effect.



- 4) After entering the Linux system, first check if a UART device node exists under **/dev**.

```
orange@orange:~$ ls /dev/ttyS*
```



```
/dev/ttyS0 /dev/ttyS1 /dev/ttyS2 /dev/ttyS6 /dev/ttyS7
```

5) Then start testing the UART interface. First, use DuPont wires to short-circuit the RX and TX pins of the UART interface to be tested.

6) Use the **gpio serial** command to test the loopback function of the serial port as shown below. If you see the following printout, it means the serial communication is normal.

Note that in the command `gpio serial /dev/ttySx`, the last 'x' needs to be replaced with the serial number of the corresponding UART device node.

```
orange@orange:~$ gpio serial /dev/ttySx
```

```
Out: 0: -> 0
Out: 1: -> 1
Out: 2: -> 2
Out: 3: -> 3^C
```

3. 16. 6. Methods for testing PWM using /sys/class/pwm/

1) As shown in the table below, the Orange Pi Zero 3w has 7 available PWM channels: pwm0_0, pwm0_1, pwm0_2, pwm0_3, pwm0_4, pwm0_8, and pwm0_9.

复用功能	复用功能	复用功能	GPIO	GPIO序号	引脚序号	引脚序号	GPIO序号	GPIO	复用功能	复用功能
			3.3V	1	2			5V		
		TWI0-SDA	PE3	35	3	4		5V		
		TWI0-SCK	PE2	34	5	6		GND		
		PWM0-0	PE4	36	7	8	41	PE9	UART0-TX	
			GND		9	10	42	PE10	UART0-RX	
		UART2-TX	PE0	32	11	12	37	PE5		
		UART2-RX	PE1	33	13	14		GND		
			PE11	139	15	16	354	PL2	UART7-TX	
			3.3V		17	18	355	PL3	UART7-RX	
			PE2	130	19	20		GND		
	TWI2-SDA	SPI3-MOSI	PE3	131	21	22	96	PD0		
	TWI3-SCK	SPI3-MISO	PE1	129	23	24	128	PE0	SPI3-CS0	UART6-TX
UART6-RX	TWI2-SCK	SPI3-CLK	GND		25	26	132	PE4	SPI3-CS1	TWI3-SDA
			PL5	357	27	28	356	PL4		
			PE12	140	29	30		GND		
			PE13	141	31	32	97	PD1	PWM0-1	
		PWM0-3	PD3	99	33	34		GND		
		PWM0-8	PE6	38	35	36	98	PD2	PWM0-2	
		PWM0-4	PD4	100	37	38	40	PE8	TWI1-SDA	
			GND		39	40	39	PE7	TWI1-SCL	PWM0-9

2) The pins corresponding to PWM in the 40-pin configuration are shown in the table below.

PWM bus	Corresponding to 40 pins	dtbo corresponding configuration
PWM0-0	Pin 7	pwm0
PWM0-1	Pin 32	pwm1
PWM0-2	Pin 36	pwm2
PWM0-3	Pin 33	pwm3



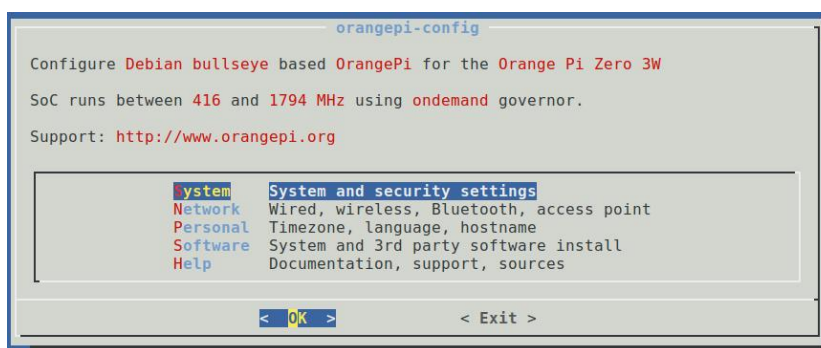
PWM0-4	Pin 37	pwm4
PWM0-8	Pin 35	pwm8
PWM0-9	Pin 40	pwm9

3) In Linux systems, the PWM in the 40-pin connector is disabled by default and needs to be manually enabled to be used. Detailed steps are as follows:

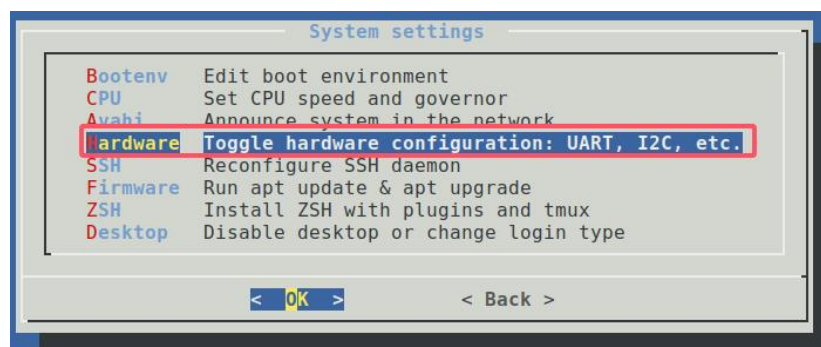
- a. First, run **orange-pi-config**. Regular users should remember to grant **sudo** privileges.

```
orange-pi@orange-pi:~$ sudo orange-pi-config
```

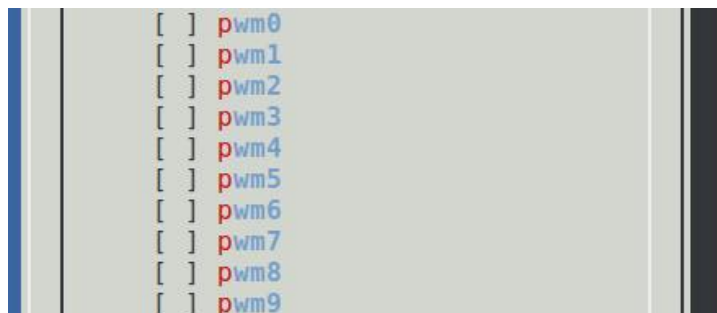
- b. Then select **System**



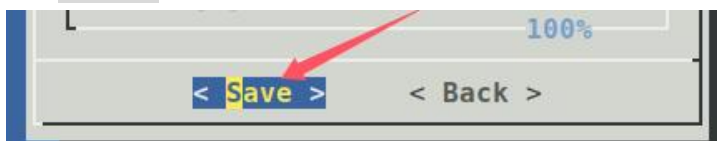
- c. Then select **Hardware**



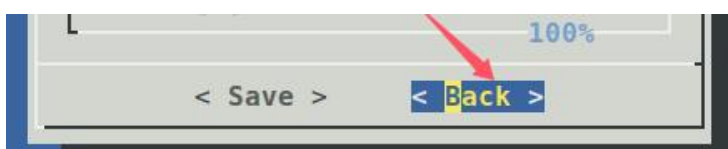
- d. Then use the arrow keys on your keyboard to navigate to the location shown in the image below, and then use the spacebar to select the open PWM configuration.



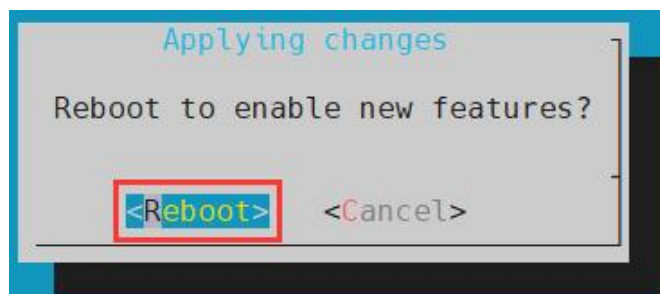
- e. Then select **<Save>** to save.



- f. Then select **<Back>**



- g. Then select **<Reboot>** to restart the system and make the configuration take effect.



- 4) After restarting, you can start the PWM test.

The following commands should be executed as the root user.

- a. Entering the following command in the command line will make pwm0 output a 50Hz square wave.

```
root@orangepi:~# echo 0 > /sys/class/pwm/pwmchip0/export
root@orangepi:~# echo 20000000 > /sys/class/pwm/pwmchip0/pwm0/period
root@orangepi:~# echo 1000000 > /sys/class/pwm/pwmchip0/pwm0/duty_cycle
root@orangepi:~# echo 1 > /sys/class/pwm/pwmchip0/pwm0/enable
```

- b. Entering the following command in the command line will make the PWM1 output a 50Hz square wave.



```
root@orangepi:~# echo 1 > /sys/class/pwm/pwmchip0/export
root@orangepi:~# echo 20000000 > /sys/class/pwm/pwmchip0/pwm1/period
root@orangepi:~# echo 1000000 > /sys/class/pwm/pwmchip0/pwm1/duty_cycle
root@orangepi:~# echo 1 > /sys/class/pwm/pwmchip0/pwm1/enable
```

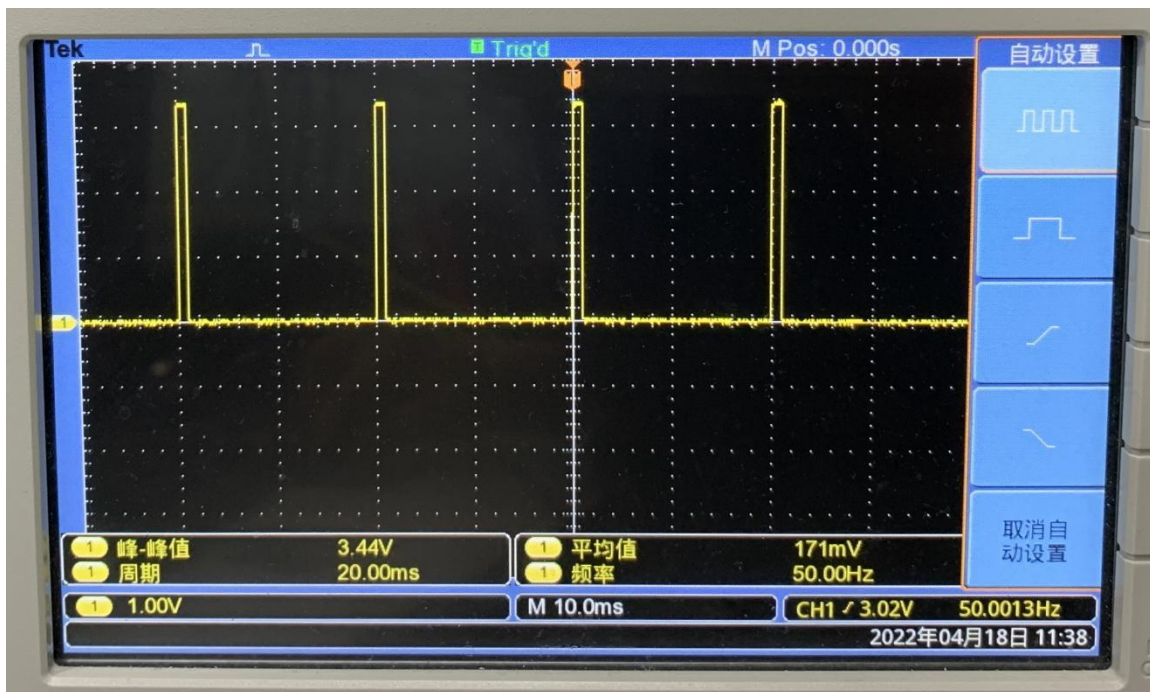
- c. Entering the following command in the command line will make the PWM2 output a 50Hz square wave.

```
root@orangepi:~# echo 2 > /sys/class/pwm/pwmchip0/export
root@orangepi:~# echo 20000000 > /sys/class/pwm/pwmchip0/pwm2/period
root@orangepi:~# echo 1000000 > /sys/class/pwm/pwmchip0/pwm2/duty_cycle
root@orangepi:~# echo 1 > /sys/class/pwm/pwmchip0/pwm2/enable
```

- d. Entering the following command in the command line will make the PWM3 output a 50Hz square wave.

```
root@orangepi:~# echo 3 > /sys/class/pwm/pwmchip0/export
root@orangepi:~# echo 20000000 > /sys/class/pwm/pwmchip0/pwm3/period
root@orangepi:~# echo 1000000 > /sys/class/pwm/pwmchip0/pwm3/duty_cycle
root@orangepi:~# echo 1 > /sys/class/pwm/pwmchip0/pwm3/enable
```

- e. Other PWM testing methods are similar and will not be elaborated here.





3. 17. How to install and use wiringOP-Python

wiringOP-Python is a Python version of the **wiringOP** library, used to manipulate hardware resources such as **GPIO**, **I2C**, **SPI**, and **UART** on the development board within Python programs.

Also note that all the commands below are performed under the **root user.**

3. 17. 1. Installation method of wiringOP-Python

1) First, install the dependency packages.

```
root@orangepi:~# sudo apt-get update
root@orangepi:~# sudo apt-get -y install git swig python3-dev python3-setuptools
```

2) Then use the following command to download the source code of wiringOP-Python.

Note that the `git clone --recursive` command below will automatically download the wiringOP source code, as wiringOP-Python depends on wiringOP. Please ensure that the download process does not encounter errors due to network issues.

If you encounter problems downloading the code from GitHub, you can directly use the wiringOP-Python source code that comes with the Linux image, located at: **/usr/src/wiringOP-Python.**

```
root@orangepi:~# git clone --recursive https://github.com/orangepi-xunlong/wiringOP-Python -b next
root@orangepi:~# cd wiringOP-Python
root@orangepi:~/wiringOP-Python# git submodule update --init --remote
```

3) Then use the following command to compile wiringOP-Python and install it onto the Linux system of the development board.

```
root@orangepi:~# cd wiringOP-Python
root@orangepi:~/wiringOP-Python# python3 generate-bindings.py > bindings.i
root@orangepi:~/wiringOP-Python# sudo python3 setup.py install
```

4) Then enter the following command. If helpful information is output, it means wiringOP-Python is installed successfully. Press the **q** key to exit the help information interface.

```
root@orangepi:~/wiringOP-Python# python3 -c "import wiringpi; help(wiringpi)"
```




Help on module wiringpi:

NAME

wiringpi

DESCRIPTION

```
# This file was automatically generated by SWIG (http://www.swig.org).
# Version 4.0.2
#
# Do not make changes to this file unless you know what you are doing--modify
# the SWIG interface file instead.
```

5) The steps to test whether wiringOP-Python is installed successfully in the Python command line are as follows:

- a. First, use the python3 command to enter the Python 3 command-line mode.

```
root@orangePi:~# python3
```

- b. Then import the wiringpi Python module.

```
>>> import wiringpi;
```

- c. Finally, enter the following command to view the help information for wiringOP-Python. Press the **q** key to exit the help information interface.

```
>>> help(wiringpi)
```

Help on module wiringpi:

NAME

wiringpi

DESCRIPTION

```
# This file was automatically generated by SWIG (http://www.swig.org).
# Version 4.0.2
#
# Do not make changes to this file unless you know what you are doing--modify
# the SWIG interface file instead.
```

CLASSES

builtins.object



```

GPIO
I2C
Serial
nes

class GPIO(builtins.object)
|   GPIO(pinmode=0)
|
>>>

```

3. 17. 2. 40-pin GPIO port test

Like wiringOP, wiringOP-Python can also determine which GPIO pin to operate by specifying the wPi number. Since wiringOP-Python does not have a command to view the wPi number, the correspondence between the board's wPi number and the physical pin can only be viewed through the gpio command in wiringOP.

```
orange@orangezero3w:~$ gpio readall
```

ZER03W											
GPIO	wPi	Name	Mode	V	Physical	V	Mode	Name	wPi	GPIO	
		3.3V			1	2		5V			
35	0	SDA.0	OFF	0	3	4		5V			
34	1	SCL.0	OFF	0	5	6		GND			
36	2	PWM0-0	OFF	0	7	8	0	ALT2	TXD.0	3	41
		GND			9	10	0	ALT2	RXD.0	4	42
32	5	PB0	ALT14	1	11	12	0	OFF	PB5	6	37
33	7	PB1	ALT14	1	13	14		GND			
137	8	PE9	OFF	0	15	16	0	OFF	PL2	9	354
		3.3V			17	18	0	OFF	PL3	10	355
130	11	MOSI.3	OFF	0	19	20		GND			
131	12	MISO.3	OFF	0	21	22	0	OFF	PD0	13	96
129	14	SCLK.3	OFF	0	23	24	0	OFF	CE.0	15	128
		GND			25	26	0	OFF	PE4	16	132
357	17	PL5	OFF	0	27	28	0	OFF	PL4	18	356
140	19	PE12	OFF	0	29	30		GND			
141	20	PE13	OFF	0	31	32	0	OFF	PD1	21	97
99	22	PD3	OFF	0	33	34		GND			
38	23	PB6	OFF	0	35	36	0	OFF	PD2	24	98
100	25	PD4	OFF	0	37	38	0	OFF	PB8	26	40
		GND			39	40	0	OFF	PB7	27	39
GPIO	wPi	Name	Mode	V	Physical	V	Mode	Name	wPi	GPIO	
ZER03W											

1) The following example uses pin 7—corresponding to GPIO PB4 and wPi number 2—to demonstrate how to set the high and low levels of a GPIO port.



```
orange_pi@orange_pi3w:~$ gpio readall
```

ZER03W											
GPIO	wPi	Name	Mode	V	Physical	V	Mode	Name	wPi	GPIO	
		3.3V			1	2		5V			
35	0	SDA.0	OFF	0	3	4		5V			
34	1	SCL.0	OFF	0	5	6		GND			
36	2	PWM0-0	OFF	0	7	8	0	ALT2	TXD.0	3	41
		GND			9	10	0	ALT2	RXD.0	4	42

2) The steps for testing directly using commands are as follows:

- First, set the GPIO port to output mode. The first parameter of the **pinMode** function is the wPi serial number corresponding to the pin, and the second parameter is the GPIO mode.

```
root@orange_pi:~/wiringOP-Python# python3 -c "import wiringpi; \
from wiringpi import GPIO; wiringpi.wiringPiSetup() ; \
wiringpi.pinMode(2, GPIO.OUTPUT) ; "
```

- Then set the GPIO port output to low level. After setting, you can use a multimeter to measure the voltage value of the pin. If it is 0V, it means that the low level setting was successful.

```
root@orange_pi:~/wiringOP-Python# python3 -c "import wiringpi; \
from wiringpi import GPIO; wiringpi.wiringPiSetup() ; \
wiringpi.digitalWrite(2, GPIO.LOW)"
```

- Then set the GPIO port to output a high level. After setting it, you can use a multimeter to measure the voltage value of the pin. If it is 3.3V, it means that the high level setting was successful.

```
root@orange_pi:~/wiringOP-Python# python3 -c "import wiringpi; \
from wiringpi import GPIO; wiringpi.wiringPiSetup() ; \
wiringpi.digitalWrite(2, GPIO.HIGH)"
```

3) The steps for testing in the Python 3 command line are as follows:

- First, use the python3 command to enter the Python 3 command-line mode.

```
root@orange_pi:~# python3
```

- Then import the wiringpi Python module.

```
>>> import wiringpi
>>> from wiringpi import GPIO
```

- Then set the GPIO port to output mode. The first parameter of the **pinMode** function is the wPi number corresponding to the pin, and the second parameter is the GPIO mode.



```
>>> wiringpi.wiringPiSetup()
0
>>> wiringpi.pinMode(2, GPIO.OUTPUT)
```

- d. Then set the GPIO port output to low level. After setting, you can use a multimeter to measure the voltage value of the pin. If it is 0V, it means that the low level setting was successful.

```
>>> wiringpi.digitalWrite(2, GPIO.LOW)
```

- e. Then set the GPIO port output to high level. After setting it, you can use a multimeter to measure the voltage value of the pin. If it is 3.3V, it means that the high level setting was successful.

```
>>> wiringpi.digitalWrite(2, GPIO.HIGH)
```

4) For setting GPIO high and low levels in Python code using wiringOP-Python, please refer to the **blink.py** test program in the examples. The **blink.py** test program will continuously change the voltage of all 40 GPIO pins on the development board.

```
root@orangeypi:~/wiringOP-Python# cd examples
root@orangeypi:~/wiringOP-Python/examples# ls blink.py
blink.py
root@orangeypi:~/wiringOP-Python/examples# python3 blink.py
```

3. 17. 3. 40-pin SPI test

1) As shown in the figure below, the SPI available for Orange Pi Zero 3w is SPI3.

复用功能	复用功能	复用功能	GPIO	GPIO序号	引脚序号	引脚序号	GPIO序号	GPIO	复用功能	复用功能
			3.3V		1	2		5V		
		TWI0-SDA	PE3	35	3	4		5V		
		TWI0-SCK	PE2	34	5	6		GND		
		PWM0-0	PE4	36	7	8	41	PE9	UART0-TX	
			GND		9	10	42	PE10	UART0-RX	
		UART2-TX	PE0	32	11	12	37	PE5		
		UART2-RX	PE1	33	13	14		GND		
			PE11	139	15	16	354	PL2	UART7-TX	
			3.3V		17	18	355	PL3	UART7-RX	
		TWI2-SDA	PE2	130	19	20		GND		
		TWI3-SCK	PE3	131	21	22	96	PD0		
		TWI2-SCK	PE1	129	23	24	128	PE0	SPI3-CS0	UART6-TX
UART6-RX		SPI3-MOSI	GND		25	26	132	PE4	SPI3-CS1	TWI3-SDA
		SPI3-MISO	PL5	357	27	28	356	PL4		
		SPI3-CLK	PE12	140	29	30		GND		
			PE13	141	31	32	97	PD1	PWM0-1	
			PD3	99	33	34		GND		
		PWM0-3	PE6	38	35	36	98	PD2	PWM0-2	
		PWM0-8	PD4	100	37	38	40	PE8	TWI1-SDA	
		PWM0-4	GND		39	40	39	PE7	TWI1-SCL	PWM0-9

2) The pins corresponding to SPI3 at pin 40 are shown in the table below.

	SPI3 corresponds to 40 pins
MOSI	Pin 19
MISO	Pin 21
CLK	Pin 23
CS0	Pin 24



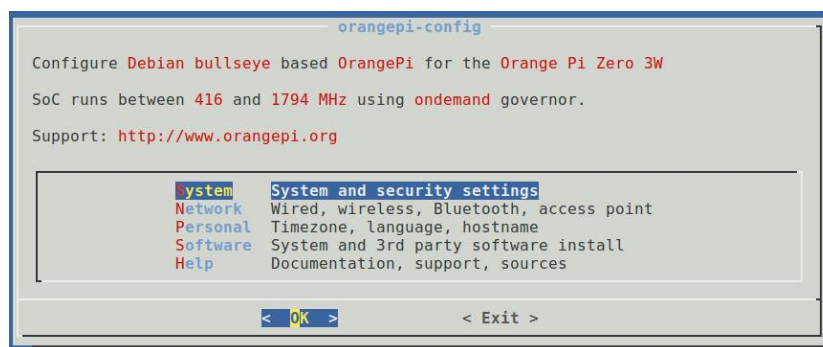
CS1	Pin 26
-----	--------

3) In Linux systems, the SPI interface on the 40-pin connector is disabled by default and needs to be manually enabled to function. Detailed steps are as follows:

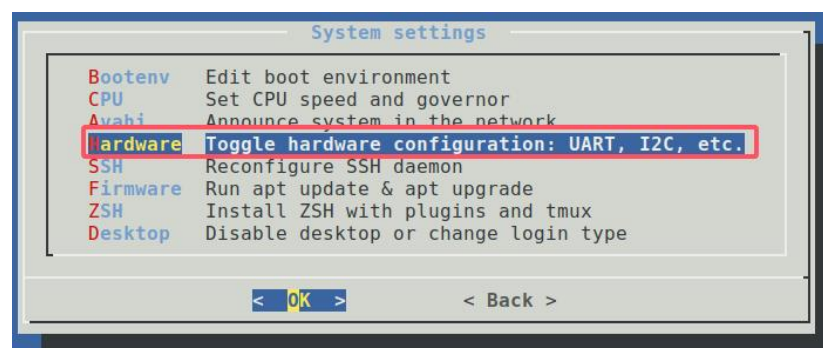
- a. First, run **orangepi-config**. Regular users should remember to grant **sudo** privileges.

```
orangepi@orangepi:~$ sudo orangepi-config
```

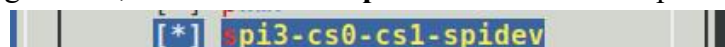
- b. Then select **System**



- c. Then select **Hardware**



- d. Then use the arrow keys on your keyboard to navigate to the location shown in the image below, and then use the **spacebar** to select the open SPI configuration.



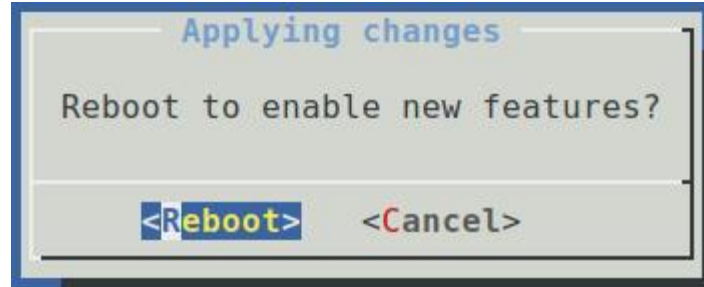
- e. Then select **<Save>** to save.



- f. Then select **<Back>**



- g. Then select **<Reboot>** to restart the system and make the configuration take effect.



- 4) Next, check if the **spidevx.x** device node exists in the Linux system. If it does, it means the SPI configuration has taken effect.

```
orangeypi@orangeypi:~$ ls /dev/spidev*
/dev/spidev3.0
```

- 5) Then you can use the **spidev_test.py** program in the examples to test the SPI loopback function. The **spidev_test.py** program requires the following two parameters:

- channel:** Specify the SPI channel number
- port:** Specify the SPI port number

- 6) Without shorting the SPI's mosi and miso pins, the output of **spidev_test.py** is shown below. It can be seen that the TX and RX data are inconsistent.

The x after the --channel and --port parameters needs to be replaced with the specific SPI channel number and SPI port number.

```
root@orangeypi:~/wiringOP-Python# cd examples
root@orangeypi:~/wiringOP-Python/examples# python3 spidev_test.py \
--channel x --port x
spi mode: 0x0
max speed: 500000 Hz (500 KHz)
Opening device /dev/spidev1.1
TX | FF FF FF FF FF FF 40 00 00 00 00 95 FF FF FF FF FF FF FF FF FF FF FF
FF FF FF FF FF FF F0 0D |.....@.....|
RX | FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
```




```
FF FF FF FF FF FF FF FF FF |.....|
```

7) Then, shorting the txd and rxd pins of the SPI using DuPont wires and running spidev_test.py again, the output is as follows. You can see that the transmitted and received data are the same, indicating that the SPI loopback test is normal.

The x after the --channel and --port parameters needs to be replaced with the specific SPI channel number and SPI port number.

```
root@orangeypi:~/wiringOP-Python# cd examples
root@orangeypi:~/wiringOP-Python/examples# python3 spidev_test.py \
--channel x --port x
spi mode: 0x0
max speed: 500000 Hz (500 KHz)
Opening device /dev/spidev1.1
TX | FF FF FF FF FF FF FF 40 00 00 00 00 95 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
FF FF FF FF FF F0 0D |.....@.....|
RX | FF FF FF FF FF FF FF 40 00 00 00 00 95 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
FF FF FF FF FF F0 0D |.....@.....|
```

3. 17. 4. 40-pin I2C test

1) As shown in the table below, the Orange Pi Zero 3w supports four I2C bus types: i2c0, i2c1, i2c2, and i2c3.

复用功能	复用功能	复用功能	GPIO	GPIO序号	引脚序号	引脚序号	GPIO序号	GPIO	复用功能	复用功能
			3.3V		1	2		5V		
		TWI0-SDA	PE3	35	3	4		5V		
		TWI0-SCK	PE2	34	5	6		GND		
		PWM0-0	PE4	36	7	8	41	PE9	UART0-TX	
			GND		9	10	42	PE10	UART0-RX	
		UART2-TX	PE0	32	11	12	37	PE5		
		UART2-RX	PE1	33	13	14		GND		
			PE11	139	15	16	354	PL2	UART7-TX	
			3.3V		17	18	355	PL3	UART7-RX	
		TWI2-SDA	PE2	130	19	20		GND		
		TWI3-SCK	PE3	131	21	22	96	PD0		
		SPI3-MISO	PE1	129	23	24	128	PE0	SPI3-CS0	UART6-TX
UART6-RX	TWI2-SCK	SPI3-CLK			25	26	132	PE4	SPI3-CS1	TWI3-SDA
			GND		27	28	356	PL4		
			PL5	357	29	30		GND		
			PE12	140	31	32	97	PD1	PWM0-1	
			PE13	141	33	34		GND		
		PWM0-3	PD3	99	35	36	98	PD2	PWM0-2	
		PWM0-8	PE6	38	37	38	40	PE8	TWI1-SDA	
		PWM0-4	PD4	100	39	40	39	PE7	TWI1-SCL	PWM0-9

2) The pins corresponding to the three I2C bus groups in the 40-pin configuration are shown in the table below.

I2C bus	SDA corresponds to 40 pins	SCL corresponds to 40 pins	dtbo corresponding configuration
I2C0	Pin 3	Pin 5	i2c0
I2C1	Pin 38	Pin 40	i2c1
I2C2	Pin 19	Pin 23	i2c2



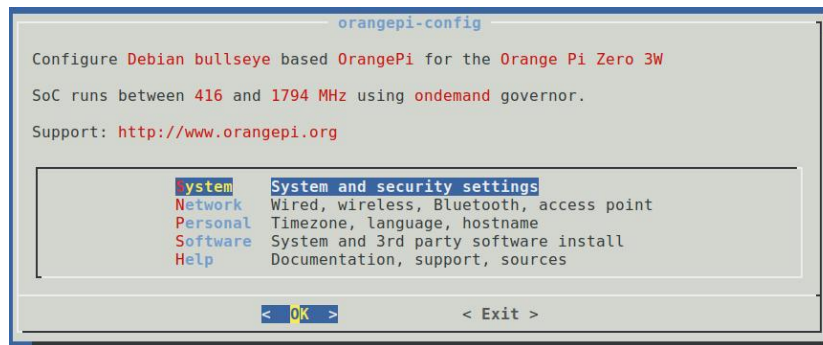
I2C3	Pin 26	Pin 21	i2c3
------	--------	--------	------

3) In Linux systems, the I2C pin 40 is disabled by default and needs to be manually enabled to function. Detailed steps are as follows:

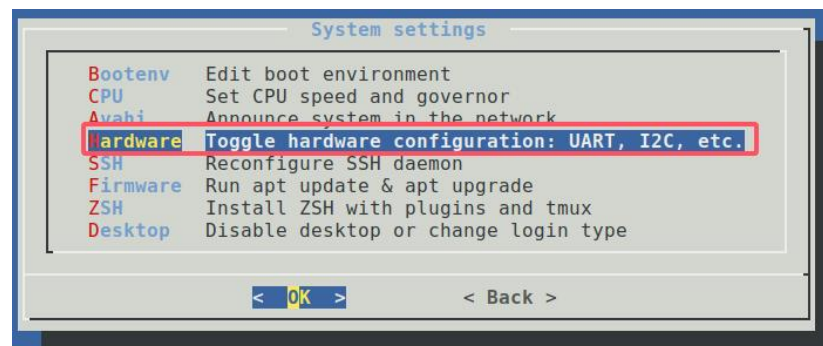
- a. First, run **orange-pi-config**. Regular users should remember to grant **sudo** privileges.

```
orange-pi@orange-pi:~$ sudo orange-pi-config
```

- b. Then select **System**



- c. Then select **Hardware**



- d. Then use the arrow keys on your keyboard to navigate to the location shown in the image below, and then use the **spacebar** to select the open I2C configuration.



- e. Then select **<Save>** to save.

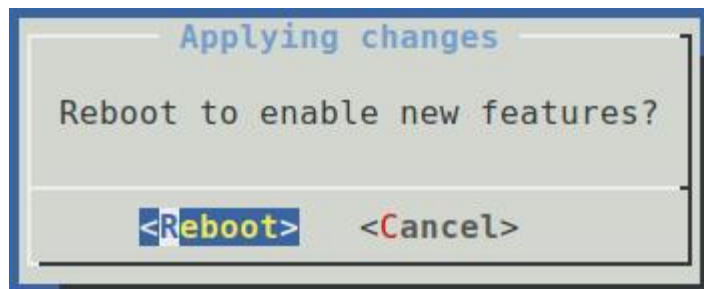




- f. Then select **<Back>**



- g. Then select **<Reboot>** to restart the system and make the configuration take effect.



- 4) After booting the Linux system, first confirm that there are I2C device nodes under /dev.

```
orangepi@orangepi:~$ ls /dev/i2c-*
/dev/i2c-0 /dev/i2c-1 /dev/i2c-13 /dev/i2c-15 /dev/i2c-2 /dev/i2c-20
/dev/i2c-3 /dev/i2c-5 /dev/i2c-7 /dev/i2c-8 /dev/i2c-9
```

- 5) Then start testing I2C. First, install i2c-tools.

```
orangepi@orangepi:~$ sudo apt-get update
orangepi@orangepi:~$ sudo apt-get install -y i2c-tools
```

- 6) Then connect an I2C device to the I2C pin of the 40-pin connector. Here, we take the DS1307 RTC module as an example.



- 7) Then use the **i2cdetect -y** command. If the address of the connected I2C device can be detected, it means that the I2C device is connected correctly.



```

orange@orange:~$ sudo i2cdetect -y -r 0      #i2c0 commands
orange@orange:~$ sudo i2cdetect -y -r 1      #i2c1 commands
orange@orange:~$ sudo i2cdetect -y -r 2      #i2c2 commands
orange@orange:~$ sudo i2cdetect -y -r 3      #i2c3 commands

```

8) Then you can run the **ds1307.py** test program in the **examples** to read the RTC time.

```

root@orange:~/wiringOP-Python# cd examples
root@orange:~/wiringOP-Python/examples# python3 ds1307.py --device \
"/dev/i2c-1"
Thu 2022-06-16 04:35:46
Thu 2022-06-16 04:35:47
Thu 2022-06-16 04:35:48
^C
exit

```

3. 17. 5. 40-pin UART test

1) As shown in the table below, the Orange Pi Zero 3w can use three UART buses: UART2, UART6, and UART7.

复用功能	复用功能	复用功能	GPIO	GPIO序号	引脚序号	引脚序号	GPIO序号	GPIO	复用功能	复用功能
			3.3V		1	2		5V		
		TWI0-SDA	PE3	35	3	4		5V		
		TWI0-SCK	PE2	34	5	6		GND		
		PWM0-0	PE4	36	7	8	41	PE9	UART0-TX	
			GND		9	10	42	PE10	UART0-RX	
		UART2-TX	PE0	32	11	12	37	PE5		
		UART2-RX	PE1	33	13	14		GND		
			PE11	139	15	16	354	PL2	UART7-TX	
			3.3V		17	18	355	PL3	UART7-RX	
								GND		
		TWI2-SDA	PE2	130	19	20				
		TWI3-SCK	PE3	131	21	22	96	PD0		
UART6-RX	TWI2-SCK	SPI3-CLK	PE1	129	23	24	128	PE0	SPI3-CS0	UART6-TX
			GND		25	26	132	PE4	SPI3-CS1	TWI3-SDA
			PL5	357	27	28	356	PL4		
			PE12	140	29	30		GND		
			PE13	141	31	32	97	PD1	PWM0-1	
								GND		
		PWM0-3	PD3	99	33	34				
		PWM0-8	PE6	38	35	36	98	PD2	PWM0-2	
		PWM0-4	PD4	100	37	38	40	PE8	TWI1-SDA	
			GND		39	40	39	PE7	TWI1-SCL	PWM0-9

2) The pins corresponding to the six UART bus groups in the 40-pin configuration are shown in the table below.

UART bus	RX corresponds to 40 pin	TX corresponds to 40 pins	dtbo corresponding configuration
UART2	Pin 13	Pin 11	uart2
UART6	Pin 23	Pin 24	uart6
UART7	Pin 18	Pin 16	uart7

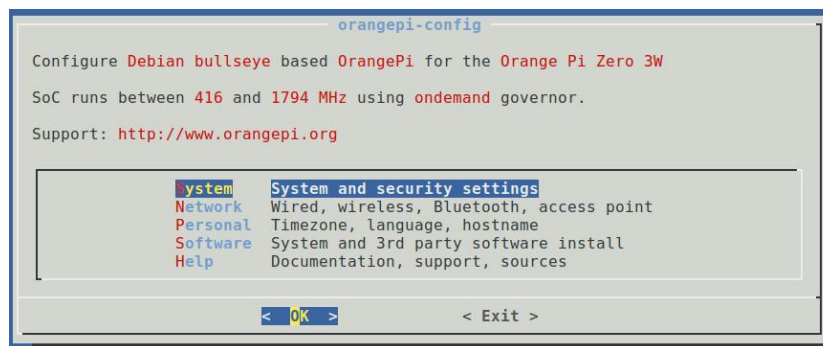
3) In Linux systems, the UART pin 40 is disabled by default and needs to be manually enabled to be used. Detailed steps are as follows:



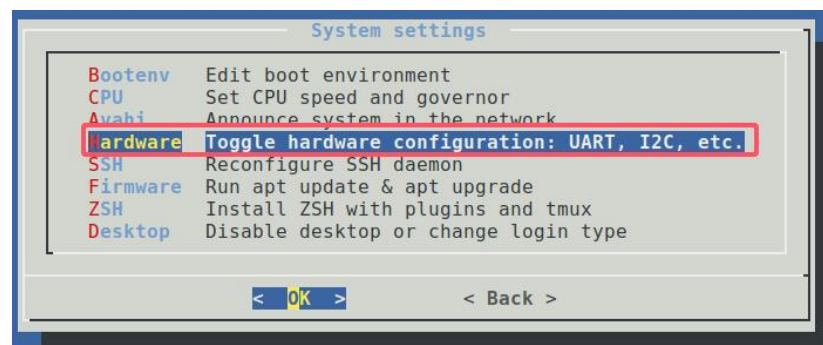
- a. First, run **orange-pi-config**. Regular users should remember to grant **sudo** privileges.

```
orange-pi@orange-pi:~$ sudo orange-pi-config
```

- b. Then select **System**



- c. Then select **Hardware**



- d. Then use the arrow keys on your keyboard to navigate to the location shown in the image below, and then use the **spacebar** to select the open UART configuration.



- e. Then select **<Save>** to save.

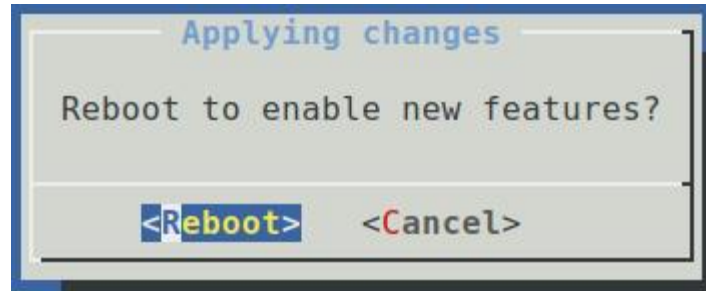


- f. Then select **<Back>**





- g. Then select **<Reboot>** to restart the system and make the configuration take effect.



- 4) After entering the Linux system, first check if a UART device node exists under **/dev**.

```
orangepi@orangepi:~$ ls /dev/ttyS*
/dev/ttyS0 /dev/ttyS1 /dev/ttyS2 /dev/ttyS6 /dev/ttyS7
```

- 5) Then, we started testing the UART interface. First, we used DuPont wires to short-circuit the RX and TX pins of the UART interface to be tested.

- 6) Finally, you can run the **serialTest.py** program in the examples to test the serial port loopback function. If you see the following printout, it means the serial port loopback test is normal.

Note that in the command `gpio serial /dev/ttySx`, the last 'x' needs to be replaced with the serial number of the corresponding UART device node.

```
root@orangepi:~/wiringOP-Python# cd examples
root@orangepi:~/wiringOP-Python/examples# python3 serialTest.py --device /dev/ttySx
Out: 0: -> 0
Out: 1: -> 1
Out: 2: -> 2
Out: 3: -> 3
Out: 4: ^C
exit
```

3. 18. Hardware Watchdog Test

The `watchdog_test` program comes pre-installed in the Linux system released with Orange Pi, which can be used for direct testing.



The method to run the watchdog_test program is as follows:

- The second parameter, 10, represents the watchdog's countdown time. If the watchdog is not fed within this time, the system will restart.
- We can feed the dog by pressing any key on the keyboard (except ESC). After feeding the dog, the program will print a line "keep alive" to indicate that feeding the dog was successful.

```
orange@orange:~$ sudo watchdog_test 10
open success
options is 33152,identity is sunxi-wdt
put_usr return,if 0,success:0
The old reset time is: 16
return ENOTTY,if -1,success:0
return ENOTTY,if -1,success:0
put_usr return,if 0,success:0
put_usr return,if 0,success:0
keep alive
keep alive
keep alive
```

3. 19. View the chipid of the A733 chip

The command to view the chip ID of the A733 chip is shown below. Each chip has a different chip ID, so you can use the chip ID to distinguish between multiple development boards.

```
orange@orange:~# cat /sys/class/sunxi_info/sys_info |grep sunxi_serial
sunxi_serial      : 147d208d0161172000007c0000000000
```

3. 20. Python Related Notes

3. 20. 1. Methods for compiling and installing Python from source code

If the Python version in the software repository of your Ubuntu or Debian system does not meet your development requirements and you want to use the latest version of Python, you can use the following method to download the Python source



package and compile and install the latest version of Python.

The following demonstration shows how to compile and install the latest version of Python 3.9. If you want to compile and install other versions of Python, the method is the same (you need to download the source code corresponding to the Python you want to install).

1) First, install the dependency packages required to compile Python.

```
orangePi@orangePi:~$ sudo apt-get update
orangePi@orangePi:~$ sudo apt-get install -y build-essential zlib1g-dev \
libncurses5-dev libgdbm-dev libnss3-dev libssl-dev libsqlite3-dev \
libreadline-dev libffi-dev curl libbz2-dev
```

2) Then download the latest version of Python 3.9 source code and extract it.

```
orangePi@orangePi:~$ wget \
https://www.python.org/ftp/python/3.9.10/Python-3.9.10.tgz
orangePi@orangePi:~$ tar xvf Python-3.9.10.tgz
```

3) Then run the configuration command.

```
orangePi@orangePi:~$ cd Python-3.9.10
orangePi@orangePi:~/Python-3.9.10$ ./configure --enable-optimizations
```

4) Then compile and install Python 3.9. The compilation process will take about half an hour.

```
orangePi@orangePi:~/Python-3.9.10$ make -j4
orangePi@orangePi:~/Python-3.9.10$ sudo make altinstall
```

5) After installation, you can use the following command to check the version number of the Python you just installed.

```
orangePi@orangePi:~/Python-3.9.10$ python3.9 --version
Python 3.9.10
```

6) Then update pip

```
orangePi@orangePi:~$ /usr/local/bin/python3.9 -m pip install --upgrade pip
```

3. 20. 2. Methods for changing the pip source in Python

Linux systems use pip by default to the official Python repository. However,



accessing the official Python repository from within China is very slow, and Python package installations often fail due to network issues. Therefore, when using pip to install Python libraries, please remember to change the pip repository.

1) First install **python3-pip**

```
orange@orange:~$ sudo apt-get update
orange@orange:~$ sudo apt-get install -y python3-pip
```

2) How to permanently change the pip source in Linux

- a. First, create a new directory named **~/.pip**, then add a **pip.conf** configuration file and set the pip source to Tsinghua University's mirror.

```
orange@orange:~$ mkdir -p ~/.pip
orange@orange:~$ cat <<EOF > ~/.pip/pip.conf
[global]
timeout = 6000
index-url = https://pypi.tuna.tsinghua.edu.cn/simple
trusted-host = pypi.tuna.tsinghua.edu.cn
EOF
```

- b. Then, using pip3 to install Python libraries will be much faster.

3) A method to temporarily change the pip source in Linux, where **<packagename>** needs to be replaced with the specific package name.

```
orange@orange:~$ pip3 install <packagename> -i \
https://pypi.tuna.tsinghua.edu.cn/simple --trusted-host pypi.tuna.tsinghua.edu.cn
```

3. 21. How to install Docker

The Linux image provided by Orange Pi comes pre-installed with Docker, but the Docker service is not enabled by default. You can enable the Docker service using the **enable_docker.sh** script, after which you can start using Docker commands, and the Docker service will automatically start the next time the system boots.

```
orange@orange:~$ enable_docker.sh
```

You can use the following command to test Docker. If you can run **hello-world**, it means that Docker is working properly.



```
orange@orange:~$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
256ab8fe8778: Pull complete
Digest:
sha256:7f0a9f93b4aa3022c3a4c147a449ef11e0941a1fd0bf4a8e6c9408b2600777c5
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.
.....
```

If you encounter a **permission denied** error when using the docker command, add the current user to the docker user group. This will allow you to run the docker command without needing to use sudo.

```
orange@orange:~$ sudo usermod -aG docker $USER
```

Note: You need to log out and log back into the system for the changes to take effect, or you can restart the system.

3. 22. How to install Home Assistant

Note that this only provides instructions on installing Home Assistant on Ubuntu or Debian systems. For detailed instructions on using Home Assistant, please refer to the official documentation or relevant books.

3. 22. 1. Installation via Docker

1) First, please install Docker and ensure that Docker is running normally. For Docker installation steps, please refer to the instructions in the [How to Install Docker](#) section.

2) Then you can search for the Home Assistant Docker image.

```
orange@orange:~$ docker search homeassistant
```

3) Then use the following command to download the Home Assistant Docker image to your local machine. The image size is approximately 1GB, so the download will take a



while. Please wait patiently for the download to complete.

```
orangepi@orangepi:~$ docker pull homeassistant/home-assistant
Using default tag: latest
latest: Pulling from homeassistant/home-assistant
be307f383ecc: Downloading
5fbc4c07ac88: Download complete
..... (Omitted output)
3cc6a1510c9f: Pull complete
7a4e4d5b979f: Pull complete
Digest:
sha256:81d381f5008c082a37da97d8b08dd8b358dae7ecf49e62ce3ef1eeafc4381bb
Status: Downloaded newer image for homeassistant/home-assistant:latest
docker.io/homeassistant/home-assistant:latest
```

4) Then you can use the following command to view the Home Assistant Docker image you just downloaded.

```
orangepi@orangepi:~$ docker images homeassistant/home-assistant
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
homeassistant/home-assistant	latest	bfa0ab9e1cf5	2 months ago	1.17GB

5) Now you can run the Home Assistant Docker container.

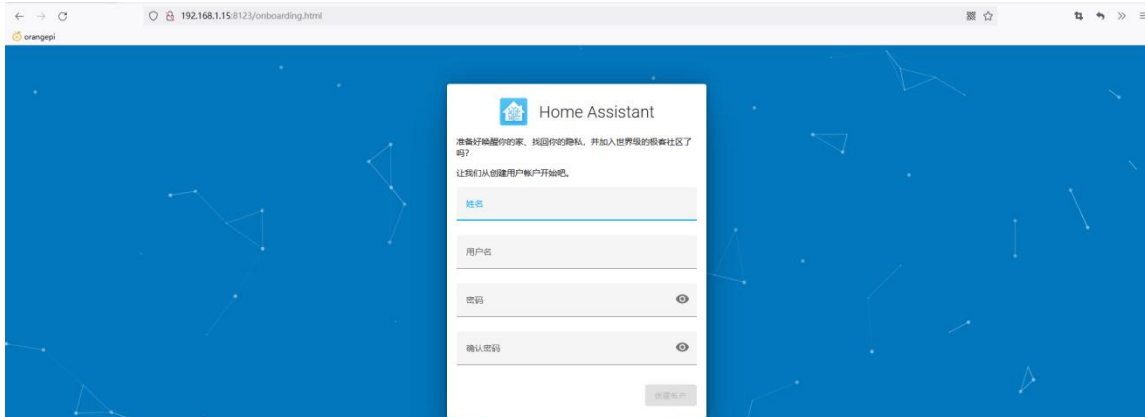
```
orangepi@orangepi:~$ docker run -d \
--name homeassistant \
--privileged \
--restart=unless-stopped \
-e TZ=Asia/Shanghai \
-v /home/orangepi/home-assistant:/config \
--network=host \
homeassistant/home-assistant:latest
```

6) Then, enter [the development board's IP address: 8123] in your browser to see the Home Assistant interface.

The Home Assistant container takes some time to start. If the interface below does not display correctly, please wait a few seconds and then refresh. If the interface still does not display correctly after more than one minute, it indicates a

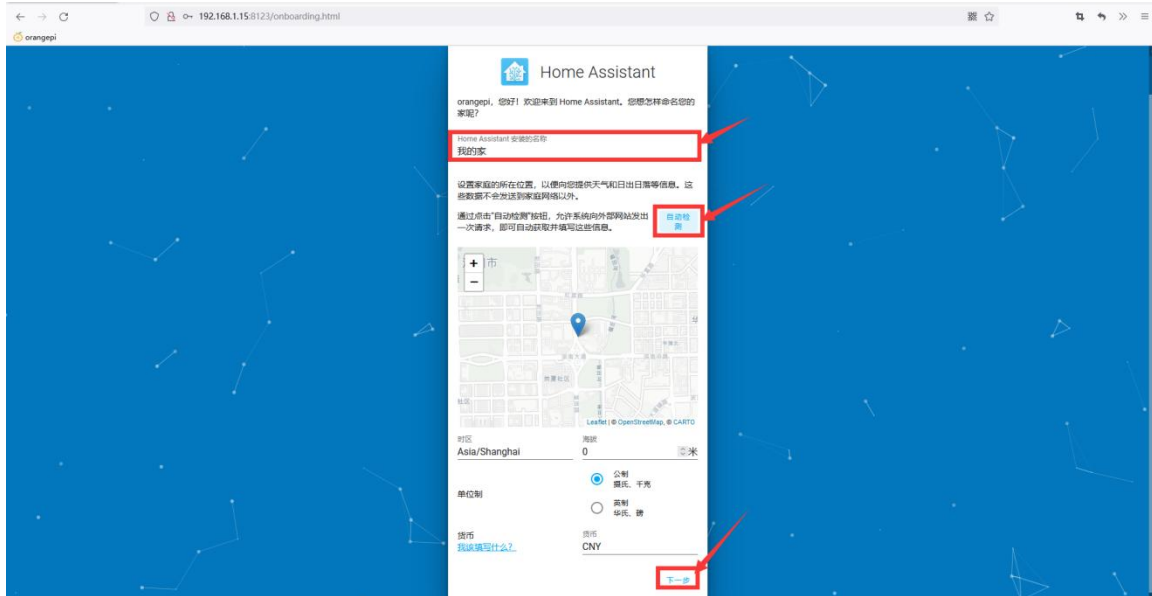


problem with the Home Assistant installation. In this case, you need to check if there were any issues with the previous installation and setup process.

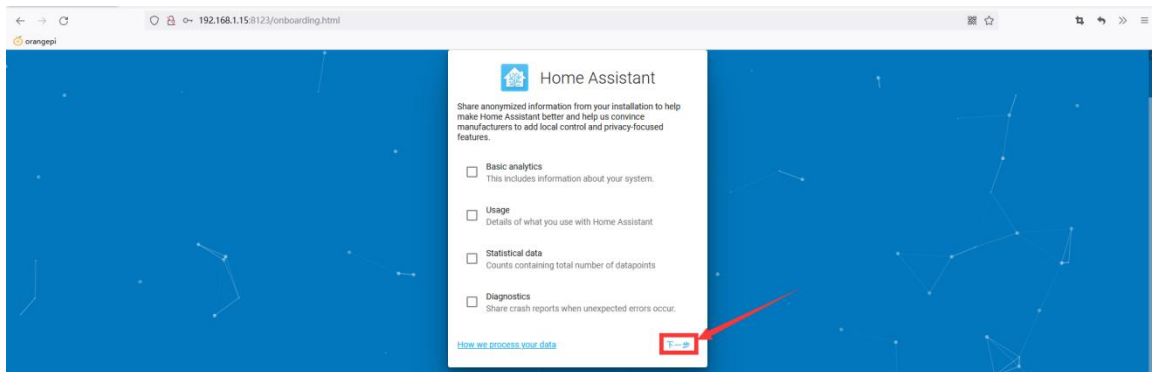


7) Then enter your **name**, **username**, and **password**, and click **Create Account**.

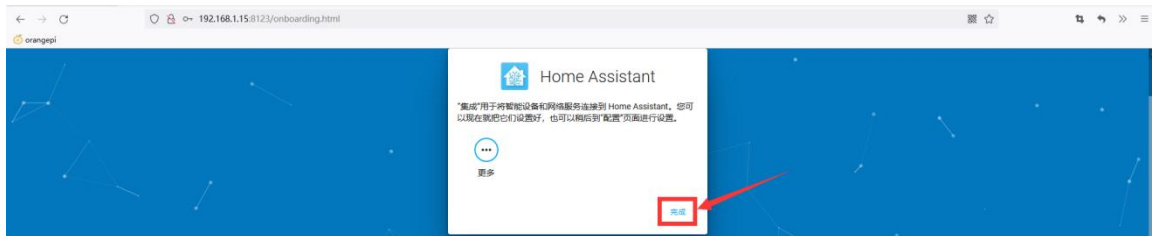
8) Then follow the on-screen instructions to set it according to your preferences, and then click Next.



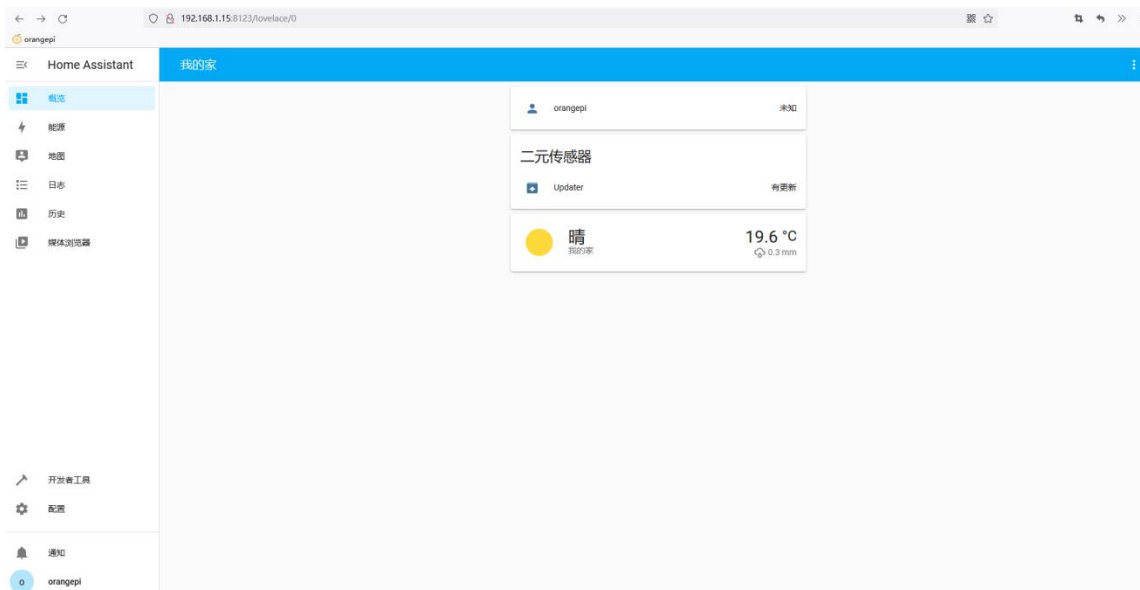
9) Then click Next.



10) Then click Finish.



11) The final main interface displayed by Home Assistant is shown in the following figure.



12) How to stop the Home Assistant container

- a. The command to view Docker containers is as follows:

```
orangePi@orangePi:~$ docker ps -a
```

- b. The command to stop the Home Assistant container is as follows:

```
orangePi@orangePi:~$ docker stop homeassistant
```

- c. The command to delete the Home Assistant container is as follows:

```
orangePi@orangePi:~$ docker rm homeassistant
```

3. 22. 2. Install via Python

Before installation, please change the pip source to a domestic source to speed up the installation of Python packages. For configuration instructions, please refer to the section on [changing the pip source in Python](#).

- 1) First, install the dependency packages.

```
orangePi@orangePi:~$ sudo apt-get update
orangePi@orangePi:~$ sudo apt-get install -y python3 python3-dev python3-venv \
python3-pip libffi-dev libssl-dev libjpeg-dev zlib1g-dev autoconf build-essential \
libopenjp2-7 libtiff5 libturbojpeg0-dev tzdata
```

If you are using Debian 12, please use the following command.:

```
orangePi@orangePi:~$ sudo apt-get update
```



```
orange@orange:~$ sudo apt-get install -y python3 python3-dev python3-venv \
python3-pip libffi-dev libssl-dev libjpeg-dev zlib1g-dev autoconf build-essential \
libopenjp2-7 libturbojpeg0-dev tzdata
```

2) Then you need to compile and install Python 3.9. Please refer to the section on [compiling and installing Python from source code for instructions](#).

The default Python version for Ubuntu Jammy is Python 3.10, so no compilation or installation is required.

Debian Bullseye comes with Python 3.9 by default, so no compilation or installation is required.

The default Python version for Debian Bookworm is Python 3.11, so there is no need to compile or install it.

3) Then create a Python virtual environment.

The Debian Bookworm uses Python 3.11; please remember to replace the corresponding commands.

```
orange@orange:~$ sudo mkdir /srv/homeassistant
orange@orange:~$ sudo chown orange:orange /srv/homeassistant
orange@orange:~$ cd /srv/homeassistant
orange@orange:/srv/homeassistant$ python3.9 -m venv .
orange@orange:/srv/homeassistant$ source bin/activate
(homeassistant) orange@orange:/srv/homeassistant$
```

4) Then install the necessary Python packages.

```
(homeassistant) orange@orange:/srv/homeassistant$ python3 -m pip install wheel
```

5) Then you can install Home Assistant Core.

```
(homeassistant) orange@orange:/srv/homeassistant$ pip3 install homeassistant
```

6) Then enter the following command to run Home Assistant Core.

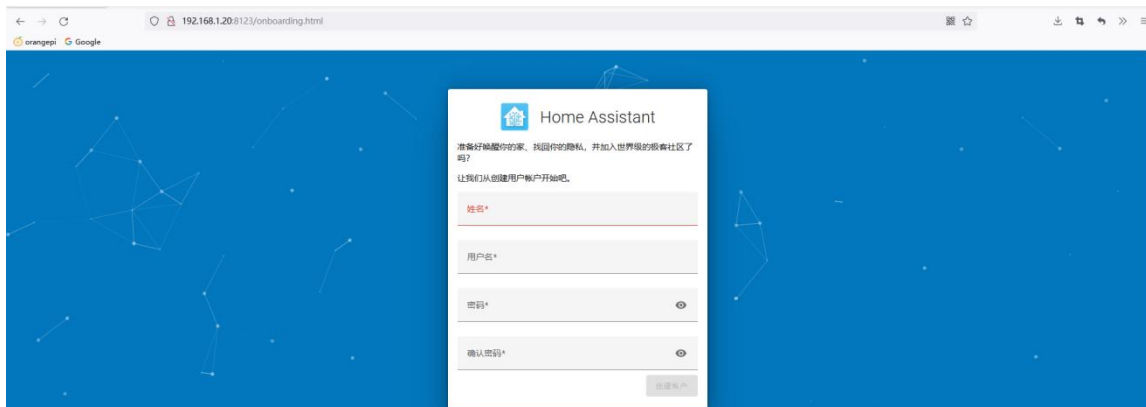
```
(homeassistant) orange@orange:/srv/homeassistant$ hass -v
```

7) Then enter the **development board's IP address: 8123** in your browser to see the Home Assistant interface.

The first time you run the hass command, it will download, install, and cache



some necessary libraries and dependencies. This process may take several minutes. Note that the Home Assistant interface will not be visible in your browser during this time; please wait a while and then refresh the page.



3. 23. How to install OpenCV

3. 23. 1. Install OpenCV using apt

1) The installation command is as follows:

```
orangePi@orangePi:~$ sudo apt-get update
orangePi@orangePi:~$ sudo apt-get install -y libopencv-dev python3-opencv
```

2) Then use the following command to print the OpenCV version number. If the output is normal, it means that OpenCV was installed successfully.

a. The OpenCV version in Ubuntu 22.04 is shown below:

```
orangePi@orangePi:~$ python3 -c "import cv2; print(cv2.__version__)"
4.5.4
```

b. The OpenCV version in Debian 11 is shown below:

```
orangePi@orangePi:~$ python3 -c "import cv2; print(cv2.__version__)"
4.5.1
```

c. The OpenCV version in Debian 12 is shown below:

```
orangePi@orangePi:~$ python3 -c "import cv2; print(cv2.__version__)"
4.6.0
```



3.24. Setting up a Chinese environment and installing a Chinese input method

Note that before installing the Chinese input method, please ensure that the Linux system used on the development board is a desktop version.

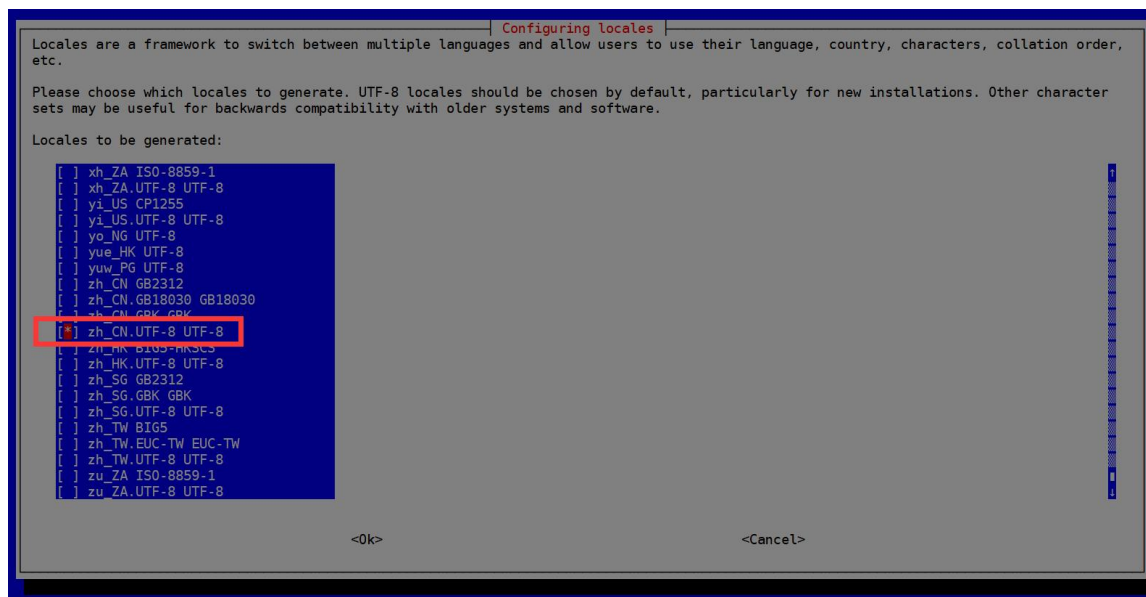
3.24.1. Debian system installation method

1) First, set the default **locale** to Chinese.

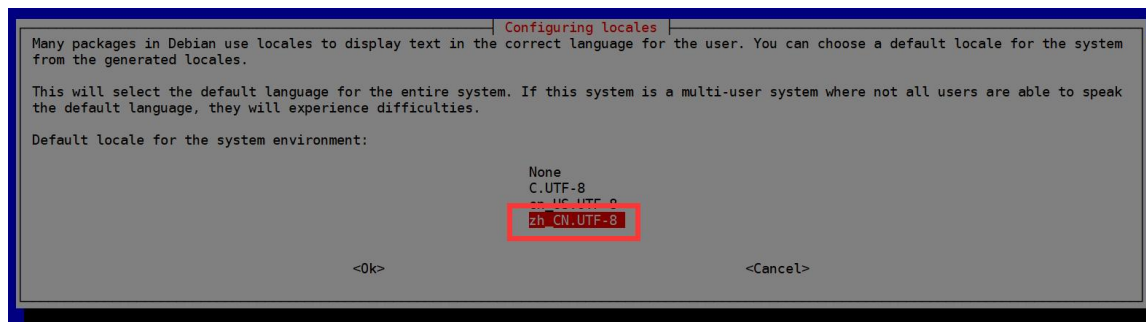
a. Enter the following command to begin configuring the **locale**.

```
orange@orange:~$ sudo dpkg-reconfigure locales
```

b. Then, in the pop-up window, select **zh_CN.UTF-8 UTF-8** (use the up and down arrow keys on your keyboard to move up and down, use the space bar to select, and finally use the Tab key to move the cursor to **<OK>**, then press Enter).



c. Then set the default **locale** to **zh_CN.UTF-8**



d. After exiting the interface, the **locale** settings will begin. The command line output will be as follows.



```
orange@orange:~$ sudo dpkg-reconfigure locales
```

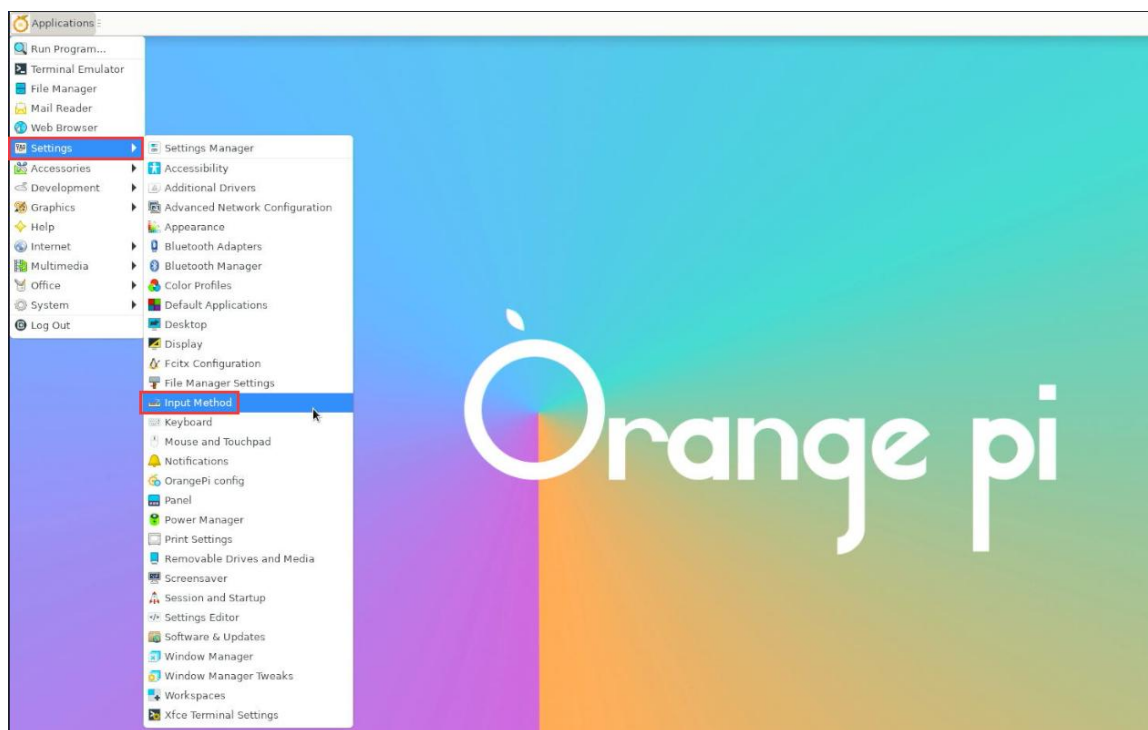
Generating locales (this might take a while)...

en_US.UTF-8... done

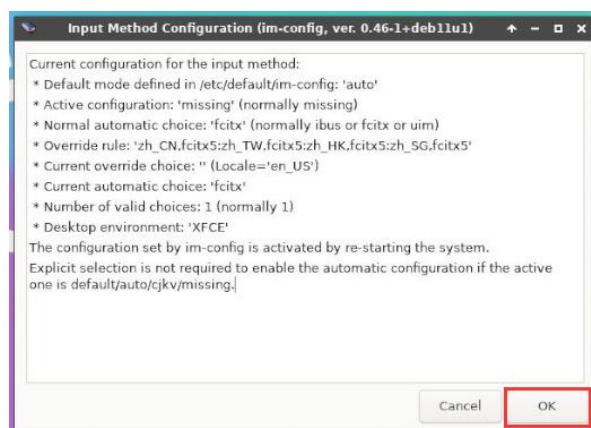
zh_CN.UTF-8... done

Generation complete.

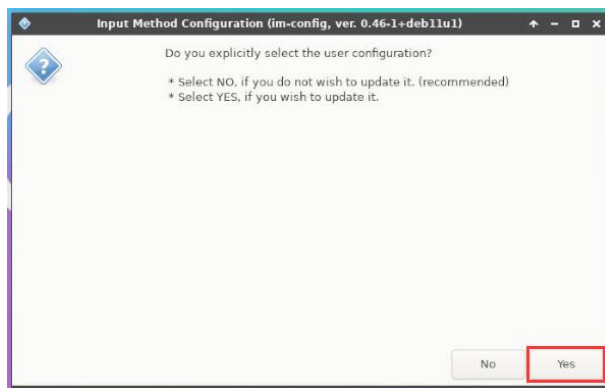
2) Then open **Input Method**



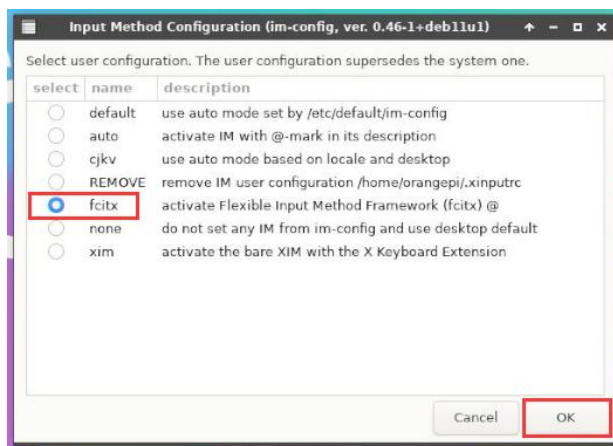
3) Then select **OK**



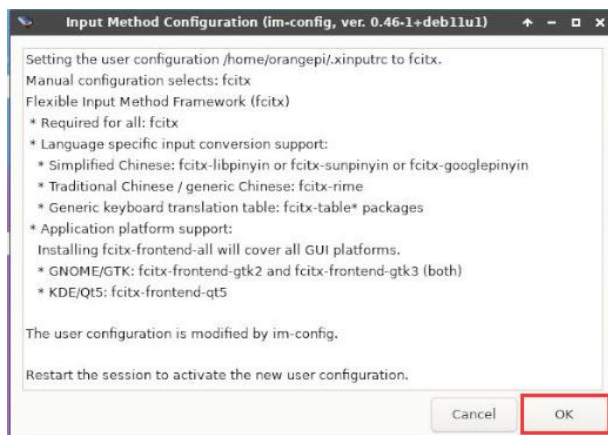
4) Then select **Yes**



5) Then select **fcitx**



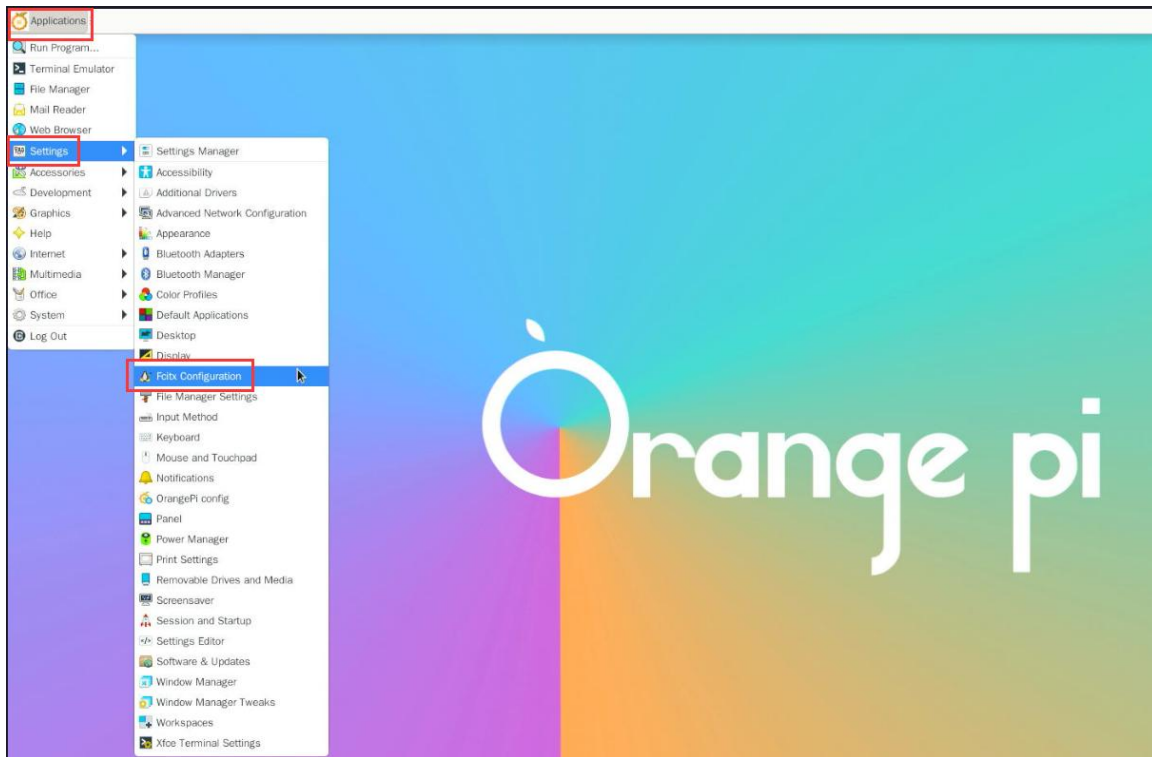
6) Then select **OK**



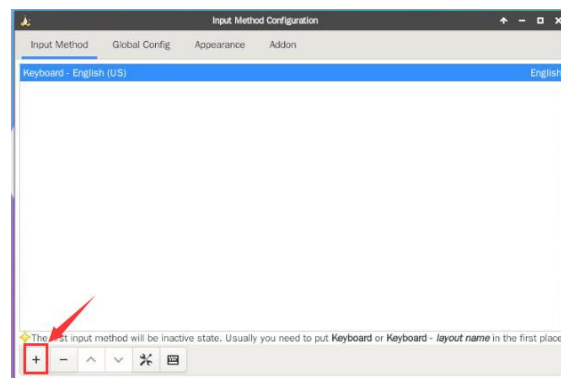
7) **Then you need to restart the Linux system for the configuration to take effect.**



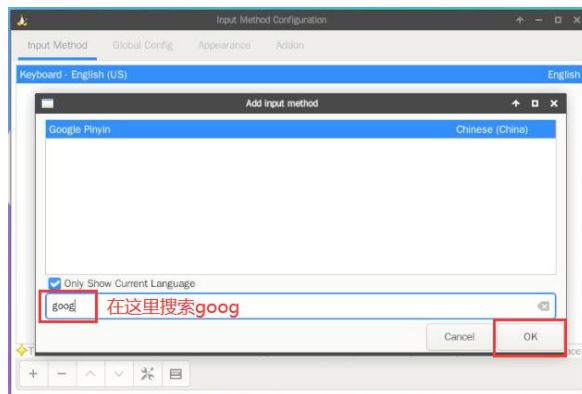
8) Then open **Fcitx configuration**



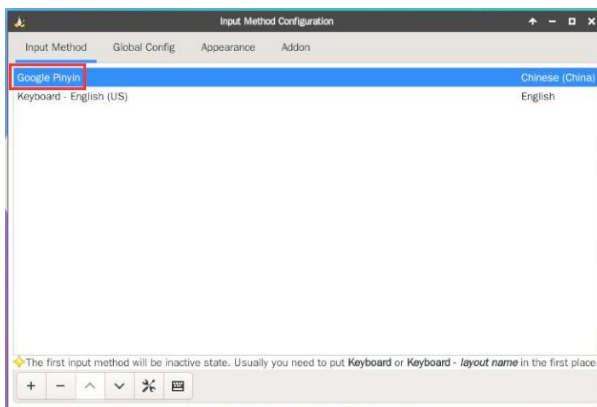
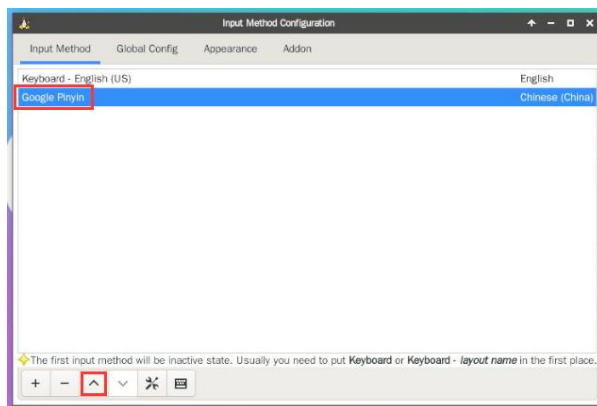
9) Then click the + sign as shown in the image below.



10) Then search **Google Pinyin** and click **OK**.



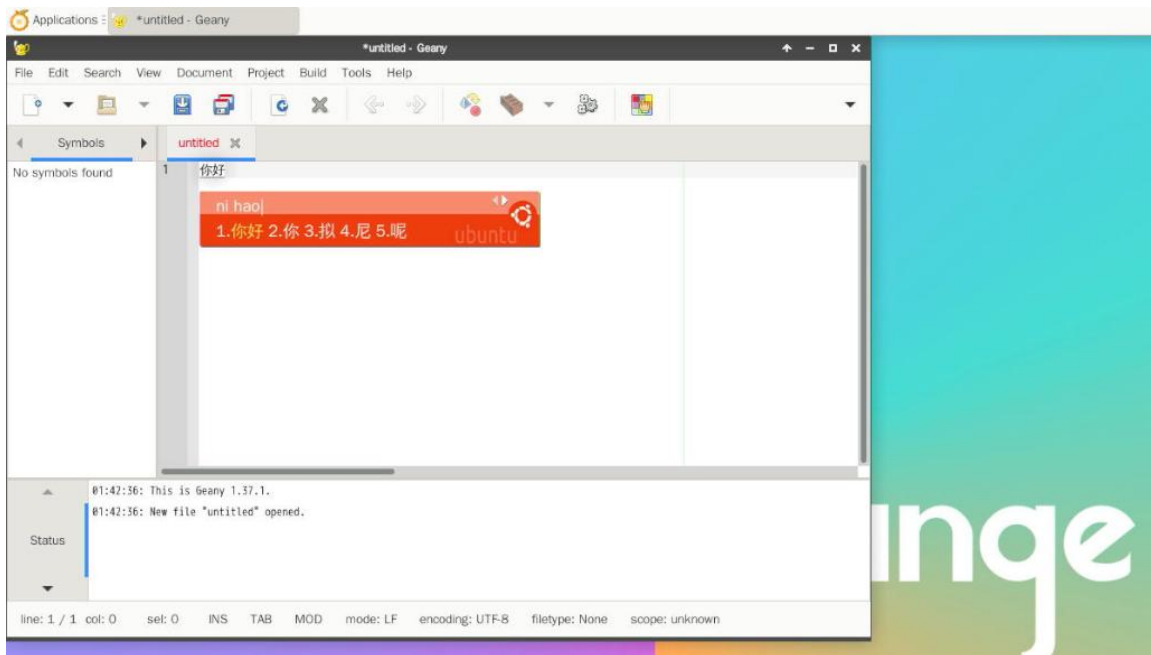
11) Then put **Google Pinyin** at the very beginning.



12) Then open the **Geany** editor to test the Chinese input method.



13) The Chinese input method test is shown below.



14) You can switch between Chinese and English input methods using the **Ctrl+Space** shortcut.

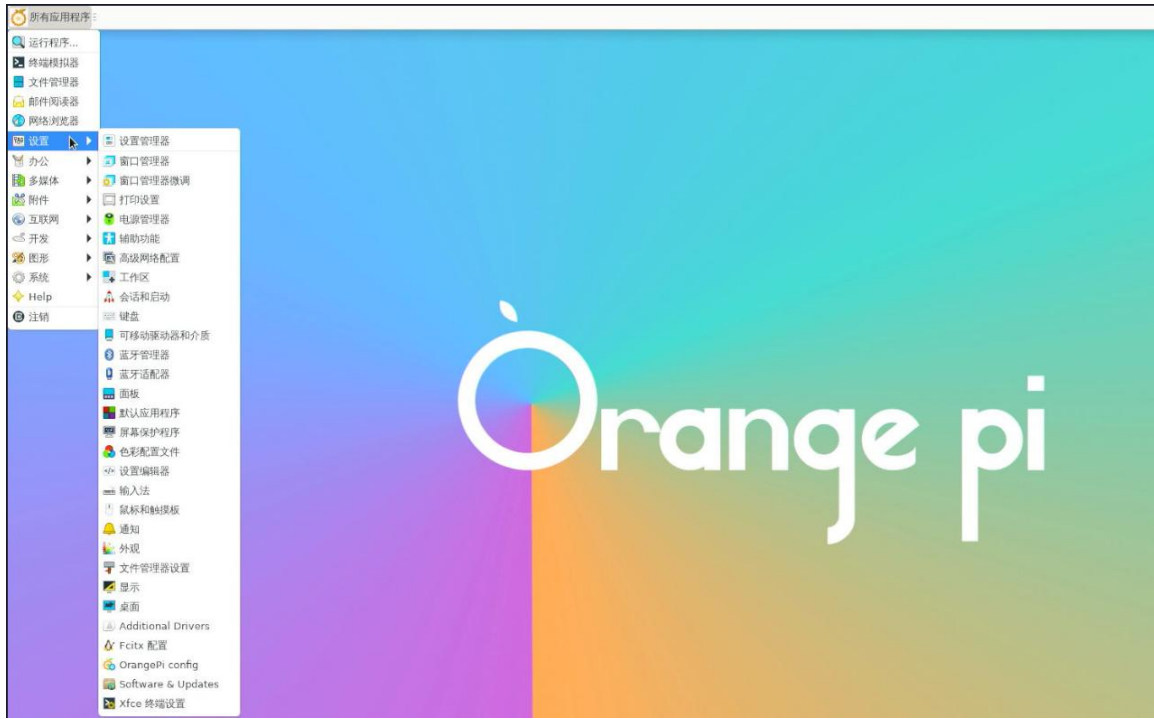
15) If you need the entire system to display in Chinese, you can set all variables in **/etc/default/locale** to **zh_CN.UTF-8**.

```
orange@orange:~$ sudo vim /etc/default/locale
# File generated by update-locale
LC_MESSAGES=zh_CN.UTF-8
LANG=zh_CN.UTF-8
```



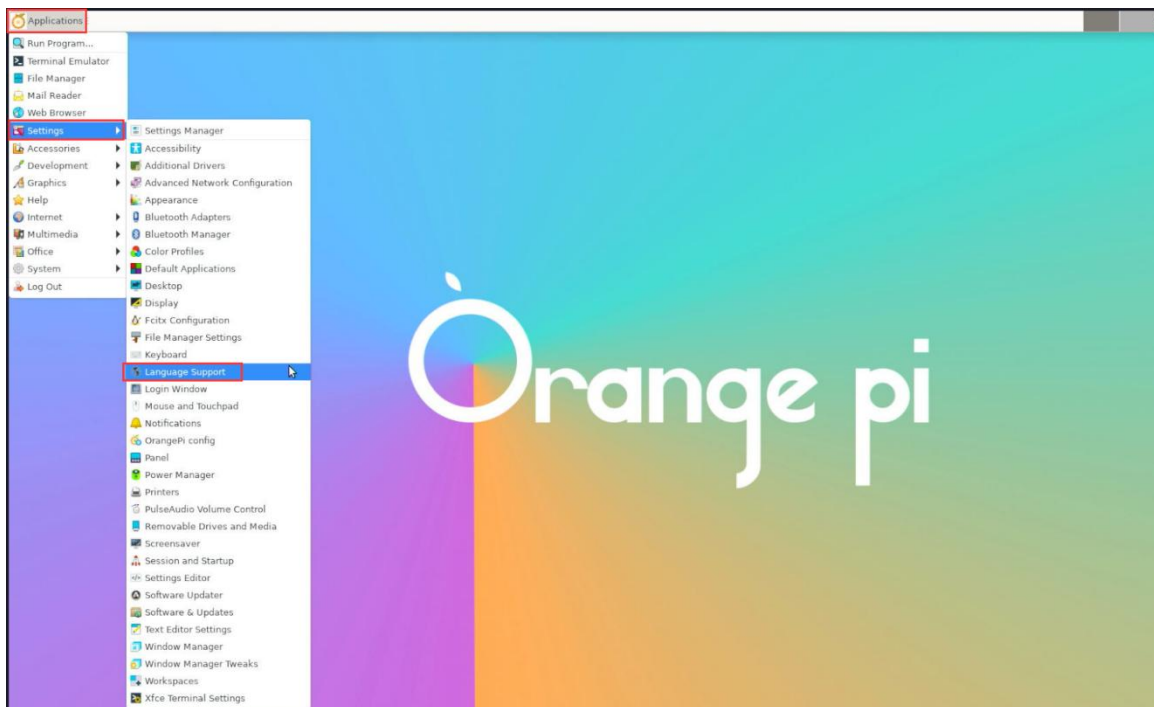

LANGUAGE=zh_CN.UTF-8

16) Then **restart the system** and you will see that the system is displayed in Chinese.



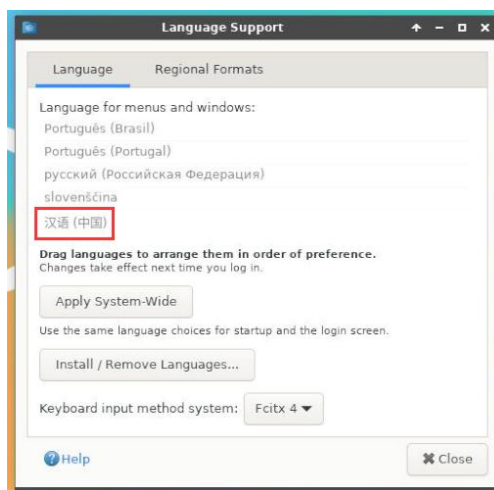
3. 24. 2. How to install Ubuntu 22.04

1) First open **Language Support**

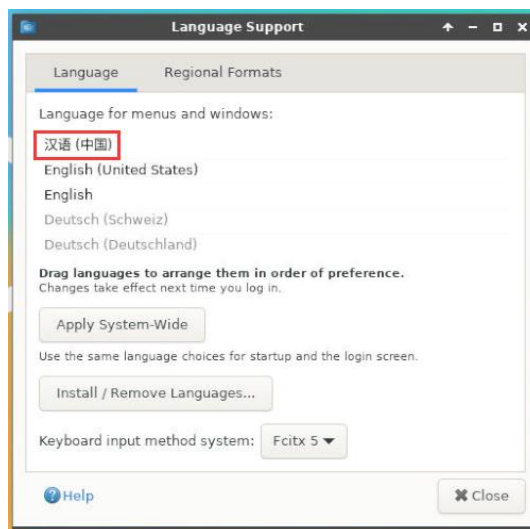




2) Then find the **Chinese (China)** option.

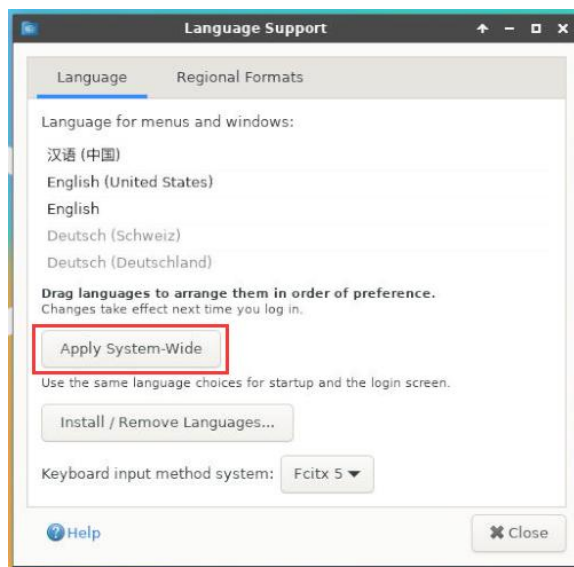


3) Then, use the left mouse button to select the **Chinese character (China)** and hold it down. Then drag it upwards to the beginning position. The result is shown in the image below:



Note that this step is not easy to drag; please try a few more times.

4) Then select **Apply System-Wide** to apply the Chinese language settings to the entire system.

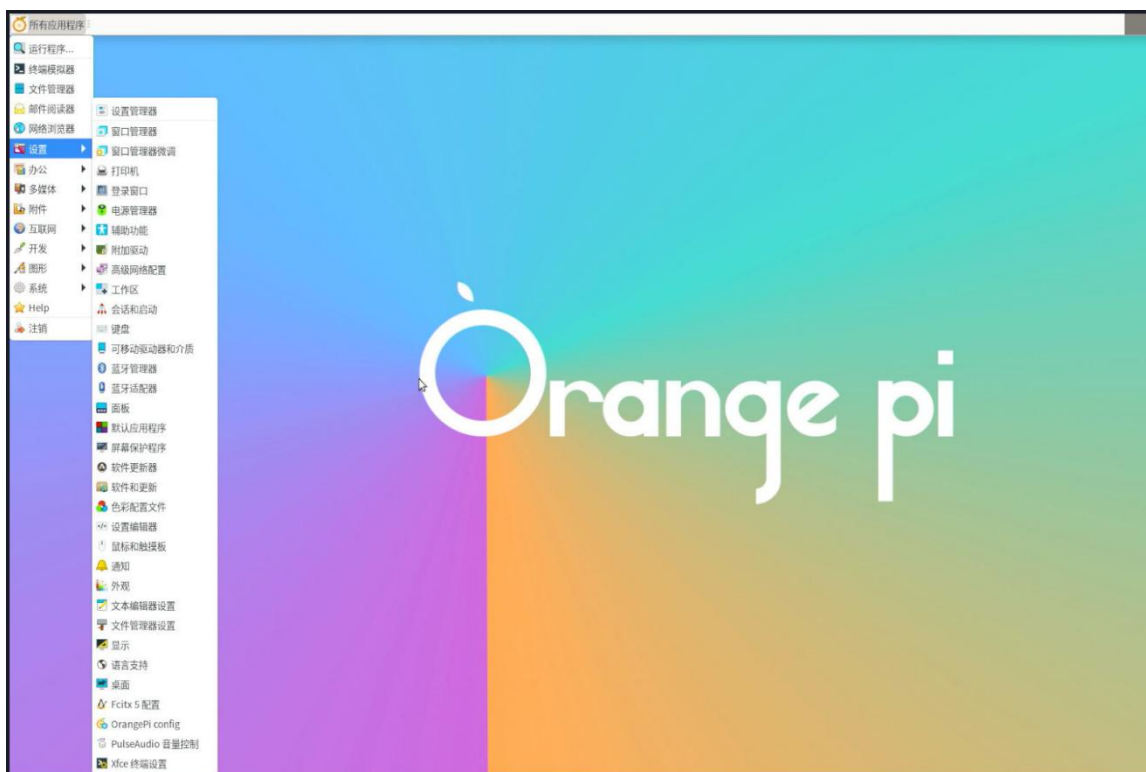


5) **Then restart the Linux system for the configuration to take effect.**

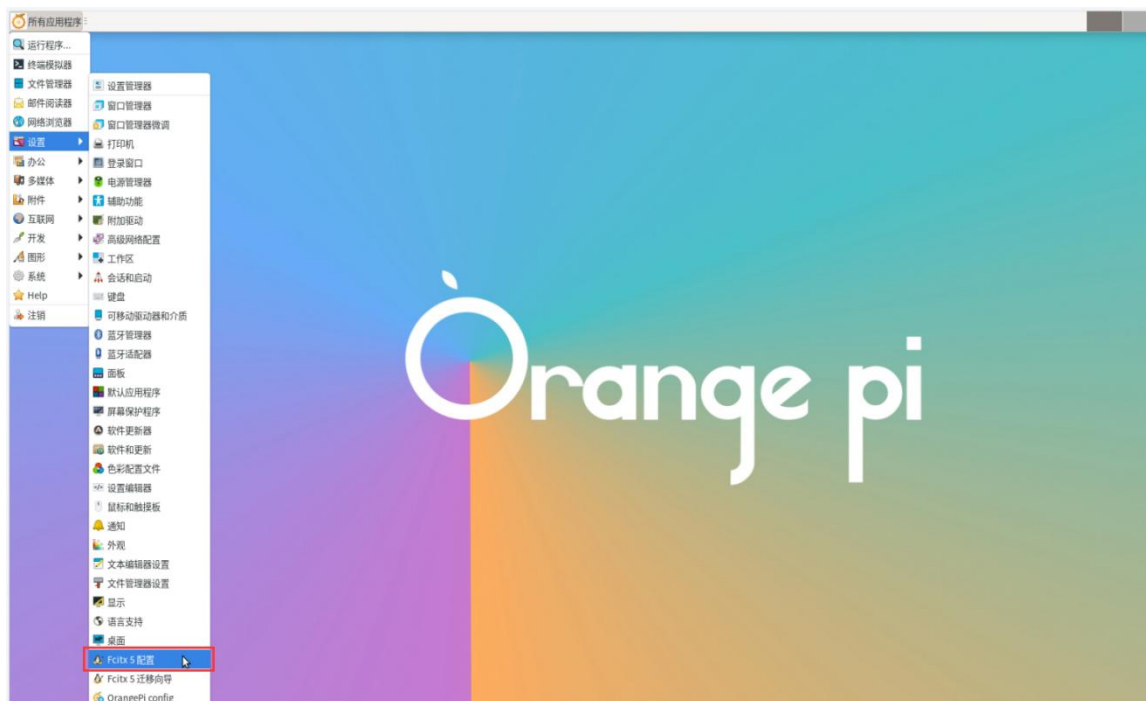
6) After re-entering the system, in the following interface, please select **Do not ask me again**, and then decide whether the standard folder should also be updated to Chinese according to your preference.



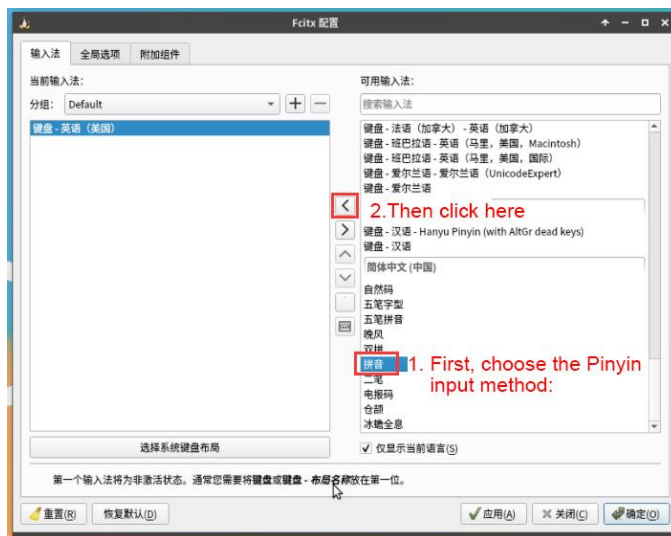
7) Then you can see that the desktop is displayed in Chinese.



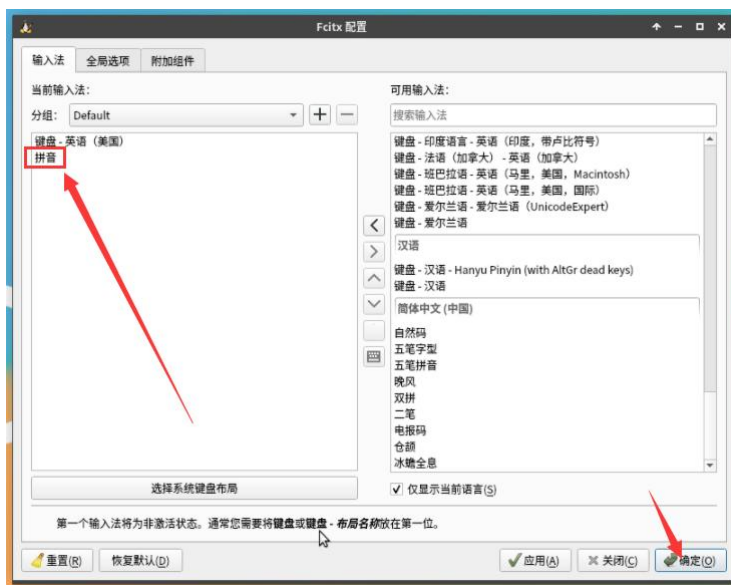
8) Then open the FcitiX5 configuration program.



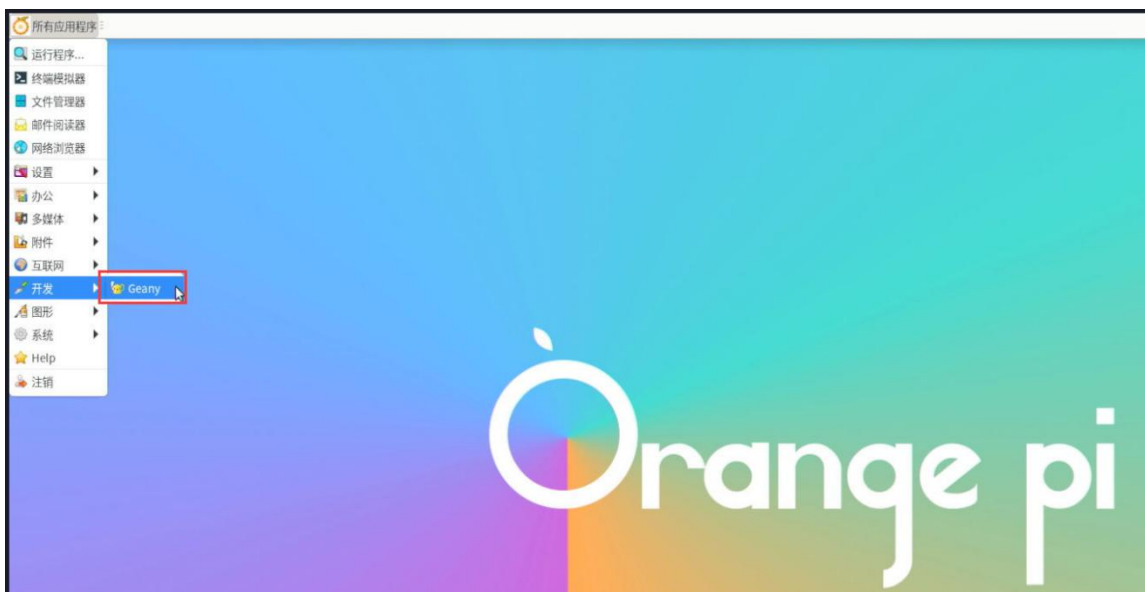
9) Then select to use Pinyin input method



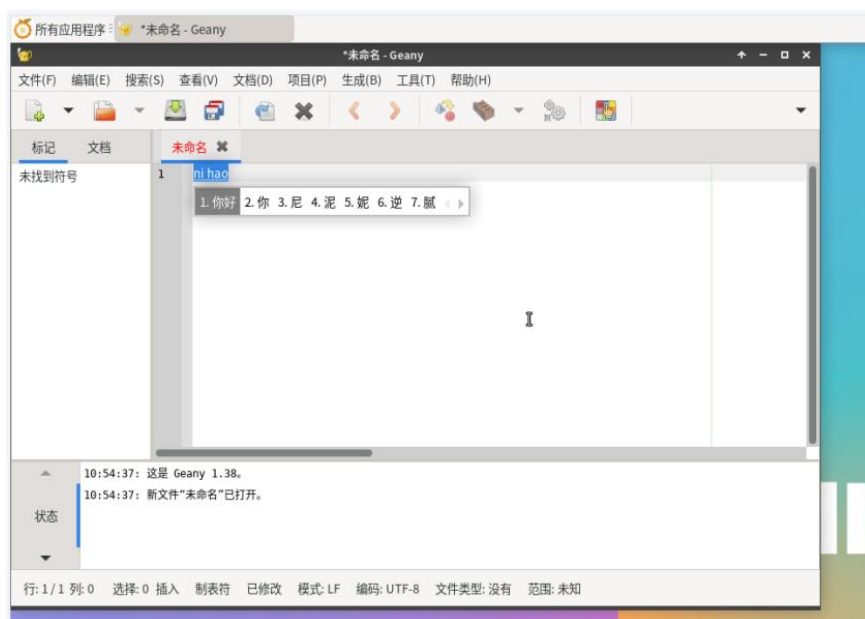
10) The interface after selection is shown below. Then click OK.



11) Then we can open **Geany** to test the Chinese input method. The opening method is shown in the image below.



12) After opening **Geany**, the default input method is English. You can switch to Chinese input method using the shortcut **Ctrl+Space**, and then you can type Chinese.



3. 25. Installation Method of Baota Linux Panel

BT Panel is a server management software that improves operational efficiency, supporting over 100 server management functions including one-click LAMP/LNMP/cluster/monitoring/website/FTP/database/JAVA, etc. (Excerpt from

**BT Panel official website)**

1) First, you need to increase the size of the **/tmp** space. After that, you need to **restart the Linux system on the development board**. The command is as follows:

```
orangePi@orangePi:~$ sudo sed -i 's/nosuid/&,size=2G/' /etc/fstab
orangePi@orangePi:~$ sudo reboot
```

2) After restarting, you can see that the size of **/tmp** has become 2GB.

```
orangePi@orangePi:~$ df -h | grep "/tmp"
tmpfs          2.0G    12K   2.0G    1% /tmp
```

3) Then, enter the following command in the Linux system to begin the installation of BT Panel.

```
orangePi@orangePi:~$ sudo install_bt_panel.sh
```

4) Then the BT Panel installer will prompt you whether to install **Bt-Panel** to the **/www** folder. Enter y at this point.

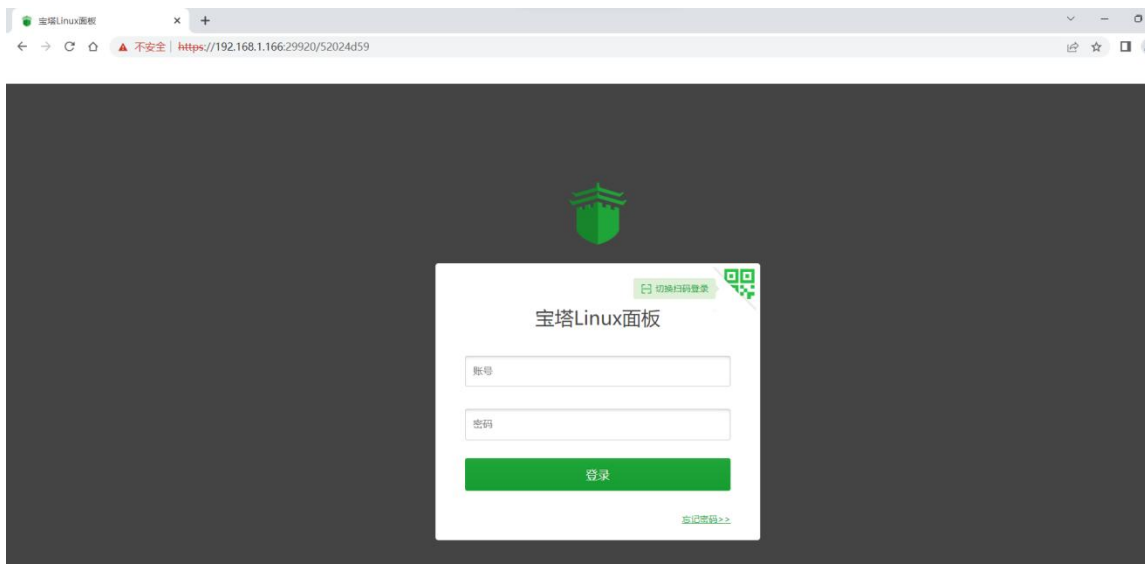
```
+-----+
| Bt-WebPanel FOR CentOS/Ubuntu/Debian
+-----+
| Copyright © 2015-2099 BT-SOFT(http://www.bt.cn) All rights reserved.
+-----+
| The WebPanel URL will be http://SERVER_IP:8888 when installed.
+-----+
Do you want to install Bt-Panel to the /www directory now?(y/n): y
```

5) Then all you need to do is wait patiently. When you see the following printout in the terminal, it means that the installation of BT Panel is complete. The entire installation process takes about 34 minutes, but this may vary depending on your network speed.

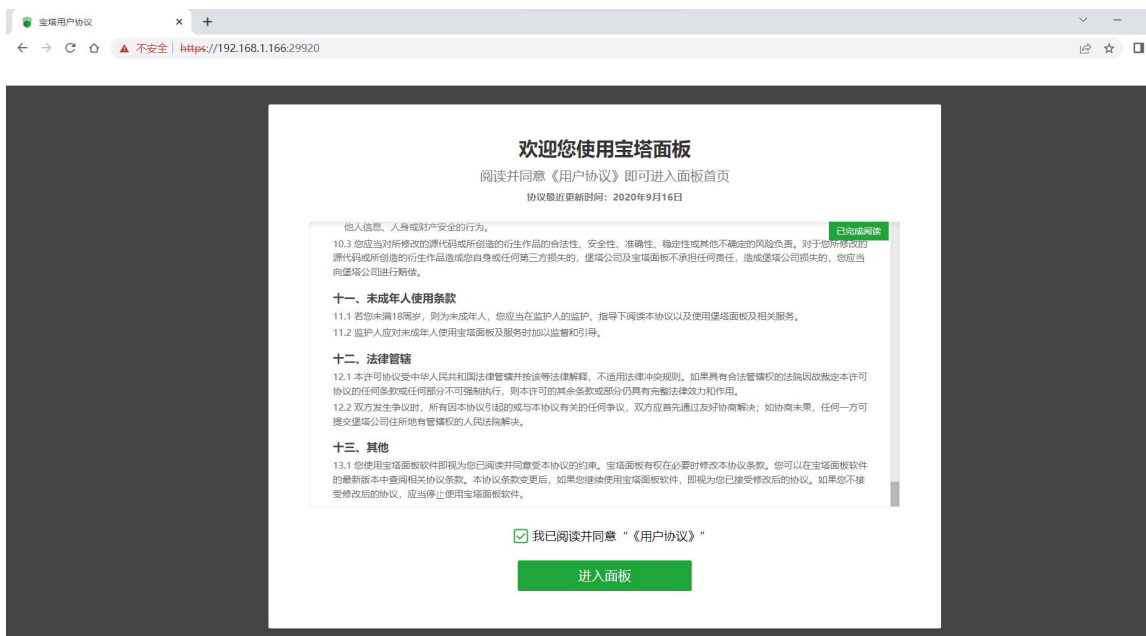


```
=====
Congratulations! Installed successfully!
=====
外网面板地址: https://183.15.204.194:29920/52024d59
内网面板地址: https://192.168.1.166:29920/52024d59
username: 4qhagfrc
password: 27b2d026
If you cannot access the panel,
release the following panel port [29920] in the security group
若无法访问面板, 请检查防火墙/安全组是否有放行面板[29920]端口
因已开启面板自签证书, 访问面板会提示不匹配证书, 请参考以下链接配置证书
https://www.bt.cn/bbs/thread-105443-1-1.html
=====
Time consumed: 34 Minute!
orange@orange:~$
```

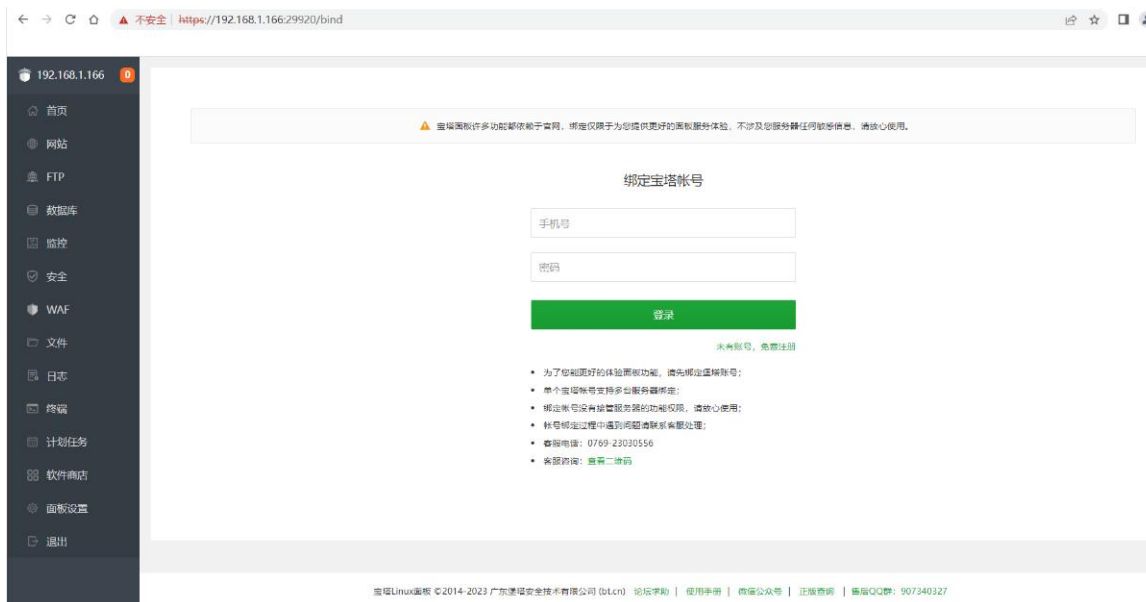
6) Now, enter the **panel address** shown above in your browser to open the BT Panel Linux login interface. Then, enter the **username** and **password** shown in the image above in the corresponding fields to log in to BT Panel.



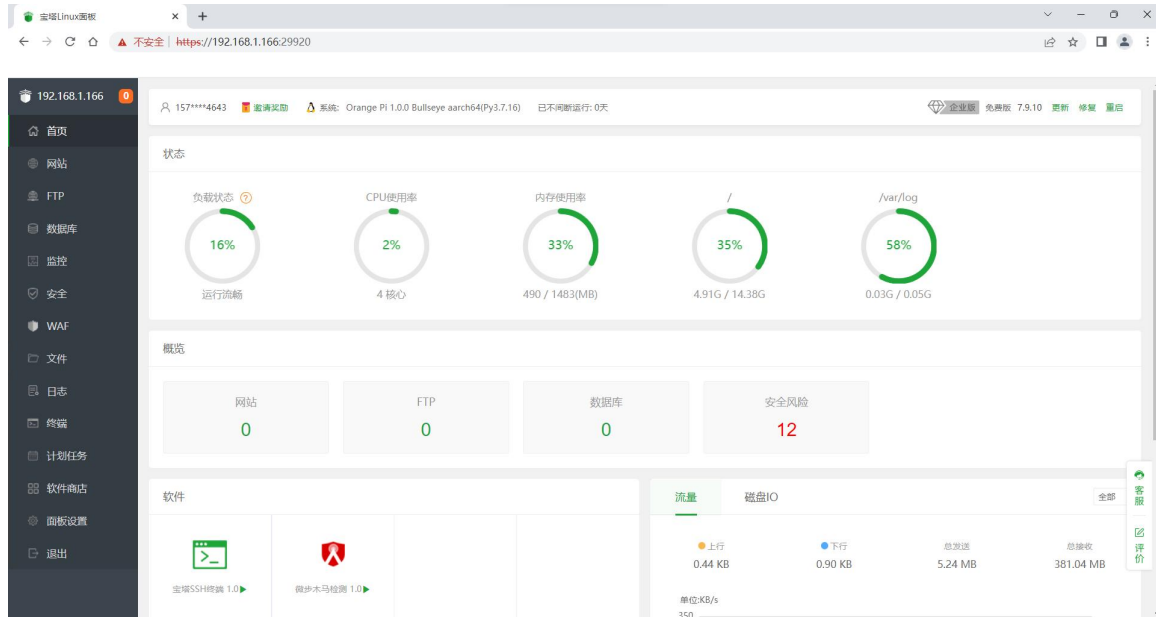
7) After successfully logging into BT Panel, the following welcome screen will pop up. First, please read the User Notice in the middle and scroll to the bottom. Then, you can select "I agree to and have read the User Agreement," and then click "Enter Panel" to enter BT Panel.



8) After entering the BT Panel, you will be prompted to bind your BT Panel official website account. If you don't have an account, you can register one on the BT Panel official website (<https://www.bt.cn>).



9) The final interface is shown in the figure below. It clearly displays some status information of the Linux system on the development board, such as load status, CPU utilization, memory utilization, and storage space usage.



10) For more functions of the pagoda, please refer to the information below and explore them on your own.

User Manual: <http://docs.bt.cn>

Forum address: <https://www.bt.cn/bbs>

GitHub link: <https://github.com/aaPanel/BaoTa>

3. 26. QT installation method

1) The following script can be used to install QT5 and QT Creator.

```
orangepi@orangepi:~$ install_qt.sh
```

2) The QT version number will be printed automatically after installation.

a. Ubuntu 22.04 comes with QT version **5.15.3**.

```
orangepi@orangepi:~$ install_qt.sh
```

.....

QMake version 3.1

Using Qt version **5.15.3** in /usr/lib/aarch64-linux-gnu

b. Debian 11 comes with QT version **5.15.2**

```
orangepi@orangepi:~$ install_qt.sh
```

.....

QMake version 3.1



Using Qt version **5.15.2** in /usr/lib/aarch64-linux-gnu

c. Debian 12 comes with QT version **5.15.8**

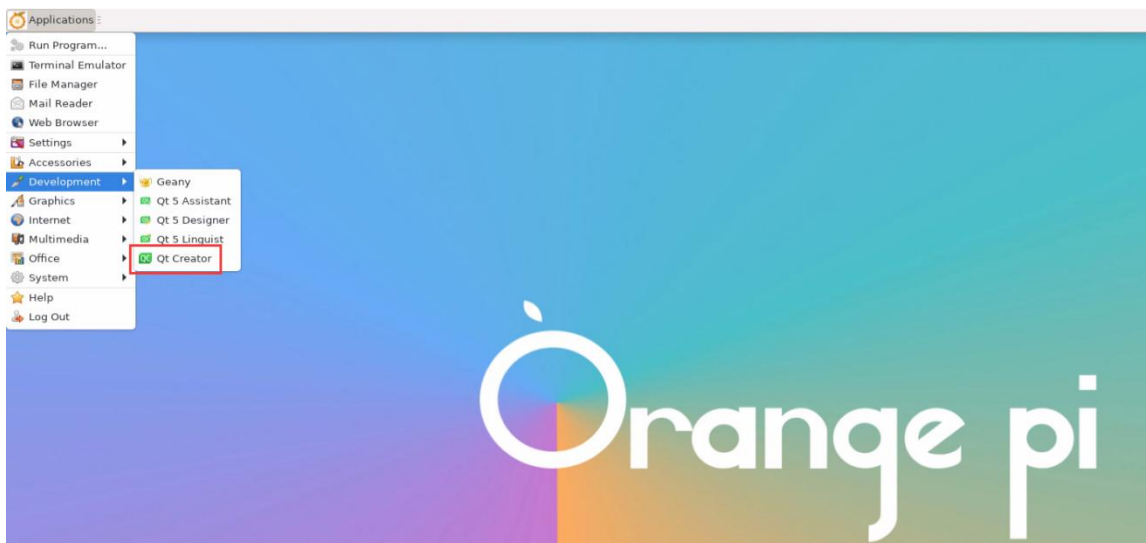
```
orangepi@orangepi:~$ install_qt.sh
```

```
.....
```

```
QMake version 3.1
```

```
Using Qt version 5.15.8 in /usr/lib/aarch64-linux-gnu
```

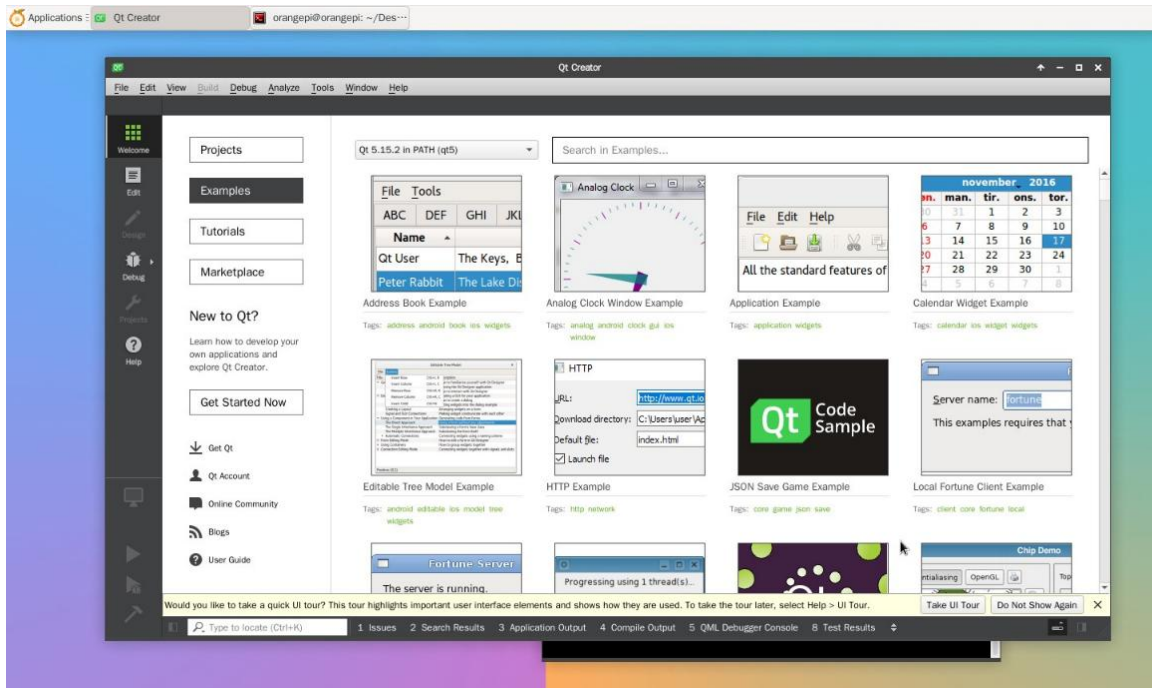
3) Then you can see the QT Creator launch icon in the **Applications** list.



You can also use the following command to open QT Creator

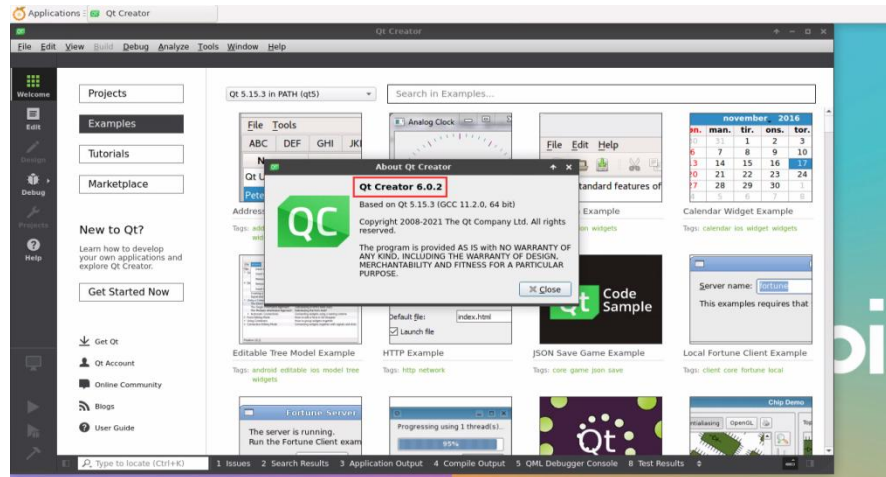
```
orangepi@orangepi:~$ qtcreator
```

4) The interface after opening QT Creator is shown below

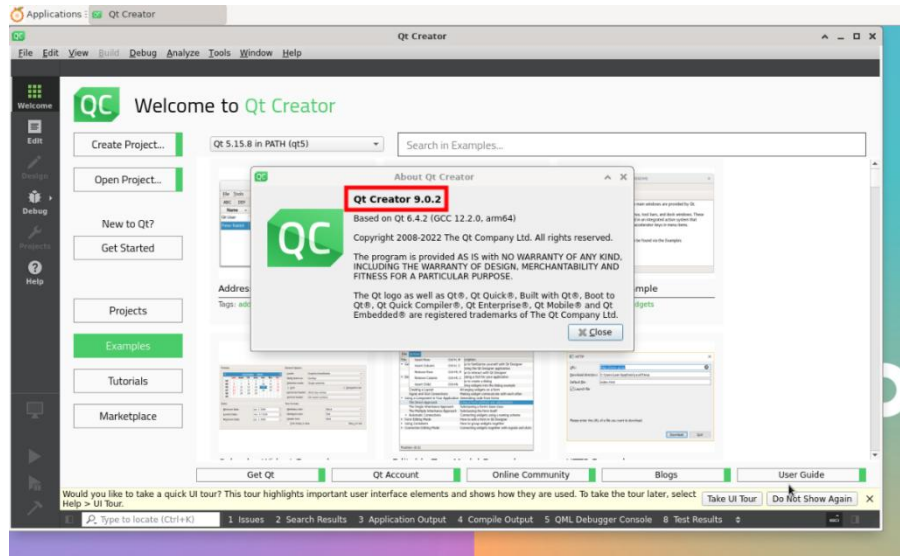


5) The following are versions of QT Creator

a. The default version of QT Creator in **Ubuntu 22.04** is shown below

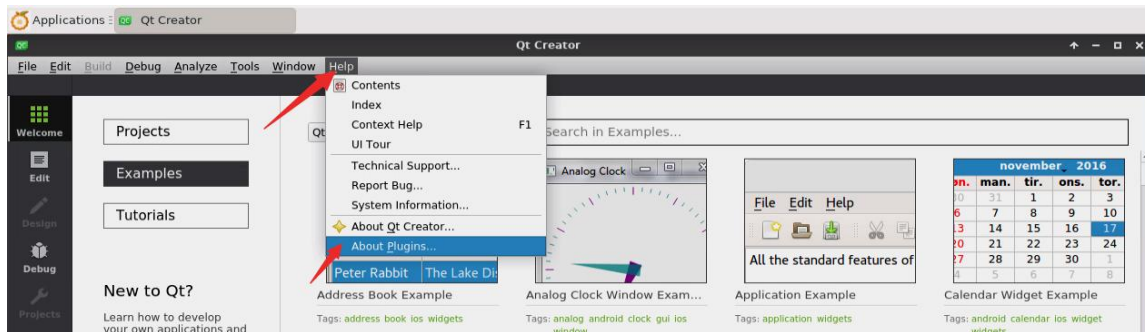


b. The default version of QT Creator in **Debian 12** is shown below

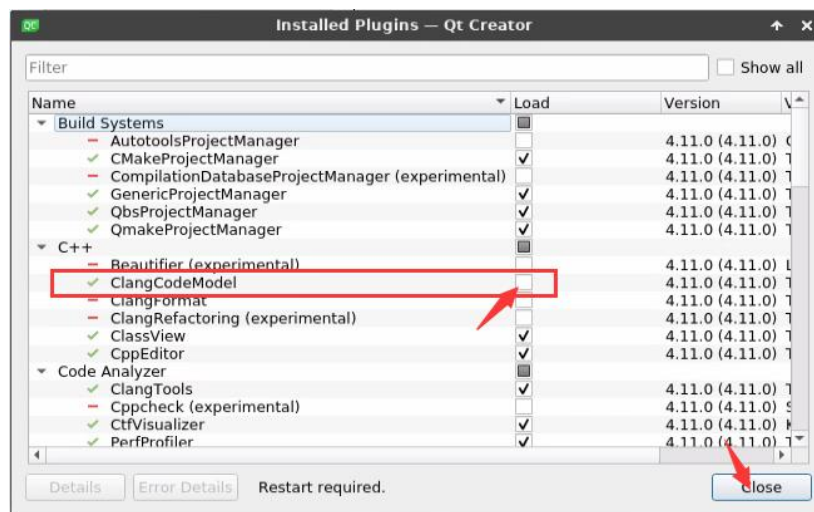


6) Then configure QT

a. First, open **Help->About Plugins...**



b. Then uncheck the **ClangCodeModel** box



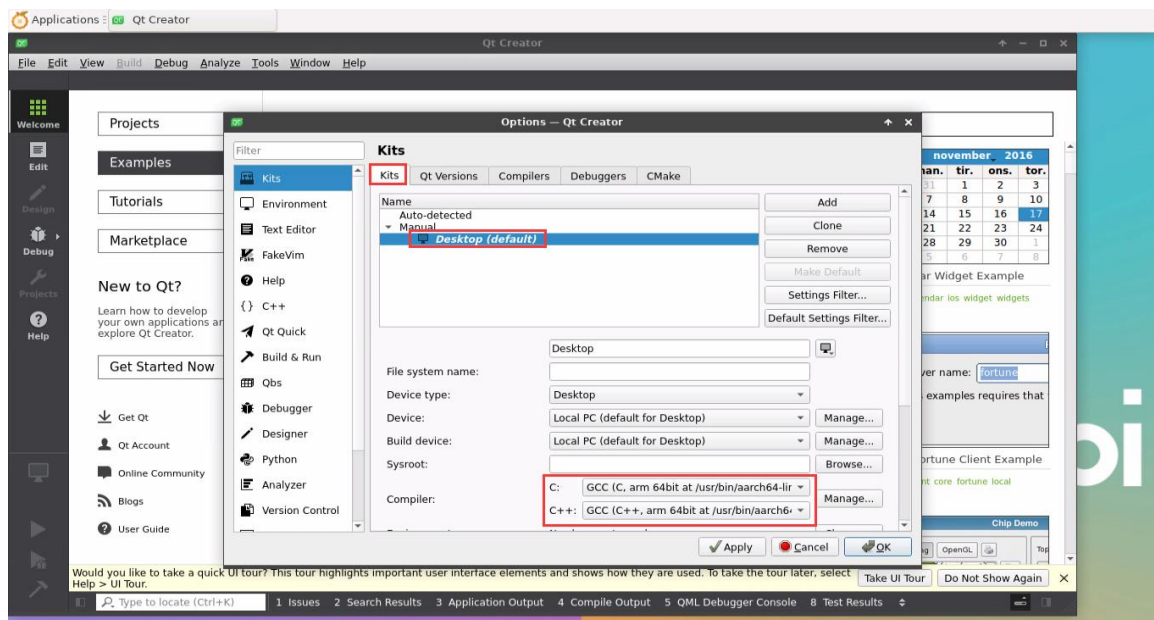
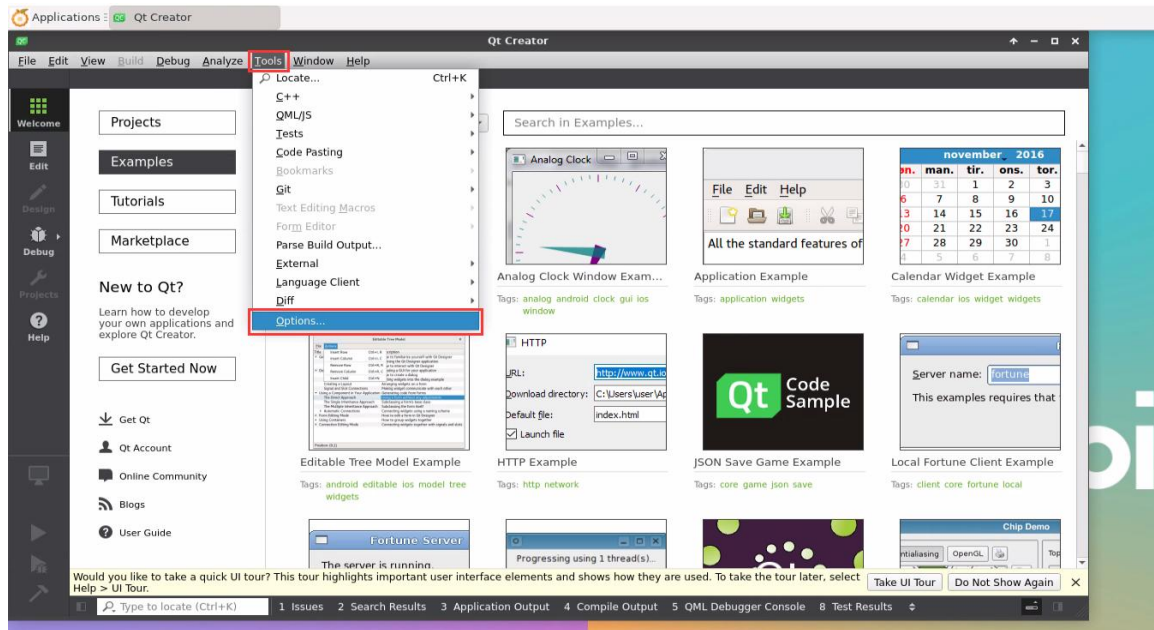
c. **After setting it up, you need to restart QT Creator**

d. Then ensure that QT Creator is using the GCC compiler. If it defaults to Clang,

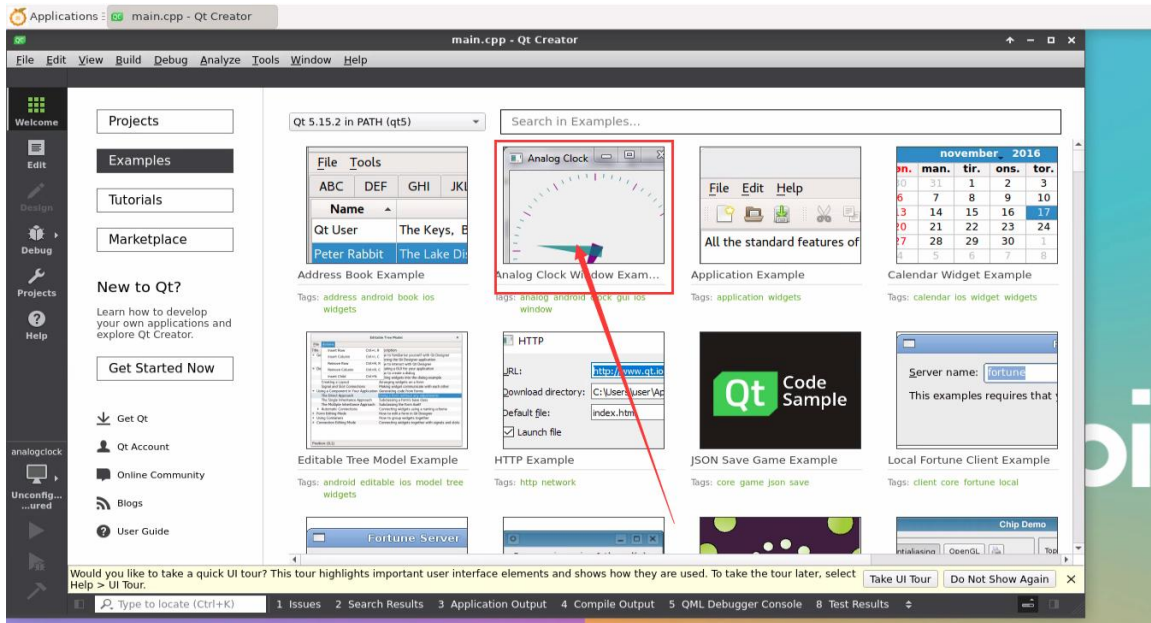


please change it to GCC

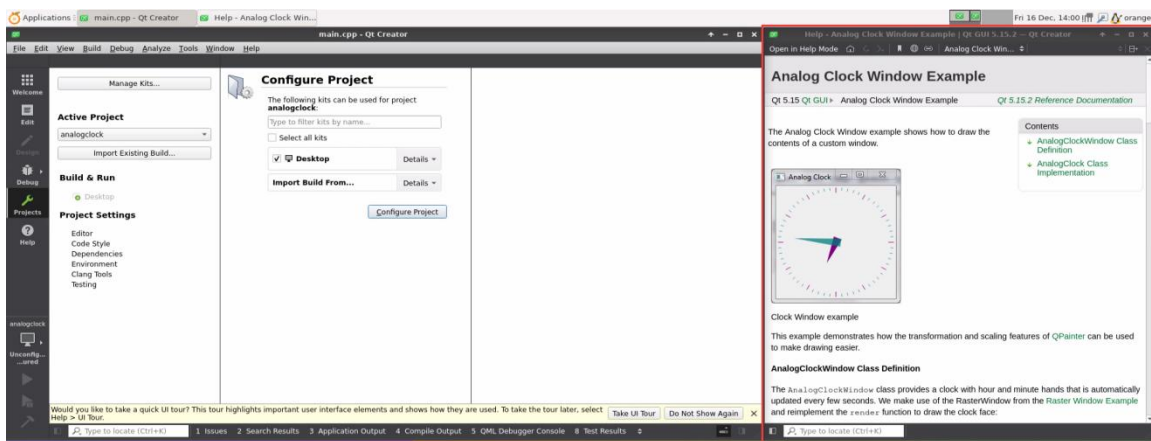
For Debian 12 users, please skip this step



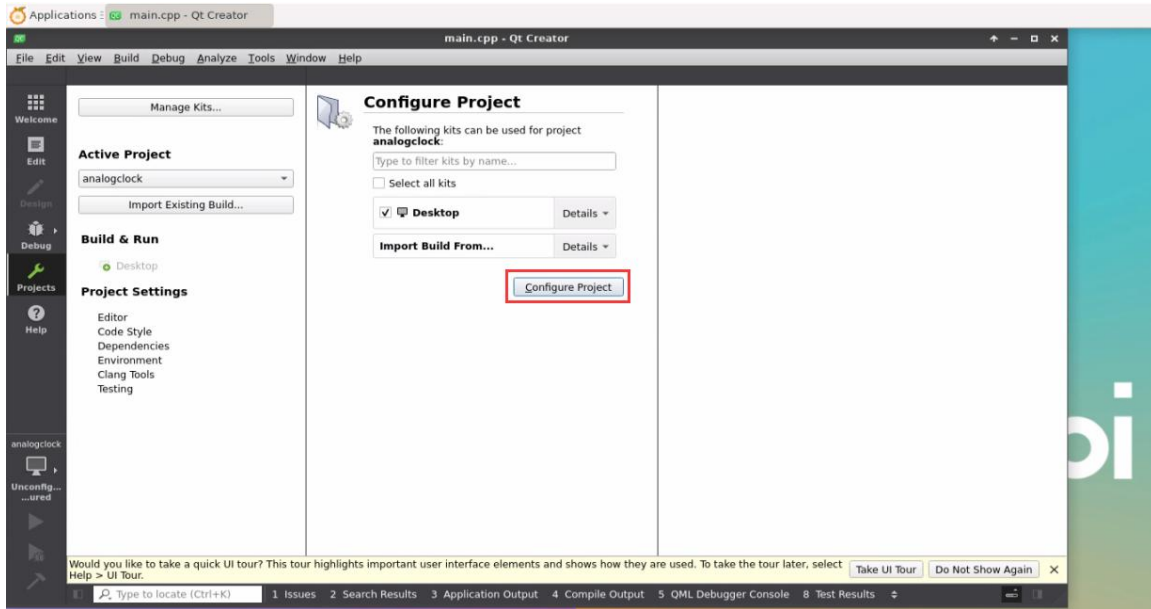
7) Then you can open a sample code



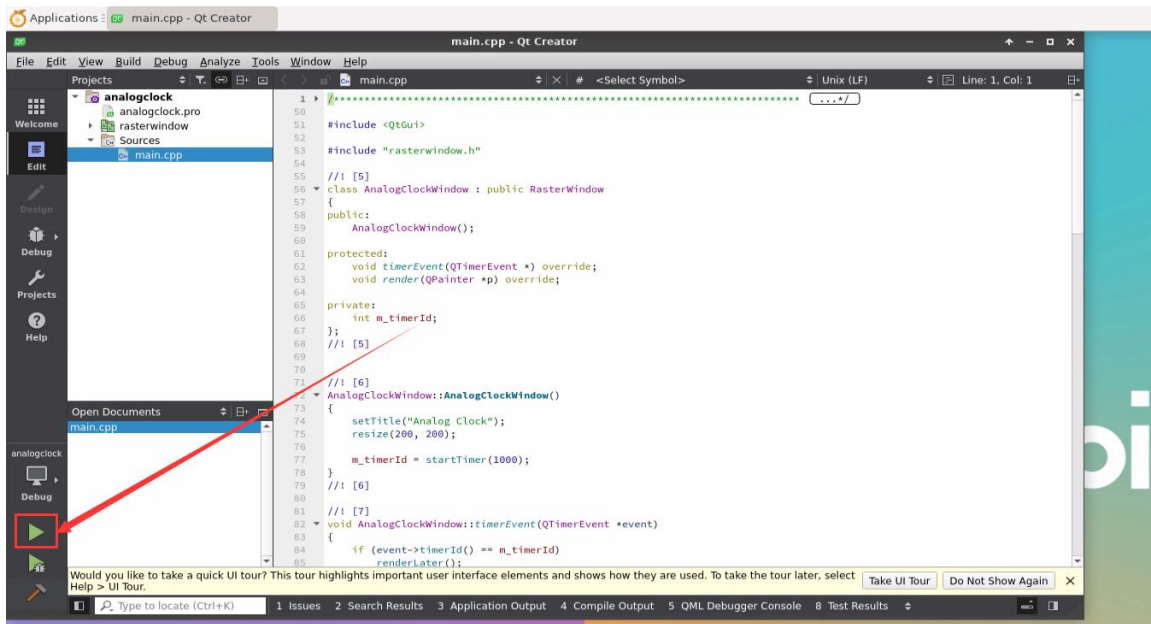
8) Clicking on the example code will automatically open the corresponding documentation, where you can carefully read the usage instructions.



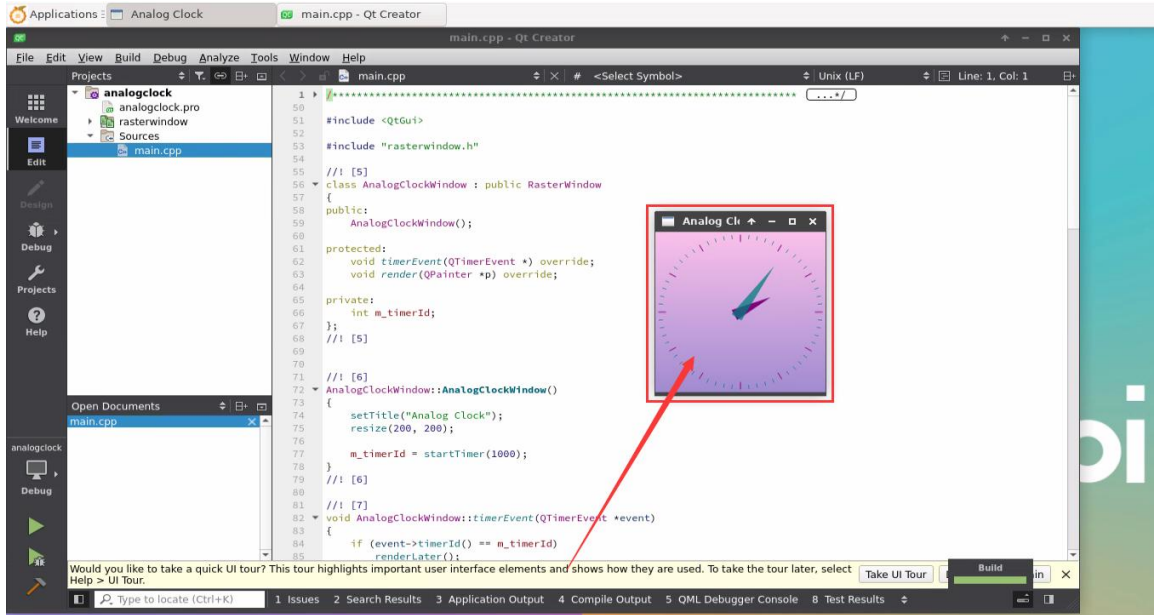
9) Then click below **Configure Project**



10) Then click the green triangle in the lower left corner to compile and run the example code



11) After waiting for a while, the interface shown in the image below will pop up, which indicates that QT can be compiled and run normally



12) References

https://wiki.qt.io/Install_Qt_5_on_Ubuntu

<https://download.qt.io/archive/qtcreator>

<https://download.qt.io/archive/qt>

3. 27. ROS Installation Method

3. 27. 1. How to install ROS 2 Humble on Ubuntu 22.04

1) The **install_ros.sh** script can be used to install ROS2

```
orange@orange:~$ install_ros.sh ros2
```

2) After the **install_ros.sh** script installs ROS2, it will automatically run the **ros2 -h** command. If you see the following output, it means that ROS2 is installed successfully

```
usage: ros2 [-h] Call 'ros2 <command> -h' for more detailed usage. ...
```

ros2 is an extensible command-line tool for ROS 2.

optional arguments:

-h, --help show this help message and exit

Commands:



action	Various action related sub-commands
bag	Various rosbag related sub-commands
component	Various component related sub-commands
daemon	Various daemon related sub-commands
doctor	Check ROS setup and other potential issues
interface	Show information about ROS interfaces
launch	Run a launch file
lifecycle	Various lifecycle related sub-commands
multicast	Various multicast related sub-commands
node	Various node related sub-commands
param	Various param related sub-commands
pkg	Various package related sub-commands
run	Run a package specific executable
security	Various security related sub-commands
service	Various service related sub-commands
topic	Various topic related sub-commands
wtf	Use `wtf` as alias to `doctor`

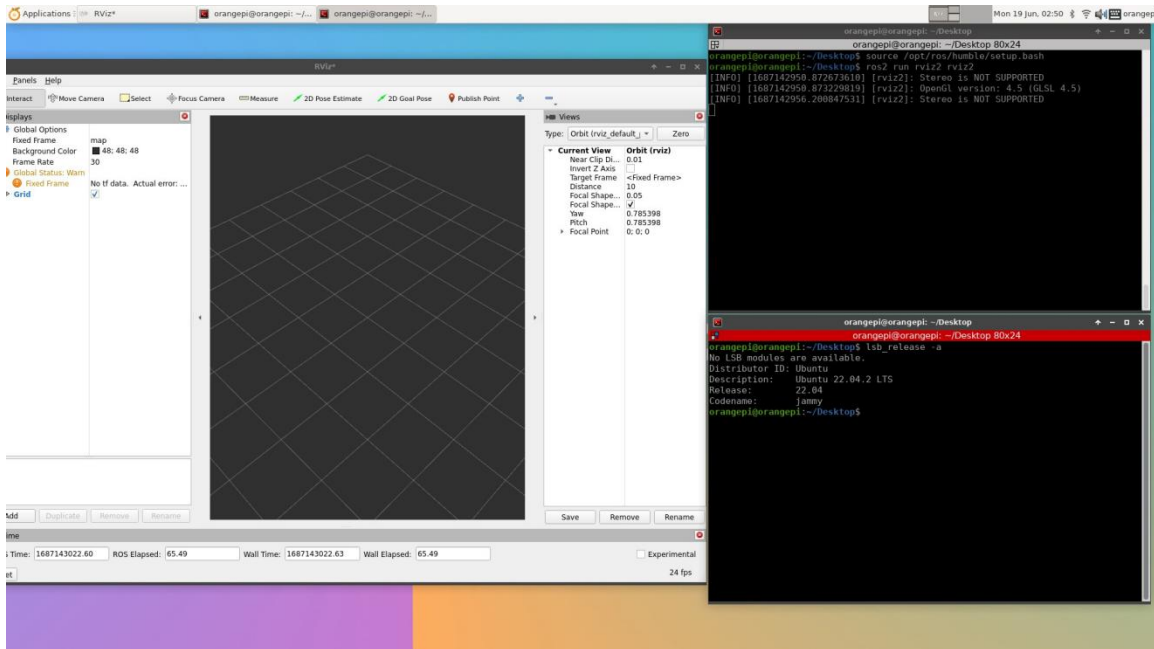
Call `ros2 <command> -h` for more detailed usage.

- 3) Then you can use the **test_ros.sh** script to test whether ROS 2 is installed successfully. If you see the following printout, it means that ROS 2 is running normally

```
orangeypi@orangeypi:~$ test_ros.sh
[INFO] [1671174101.200091527] [talker]: Publishing: 'Hello World: 1'
[INFO] [1671174101.235661048] [listener]: I heard: [Hello World: 1]
[INFO] [1671174102.199572327] [talker]: Publishing: 'Hello World: 2'
[INFO] [1671174102.204196299] [listener]: I heard: [Hello World: 2]
[INFO] [1671174103.199580322] [talker]: Publishing: 'Hello World: 3'
[INFO] [1671174103.204019965] [listener]: I heard: [Hello World: 3]
```

- 4) Running the following command will open rviz2

```
orangeypi@orangeypi:~$ source /opt/ros/humble/setup.bash
orangeypi@orangeypi:~$ ros2 run rviz2 rviz2
```



5) Reference documents

<http://docs.ros.org/en/humble/index.html>

<http://docs.ros.org/en/humble/Installation/Ubuntu-Install-Debians.html>

3. 28. Methods for installing kernel header files

1) Linux images released by OPi come with a deb package containing the kernel header files by default, located at **/opt/**

```
orange@orange:~$ ls /opt/linux-headers*
/opt/linux-headers-xxx-sun60iw2_x.x.x_arm64.deb
```

2) The following command can be used to install the kernel header file deb package

```
orange@orange:~$ sudo dpkg -i /opt/linux-headers*.deb
```

3) After installation, you can find the folder containing the kernel header files under **/usr/src**

```
orange@orange:~$ ls /usr/src
linux-headers-x.x.x
```

4) Then you can compile the source code of the hello kernel module that comes with the



Linux image. The source code of the hello module is located in **/usr/src/hello**. After entering this directory, you can compile it using the make command

```
orangeypi@orangeypi:~$ cd /usr/src/hello/
orangeypi@orangeypi:/usr/src/hello$ sudo make
make -C /lib/modules/5.15.147-sun60iw2/build M=/usr/src/hello modules
make[1]: Entering directory '/usr/src/linux-headers-5.15.147-sun60iw2'
  CC [M]  /usr/src/hello/hello.o
  MODPOST /usr/src/hello/Module.symvers
  CC [M]  /usr/src/hello/hello.mod.o
  LD [M]  /usr/src/hello/hello.ko
make[1]: Leaving directory '/usr/src/linux-headers-5.15.147-sun60iw2'
```

5) After compilation, the **hello.ko** kernel module will be generated

```
orangeypi@orangeypi:/usr/src/hello$ ls *.ko
hello.ko
```

6) The **insmod** command can be used to insert the **hello.ko** kernel module into the kernel

```
orangeypi@orangeypi:/usr/src/hello$ sudo insmod hello.ko
```

7) Then, use the **dmesg** command to check the output of the **hello.ko** kernel module. If you see the following output, it means the **hello.ko** kernel module has been loaded correctly

```
orangeypi@orangeypi:/usr/src/hello$ sudo dmesg | grep "Hello"
[ 2871.893988] Hello Orange Pi -- init
```

8) The **rmmod** command can be used to uninstall the **hello.ko** kernel module

```
orangeypi@orangeypi:/usr/src/hello$ sudo rmmod hello
orangeypi@orangeypi:/usr/src/hello$ dmesg | grep "Hello"
[ 2871.893988] Hello Orange Pi -- init
[ 3173.800892] Hello Orange Pi -- exit
```



3. 29. Testing of Some Programming Languages Supported by Linux Systems

3. 29. 1. Debian Bullseye System

1) Debian Bullseye comes with the gcc compiler toolchain pre-installed by default, allowing direct compilation of C language programs in the Linux system on the development board

- a. The version of gcc is as follows

```
orangepi@orangepi:~$ gcc --version
gcc (Debian 10.2.1-6) 10.2.1 20210110
Copyright (C) 2020 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR
PURPOSE.
```

- b. Write a C language program called **hello_world.c**

```
orangepi@orangepi:~$ vim hello_world.c
#include <stdio.h>

int main(void)
{
    printf("Hello World!\n");

    return 0;
}
```

- c. Then compile and run **hello_world.c**

```
orangepi@orangepi:~$ gcc -o hello_world hello_world.c
orangepi@orangepi:~$ ./hello_world
Hello World!
```

2) Debian Bullseye comes with Python 3 installed by default

- a. The specific version of Python is as follows

```
orangepi@orangepi:~$ python3
Python 3.9.2 (default, Feb 28 2021, 17:03:44)
[GCC 10.2.1 20210110] on linux
Type "help", "copyright", "credits" or "license" for more information.
```




```
>>>
```

Use the Ctrl+D shortcut key to exit Python's interactive mode.

- b. Write the **hello_world.py** program in Python language

```
orangepi@orangepi:~$ vim hello_world.py  
print('Hello World!')
```

- c. The result of running **hello_world.py** is as follows

```
orangepi@orangepi:~$ python3 hello_world.py  
Hello World!
```

3) Debian Bullseye does not have Java compilation tools and runtime environment installed by default

- a. Use the following command to install openjdk. The latest version in Debian Bullseye is openjdk-17

```
orangepi@orangepi:~$ sudo apt install -y openjdk-17-jdk
```

- b. After installation, you can check the version of Java

```
orangepi@orangepi:~$ java --version
```

- c. Write a Java version of **hello_world.java**

```
orangepi@orangepi:~$ vim hello_world.java  
public class hello_world  
{  
    public static void main(String[] args)  
    {  
        System.out.println("Hello World!");  
    }  
}
```

- d. Then compile and run **hello_world.java**

```
orangepi@orangepi:~$ javac hello_world.java  
orangepi@orangepi:~$ java hello_world  
Hello World!
```

3. 29. 2. Debian Bookworm System

1) Debian Bookworm comes with the gcc compilation toolchain installed by default, which allows direct compilation of C language programs on the Linux system of the development board

- a. The version of gcc is as follows

```
orangepi@orangepi:~$ gcc --version
```



```
gcc (Debian 12.2.0-14) 12.2.0
```

```
Copyright (C) 2022 Free Software Foundation, Inc.
```

```
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR
PURPOSE.
```

- b. Write a C language program called **hello_world.c**

```
orangepi@orangepi:~$ vim hello_world.c
```

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    printf("Hello World!\n");
```

```
    return 0;
```

```
}
```

- c. Then compile and run **hello_world.c**

```
orangepi@orangepi:~$ gcc -o hello_world hello_world.c
```

```
orangepi@orangepi:~$ ./hello_world
```

```
Hello World!
```

2) Debian Bookworm comes with Python 3 installed by default

- a. The specific version of Python is as follows

```
orangepi@orangepi:~$ python3
```

```
Python 3.11.2 (main, Apr 28 2025, 14:11:48) [GCC 12.2.0] on linux
```

```
Type "help", "copyright", "credits" or "license" for more information.
```

```
>>>
```

Use the Ctrl+D shortcut key to exit Python's interactive mode.

- b. Write the **hello_world.py** program in Python language

```
orangepi@orangepi:~$ vim hello_world.py
```

```
print('Hello World!')
```

- c. The result of running **hello_world.py** is as follows

```
orangepi@orangepi:~$ python3 hello_world.py
```

```
Hello World!
```

3) Debian Bookworm does not have Java compilation tools and runtime environment



installed by default

- a. You can use the following command to install openjdk. The latest version in Debian Bookworm is openjdk-17

```
orange@orange:~$ sudo apt install -y openjdk-17-jdk
```

- b. After installation, you can check the version of Java

```
orange@orange:~$ java --version
```

- c. Write a Java version of **hello_world.java**

```
orange@orange:~$ vim hello_world.java
public class hello_world
{
    public static void main(String[] args)
    {
        System.out.println("Hello World!");
    }
}
```

- d. Then compile and run **hello_world.java**

```
orange@orange:~$ javac hello_world.java
orange@orange:~$ java hello_world
Hello World!
```

3. 29. 3. Ubuntu Jammy System

1) Ubuntu Jammy comes with the gcc compilation toolchain installed by default, which allows direct compilation of C language programs on the Linux system of the development board

- a. The version of gcc is as follows

```
orange@orange:~$ gcc --version
gcc (Ubuntu 11.4.0-1ubuntu1~22.04) 11.4.0
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR
PURPOSE.
```

- b. Write a C language program called **hello_world.c**

```
orange@orange:~$ vim hello_world.c
#include <stdio.h>

int main(void)
```



```
{  
    printf("Hello World!\n");  
  
    return 0;  
}
```

- c. Then compile and run **hello_world.c**

```
orange@orange:~$ gcc -o hello_world hello_world.c  
orange@orange:~$ ./hello_world  
Hello World!
```

2) Ubuntu Jammy comes with Python 3 installed by default

- a. The specific version of Python3 is as follows

```
orange@orange:~$ python3  
Python 3.10.12 (main, Jul 29 2024, 16:56:48) [GCC 11.4.0] on linux  
Type "help", "copyright", "credits" or "license" for more information.  
>>>
```

Use the Ctrl+D shortcut key to exit Python's interactive mode.

- b. Write the **hello_world.py** program in Python language

```
orange@orange:~$ vim hello_world.py  
print('Hello World!')
```

- c. The result of running **hello_world.py** is as follows

```
orange@orange:~$ python3 hello_world.py  
Hello World!
```

3) Ubuntu Jammy does not have Java compilation tools and runtime environment installed by default

- a. You can use the following command to install openjdk-18

```
orange@orange:~$ sudo apt install -y openjdk-18-jdk
```

- b. After installation, you can check the version of Java

```
orange@orange:~$ java --version  
openjdk 18.0.2-ea 2022-07-19  
OpenJDK Runtime Environment (build 18.0.2-ea+9-Ubuntu-22.04)  
OpenJDK 64-Bit Server VM (build 18.0.2-ea+9-Ubuntu-22.04, mixed mode, sharing)
```

- c. Write a Java version of **hello_world.java**

```
orange@orange:~$ vim hello_world.java
```



```
public class hello_world
{
    public static void main(String[] args)
    {
        System.out.println("Hello World!");
    }
}
```

d. Then compile and run **hello_world.java**

```
orange@orange:~$ javac hello_world.java
orange@orange:~$ java hello_world
Hello World!
```

3. 30. Method of uploading files to the Linux system of the development board

3. 30. 1. Method for uploading files to the Linux system on the development board in Ubuntu PC

3. 30. 1. 1. Method of uploading files using the scp command

1) The scp command can be used to upload files from Ubuntu PC to the Linux system of the development board. The specific command is as follows

- a. **file_path**: Need to replace with the path of the file to be uploaded
- b. **orange@orange**: The username for the Linux system on the development board can also be replaced with another one, such as root
- c. **192.168.xx.xx**: Please modify the IP address of the development board according to the actual situation
- d. **/home/orange@orange**: The path in the Linux system of the development board can also be modified to other paths

```
test@test:~$ scp file_path orange@192.168.xx.xx:/home/orange@orange/
```

2) If you want to upload a folder, you need to add the -r parameter

```
test@test:~$ scp -r dir_path orange@192.168.xx.xx:/home/orange@orange/
```



3) scp has more uses, please use the following command to view the man manual

```
test@test:~$ man scp
```

3. 30. 1. 2. Method of uploading files using Filezilla

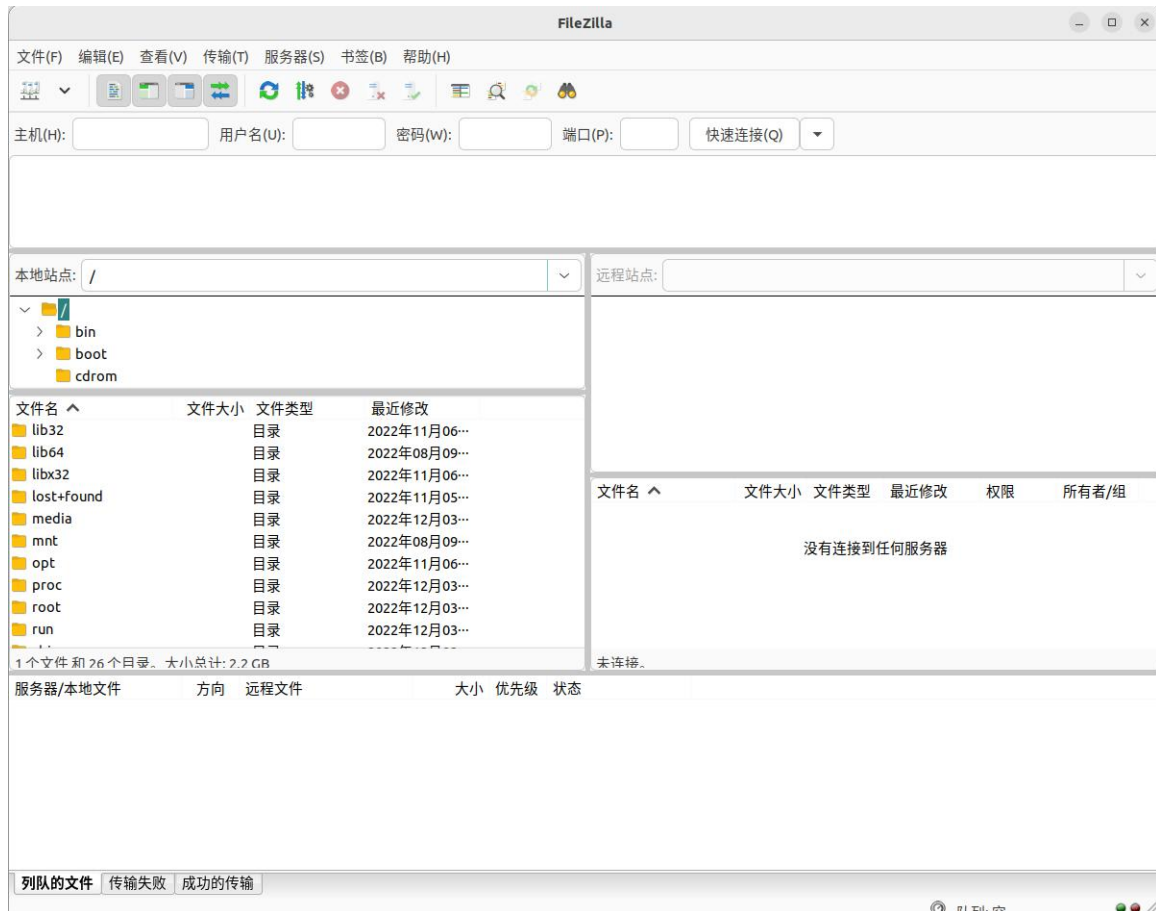
1) First, install Filezilla on Ubuntu PC

```
test@test:~$ sudo apt install -y filezilla
```

2) Then use the following command to open Filezilla

```
test@test:~$ filezilla
```

3) The interface of Filezilla after opening is shown below. At this time, the remote site on the right is displayed as empty



4) The method of connecting the development board is shown in the following figure



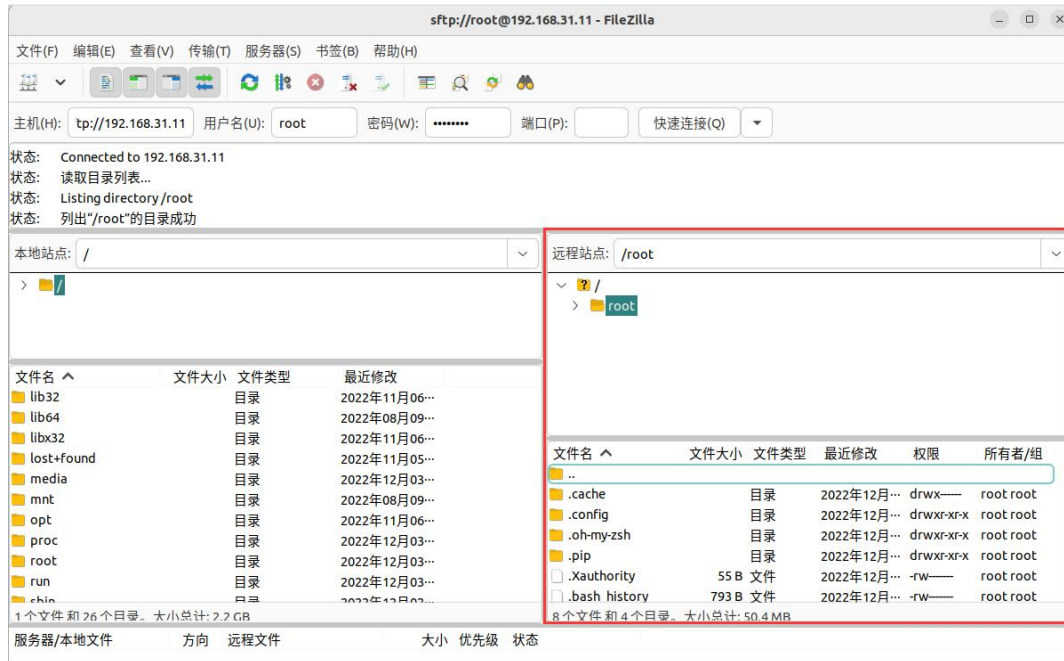
5) Then select 'Save Password' and click 'OK'



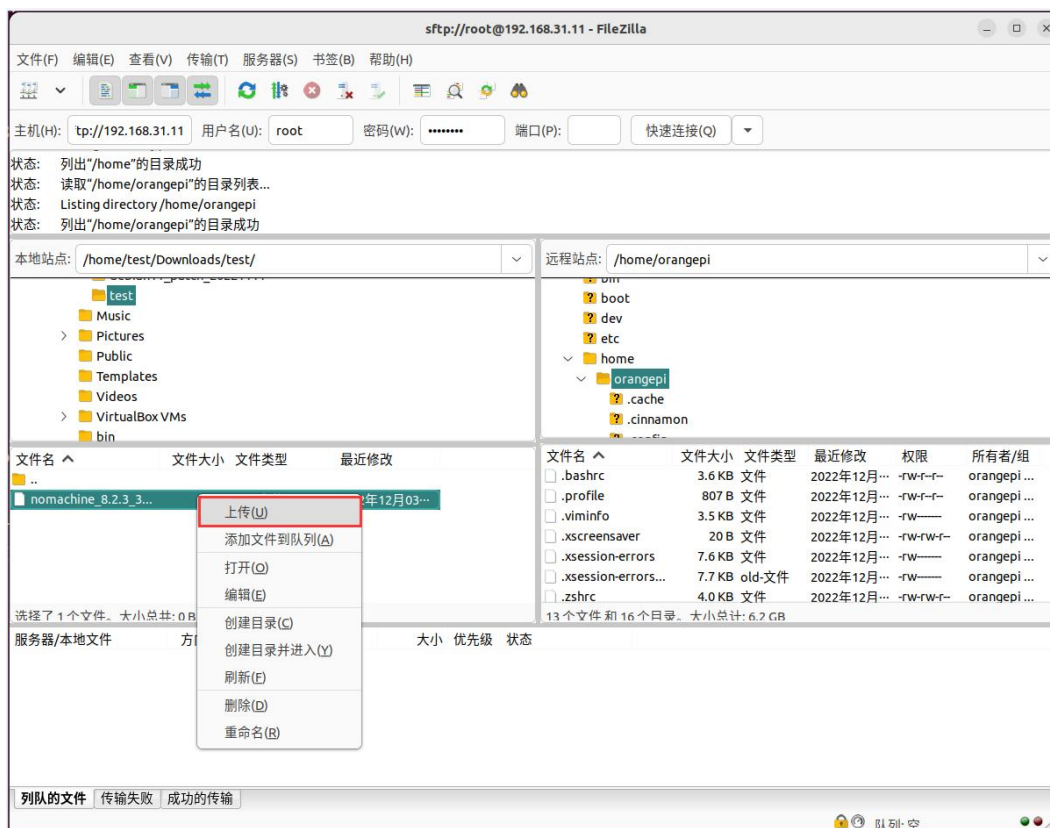
6) Then choose to always trust the host, and click OK



7) After successful connection, the directory structure of the Linux file system on the development board can be seen on the right side of the Filezilla software



8) Then select the path to upload to the development board on the right side of the Filezilla software, and then select the file to upload to Ubuntu PC on the left side of the Filezilla software. Right click and select the upload option to start uploading the file to the development board.



9) After uploading, you can go to the corresponding path in the Linux system of the development board to view the uploaded files

10) The method of uploading folders is the same as the method of uploading files, so it will not be repeated here

3. 30. 2. Method for uploading files from Windows PC to development board Linux system

3. 30. 2. 1. Method of uploading files using Filezilla

1) Firstly, download the installation file for the Windows version of Filezilla software. The download link is shown below

<https://filezilla-project.org/download.php?type=client>



FileZilla The free FTP solution

Home
FileZilla
Features
Screenshots
Download
Documentation
FileZilla Pro
FileZilla Server
Download
Community
Forum
Wiki
General
FAQ
Support
Contact
License
Privacy Policy
Trademark Policy
Development
Source code
Nightly builds
Translations
Version history
Changelog
Issue tracker
Other projects

Download FileZilla Client for Windows (64bit x86)
The latest stable version of FileZilla Client is 3.62.2.
Please select the file appropriate for your platform below.

Windows (64bit x86)

Download FileZilla Client [Click here to download](#)

This installer may include bundled offers. Check below for more options.
The 64bit versions of Windows 8.1, 10 and 11 are supported.

More download options
Other platforms:
Not what you are looking for?
[Show additional download options](#)

Please select your edition of FileZilla Client

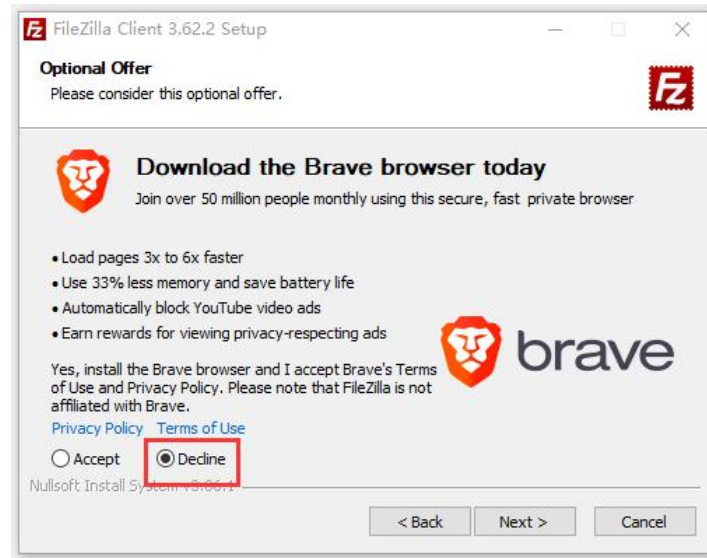
	FileZilla	FileZilla with manual	FileZilla Pro	FileZilla Pro + CLI
Standard FTP	Yes	Yes	Yes	Yes
FTP over TLS	Yes	Yes	Yes	Yes
SFTP	Yes	Yes	Yes	Yes
Comprehensive PDF manual	-	Yes	Yes	Yes
Amazon S3	-	-	Yes	Yes
Backblaze B2	-	-	Yes	Yes
Dropbox	-	-	Yes	Yes
Microsoft OneDrive	-	-	Yes	Yes
Google Drive	-	-	Yes	Yes
Google Cloud Storage	-	-	Yes	Yes
Microsoft Azure Blob + File Storage	-	-	Yes	Yes
WebDAV	-	-	Yes	Yes
OpenStack Swift	-	-	Yes	Yes
Box	-	-	Yes	Yes
Site Manager synchronization	-	-	Yes	Yes
Command-line interface	-	-	-	Yes
Batch transfers	-	-	-	Yes

Then select here to download [Download](#) [Select](#) [Select](#) [Select](#)

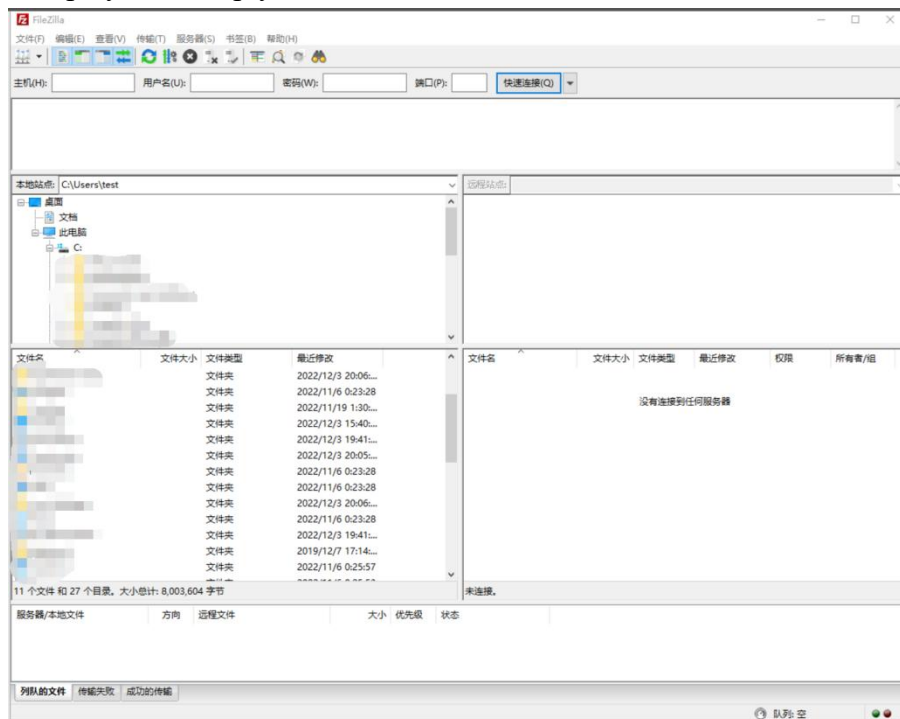
2) The downloaded installation package is shown below, then double-click to install directly

FileZilla_Server_1.5.1_win64-setup.exe

During the installation process, please select **Decline** on the installation interface below, and then select **Next>**



3) The interface of Filezilla after opening is shown below. At this time, the remote site on the right is displayed as empty



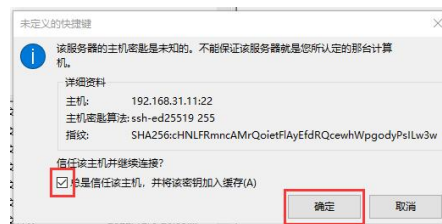
4) The method of connecting the development board is shown in the following figure:



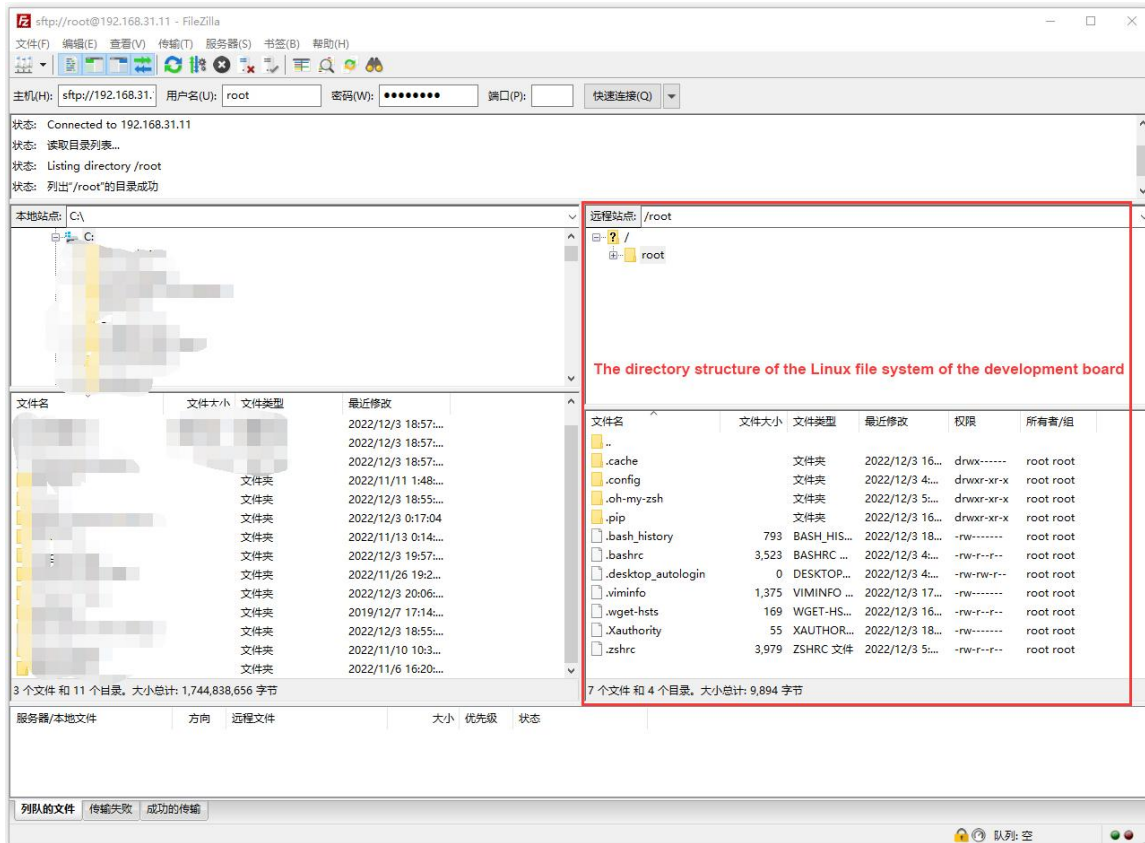
5) Then select 'Save Password' and click 'OK'



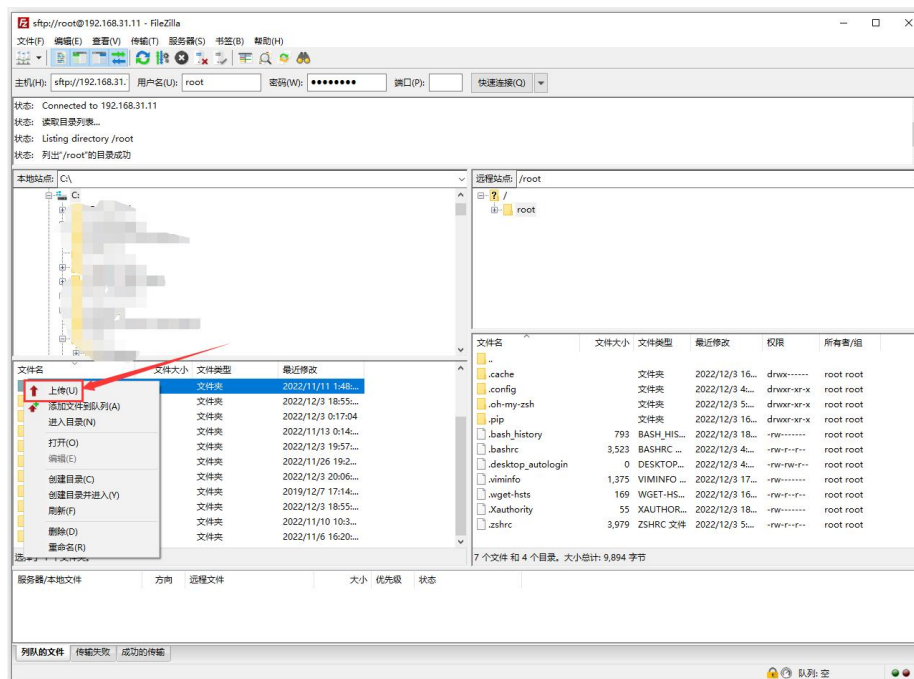
6) Then choose to always trust the host, and click OK



7) After successful connection, the directory structure of the Linux file system on the development board can be seen on the right side of the Filezilla software



8) Then select the path to upload to the development board on the right side of the Filezilla software, and then select the file to be uploaded to the Windows PC on the left side of the Filezilla software. Right click and select the upload option to start uploading the file to the development board



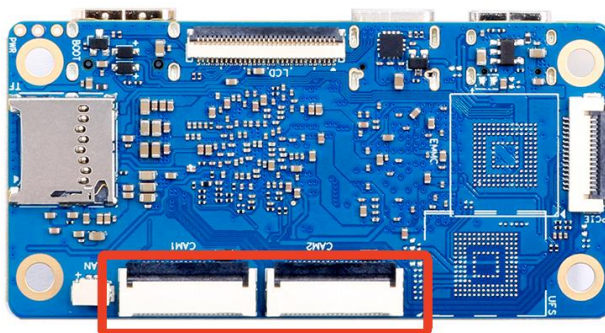
9) After uploading, you can go to the corresponding path in the Linux system of the development board to view the uploaded files

10) The method of uploading folders is the same as the method of uploading files, so it will not be repeated here

3. 31. Instructions for using IMX219 MIPI camera

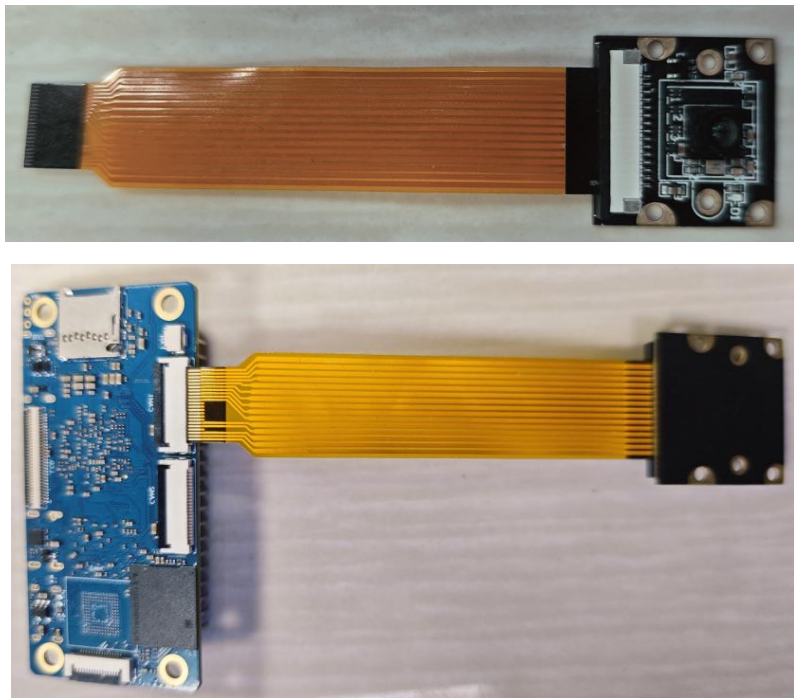
3. 31. 1. Installation method of camera

1) There are a total of 2 camera interfaces on the development board, currently only supporting the connection of IMX219 cameras. The positions of CAM1 and CAM2 are shown below:





- 2) The camera interface specification is 22PIN 0.5mm spacing, and the IMX219 camera needs to be connected to the development board through a 22PIN to 15PIN ribbon cable. The CAM1 and CAM2 connections are the same



3. 31. 2. Method of opening the camera through Gstreamer

Note that currently only the Debian bullseye system supports this method, and other versions of the system do not currently support it.

- 1) First, install the camera driver

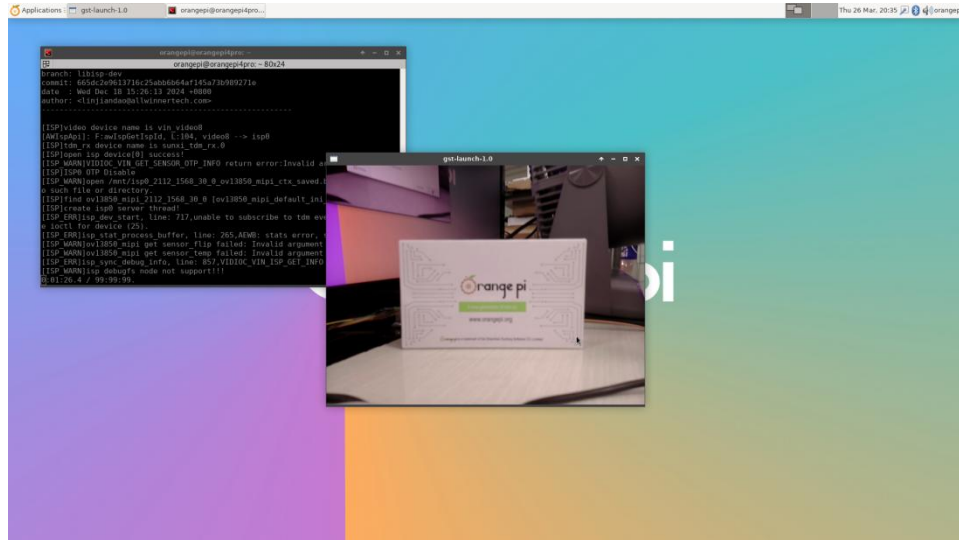
```
orange@orange:~$ sudo modprobe vin_v4l2
```

- 2) Then execute the following command to confirm the device node of the camera. Currently, two MIPI cameras are connected, so there will be two nodes displayed.

```
orange@orange:~$ ls /dev/video*  
/dev/video0  /dev/video8
```

- 3) The system comes pre installed with the **test_camera.sh** script, which can be executed to turn on the camera.

```
orange@orange:~$ test_camera.sh /dev/video0  #打开 CAM1  
orange@orange:~$ test_camera.sh /dev/video8  #打开 CAM2
```



3. 31. 3. Method of opening MIPI camera through OpenCV program

1) First, you need to install the dependency package.

```
orangepi@orangepi:~$ sudo apt install python3-pip python3-opencv libopencv-dev
orangepi@orangepi:~$ sudo apt install python3-pybind11 python3-dev
```

2) Then enter the directory where **v4l2_opencv_demo** is located, and you can see that it contains the following files.

```
orangepi@orangepi:~$ cd /opt/v4l2_opencv_demo/
orangepi@orangepi:/opt/v4l2_opencv_demo$ ls
Makefile  test_camera.cpp  test_camera.py  v4l2_camera_bind.cpp
v4l2_camera.cpp  v4l2_camera.h
```

3) Change the owner of **v4l2_opencv_demo** to a regular user so that regular users can operate files

```
orangepi@orangepi:~$ sudo chown orangepi:orangepi /opt/v4l2_opencv_demo/ -R
```

4) Example file description:

- Makefile:** Build the compilation rule file for the C++ sample program.
- test_camera.cpp:** C++ testing program, using OpenCV to preview the camera in real-time.
- test_camera.py :** Python sample program, using OpenCV to preview camera images.
- v4l2_camera_bind.cpp:** Use pybind11 to encapsulate the V4L2Camera class for



Python to call.

- e. **v4l2_camera.cpp** : V4L2Camera class implementation, encapsulating camera initialization and frame capture logic.
- f. **v4l2_camera.h** : V4L2Camera class header file, defining interfaces and member variables.

5) Then install the camera driver

```
orange@orange:~$ sudo modprobe vin_v4l2
```

6) Then execute the following command to confirm the device node of the camera. Currently, two MIPI cameras are connected, so there will be two nodes displayed.

```
orange@orange:~$ ls /dev/video*
/dev/video0  /dev/video8
```

7) The steps to open a MIPI camera using a C++example are as follows:

- a. First, execute the following command to compile the C++example.

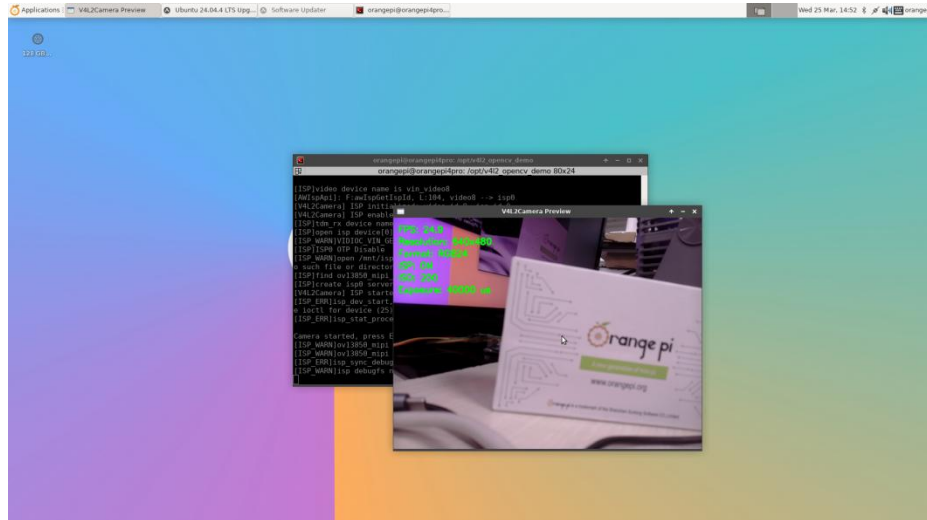
```
orange@orange:~$ cd /opt/v4l2_opencv_demo/
orange@orange:/opt/v4l2_opencv_demo$ make clean && make
```

- b. After compilation, the **cam_test** executable file will be generated in the current directory.

```
orange@orange:/opt/v4l2_opencv_demo$ ls cam_test
cam_test
```

- c. Then execute the **cam_test** program to open the MIPI camera (**note that the camera node should be selected according to the actual situation**):

```
orange@orange:/opt/v4l2_opencv_demo$ ./cam_test /dev/video0  #Open CAM1
orange@orange:/opt/v4l2_opencv_demo$ ./cam_test /dev/video8  #Open CAM2
```



- d. The camera resolution can be set by modifying the following code in the **test_camera.cpp** file. After modification, please note that the code needs to be recompiled.

```
int main(int argc, char* argv[])
{
    const char* device = "/dev/video8";
    int width = 640;
    int height = 480;
```

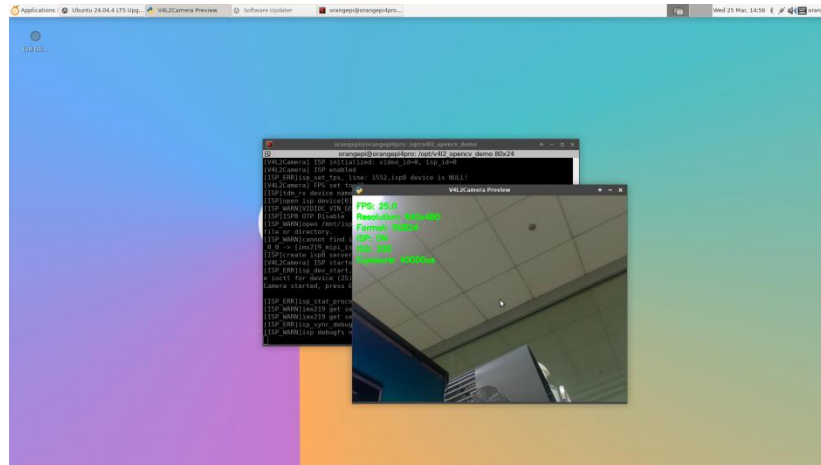
- 8) The steps to open a MIPI camera using a Python example are as follows:

- a. First, execute the following command to compile the **python_module**

```
orangeapi@orangeapi:~$ cd /opt/v4l2_opencv_demo/
orangeapi@orangeapi:/opt/v4l2_opencv_demo$ make python_module
```

- b. Then execute the test_camera.py script to open the MIPI camera.

```
orangeapi@orangeapi:/opt/v4l2_opencv_demo$ python3 test_camera.py -d /dev/video0 #Open CAM1
orangeapi@orangeapi:/opt/v4l2_opencv_demo$ python3 test_camera.py -d /dev/video8 #Open CAM2
```



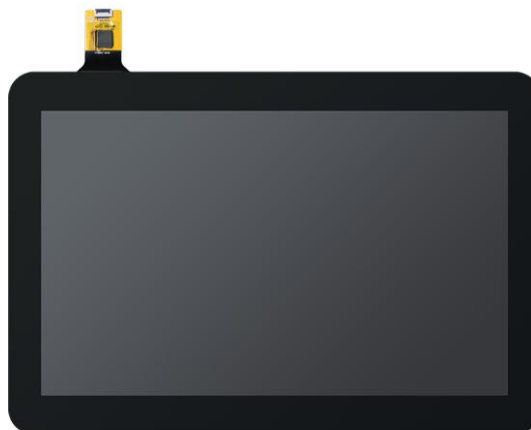
- c. The camera resolution can be set by modifying the following code in the test_camera.py file:

```
parser.add_argument('--width', '-W', type=int, default=640,
                    help='Image width (default: 640)')
parser.add_argument('--height', '-H', type=int, default=480,
                    help='Image height (default: 480)')
```

3. 32. 10.1 inch MIPI LCD screen usage method

3. 32. 1. Assembly method for 10.1-inch MIPI screen

- 1) First, prepare the necessary accessories
 - a. 10.1-inch MIPI LCD display screen+touch screen



- b. Screen adapter board+31pin to 40pin ribbon cable



c. 30pin MIPI ribbon cable



d. 12pin touchscreen cable



2) Connect the 12pin touch screen ribbon cable, 31pin to 40pin ribbon cable, and 30pin MIPI ribbon cable to the screen adapter board according to the following diagram. **Note that the blue insulation surface of the touch screen ribbon cable is facing downwards**, and the insulation surfaces of the other two ribbon cables are facing upwards. If connected incorrectly, it may cause problems such as no display or inability to touch



3) Place the adapter board connected to the ribbon cable on top of the MIPI LCD screen according to the following diagram, and connect the MIPI LCD screen to the adapter board through a 31 pin to 40 pin ribbon cable



- 4) Then connect the touch screen to the adapter board through a 12 pin touch screen cable, paying attention to the orientation of the insulation surface



- 5) Finally, connect the 30pin MIPI ribbon cable to the LCD interface of the development board



3. 32. 2. Method for configuring a 10.1-inch MIPI LCD screen

- 1) The default configuration for Linux images is not to enable the mipi LCD screen. If



you need to use the mipi LCD screen, you need to manually enable it.

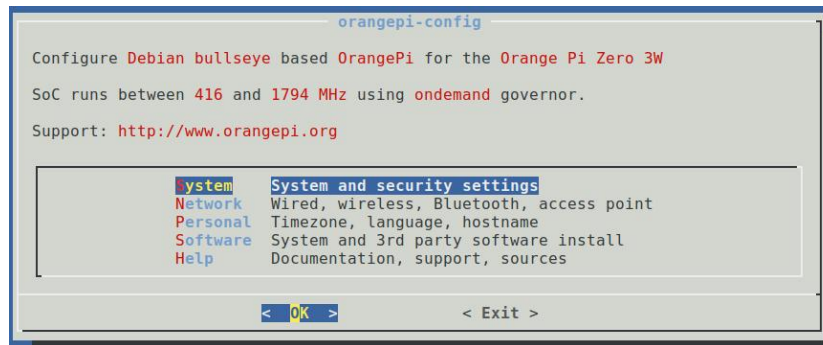
2) The interface of the mipi LCD screen on the development board is shown in the following figure:

3) The method to open mipi LCD configuration is as follows:

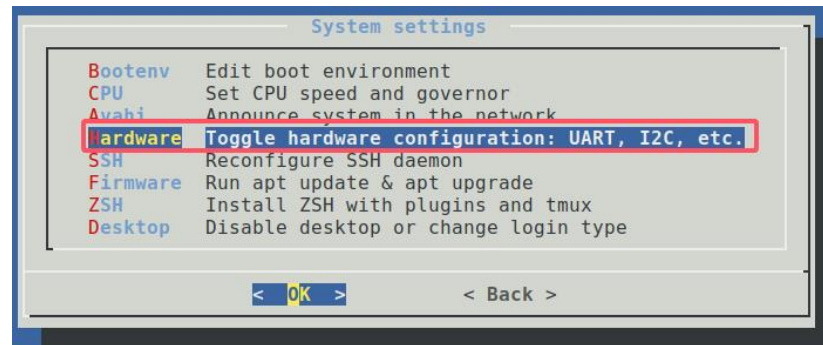
- a. First, run **orangepi-config**. Regular users should remember to add **sudo** privileges

```
orangepi@orangepi:~$ sudo orangepi-config
```

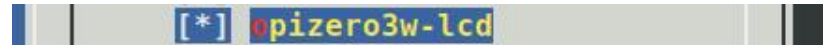
- b. Then select **System**



- c. Then select **Hardware**



- d. Then use the directional keys on the keyboard to locate the **opizero3w-lcd**, and use a **space** to select it



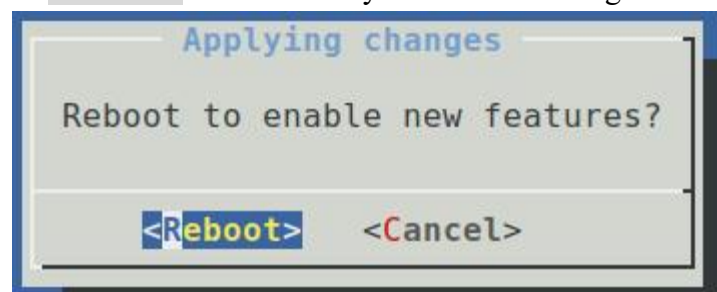
- e. Then select **<Save>** to save



- f. Then select **<Back>**



- g. Then select **<Reboot>** to restart the system for the configuration to take effect



The above settings will eventually add **overlays=opizero3w-lcd** in **/boot/orangepiEnv.txt**. After setting it up, you can check it first. If this line of configuration does not exist, then there is a problem with the settings.

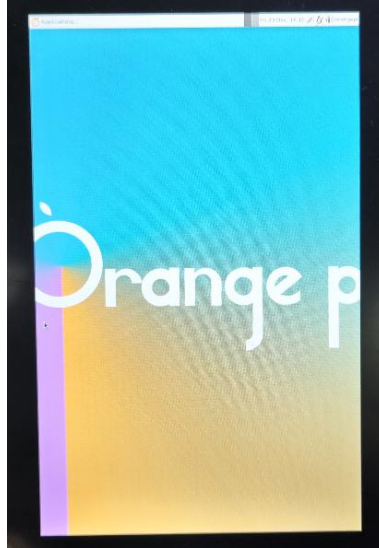
If you find it troublesome to use **orangepi-config**, you can also use the Vim editor to open **/boot/orangepiEnv.txt** and add the **overlays=opizero3w-lcd** configuration line.

```
orangepi@orangepi:~$ cat /boot/orangepiEnv.txt | grep "lcd"
```

```
overlays=opizero3w-lcd    #Example configuration
```

4) Then restart the Linux system

5) After restarting, you can see the following display on the LCD screen (default is vertical):



3. 32. 3. Method for Rotating Display Direction of Server Version Mirrors

1) Adding the configuration sentence '**extraargs=fbcon=rotate: direction to rotate**' in **/boot/orangepiEnv.txt** can set the direction displayed by the server version of the Linux system, where **fbcon=rotate:** The number after it can be set to:

- a. 0: Normal screen (default is vertical screen)
- b. 1: Rotate 90 degrees clockwise
- c. 2: Flip 180 degrees
- d. 3: Clockwise turn 270 degrees

```
orangepi@orangepi:~$ sudo vim /boot/orangepiEnv.txt
overlays=lcd
extraargs=fbcon=rotate:3
```

2) Then **restart** the Linux system and you will see that the direction of the LCD screen display has rotated

3. 33. Instructions for Hard Debugging Test of Debian Bulleye System

Note that currently only the Debian bullseye system supports hard cracking, and other versions of the system do not currently support it.

1) Both VLC player and Gstreamer support hardware decoding, and the currently tested formats are as follows:



- a. Video hardware decoding - HEVC
- b. Video hardware decoding - H.264
- c. Video hardware decoding - VP8
- d. Video hardware decoding - VP9
- e. Video hardware decoding - MPEG2

2) Firstly, prepare a test video, which can be downloaded from the following website or from the official tool's Videos directory.

<http://bbb3d.renderfarming.net/download.html>

3) Then open the command-line terminal on the desktop of the Debian system and run the gstreamer command below.

```
orangeipi@orangeipi:~$ gst-play-1.0 bbb_sunflower_1080p_60fps_normal.mp4
```

4) Alternatively, you can use the following command.

```
orangeipi@orangeipi:~$ gst-launch-1.0 filesrc location=bbb_sunflower_1080p_60fps_normal.mp4 ! qtdemux ! h264parse ! omxh264dec ! xvimagesink
```

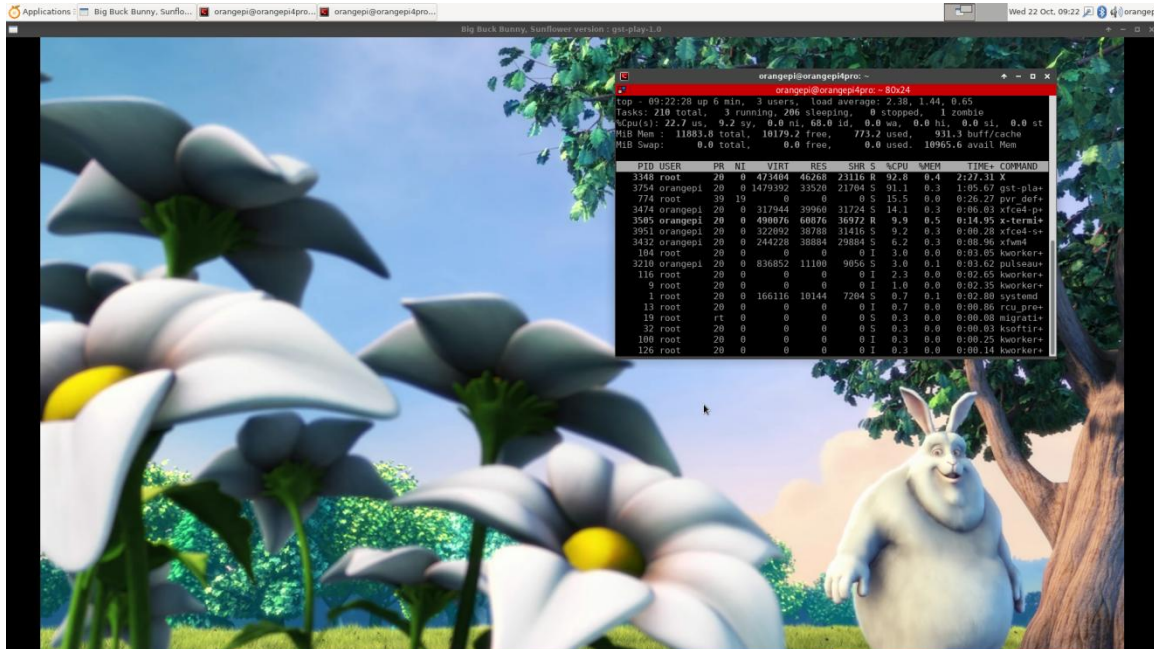
5) If using VLC player, simply right-click on the video file and select VLC player to open.

6) You can verify whether hardware decoding is called by using the following command. If you see an increasing number of interrupts, it means that hardware decoding is already being used.

```
orangeipi@orangeipi:~# cat /proc/interrupts |grep cedar_dev
```

```
522:      185      0      0      0      0      0      0      0 wakeupgen 49 Level cedar_dev
524:       0      0      0      0      0      0      0      0 wakeupgen 48 Level cedar_dev_ve2
```

7) In the case of hardware decoding, executing the htop command can result in lower CPU usage.



3. 34. GPU Testing Instructions for Debian Bullseye System

Note that currently only the Debian bullseye system supports GPU acceleration, while other versions of the system do not currently support it.

1) First, open the command-line terminal on the desktop of the Debian system, and then run **glmark2-es2**. Choose one of the following commands:

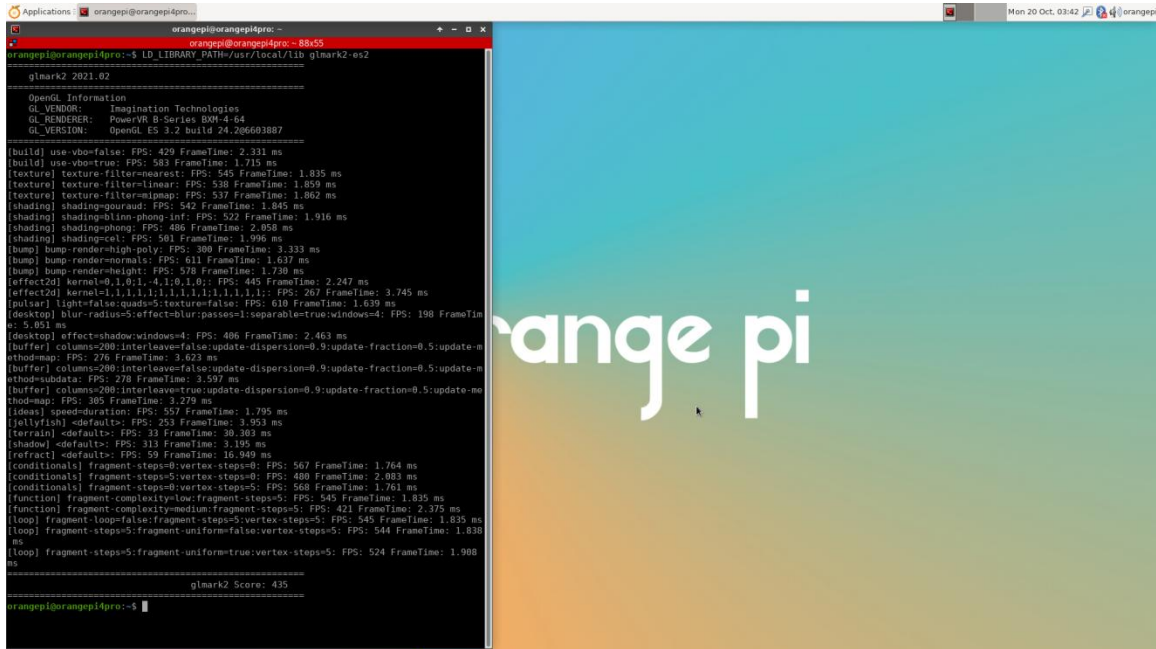
Note that **the following command needs to be run on the desktop.**

```
orangepi@orangepi:~$ LD_LIBRARY_PATH=/usr/local/lib glmark2-es2
```

```
orangepi@orangepi:~$ LD_LIBRARY_PATH=/usr/local/lib glmark2-es2 --off-screen
```

2) **glmark2-es2** is a benchmark testing tool for OpenGL (ES) 2.0, which can be used to test the performance of GPU OpenGL ES 2.0

a. The test scores for the **glmark2-es2** command are as follows

```

orange@orange-pi:~$ LD_LIBRARY_PATH=/usr/lib glmark2-es2
glmark2 2021.02

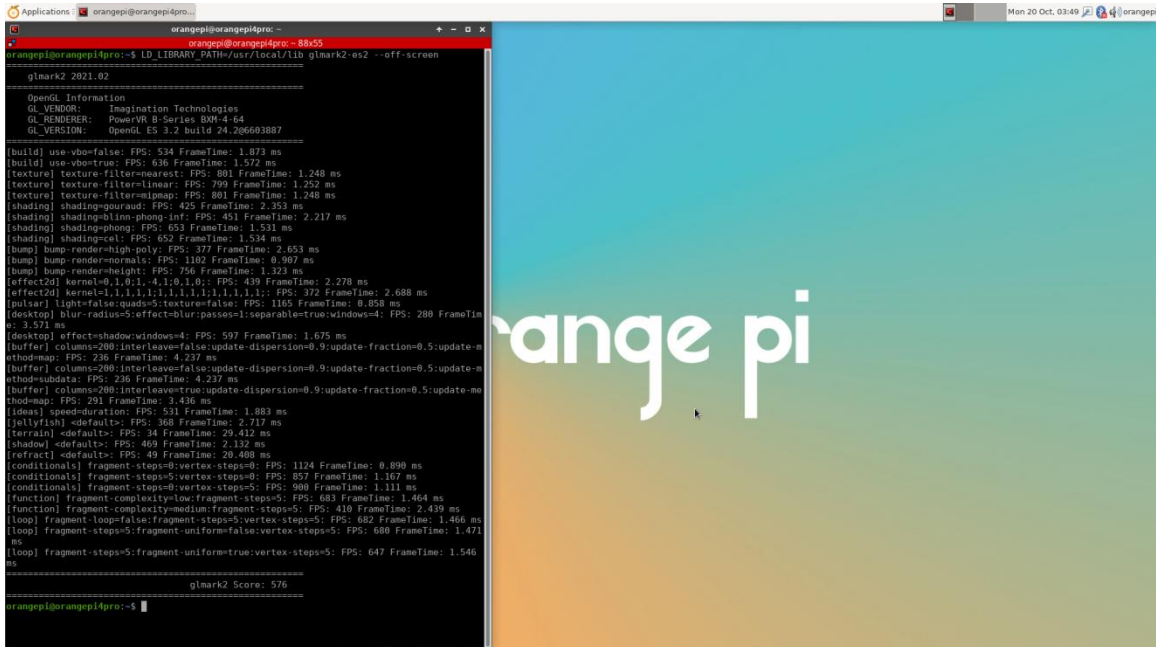
OpenGL Information
GL_VENDOR:    Imagination Technologies
GL_RENDERER:  PowerVR 8-Series D00-4-64
GL_VERSION:   OpenGL ES 3.2 build 24.206603887

[build] use-vertex=false: FPS: 429 FrameTime: 2.331 ms
[build] use-vertex=true: FPS: 583 FrameTime: 1.715 ms
[texture] texture-filter-nearest: FPS: 545 FrameTime: 1.835 ms
[texture] texture-filter-linear: FPS: 538 FrameTime: 1.859 ms
[texture] texture-filter-mipmap: FPS: 537 FrameTime: 1.862 ms
[shading] shading-gouraud: FPS: 542 FrameTime: 1.845 ms
[shading] shading-blend: FPS: 522 FrameTime: 1.916 ms
[shading] shading-phong: FPS: 486 FrameTime: 2.058 ms
[shading] shading-cel: FPS: 501 FrameTime: 1.996 ms
[bump] bump-render-high-poly: FPS: 308 FrameTime: 3.333 ms
[bump] bump-render-normal: FPS: 611 FrameTime: 1.637 ms
[bump] bump-render-height: FPS: 578 FrameTime: 1.730 ms
[effect2d] kernel=0,1,0,1,1,4,1,0,1,0: FPS: 445 FrameTime: 2.247 ms
[effect2d] kernel=1,1,1,1,1,1,1,1,1,1,1,1: FPS: 267 FrameTime: 3.745 ms
[pulsar] light=false: FPS: 610 FrameTime: 1.639 ms
[desktop] blur-radius=5: FPS: 198 FrameTime: 5.461 ms
[desktop] effect-shadow-windows=4: FPS: 406 FrameTime: 2.463 ms
[buffer] columns=200: FPS: 226 FrameTime: 3.623 ms
[buffer] columns=200: FPS: 278 FrameTime: 3.597 ms
[buffer] columns=200: FPS: 305 FrameTime: 3.279 ms
[ideas] speed-duration: FPS: 557 FrameTime: 1.795 ms
[jellyfish] <default>: FPS: 283 FrameTime: 3.903 ms
[terrain] <default>: FPS: 33 FrameTime: 30.383 ms
[shadow] <default>: FPS: 313 FrameTime: 3.195 ms
[refract] <default>: FPS: 59 FrameTime: 16.949 ms
[conditionals] fragment-steps=0: FPS: 567 FrameTime: 1.764 ms
[conditionals] fragment-steps=5: FPS: 480 FrameTime: 2.083 ms
[conditionals] fragment-steps=10: FPS: 568 FrameTime: 1.761 ms
[function] fragment-complexity=low: FPS: 545 FrameTime: 1.835 ms
[function] fragment-complexity=medium: FPS: 421 FrameTime: 2.375 ms
[loop] fragment-loop=false: FPS: 545 FrameTime: 1.835 ms
[loop] fragment-loop=true: FPS: 544 FrameTime: 1.838 ms
[loop] fragment-steps=5: FPS: 524 FrameTime: 1.988 ms

glmark2 Score: 435

```

b. The test scores for the **glmark2-es2 --off-screen** command are as follows



```

orange@orange-pi:~$ LD_LIBRARY_PATH=/usr/lib glmark2-es2 --off-screen
glmark2 2021.02

OpenGL Information
GL_VENDOR:    Imagination Technologies
GL_RENDERER:  PowerVR 8-Series D00-4-64
GL_VERSION:   OpenGL ES 3.2 build 24.206603887

[build] use-vertex=false: FPS: 534 FrameTime: 1.873 ms
[build] use-vertex=true: FPS: 636 FrameTime: 1.572 ms
[texture] texture-filter-nearest: FPS: 801 FrameTime: 1.248 ms
[texture] texture-filter-linear: FPS: 799 FrameTime: 1.252 ms
[texture] texture-filter-mipmap: FPS: 801 FrameTime: 1.248 ms
[shading] shading-gouraud: FPS: 425 FrameTime: 2.353 ms
[shading] shading-blend: FPS: 451 FrameTime: 2.217 ms
[shading] shading-phong: FPS: 653 FrameTime: 1.531 ms
[shading] shading-cel: FPS: 652 FrameTime: 1.534 ms
[bump] bump-render-high-poly: FPS: 377 FrameTime: 2.653 ms
[bump] bump-render-normal: FPS: 1102 FrameTime: 0.987 ms
[bump] bump-render-height: FPS: 756 FrameTime: 1.323 ms
[effect2d] kernel=0,1,0,1,1,4,1,0,1,0: FPS: 439 FrameTime: 2.278 ms
[effect2d] kernel=1,1,1,1,1,1,1,1,1,1,1,1: FPS: 372 FrameTime: 2.688 ms
[pulsar] light=false: FPS: 1165 FrameTime: 0.858 ms
[desktop] blur-radius=5: FPS: 280 FrameTime: 3.571 ms
[desktop] effect-shadow-windows=4: FPS: 597 FrameTime: 1.675 ms
[buffer] columns=200: FPS: 226 FrameTime: 4.237 ms
[buffer] columns=200: FPS: 236 FrameTime: 4.237 ms
[buffer] columns=200: FPS: 291 FrameTime: 3.436 ms
[ideas] speed-duration: FPS: 531 FrameTime: 1.883 ms
[jellyfish] <default>: FPS: 308 FrameTime: 2.717 ms
[terrain] <default>: FPS: 34 FrameTime: 29.412 ms
[shadow] <default>: FPS: 469 FrameTime: 2.132 ms
[refract] <default>: FPS: 49 FrameTime: 29.468 ms
[conditionals] fragment-steps=0: FPS: 1124 FrameTime: 0.898 ms
[conditionals] fragment-steps=5: FPS: 857 FrameTime: 1.167 ms
[conditionals] fragment-steps=10: FPS: 980 FrameTime: 1.111 ms
[function] fragment-complexity=low: FPS: 683 FrameTime: 1.464 ms
[function] fragment-complexity=medium: FPS: 410 FrameTime: 2.439 ms
[loop] fragment-loop=false: FPS: 682 FrameTime: 1.466 ms
[loop] fragment-loop=true: FPS: 680 FrameTime: 1.471 ms
[loop] fragment-steps=5: FPS: 647 FrameTime: 1.546 ms

glmark2 Score: 576

```




3. 35. Instructions for using NPU

3. 35. 1. PC Development Environment Configuration

3. 35. 1. 1. Installing Docker Environment

1) Firstly, install the Docker software on your Ubuntu computer. After installing Docker, if you can see the version number using the `docker -v` command, it means the installation is successful.

```
$ sudo apt update
$ sudo apt install -y docker.io
$ docker -v
```

2) Then download the ACUITY docker compressed file from [Quanzhi Netdisk](#) and unzip it.

```
$ unzip docker_images_v2.0.x.zip
```

3) Then execute the following command to import the Docker image containing the All Star NPU development environment.

```
$ cd docker_images_v2.0.x
$ unzip ubuntu-npu_v2.0.10.tar.zip
$ sudo docker load -i ubuntu-npu_v2.0.10.tar
```

4) Then download the `ai-sdk.tar.gz` compressed file from the official tool on the Orange Pi website and extract it to the `docker_data` directory.

```
$ mkdir docker_data
$ tar -xvf ai-sdk.tar.gz -C docker_data
```

5) Then create and enter the Docker container.

```
$ cd docker_data
$ sudo docker run --ipc=host -itd -v ${PWD}:/workspace --name allwinner_v2.0.10
ubuntu-npu:v2.0.10 /bin/bash
$ sudo docker exec -it allwinner_v2.0.10 /bin/bash
```




3.35.1.2. Model Conversion and Quantification

1) Before starting the model transformation, it is necessary to configure the model compilation script

```
# cd /workspace/ai-sdk/models
# source env.sh v3
# cp ../scripts/* .
```

2) Multiple conversion examples of framework models are provided in the **ai-sdk/models** directory. Taking the **yolov5s** model as an example, we will introduce the usage of the model deployment script.

3) The directory structure of **yolov5s**:

```
# tree yolov5s-sim
yolov5s-sim
|-- channel_mean_value.txt
|-- convert_export.sh
|-- dataset.txt
|-- images
|   |-- COCO_train2014_0000000000529.jpg
|   |-- COCO_train2014_0000000001183.jpg
|   |-- COCO_train2014_0000000002349.jpg
|   |-- COCO_train2014_0000000003685.jpg
|   |-- COCO_train2014_0000000004463.jpg
|   `-- dog.jpg
|-- inference_compare.sh
|-- inputs_outputs.txt
`-- yolov5s-sim.onnx      #The onnx model file of yolov5s has fixed the input size to (1,3640640)

1 directory, 12 files
```

4) Generate NBG files with uint8 quantization.

a. Import the model.

```
# ./pegasus_import.sh yolov5s-sim
```

b. Modify the **scale** of the yolov5s-sim_inputmeta.yml file to 0.00392157



```
# vim yolov5s-sim/yolov5s-sim_inputmeta.yml
```

c. Execute quantization, default to using uint8 quantization.

```
# ./pegasus_quantize.sh yolov5s-sim uint8
```

d. Run simulation verification.

```
# ./pegasus_inference.sh yolov5s-sim uint8
```

e. Export NBG model files available for NPU

```
# ./pegasus_export_ovx.sh yolov5s-sim uint8
```

f. After execution, the path of the quantized NBG file is as follows:

```
yolov5s-sim/wksp/yolov5s-sim_uint8_nbg_unify/network_binary.nb
```

3. 35. 2. On-board Model Deployment

3. 35. 2. 1. Preparation of On-board Environment

1) Firstly, the development board needs to install OpenCV and CMAKE

```
orangeypi@orangeypi:~$ sudo apt update
```

```
orangeypi@orangeypi:~$ sudo apt install libopencv-dev cmake
```

2) Then download the **ai-sdk.tar.gz** compressed file from the official tool on the Orange Pi website.

3) Then upload the compressed file to the development board and decompress it.

```
orangeypi@orangeypi:~$ tar -xvf ai-sdk.tar.gz
```

4) After decompressing the compressed file, the directory structure is as follows.

```
orangeypi@orangeypi:~$ cd ai-sdk
```

```
orangeypi@orangeypi:~/ai-sdk$ ls
```

```
examples machinfo Makefile models scripts tests tools unified-android
unified-tina viplite-android viplite-tina
```

3. 35. 2. 2. Running yolov5 Object Detection Example

1) First, compile the yolov5 example.

```
orangeypi@orangeypi:~/ai-sdk$ cd examples/yolov5
```

```
orangeypi@orangeypi:~/ai-sdk/examples/yolov5$ mkdir build
```

```
orangeypi@orangeypi:~/ai-sdk/examples/yolov5$ cd build
```



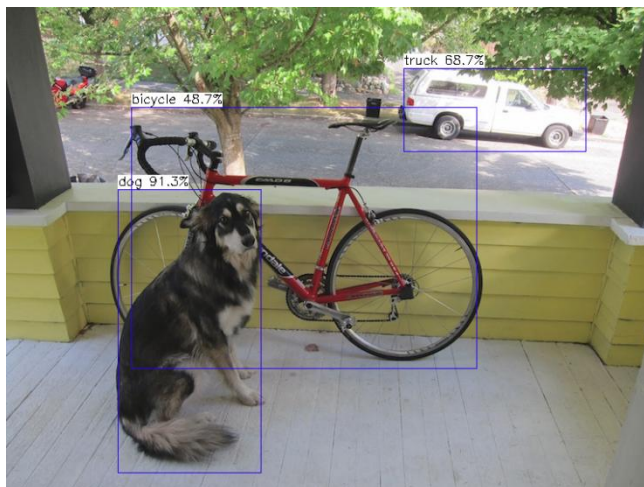
```
orangeapi@orangeapi:~/ai-sdk/examples/yolov5/build$ cmake ..  
orangeapi@orangeapi:~/ai-sdk/examples/yolov5/build$ make
```

2) Then execute the following command to run the example.

```
orangeapi@orangeapi:~/ai-sdk/examples/yolov5/build$ ./yolov5 ../model/v3/yolov5.nb ../input_data/dog.jpg ./yolov5 nbg input  
VIPLite driver software version 2.0.3.2-AW-2024-08-30  
yolov5_preprocess.cpp run.  
yolov5_postprocess.cpp run.  
detection num: 3  
16: 87%, [ 134, 216, 308, 552], dog  
2: 54%, [ 463, 77, 693, 170], car  
1: 49%, [ 160, 131, 562, 423], bicycle
```

3) The output result will be saved as the result. png in the current directory

```
orangeapi@orangeapi:~/ai-sdk/examples/yolov5/build$ ls result.png  
result.png
```



3. 35. 2. 3. Running resnet50 Image Classification Example

1) First, compile the resnet50 example.

```
orangeapi@orangeapi:~/ai-sdk$ cd examples/resnet50  
orangeapi@orangeapi:~/ai-sdk/examples/resnet50$ mkdir build  
orangeapi@orangeapi:~/ai-sdk/examples/resnet50$ cd build  
orangeapi@orangeapi:~/ai-sdk/examples/resnet50/build$ cmake ..
```



```
orange@orange:~/ai-sdk/examples/resnet50/build$ make
```

2) Then execute the following command to run the example.

```
orange@orange:~/ai-sdk/examples/resnet50/build$ ./resnet50 ../model/v3/resnet50.nb ../input_data/dog_224_224.jpg
```

3) The output result information is shown below, which displays the top 5 predicted by the model, with the most likely category being collie

```
orange@orange:~/ai-sdk/examples/resnet50/build$ ./resnet50 ../model/v3/resnet50.nb ../input_data/dog_224_224.jpg
...
===== top5 =====
class id: 231, prob: 10.750525, label: collie
class id: 160, prob: 10.616143, label: Afghan hound, Afghan
class id: 169, prob: 10.481762, label: borzoi, Russian wolfhound
class id: 230, prob: 10.347381, label: Shetland sheepdog, Shetland sheep dog, Shetland
class id: 176, prob: 6.987841, label: Saluki, gazelle hound
class_postprocess success.
```

3.35.2.4. Running struct2depth deep detection example

1) First, compile the struct2depth example

```
orange@orange:~/ai-sdk$ cd examples/struct2depth
orange@orange:~/ai-sdk/examples/struct2depth$ mkdir build
orange@orange:~/ai-sdk/examples/struct2depth$ cd build
orange@orange:~/ai-sdk/examples/struct2depth/build$ cmake ..
orange@orange:~/ai-sdk/examples/struct2depth/build$ make
```

2) Then execute the following command to run the example

```
orange@orange:~/ai-sdk/examples/struct2depth/build$ ./struct2depth -b ../model/v3/struct2depth.nb -i ../input_data/0015.jpg
```

3) The depth information of model inference will be saved in jpg and txt files.

```
orange@orange:~/ai-sdk/examples/struct2depth/build$ ls disp_* output_*
disp_color.jpg  disp_show.jpg  output_1.txt  output_3.txt
disp_gray.jpg   output_0.txt   output_2.txt
```



3. 35. 2. 5. Running transformer_cls image classification example

1) First, compile the transformer_cls example.

```
orange@orange:~/ai-sdk$ cd examples/transformer_cls
orange@orange:~/ai-sdk/examples/transformer_cls$ mkdir build
orange@orange:~/ai-sdk/examples/transformer_cls$ cd build
orange@orange:~/ai-sdk/examples/transformer_cls/build$ cmake ..
orange@orange:~/ai-sdk/examples/transformer_cls/build$ make
```

2) Then execute the following command to run the example.

```
orange@orange:~/ai-sdk/examples/transformer_cls/build$ ./transformer_cls ./model/v3/transformer_cls.nb ./input_data/cat_224_224.jpg
```

3) The output results of image classification are shown below.

```
orange@orange:~/ai-sdk/examples/transformer_cls/build$ ./transformer_cls ./model/v3/transformer_cls.nb ./input_data/cat_224_224.jpg
./transformer_cls nbg input
VIPLite driver software version 2.0.3.2-AW-2024-08-30
get jpeg success.
trans data success.
awnn_set_input_buffers success.
awnn_run success.
class_postprocess.cpp run.
===== top5 =====
class id: 281, prob: 8.171444, label: tabby, tabby cat
class id: 285, prob: 6.455214, label: Egyptian cat
class id: 282, prob: 6.139260, label: tiger cat
class id: 287, prob: 3.832908, label: lynx, catamount
class id: 756, prob: 3.263942, label: rain barrel
class_postprocess success.
```

3. 35. 2. 6. Running yolact object detection segmentation example

1) First, compile the yolact example.



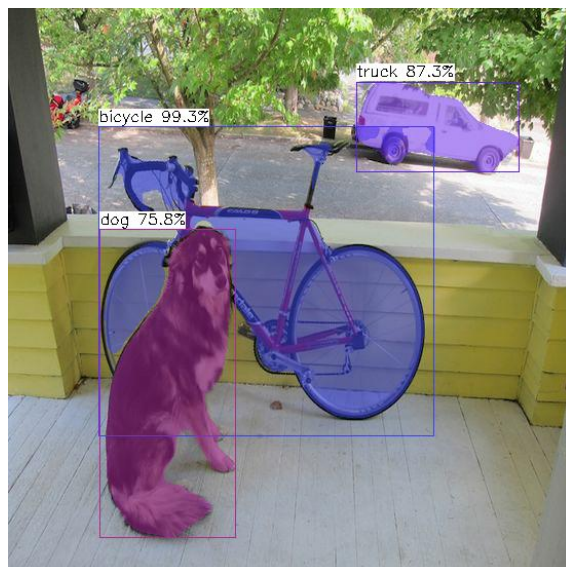
```
orangepi@orangepi:~/ai-sdk$ cd examples/yolact
orangepi@orangepi:~/ai-sdk/examples/yolact$ mkdir build
orangepi@orangepi:~/ai-sdk/examples/yolact$ cd build
orangepi@orangepi:~/ai-sdk/examples/yolact/build$ cmake ..
orangepi@orangepi:~/ai-sdk/examples/yolact/build$ make
```

2) Then execute the following command to run the example.

```
orangepi@orangepi:~/ai-sdk/examples/yolact/build$ ./yolact ../model/v3/yolact.nb ../input_data/dog_550_550.jpg
```

3) The output result will be saved as the result.png in the current directory

```
orangepi@orangepi:~/ai-sdk/examples/yolact/build$ ls result.png
result.png
```



3. 35. 2. 7. Running deepspeech2 Speech Recognition Example

1) First, compile the DeepSpeech2 example.

```
orangepi@orangepi:~/ai-sdk$ cd examples/deepspeech2
orangepi@orangepi:~/ai-sdk/examples/deepspeech2$ mkdir build
orangepi@orangepi:~/ai-sdk/examples/deepspeech2$ cd build
orangepi@orangepi:~/ai-sdk/examples/deepspeech2/build$ cmake ..
orangepi@orangepi:~/ai-sdk/examples/deepspeech2/build$ make
```



2) Then execute the following command to run the example.

```
orange@orange:~/ai-sdk/examples/deepspeech2/build$ ./deepspeech2 ../model/v3/deepspeech2.nb ../input_data/1188-133604-0010.flac_756_161_1.tensor
```

3) The output result is shown below.

```
orange@orange:~/ai-sdk/deepspeech2/build$ ./deepspeech2 ../model/v3/deepspeech2.nb ../input_data/1188-133604-0010.flac_756_161_1.tensor
input tensor_file = ../input_data/1188-133604-0010.flac_756_161_1.tensor
VIPLite driver software version 2.0.3.2-AW-2024-08-30
Original array: but in thiss vvingyet copiiedd ffrom turrnerr yoou haavve the ttwo
princiipleess brrougth oout perrfeccttllly
Modified array : but in this vingyet copied from turner you have the two principles brought out
perfectly
```

3.35.2.8. Running chineseocr Text Recognition Example

1) First, compile the chineseocr example.

```
orange@orange:~/ai-sdk$ cd examples/chineseocr
orange@orange:~/ai-sdk/examples/chineseocr$ mkdir build
orange@orange:~/ai-sdk/examples/chineseocr$ cd build
orange@orange:~/ai-sdk/examples/chineseocr/build$ cmake ..
orange@orange:~/ai-sdk/examples/chineseocr/build$ make
```

2) Then execute the following command to run the example.

```
orange@orange:~/ai-sdk/examples/chineseocr/build$ ./chineseocr -d ../model/v3/-1 dbnet_1024 -2 angle_net -3 crnn_lite_lstm_256 -4 keys.txt -i ../input_data/1.jpg
```

3) The output result is shown below, and it can be seen that the text in the image has been recognized.

```
orange@orange:~/ai-sdk/examples/chineseocr/build$ ./chineseocr -d ../model/v3/-1 dbnet_1024 -2 angle_net -3 crnn_lite_lstm_256 -4 keys.txt -i ../input_data/1.jpg
...
=====End detect=====
FullDetectTime(903.447417ms)
We are at Quanzhi Technology
AI
chip
take off 12345666666 !
```




```
run finished.
```

```
~CrnnNet.
```

```
~AngleNet.
```

```
~DbNet.
```

```
~NpuUint.
```

3. 36. Method for burning Linux images to eMMC

Note that the development board can be launched through TF card or eMMC, with TF card having higher priority than eMMC. That is to say, if the development board is inserted with a TF card and there is a system in the TF card, the system in the TF card will be started by default instead of the system in eMMC.

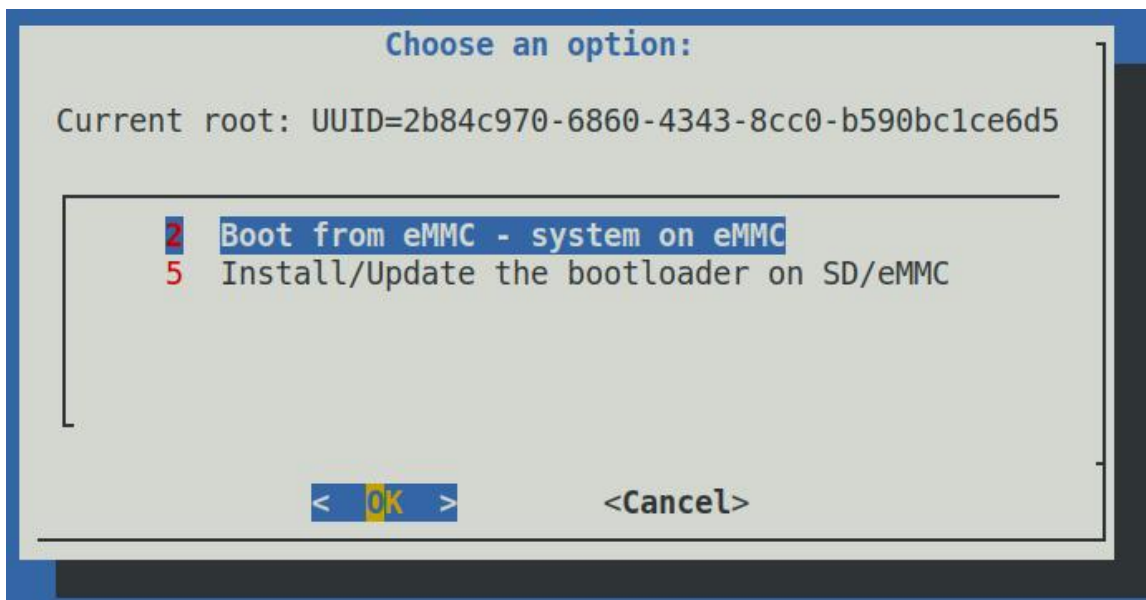
Before performing the following operations, please ensure that the eMMC module is affixed to the development board.

1) Burning a Linux image to eMMC requires the use of a TF card. First, burn the Linux image onto the TF card, and then start the development board to enter the Linux system

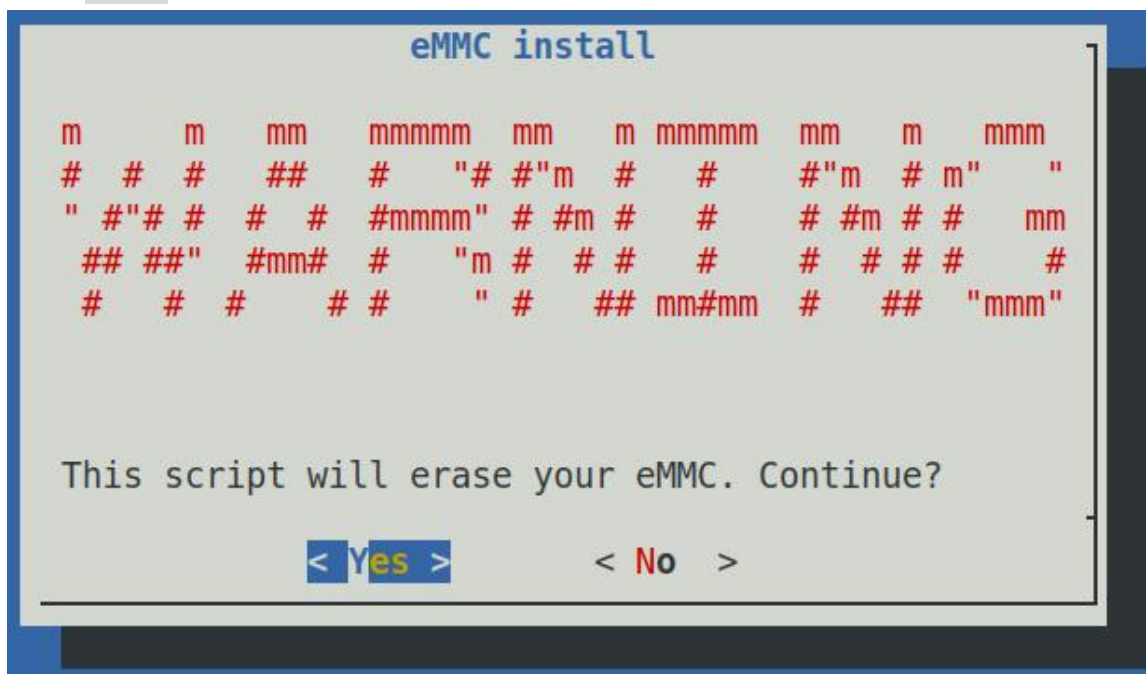
2) Then run the **nand-sata-install** script, **remember to add sudo privileges**

```
orangeypi@orangeypi:~$ sudo nand-sata-install
```

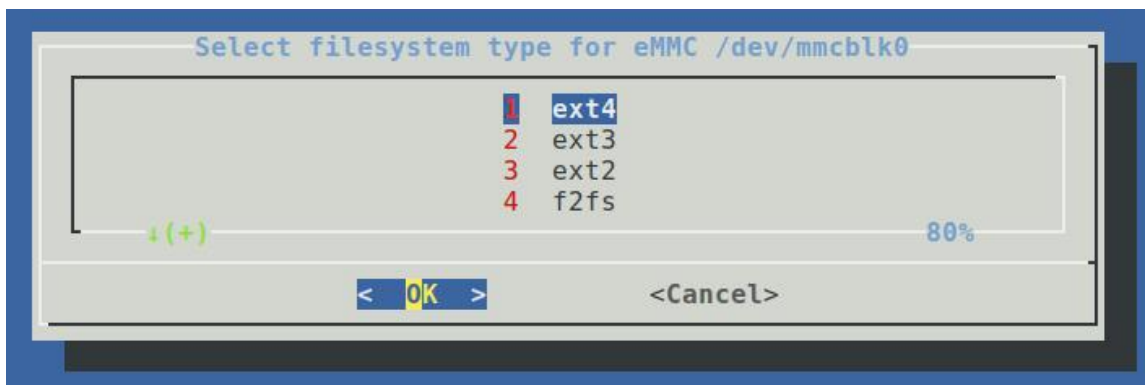
3) Then select **2 Boot from eMMC - system on eMMC**



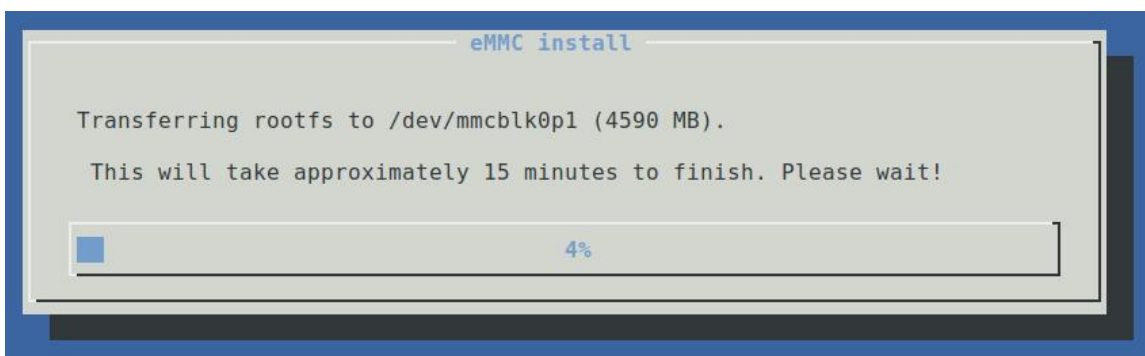
4) Then a warning will pop up, and the script will erase all data on eMMC. Select **<Yes>** to continue



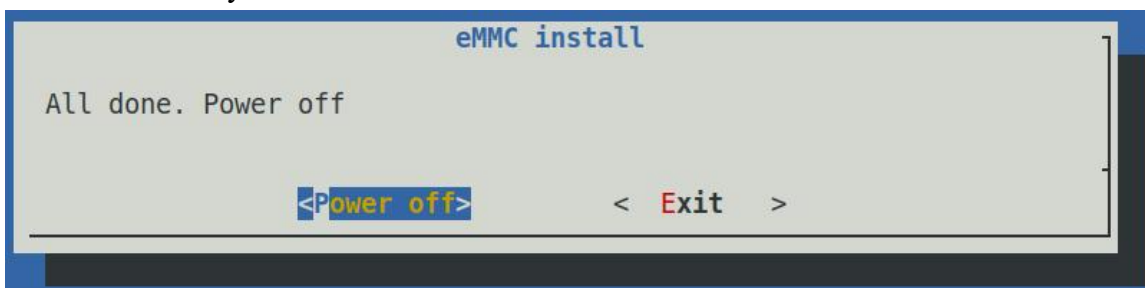
5) Then it will prompt to select the type of file system, supporting five file systems: ext2/3/4, f2fs, and btrfs



6) Then it will start formatting eMMC, and after formatting eMMC, it will start burning Linux images into eMMC



7) After burning, the following options will be prompted, you can choose **<Power off>** to shut down directly



8) Then unplug the TF card and power it on again, and the Linux system in eMMC will start up

3. 37. Method for burning Linux images to UFS

Note that the development board can be booted through TF card or UFS, with



TF card having higher priority than UFS. That is to say, if the development board is inserted with a TF card and there is a system in the TF card, the system in the TF card will be started by default instead of the system in UFS.

Before performing the following operations, please ensure that the development board is labeled with UFS.

1) Burning a Linux image to UFS requires the use of a TF card. First, burn the Linux image onto the TF card, and then start the development board to enter the Linux system

2) Then create a partition for UFS, making sure to confirm the device nodes of UFS.

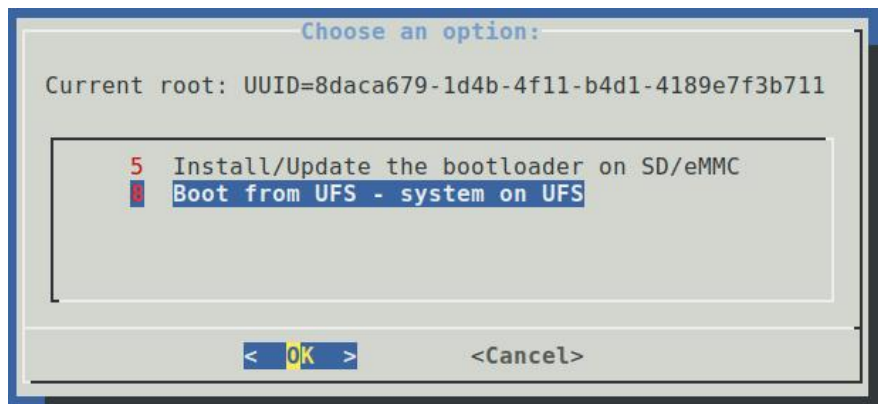
Here is **sda**

```
orangepi@orangepi:~$ sudo parted --script /dev/sda mklabel msdos mkpart primary ext4 65536s 100%
```

3) Then run the **nand-sata-install** script, **remember to add sudo privileges**

```
orangepi@orangepi:~$ sudo nand-sata-install
```

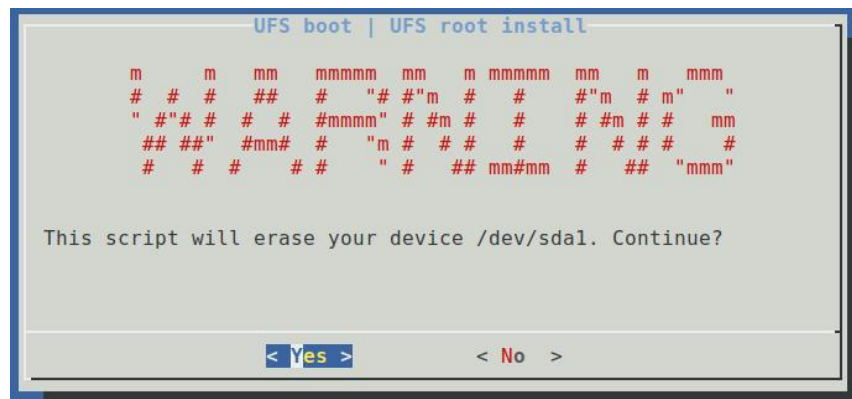
4) Then select **8 Boot from UFS - system on UFS**



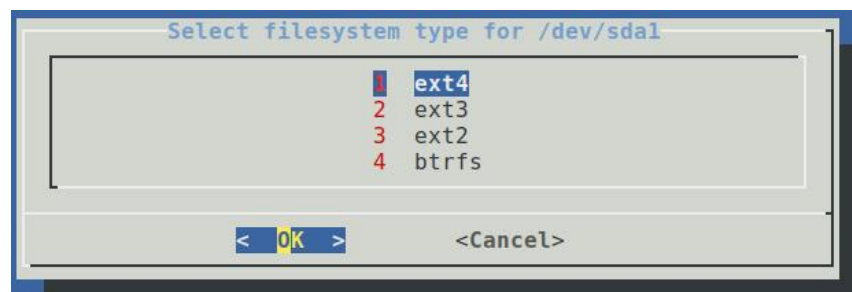
5) Select **<OK>** to continue



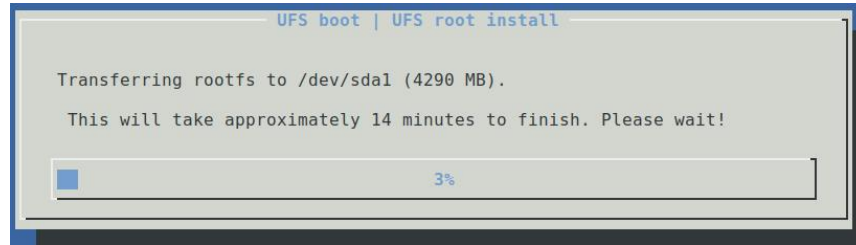
6) Then a warning will pop up, and the script will erase all data on UFS. Select **<Yes>** to continue



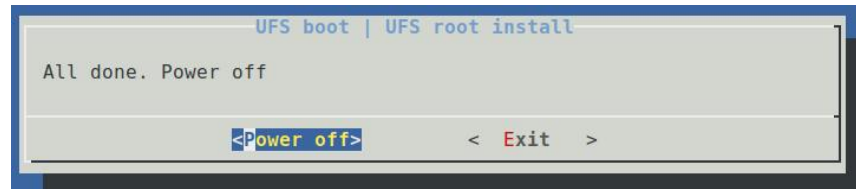
7) Then it will prompt to select the type of file system, supporting five file systems: ext2/3/4, f2fs, and btrfs



8) Then it will start formatting UFS, and after formatting UFS, it will start burning Linux images into UFS



9) After burning, the following options will be prompted, you can choose **<Power off>** to shut down directly



10) Then unplug the TF card and power it on again, and the Linux system in UFS will start up

3. 38. Instructions for using the system backup script

opi-bking

opi-bking is a system backup script designed for the Orange Pi series development board, which can help users fully backup their currently running Linux system to an image file for subsequent recovery or migration to other devices.

Before starting the backup, please ensure that there is enough space left in the system, otherwise the backup may fail

1) Run the **opi-bking** script to start backing up the system.

```
orange@orange:~$ sudo opi-bking
```

2) If there is no **opi-bking** script in the system, you can download it from the following repository by yourself.

<https://gitee.com/orangepi-xunlong/opi-bking>

3) The script will check the root file system size and create a properly sized image file, then use the **rsync** command to copy system data to the image file. The specific process



is as follows.

```
orangepi@orangepi:~$ sudo opi-bking
[ o.k. ] Starting backup to image [ /mnt/backup.img ]
[ o.k. ] Current rootfs size [ 2561 MiB ]
[ o.k. ] Creating blank image for rootfs [ 3500 MiB ]
[ .... ] dd: 3.42GiB [ 126MiB/s] [=====] 100%
[ o.k. ] Creating partitions [ root: ext4 ]
[ .... ] Creating rootfs [ ext4 on /dev/loop1p1 ]
[ .... ] Copying file to [ / ] [#####] 100%
[ .... ] Re-enabling [ orangepi-resize-filesystem ]
[ o.k. ] Writing U-boot bootloader [ /dev/loop1 ]
[ o.k. ] Unmounting [ /mnt/opi-bking.iaioMi/rootfs ]
[ o.k. ] Backup completed [ /mnt/backup.img ]
orangepi@orangepi:~$
```

4) When the script displays the message '**Backup completed**', it indicates that the backup has ended. The image generation path is shown below,

```
orangepi@orangepi:~$ ls /mnt/
backup.img
```

5) **backup.img** is the image file that can be burned.

3. 39. Usage of Linux Overlayboot

In some user scenarios, the system often shuts down by directly cutting off power instead of executing the poweroff command, which may cause file system damage and prevent normal startup. To prevent such issues, it is recommended to enable the overlay root feature and mount the root file system as read-only. This ensures that even in the event of an abnormal power outage, the original file system is not affected, thereby improving the stability and reliability of the system.

Overlayboot is a technology based on Overlayfs that can mount the Linux root file system (/) as read-only and provide writable storage through an overlay. All user modifications (such as file writes, configuration changes, etc.) will be redirected to the writable layer, while the original root file system remains unchanged, making it suitable for scenarios such as embedded devices and servers that require high system stability.

Overlayboot consists of two layers: a read-only layer (lowerdir) and a writable layer (upperdir). The read-only layer is the original root file system (/), mounted in read-only mode; The writable layer is used to store user modified data, which is



merged with the read-only layer to form the final root file system view, making users unaware of the hierarchical structure of the underlying layer during operation.

The writable layer can be stored in tmpfs or block devices. When using tmpfs, all modifications are stored in memory and lost after restart, and the system returns to its original state; When using block devices such as USB drives and TF card partitions, data can be stored persistently and will not be lost after restarting.

3. 39. 1. Configure Overlayboot to tmpfs mode

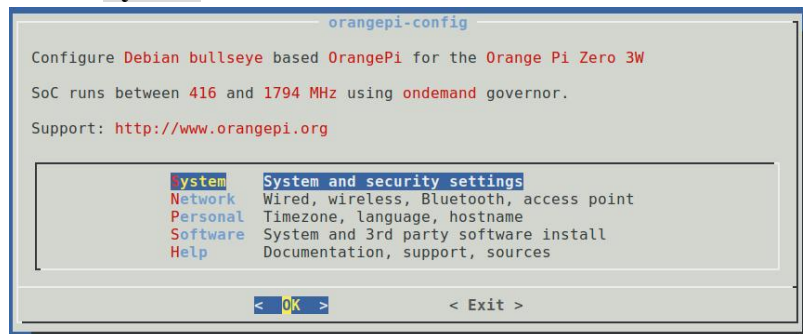
In this mode, all modifications made by the system will be stored in memory (tmpfs). After restarting, all changes will be lost and the system will return to its initial state.

1) Overlayboot is turned off by default in Linux systems and needs to be manually turned on to use. The opening steps are as follows:

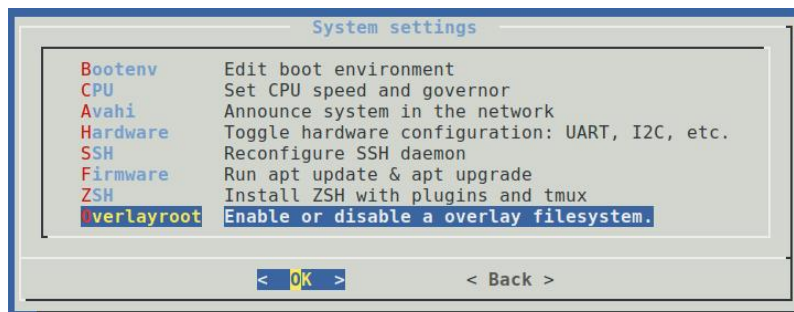
- a. Firstly, please ensure that the development board is connected to a wired or wireless network. For specific methods, please refer to the [Network Connection Testing](#) section
- b. Then run **orangepi-config**, remember to add **sudo** privileges for regular users

```
orangepi@orangepi:~$ sudo orangepi-config
```

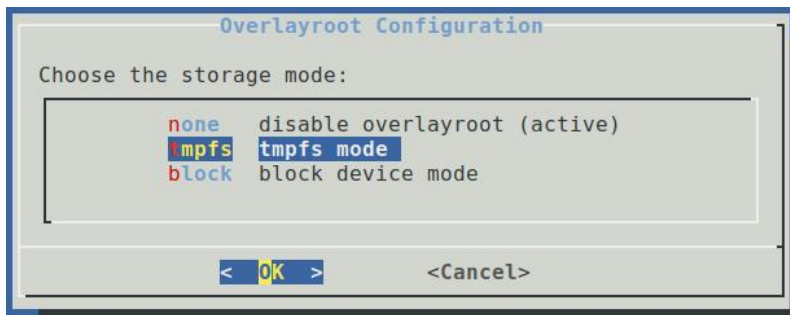
- c. Then select **System**



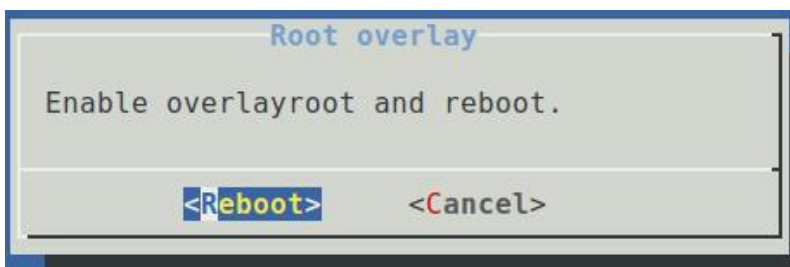
- d. Then select **Overlayroot**



- e. Then select **tmpfs** and press enter



- f. Then select **<Reboot>** to restart the system for the configuration to take effect



- 2) Then enter the **mount | grep root** command to see the mounting information of the file system

```
orange@orange:~$ mount |grep root
/dev/mmcblk0p1 on /media/root-ro type ext4 (ro,relatime)
tmpfs-root on /media/root-rw type tmpfs (rw,relatime)
overlayroot on / type overlay (rw,relatime,lowerdir=/media/root-ro,upperdir=/media/root-rw/overlay,workdir=/media/root-rw/overlay-workdir/_ )
overlayroot on /var/log.hdd type overlay (rw,relatime,lowerdir=/media/root-ro,upperdir=/media/root-rw/overlay,workdir=/media/root-rw/overlay-workdir/_ )
orange@orange:~$
```

From the mounting information, it can be seen that the file system partition/dev/mmcblkXp1 is mounted in read-only mode to/media/root ro, and tmpfs is mounted as a writable layer to/media/root rw, indicating that the overlay root configuration has taken effect.

Due to tmpfs being a memory file system, all modifications will be lost after restarting.

3. 39. 2. Configure Overlayboot to block device storage mode

In this mode, the writable layer uses specific storage devices (such as TF card partition/dev/mmcblkXp2), and data will not be lost after restarting.

Please note that after opening Overlayboot, modifications to the/boot partition (such as updating the kernel or modifying/boot/orangepiEnv. txt) will not take effect. The reason is that in block device mode, modifications are actually made to/boot in the overlay layer, while Uboot reads files from the real partition at startup, so the modifications will not take effect. To make the modifications effective, it is necessary to disable Overlayboot before performing these operations.



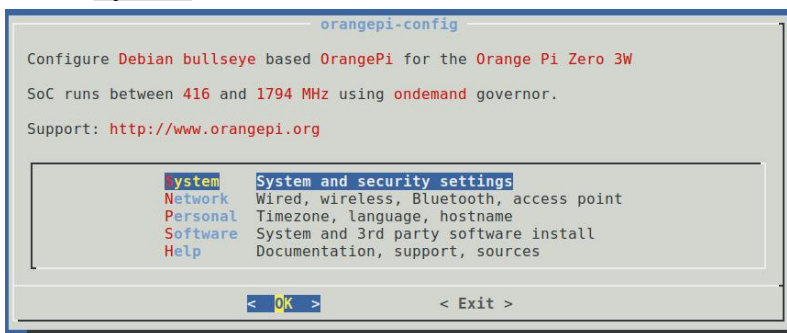
1) Overlayboot is turned off by default in Linux systems and needs to be manually turned on to use. The opening steps are as follows:

- a. Firstly, please ensure that the development board is connected to a wired or wireless network. For specific methods, please refer to the [Network Connection Testing](#) section

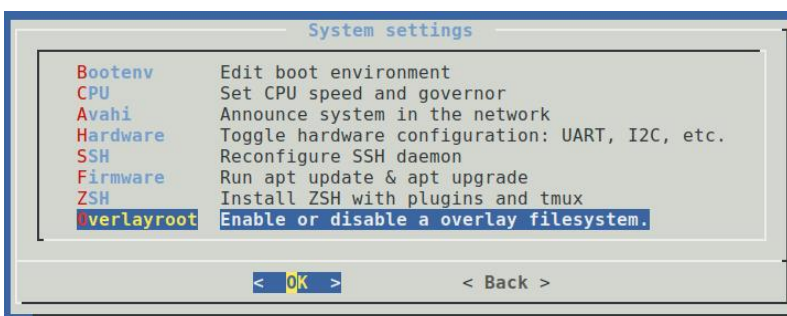
- a. Then run **orange-pi-config**, remember to add **sudo** privileges for regular users

```
orange-pi@orange-pi:~$ sudo orange-pi-config
```

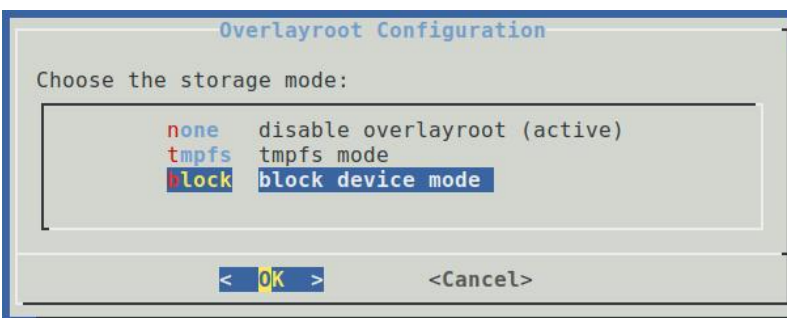
- b. Then select **System**



- c. Then select **Overlayroot**



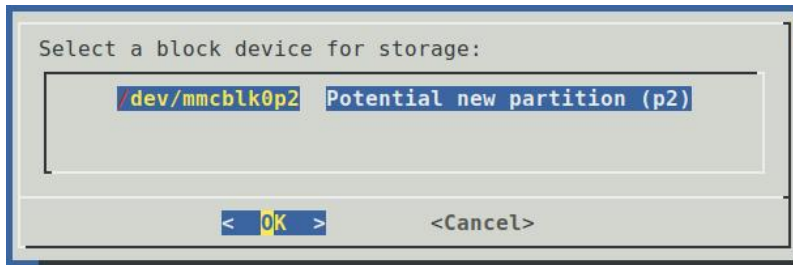
- d. Then select **block** and press enter



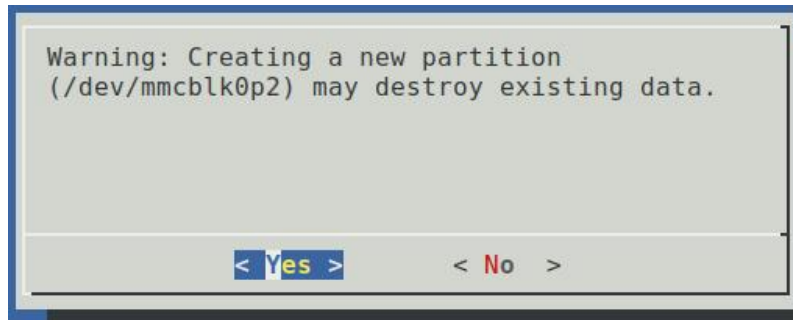
- e. Then an Overlay writable layer partition will be created. You need to select the storage device where the **Overlay writable layer** partition is located. If no other USB devices are inserted into the development board, there is only one option by default, which means creating a partition in the TF card. The size of the partition



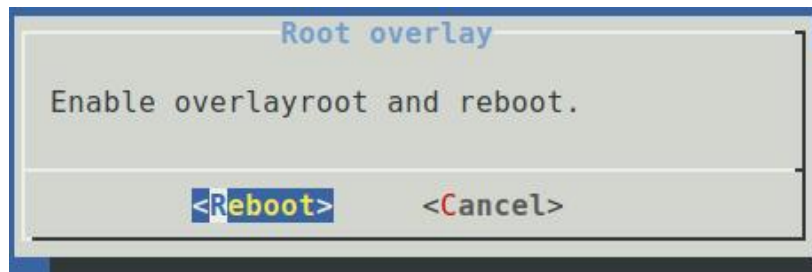
is: the total capacity of TF minus twice the capacity occupied by rootfs



- f. Then select **<Yes>** to automatically create and format partitions



- g. Then select **<Reboot>** to restart the system for the configuration to take effect



- 2) Then enter the **mount | grep root** command to see the mounting information of the file system.

```
orange@orange:~$ mount |grep root
/dev/mmcblk0p1 on /media/root-ro type ext4 (ro,relatime)
/dev/mmcblk0p2 on /media/root-rw type ext4 (rw,relatime)
overlayroot on / type overlay (rw,relatime,lowerdir=/media/root-ro,upperdir=/media/root-rw/overlay,workdir=/media/root-rw/overlay-workdir/_ )
overlayroot on /var/log.hdd type overlay (rw,relatime,lowerdir=/media/root-ro,upperdir=/media/root-rw/overlay,workdir=/media/root-rw/overlay-workdir/_ )
orange@orange:~$
```

。 From the mounting information, it can be seen that the file system partition/dev/mmcblkXp1 is mounted in read-only mode to/media/root-ro, and/dev/mmcblkXp2 is mounted as a writable layer to/media/root-rw, indicating that the overlay root configuration has taken effect.

- 3) Because all user modifications are redirected to the **/media/root-rw** directory, it is possible to restore factory settings by deleting all content unde **/media/root-rw/**(which requires a restart to take effect).



```
orange_pi@orange_pi:~$ sudo rm -rf /media/root-rw/*
orange_pi@orange_pi:~$ reboot
```

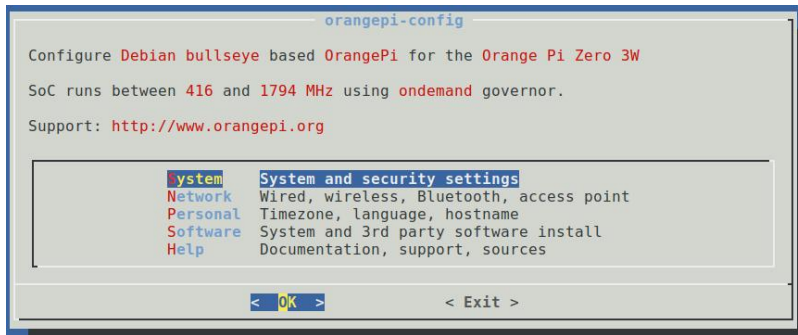
3.39.3. Methods to disable Overlayboot

1) The steps to disable Overlayboot are as follows:

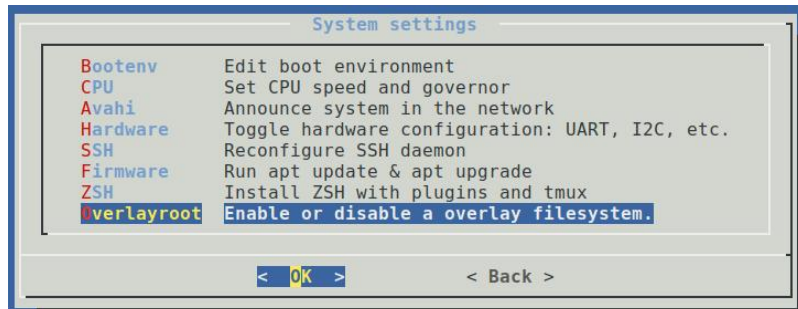
- a. First, run **orange_pi-config**. Regular users should remember to add **sudo** privileges

```
orange_pi@orange_pi:~$ sudo orange_pi-config
```

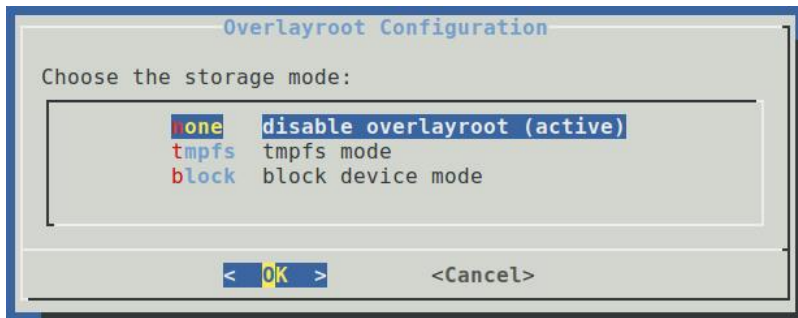
- b. Then select **System**



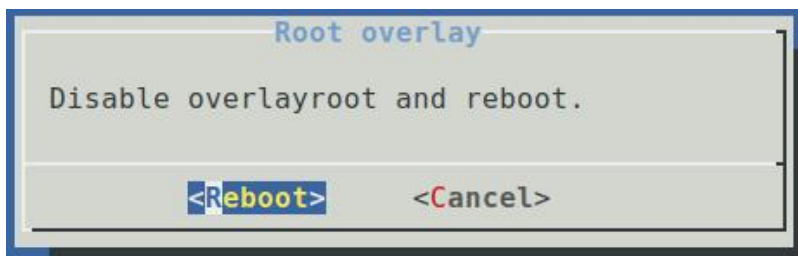
- c. Then select **Overlayroot**



- d. Then select **none** and press enter



- e. Then select **<Reboot>** to restart the system for the configuration to take effect



2) If you find using orangepi-config cumbersome, you can also follow the steps below.

- a. Firstly, it is necessary to re mount the **root-ro** directory in writable mode.

```
orangepi@orangepi:~$ sudo mount -o remount,rw /media/root-ro
```

- b. Then modify the **/etc/overlayroot.conf** file under the read-only partition, set the overlayroot variable to empty, and restart the system.

```
orangepi@orangepi:~$ sudo vim /media/root-ro/etc/overlayroot.conf
```

```
overlayroot=""
```

3.39.4. Method of backing up overlayroot image through opi-bking

The above method is only applicable to local operation of a single device and may cause damage to the system in some cases. It is recommended to use the method in this section to generate an overlay root supported image through backup, which can be easily burned to other devices. Before starting the backup, please ensure that the overlay root function is not currently enabled in the system. If it is enabled, please disable it according to the method in the previous section.

opi-bking is a system backup script designed for the Orange Pi series development board, which can help users fully backup their currently running Linux system to an image file for subsequent recovery or migration to other devices.

Before starting the backup, please ensure that there is enough space left in the system, otherwise the backup may fail.

- 1) Run the **opi-bking** script to start backing up the system, making sure to specify the **overlayfs** parameter.

```
orangepi@orangepi:~$ sudo opi-bking overlayfs
```

- 2) If there is no **opi-bking** script in the system, you can download it from the following repository by yourself.

<https://gitee.com/orangepi-xunlong/opi-bking>



3) The script will check the root file system size and create a properly sized image file, then use the **rsync** command to copy system data to the image file. The specific process is as follows.

```
orange@orange:~$ sudo opi-bking overlays
[ o.k. ] Starting backup to image [ /mnt/backup.img ]
[ o.k. ] Current rootfs size [ 5331 MiB ]
[ o.k. ] Overlayfs size [ 256MB ]
[ o.k. ] Creating blank image for rootfs [ 7500 MiB ]
[ .... ] dd: 7.32GiB [ 224MiB/s] [=====] 100%
[ o.k. ] Creating partitions [ root: ext4, overlay: ext4 ]
[ .... ] Creating rootfs [ ext4 on /dev/loop0p1 ]
[ .... ] Creating overlayfs [ ext4 on /dev/loop0p2 ]
[ .... ] Copying 178549 file to [ / ] [#####] 100%
[ .... ] Re-enabling [ orangepi-resize-filesystem ]
[ .... ] Enabling [ setup-overlay ]
[ o.k. ] Writing U-boot bootloader [ /dev/loop0 ]
[ o.k. ] Backup completed [ /mnt/backup.img ]
[ o.k. ] Unmounting [ /mnt/opi-bking.khPthG/rootfs ]
orange@orange:~$
```

4) When the script displays the message '**Backup completed**', it indicates that the backup is complete, and the image generation path is as follows.

```
orange@orange:~$ ls /mnt/
backup.img
```

5) **backup.img** is an image file that can be directly burned. After burning the image to a TF card or eMMC and powering it on, the system will automatically complete initialization. During this process, there will be a restart. After the system restarts, execute the `df -lh` command. If it can output information similar to the following image, it indicates that the system overlay root initialization is successful.

Filesystem	Size	Used	Avail	Use%	Mounted on
tmpfs	99M	4.6M	94M	5%	/run
/dev/mmcblk0p1	2.7G	2.1G	550M	80%	/media/root-ro
/dev/mmcblk0p2	25G	652K	25G	1%	/media/root-rw
overlayroot	25G	652K	25G	1%	/
tmpfs	491M	0	491M	0%	/dev/shm
tmpfs	5.0M	4.0K	5.0M	1%	/run/lock
tmpfs	491M	0	491M	0%	/tmp
/dev/zram1	47M	2.1M	42M	5%	/var/log
tmpfs	99M	0	99M	0%	/run/user/1000



3. 40. Methods for shutting down and restarting the development board

1) During the operation of the Linux system, if the power is directly unplugged and disconnected, it may cause the file system to lose some data. It is recommended to use the **poweroff** command to shut down the Linux system of the development board before unplugging the power

```
orangepi@orangepi:~$ sudo poweroff
```

2) Use the **reboot** command to restart the Linux system in the development board

```
orangepi@orangepi:~$ sudo reboot
```



4. Linux SDK——orange pi build usage instructions

4. 1. Compilation System Requirements

Linux SDK, **orange pi-build**, which only supports running on X64 computers with **Ubuntu 22.04** installed, so before downloading Orange pi build, please first ensure that the Ubuntu version installed on your computer is Ubuntu 22.04. The command to check the installed Ubuntu version on the computer is as follows. If the Release field does not display **22.04**, it means that the current Ubuntu version used does not meet the requirements. Please replace the system before performing the following operations.

```
test@test:~$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 22.04 LTS
Release:        22.04
Codename:       jammy
```

If the computer is installed with a Windows system and does not have Ubuntu 22.04 installed, you can consider using **VirtualBox** or **VMware** to install an Ubuntu 22.04 virtual machine on the Windows system. However, please note not to compile orange pi-build on the WSL virtual machine, as orange pi-build has not been tested on the WSLs virtual machine, so it cannot be guaranteed that orange pi-build can be used properly in WSLs. Additionally, please do not use orange pi-build on Linux systems on the **development board**. The installation image download address for Ubuntu 22.04 **amd64** version is:

<https://mirrors.tuna.tsinghua.edu.cn/ubuntu-releases/22.04/ubuntu-22.04-desktop-amd64.iso>

After installing Ubuntu 22.04 on your computer or virtual machine, please first set the software source for Ubuntu 22.04 to Qinghua Source (or other domestic sources that you think are faster), otherwise it is easy to encounter errors during software installation



due to network issues. The steps to replace Tsinghua Source are as follows:

- a. The method of replacing Tsinghua Source can refer to the instructions on this webpage.

<https://mirrors.tuna.tsinghua.edu.cn/help/ubuntu/>

- b. Note that Ubuntu version needs to be switched to 22.04.

Ubuntu 镜像使用帮助

Ubuntu 的软件源配置文件是 `/etc/apt/sources.list`。将系统自带的该文件做个备份，将该文件替换为下面内容，即可使用 TUNA 的软件源镜像。

选择你的ubuntu版本:

```
# 默认注释了源码镜像以提高 apt update 速度，如有需要可自行取消注释
deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy main restricted universe multiverse
# deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy main restricted universe multiverse
deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-updates main restricted universe multiverse
# deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-updates main restricted universe multiverse
deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-backports main restricted universe multiverse
# deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-backports main restricted universe multiverse
deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-security main restricted universe multiverse
# deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-security main restricted universe multiverse

# 预发布软件源，不建议启用
# deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-proposed main restricted universe multiverse
# deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-proposed main restricted universe multiverse
```

- c. The contents of the `/etc/apt/sources.list` file that needs to be replaced are:

```
test@test:~$ sudo mv /etc/apt/sources.list cat /etc/apt/sources.list.bak
test@test:~$ sudo vim /etc/apt/sources.list

# By default, the source code image has been annotated to improve the speed of apt updates. If necessary, you can
remove the annotation yourself

deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy main restricted universe multiverse
# deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy main restricted universe multiverse
deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-updates main restricted universe multiverse
# deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-updates main restricted universe multiverse
deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-backports main restricted universe multiverse
# deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-backports main restricted universe multiverse
deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-security main restricted universe multiverse
# deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-security main restricted universe multiverse

# Pre release software source, not recommended to enable
# deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-proposed main restricted universe multiverse
# deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-proposed main restricted universe multiverse
```

- d. After replacement, it is necessary to update the package information and ensure that there are no errors.



```
test@test:~$ sudo apt-get update
```

- e. In addition, since the kernel and U-boot source code are stored on GitHub, it is important to ensure that the computer can download the code from GitHub properly when compiling the image.

4. 2. Obtain the source code of Linux SDK

4. 2. 1. Download orangepi-build

The linux sdk refers to the orangepi-build code, which is modified based on the armbian build compilation system. Using orangepi-build, multiple versions of Linux images can be compiled. You can download the code for orangepi-build using the following command:

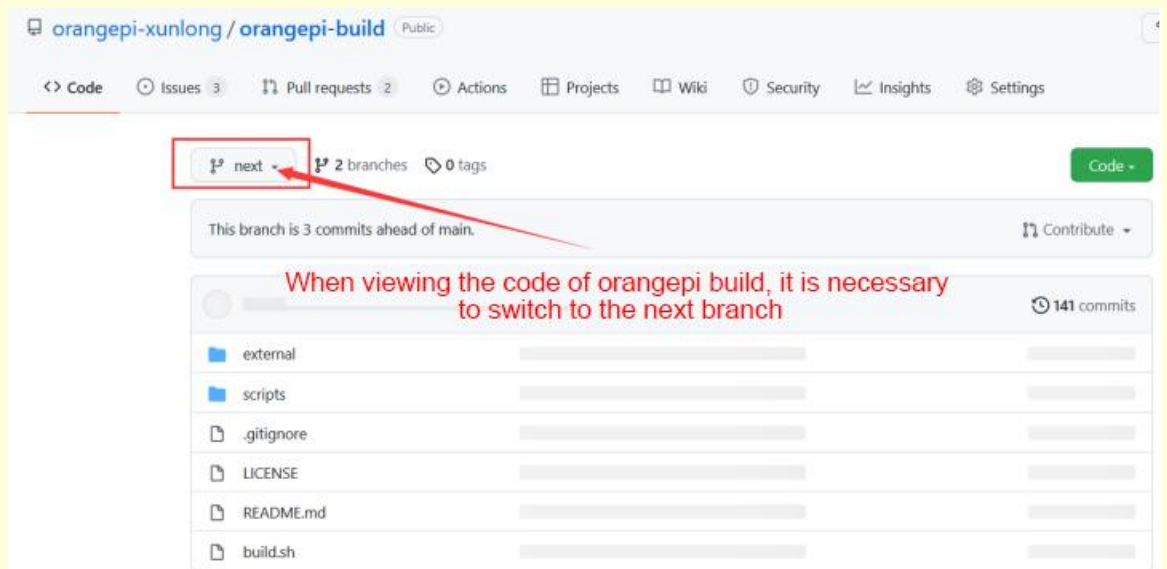
- a. The command downloaded from github is as follows:

```
test@test:~$ sudo apt-get update && sudo apt-get install -y git
test@test:~$ git clone https://github.com/orangepi-xunlong/orangepi-build.git -b next
```

- b. The command downloaded from gitee is as follows:

```
test@test:~$ sudo apt-get update && sudo apt-get install -y git
test@test:~$ git clone https://gitee.com/orangepi-xunlong/orangepi-build.git -b next
```

Note that using the A733 Soc development board requires downloading the **next** branch source code of orangepi-build. The git clone command above needs to specify the branch of orangepi-build source code as next.





Downloading the code for orangepi-build through the git clone command does not require entering the username and password of the github account (the same applies to downloading other code in this manual). If Ubuntu PC prompts for the username and password of the github account after entering the git clone command, it is usually due to an incorrect input of the address of the orangepi-build repository after git clone. Please carefully check the spelling of the command for errors, rather than thinking that we forgot to provide the username and password of the github account here.

The current u-boot and Linux kernel versions used by the A733 series development board are as follows:

branch	u-boot version	Linux kernel version
legacy	u-boot v2018.05	linux5.15
current	u-boot v2018.05	Linux6.6

The branch mentioned here and the branch of orangepi-build source code are not the same thing, please don't confuse them. This branch is mainly used to distinguish between different versions of kernel source code.

At present, the linux5.15 bsp kernel provided by Quanzhi is defined as the legacy branch, and the linux6.6 bsp kernel is defined as the current branch.

After downloading orangepi-build, it will include the following files and folders:

- build.sh**: Compile startup script
- external**: Contains configuration files required for compiling images, specific scripts, and source code for some programs, etc
- LICENSE**: GPL 2 License File
- README.md**: orangepi-build documentation
- scripts**: General script for compiling Linux images

```
test@test:~/orangepi-build$ ls
```

```
build.sh  external  LICENSE  README.md  scripts
```

If you download the code for orangepi-build from github, you may find that the orangepi-build does not include the source code for u-boot and Linux kernel, nor does it require a cross compilation toolchain to compile u-boot and Linux kernel.



This is normal because these things are stored in other separate github repositories or on certain servers (the addresses will be detailed below). **orange-pi-build** specifies the addresses of **u-boot**, **Linux kernel**, and **cross compilation toolchain** in scripts and configuration files. When running **orange-pi-build**, if it finds that these things are not available locally, it will automatically download them from the corresponding places

4. 2. 2. Download the cross compilation toolchain

When **orange-pi-build** runs for the first time, it automatically downloads the cross compilation **toolchains** and places it in the **toolchains** folder. After running the **build.sh** script of **orange-pi-build**, it checks whether all the cross compilation **toolchains** in **toolchains** exist. If they do not exist, it will restart the download. If they do exist, it will be used directly without repeated downloads.

```
[ o.k. ] Checking for external GCC compilers
[ .... ] downloading using http(s) network [ gcc-linaro-aarch64-none-elf-4.8-2013.11_linux.tar.xz ]
#8d7029 16MiB/24MiB(65%) CN:1 DL:7.9MiB ETA:1s]
[ o.k. ] Verified [ PGP ]
[ .... ] decompressing
[ .... ] gcc-linaro-aarch64-none-elf-4.8-2013.11_linux.tar.xz: 24.9MiB [14.4MiB/s] [=====] 100%
[ .... ] downloading using http(s) network [ gcc-linaro-arm-none-eabi-4.8-2014.04_linux.tar.xz ]
#e30eec 17MiB/33MiB(50%) CN:1 DL:10MiB ETA:1s]
[ o.k. ] Verified [ PGP ]
[ .... ] decompressing
[ .... ] gcc-linaro-arm-none-eabi-4.8-2014.04_linux.tar.xz: 33.9MiB [9.6MiB/s] [=====] 100%
[ .... ] downloading using http(s) network [ gcc-linaro-arm-linux-gnueabi-4.8-2014.04_linux.tar.xz ]
#041c24 48MiB/48MiB(99%) CN:1 DL:2.7MiB]
[ o.k. ] Verified [ PGP ]
[ .... ] decompressing
[ .... ] gcc-linaro-arm-linux-gnueabi-4.8-2014.04_linux.tar.xz: 48.8MiB [13.0MiB/s] [=====] 100%
[ .... ] downloading using http(s) network [ gcc-linaro-4.9.4-2017.01-x86_64_arm-linux-gnueabi.tar.xz ]
#3dee3e 72MiB/76MiB(93%) CN:1 DL:3.7MiB ETA:1s]
[ o.k. ] Verified [ MD5 ]
[ .... ] decompressing
[ .... ] gcc-linaro-4.9.4-2017.01-x86_64_arm-linux-gnueabi.tar.xz: 77.0MiB [14.2MiB/s] [=====] 100%
[ .... ] downloading using http(s) network [ gcc-linaro-7.4.1-2019.02-x86_64_arm-linux-gnueabi.tar.xz ]
#42e728 104MiB/104MiB(99%) CN:1 DL:2.8MiB]
[ o.k. ] Verified [ MD5 ]
[ .... ] decompressing
[ .... ] gcc-linaro-7.4.1-2019.02-x86_64_arm-linux-gnueabi.tar.xz: 104MiB [13.9MiB/s] [=====] 100%
[ .... ] downloading using http(s) network [ gcc-linaro-7.4.1-2019.02-x86_64_aarch64-linux-gnu.tar.xz ]
#2c065e 108MiB/111MiB(97%) CN:1 DL:3.9MiB]
[ o.k. ] Verified [ MD5 ]
[ .... ] decompressing
[ .... ] gcc-linaro-7.4.1-2019.02-x86_64_aarch64-linux-gnu.tar.xz: 111MiB [13.4MiB/s] [=====] 100%
[ .... ] downloading using http(s) network [ gcc-arm-9.2-2019.12-x86_64-arm-none-linux-gnueabi.tar.xz ]
#d232ee 250MiB/251MiB(99%) CN:1 DL:2.0MiB]
[ o.k. ] Verified [ MD5 ]
[ .... ] decompressing
[ .... ] gcc-arm-9.2-2019.12-x86_64-arm-none-linux-gnueabi.tar.xz: 251MiB [13.7MiB/s] [=====] 100%
[ .... ] downloading using http(s) network [ gcc-arm-9.2-2019.12-x86_64-aarch64-none-linux-gnu.tar.xz ]
#80b441 268MiB/269MiB(99%) CN:1 DL:0.9MiB]
[ o.k. ] Verified [ MD5 ]
[ .... ] decompressing
```

The image website of the cross compilation toolchain in China is the open source software mirror site of Tsinghua University:

https://mirrors.tuna.tsinghua.edu.cn/armbian-releases/_toolchain/

After downloading **toolchains**, multiple versions of cross compilation toolchains will be included:

```
test@test:~/orange-pi-build$ ls toolchains/
gcc-arm-11.2-2022.02-x86_64-aarch64-none-linux-gnu
gcc-linaro-4.9.4-2017.01-x86_64_aarch64-linux-gnu
gcc-linaro-7.4.1-2019.02-x86_64_arm-linux-gnueabi
gcc-arm-11.2-2022.02-x86_64-arm-none-linux-gnueabi
```



```
gcc-linaro-4.9.4-2017.01-x86_64_arm-linux-gnueabi
gcc-linaro-aarch64-none-elf-4.8-2013.11_linux
gcc-arm-9.2-2019.12-x86_64-aarch64-none-linux-gnu
gcc-linaro-5.5.0-2017.10-x86_64_arm-linux-gnueabihf
gcc-linaro-arm-linux-gnueabi-4.8-2014.04_linux
gcc-arm-9.2-2019.12-x86_64-arm-none-linux-gnueabi
gcc-linaro-7.4.1-2019.02-x86_64_aarch64-linux-gnu
gcc-linaro-arm-none-eabi-4.8-2014.04_linux
```

The cross compilation toolchain used to compile the A733 Linux kernel source code is:

- a. linux5.15

```
gcc-arm-11.2-2022.02-x86_64-aarch64-none-linux-gnu
```

The cross compilation toolchain used to compile the A733 u-boot source code is:

- a. v2018.05

```
gcc-linaro-7.4.1-2019.02-x86_64_arm-linux-gnueabi
```

4. 2. 3. Explanation of the complete directory structure of orangepi-build

2) After downloading the orangepi-build repository, it does not include the Linux kernel, u-boot source code, or cross compilation toolchain. The Linux kernel and u-boot source code are stored in separate Git repositories

- a. The git repository where the Linux kernel source code is stored is as follows:

- a) Github

```
https://github.com/orangepi-xunlong/linux-orangepi/tree/orange-pi-5.15-sun60iw2
```

- b) Gitee

```
https://gitee.com/orangepi-xunlong/orange-pi-5.15-sun60iw2
```

- b. The git repository where the u-boot source code is stored is as follows:

- a) Github

```
https://github.com/orangepi-xunlong/u-boot-orangepi/tree/v2018.05-sun60iw2
```

- b) Gitee

```
https://gitee.com/orangepi-xunlong/u-boot-orangepi/tree/v2018.05-sun60iw2
```

3) When orangepi-build is first run, it will download the cross compilation toolchain, u-boot, and Linux kernel source code. After successfully compiling the Linux image once,



the files and folders that can be seen in orangepi-build are:

- a. **build.sh**: Compile startup script
- b. **external**: Contains configuration files required for compiling the image, scripts for specific functions, and source code for some programs. The rootfs compressed file cached during the image compilation process is also stored in the external file
- c. **kernel**: Store the source code of the Linux kernel
- d. **LICENSE**: GPL 2 License File
- e. **README.md**: orangepi-build documentation
- f. **output**: Store compiled deb packages such as u-boot and Linux, compilation logs, and compiled images
- g. **scripts**: General script for compiling Linux images
- h. **toolchains**: Store cross compilation toolchain
- i. **u-boot**: Store the source code of u-boot
- j. **userpatches**: Store the configuration files required for compiling scripts

```
test@test:~/orangepi-build$ ls
build.sh  external  kernel  LICENSE  output  README.md  scripts  toolchains
u-boot   userpatches
```

4. 3. Compiling u-boot

If downloading code from GitHub is slow, you can do it at /Add the parameter **GITEE.SERVER=yes** after **build.sh**, so that it can be downloaded from the Gitee website during compilation.

The compilation command for adding the **GITEE.SERVER=yes** parameter is as follows:

```
test@test:~/orangepi-build$ sudo ./build.sh GITEE_SERVER=yes
```

- 1) Run the **build.sh** script, remember to grant **sudo** privileges

```
test@test:~/orangepi-build$ sudo ./build.sh
```

- 2) Next, select the model of the development board



```

Please choose a Board.
Choose an option
orangePi3      Allwinner H6 quad core 1GB/2GB RAM GBE WiFi/BT eMMC USB3
orangePi3-lts  Allwinner H6 quad core 2GB RAM GBE WiFi/BT-AW859A eMMC USB3
orangePiZero2  Allwinner H616 quad core 512MB/1GB RAM WiFi/BT GBE SPI
orangePiZero3  Allwinner H618 quad core 1GB/1.5GB/2GB/4GB RAM WiFi/BT GBE SPI
orangePiZero2w Allwinner H618 quad core 1GB/1.5GB/2GB/4GB RAM WiFi/BT SPI
orangePiZero3w Allwinner A733 octa core 4-16GB RAM GBE USB3 WiFi/BT
orangePi4      Rockchip RK3399 hexa core 4GB RAM GBE eMMC USB3 USB-C WiFi/BT
orangePi4a     Allwinner T527 octa core 2-4GB RAM GBE WiFi/BT NVMe eMMC

```

3) Select **U-boot package** and press Enter

```

Choose an option
Compile image | rootfs | kernel | u-boot
U-boot package
Kernel package
Rootfs and all deb packages
Full OS image for flashing

```

4) Then it will start compiling u-boot, and some of the information prompted when compiling the current branch is as follows:

a. Version of u-boot source code

```
[ o.k. ] Compiling u-boot [ v2018.05 ]
```

b. Version of cross compilation toolchain

```
[ o.k. ] Compiler version [ aarch64-linux-gnu-gcc 11 ]
```

c. The path of the compiled u-boot deb package

```
[ o.k. ] Target directory [ orangePi-build/output/debs/u-boot ]
```

d. The package name of the compiled u-boot deb package

```
[ o.k. ] File name [ linux-u-boot-current-orangepizero3w_x.x.x_arm64.deb ]
```

e. Compilation time used

```
[ o.k. ] Runtime [ 1 min ]
```

f. Repeat the command to compile u-boot, and use the following command to start compiling u-boot directly without selecting through the graphical interface

```
[ o.k. ] Repeat Build Options [ sudo ./build.sh BOARD=orangepizero3w
BRANCH=current BUILD_OPT=u-boot ]
```

5) View the compiled u-boot deb package

```
test@test:~/orangePi-build$ ls output/debs/u-boot/
linux-u-boot-current-orangepizero3w_x.x.x_arm64.deb
```



6) When the orangepi build compilation system compiles the U-boot source code, it first synchronizes the U-boot source code with the GitHub server's U-boot source code. Therefore, if you want to modify the U-boot source code, you first need to turn off the download and update function of the source code (**you need to compile the U-boot completely before turning off this function, otherwise you will prompt that the U-boot source code cannot be found**). Otherwise, the modifications made will be restored. The method is as follows:

Set the IGNOREUPDATES variable to "yes" in **userpatches/config-default.conf**

```
test@test:~/orangepi-build$ vim userpatches/config-default.conf
.....
IGNORE_UPDATES="yes"
.....
```

7) When debugging u-boot code, you can use the following method to update u-boot in the Linux image for testing

- a. Firstly, upload the compiled deb package of u-boot to the Linux system on the development board

```
test@test:~/orangepi-build$ cd output/debs/u-boot
test@test:~/orangepi_build/output/debs/u-boot$ scp \
linux-u-boot-current-orangepizero3w_x.x.x_arm64.deb root@192.168.1.xxx:/root
```

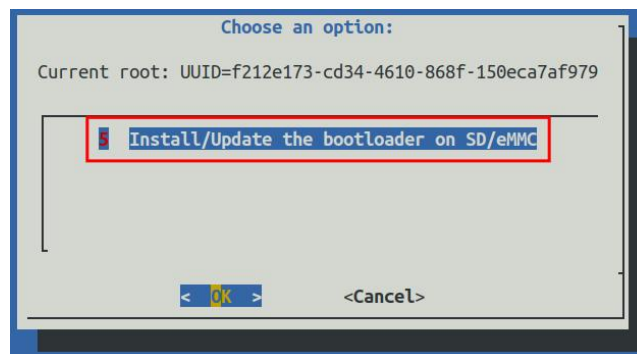
- b. Reinstall the newly uploaded deb package for u-boot

```
orangepi@orangepi:~$ sudo dpkg -i
linux-u-boot-current-orangepizero3w_x.x.x_arm64.deb
```

- c. Then run the nand sata install script

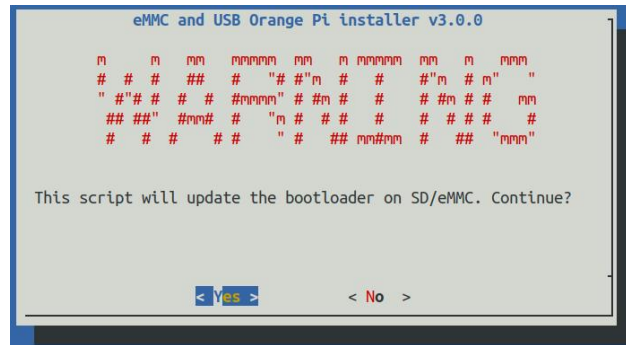
```
orangepi@orangepi:~$ sudo nand-sata-install
```

- d. Then select **5 Install/Update the bootloader on SD/eMMC**

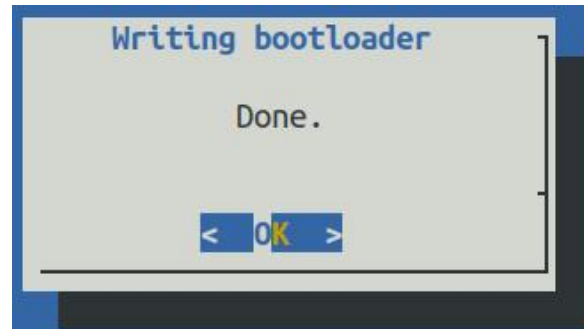




- e. After pressing the enter key, a warning will first pop up



- f. Pressing the enter key again will start updating u-boot, and after the update is complete, the following information will be displayed



- g. Then you can restart the development board to test whether the u-boot modifications have taken effect

4. 4. Compiling Linux Kernel

If downloading code from GitHub is slow, you can do it at /Add the parameter **GITEE.SERVER=yes** after **build.sh**, so that it can be downloaded from the Gitee website during compilation.

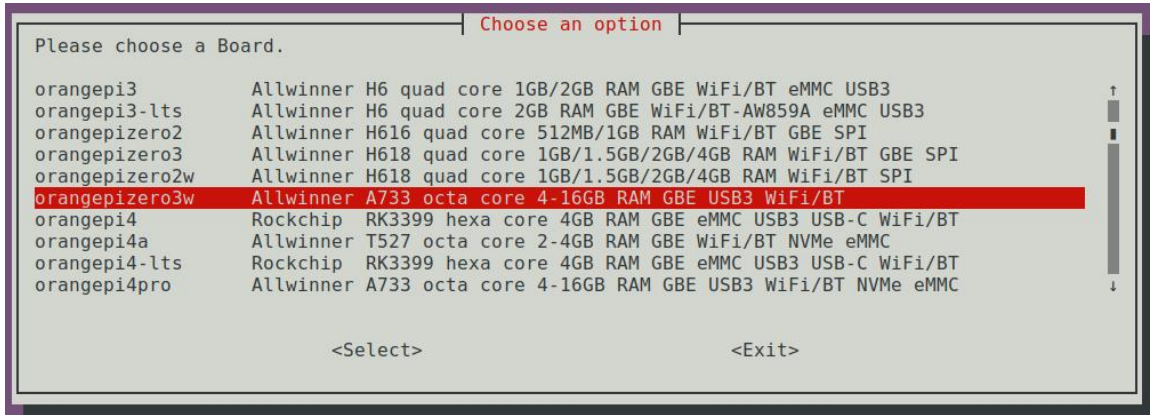
The compilation command for adding the **GITEE.SERVER=yes** parameter is as follows:

```
test@test:~/orange-pi-build$ sudo ./build.sh GITEE_SERVER=yes
```

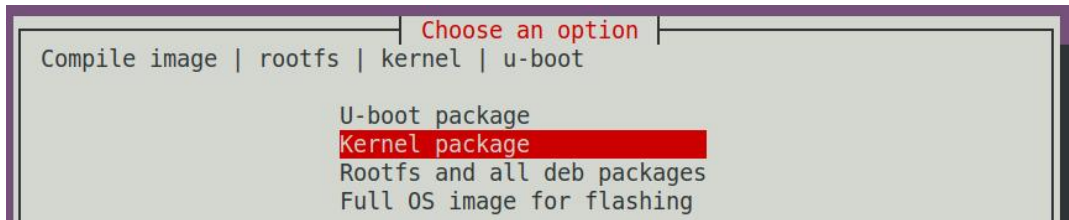
- 1) Run the **build.sh** script, remember to grant sudo privileges

```
test@test:~/orange-pi-build$ sudo ./build.sh
```

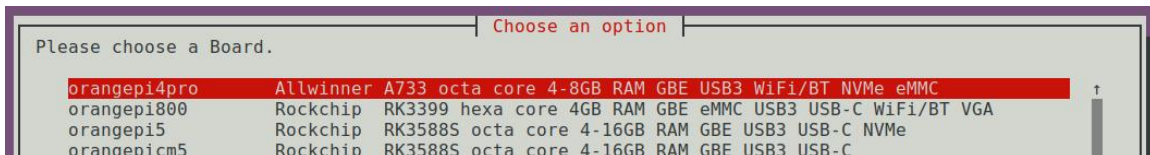
- 2) Next, select the model of the development board



3) Select **Kernel package** and press Enter

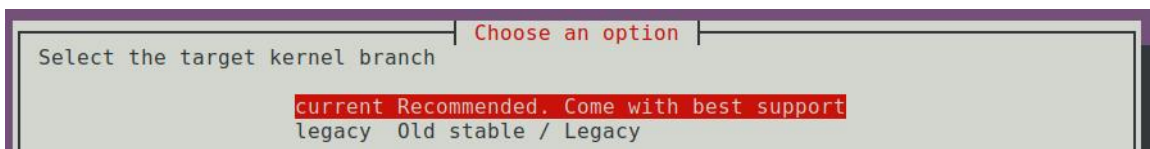


4) Then it will prompt whether the kernel configuration interface needs to be displayed. If the kernel configuration does not need to be modified, select the first one. If the kernel configuration needs to be modified, select the second one



5) Then select the branch type of the kernel source code

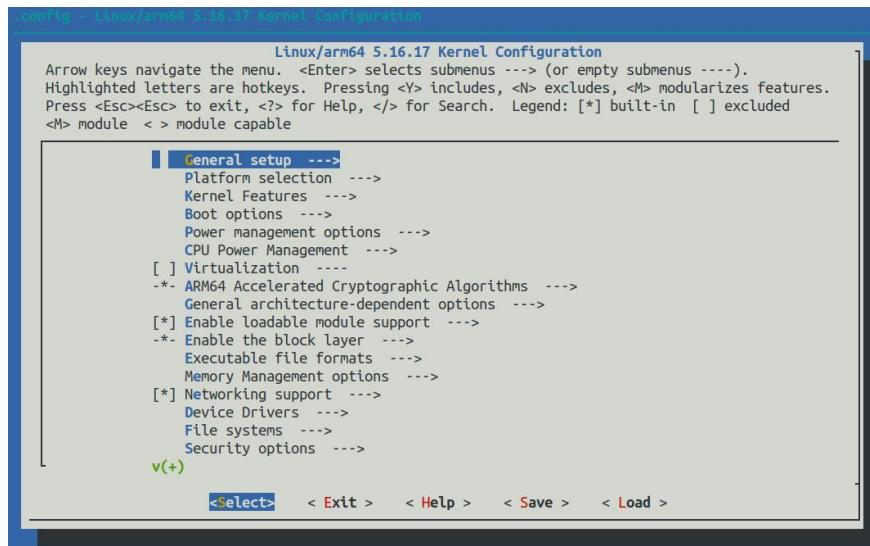
- The legacy branch will compile the Linux 5.15 kernel source code
- The current branch will compile the Linux 6.6 kernel source code



6) If step 3) selects the option to display the kernel configuration menu (second option), a kernel configuration interface opened through **make menuconfig** will pop up. At this time, you can directly modify the kernel configuration, save and exit after modification,



and the kernel source code will be compiled after exit.



- a. If there is no need to modify the configuration options of the kernel, passing **KERNEL_CONFIGURE=no** when running the build.sh script can temporarily block the pop-up kernel configuration interface

```
test@test:~/orange-pi-build$ sudo ./build.sh KERNEL_CONFIGURE=no
```

- b. You can also set **KERNEL_CONFIGURE=no** in the `orange-pi-build/userpatches/config-default.conf` configuration file to permanently disable this feature
- c. If the following error appears when compiling the kernel, it is due to the small terminal interface of Ubuntu PC, which causes the `make menuconfig` interface to not display. Please set the terminal of Ubuntu PC to its maximum size and run the build.sh script again

```
HOSTCC scripts/kconfig/mconf.o
HOSTCC scripts/kconfig/lxdialog/checklist.o
HOSTCC scripts/kconfig/lxdialog/util.o
HOSTCC scripts/kconfig/lxdialog/inputbox.o
HOSTCC scripts/kconfig/lxdialog/textbox.o
HOSTCC scripts/kconfig/lxdialog/yesno.o
HOSTCC scripts/kconfig/lxdialog/menubox.o
HOSTLD scripts/kconfig/mconf
scripts/kconfig/mconf Kconfig
Your display is too small to run Menuconfig!
It must be at least 19 lines by 80 columns.
scripts/kconfig/Makefile:28: recipe for target 'menuconfig' failed
make[1]: *** [menuconfig] Error 1
Makefile:560: recipe for target 'menuconfig' failed
make: *** [menuconfig] Error 2
[ error ] ERROR in function compile_kernel [ compilation.sh:376 ]
[ error ] Error kernel menuconfig failed
[ o.k. ] Process terminated
```

- 7) The following is a partial explanation of the information prompted when compiling



the current branch kernel source code:

- a. Version of Linux kernel source code

[o.k.] Compiling current kernel [**5.15.147**]

- b. The version of the cross compilation toolchain used

[o.k.] Compiler version [**aarch64-linux-gnu-gcc 11**]

- c. The default configuration file used by the kernel and its storage path are as follows

[o.k.] Using kernel config file

[**orange-pi-build/external/config/kernel/linux-5.15-sun60iw2-current.config**]

- d. The path of the compiled kernel related deb package

[o.k.] Target directory [**output/debs/**]

- e. The package name of the compiled kernel image deb package

[o.k.] File name [**linux-image-current-sun60iw2_x.x.x_arm64.deb**]

- f. Compilation time used

[o.k.] Runtime [**10 min**]

- g. Finally, the compilation command for the kernel selected last time will be displayed. The following command can be used to start compiling the kernel source code without selecting it through the graphical interface

[o.k.] Repeat Build Options [**sudo ./build.sh BOARD=orange-pi-zero3w
BRANCH=current BUILD_OPT=kernel KERNEL_CONFIGURE=no**]

8) View the compiled kernel related deb packages

- a. **linux-dtb-current-sun60iw2_x.x.x_arm64.deb** Contains dtb files used by the kernel
- b. **linux-headers-current-sun60iw2_x.x.x_arm64.deb** Contains kernel header files
- c. **linux-image-current-sun60iw2_x.x.x_arm64.deb** Contains kernel images and kernel modules

```
test@test:~/orange-pi-build$ ls output/debs/linux-*
output/debs/linux-dtb-current-sun60iw2_x.x.x_arm64.deb
output/debs/linux-headers-current-sun60iw2_x.x.x_arm64.deb
output/debs/linux-image-current-sun60iw2_x.x.x_arm64.deb
```

9) When the Orange-pi build compilation system compiles the Linux kernel source code, it first synchronizes the Linux kernel source code with the GitHub server's Linux kernel



source code. Therefore, if you want to modify the Linux kernel source code, you first need to turn off the source code update function (**which requires a complete compilation of the Linux kernel source code before it can be turned off, otherwise it will prompt that the Linux kernel source code cannot be found**). Otherwise, the modifications made will be restored. The method is as follows:

Set the IGNOREUPDATES variable to "yes" in **userpatches/config-default.conf**

```
test@test:~/orange-pi-build$ vim userpatches/config-default.conf
IGNORE_UPDATES="yes"
```

10) If modifications have been made to the kernel, the following method can be used to update the kernel and kernel modules of the Linux system on the development board

- a. Upload the compiled deb package of the Linux kernel to the Linux system on the development board

```
test@test:~/orange-pi-build$ cd output/debs
test@test:~/orange-pi-build/output/debs$ scp \
linux-image-current-sun60iw2_x.x.x_arm64.deb root@192.168.1.xxx:/root
```

- b. Reinstall the deb package of the new Linux kernel that was just uploaded

```
orange-pi@orange-pi:~$ sudo dpkg -i linux-image-current-sun60iw2_x.x.x_arm64.deb
```

- c. Then restart the development board and check if the kernel related modifications have taken effect

```
orange-pi@orange-pi:~$ sudo reboot
```

4.5. Compile rootfs

If downloading code from GitHub is slow, you can do it at /Add the parameter GITEE.SERVER=yes after build.sh, so that it can be downloaded from the Gitee website during compilation.

The compilation command for adding the GITEE.SERVER=yes parameter is as follows:

```
test@test:~/orange-pi-build$ sudo ./build.sh GITEE_SERVER=yes
```

- 1) Run the build.sh script, remember to grant sudo privileges

```
test@test:~/orange-pi-build$ sudo ./build.sh
```

- 2) Next, select the model of the development board



```
Choose an option
Please choose a Board.
orangePi3      Allwinner H6 quad core 1GB/2GB RAM GBE WiFi/BT eMMC USB3
orangePi3-lts  Allwinner H6 quad core 2GB RAM GBE WiFi/BT-AW859A eMMC USB3
orangePiZero2  Allwinner H616 quad core 512MB/1GB RAM WiFi/BT GBE SPI
orangePiZero3  Allwinner H618 quad core 1GB/1.5GB/2GB/4GB RAM WiFi/BT GBE SPI
orangePiZero2w Allwinner H618 quad core 1GB/1.5GB/2GB/4GB RAM WiFi/BT SPI
orangePiZero3w Allwinner A733 octa core 4-16GB RAM GBE USB3 WiFi/BT
orangePi4      Rockchip RK3399 hexa core 4GB RAM GBE eMMC USB3 USB-C WiFi/BT
orangePi4a     Allwinner T527 octa core 2-4GB RAM GBE WiFi/BT NVMe eMMC
```

3) Select **Rootfs and all deb packages**, then press enter

```
Choose an option
Compile image | rootfs | kernel | u-boot

U-boot package
Kernel package
Rootfs and all deb packages
Full OS image for flashing
```

4) Then select the branch type of the kernel source code. Currently, the rootfs maintained by the kernel source code use the same set

```
Choose an option
Select the target kernel branch

current Recommended. Come with best support
legacy Old stable / Legacy
```

5) Then select the type of rootfs

```
Choose a release package base
Select the target OS release package base

bookworm Debian 12 Bookworm
jammy Ubuntu jammy 22.04 LTS
```

6) Then select the type of image

- Image with console interface (server)** Represents a server version image with a relatively small size
- Image with desktop environment** Represents a desktop image with a relatively large volume

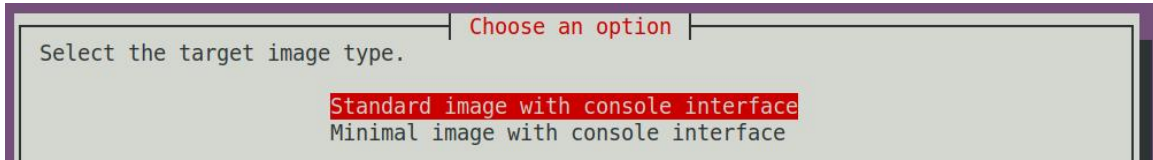
```
Choose an option
Select the target image type.

Image with console interface (server)
Image with desktop environment
```

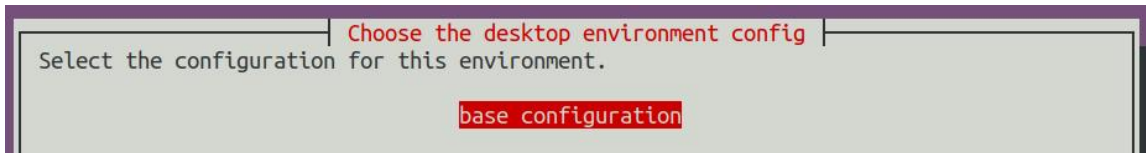
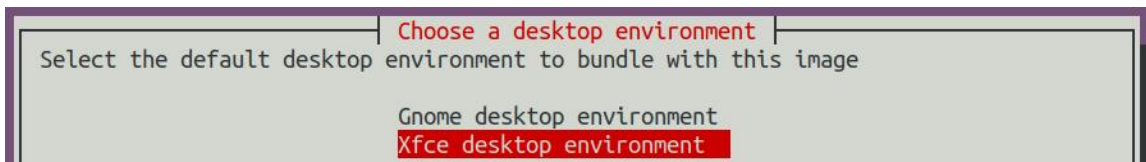
7) If you are compiling the server version image, you can also choose to compile the



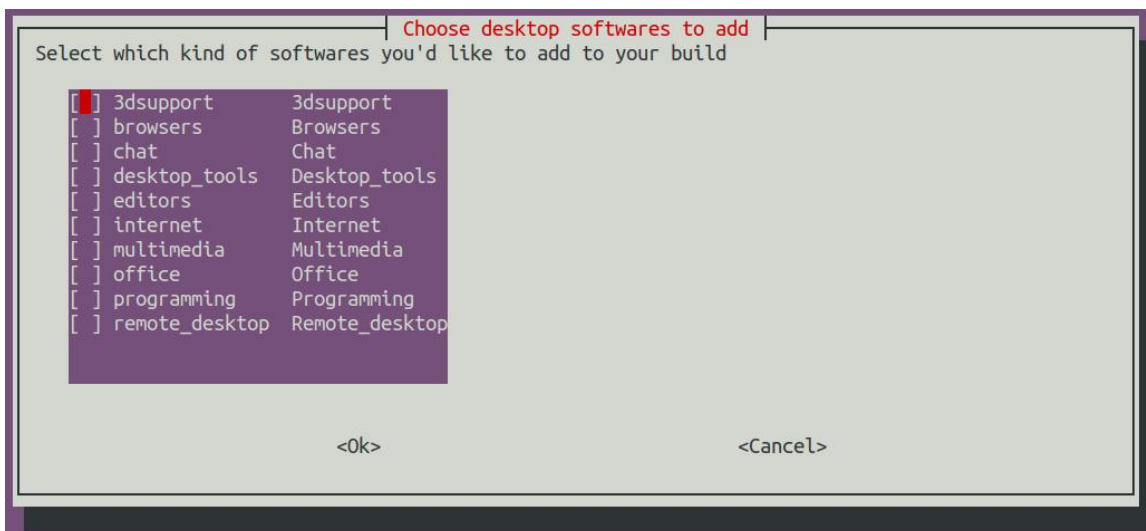
Standard version or the Minimal version. The Minimal version comes with much less pre installed software than the Standard version (**please do not choose the Minimal version unless you have special requirements, as many things are not pre installed by default and some features may not be available**)



8) If compiling a desktop version of the image, you also need to choose the type of desktop environment. Currently, only GNOME is maintained, so please choose a GNOME type desktop



Then you can choose additional software packages that need to be installed. Please press the enter key here to skip directly.





9) Then it will start compiling rootfs, and some of the information prompted during compilation is explained as follows

a. Types of rootfs

```
[ o.k. ] local not found [ Creating new rootfs cache for jammy ]
```

b. The storage path of the rootfs compressed file generated by compilation

```
[ o.k. ] Target directory [ orangepi-build/external/cache/rootfs ]
```

c. The name of the rootfs compressed file generated by compilation

```
[ o.k. ] File name [ jammy-xfce-arm64.5250ec7002de9e81a41de169f1f89721.tar.lz4 ]
```

10) View the compiled rootfs compressed file

a. **jammy-xfce-arm64.5250ec7002de9e81a41de169f1f89721.tar.lz4** is a compressed file of rootfs, and the meaning of each field in the name is

- a) **jammy** represents the type of Linux distribution of rootfs
- b) **xfce** represents rootfs as the desktop version type, and if it is **cli**, it represents the server version type
- c) **arm64** represents the architecture type of rootfs
- d) **5250ec7002de9e81a41de169f1f89721** is an MD5 hash value generated from the package names of all software packages installed by rootfs. As long as the list of software packages installed by rootfs is not modified, this value will not change. The compilation script will use this MD5 hash value to determine whether to recompile rootfs

b. **jammy-xfce-arm64.5250ec7002de9e81a41de169f1f89721.tar.lz4.list** lists the package names of all the software packages installed by rootfs

```
test@test:~/orangepi-build$ ls external/cache/rootfs/
jammy-xfce-arm64.5250ec7002de9e81a41de169f1f89721.tar.lz4
jammy-xfce-arm64.5250ec7002de9e81a41de169f1f89721.tar.lz4.current
jammy-xfce-arm64.5250ec7002de9e81a41de169f1f89721.tar.lz4.list
```

11) If the required rootfs already exist in **external/cache/rootfs**, compiling rootfs again will skip the compilation process and will not restart. When compiling the image, it will also search for available rootfs in **external/cache/rootfs**, and if so, use them directly, which can save a lot of download and compilation time



4. 6. Compiling Linux Images

If downloading code from GitHub is slow, you can do it at /Add the parameter **GITEE.SERVER=yes** after **build.sh**, so that it can be downloaded from the Gitee website during compilation.

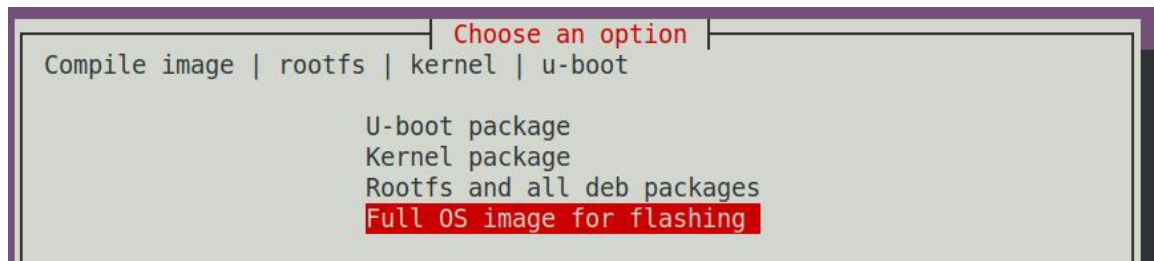
The compilation command for adding the **GITEE.SERVER=yes** parameter is as follows:

```
test@test:~/orange-pi-build$ sudo ./build.sh GITEE_SERVER=yes
```

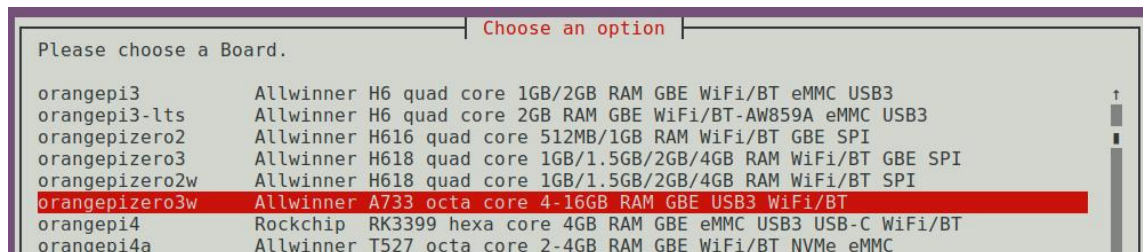
1) Run the **build.sh** script, remember to grant sudo privileges

```
test@test:~/orange-pi-build$ sudo ./build.sh
```

2) Select **Full OS image for flashing** and press Enter

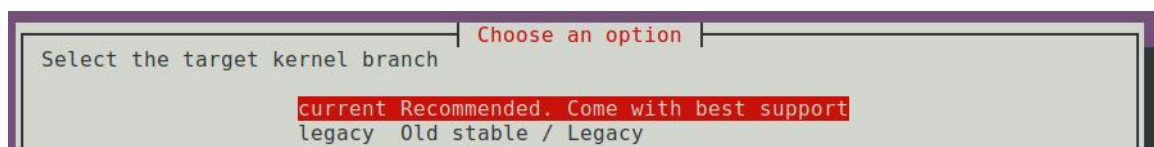


3) Then select the model of the development board



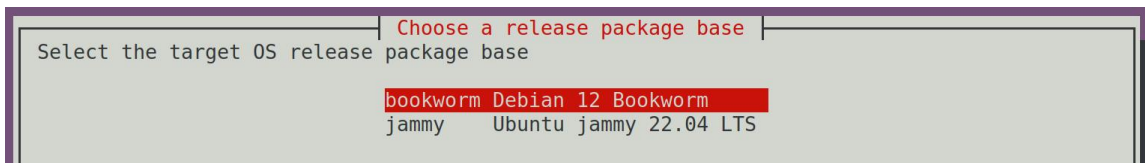
4) Then select the branch of the kernel source code

- The legacy branch will compile the Linux 5.15 kernel source code
- The current branch will compile the Linux 6.6 kernel source code



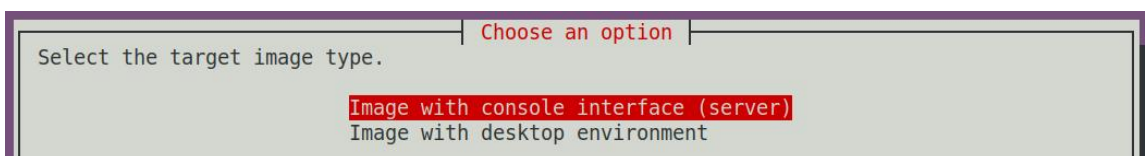


5) Then select the type of rootfs

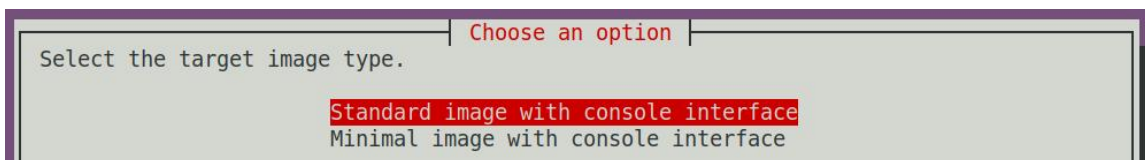


6) Then select the type of image

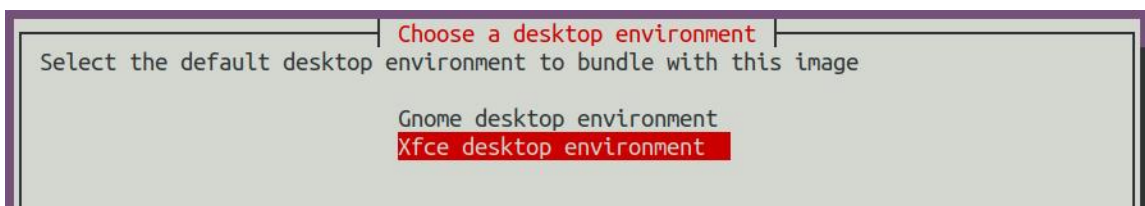
- a. **Image with console interface (server)** Represents a server version image with a relatively small size
- b. **Image with desktop environment** Represents a desktop image with a relatively large volume

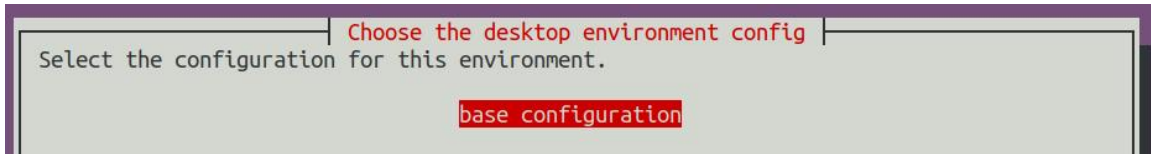


7) If you are compiling the server version image, you can also choose to compile the Standard version or the Minimal version. The Minimal version comes with much less pre installed software than the Standard version (**please do not choose the Minimal version unless you have special requirements, as many things are not pre installed by default and some features may not be available**)

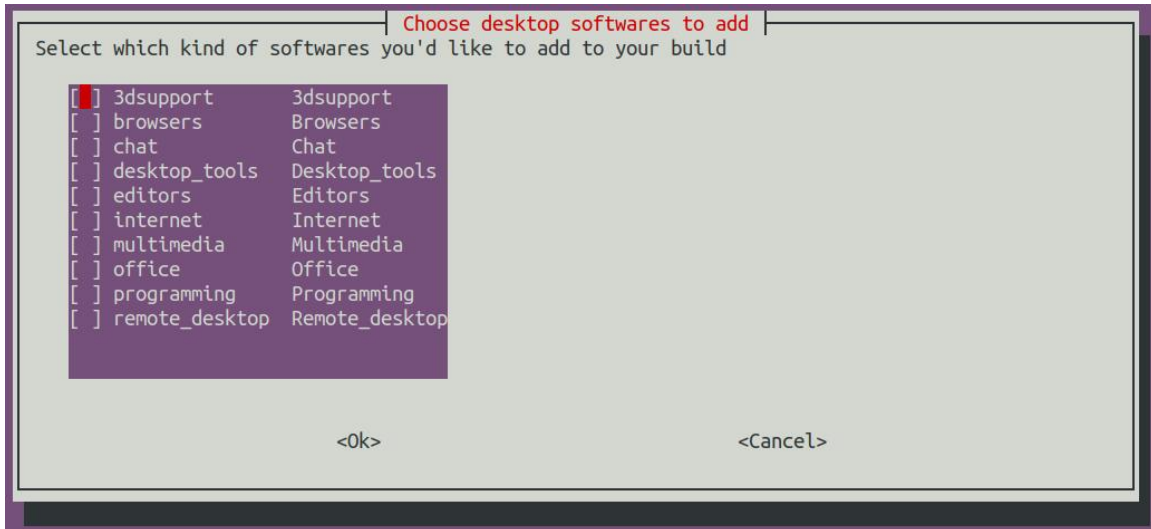


8) If compiling the desktop version of the image, you also need to choose the type of desktop environment. Currently, only XFCE is maintained, so please choose the XFCE type desktop





Then you can choose additional software packages that need to be installed. Please press the enter key here to skip directly.



9) Then it will start compiling the Linux image, and the general process of compilation is as follows

- a. Initialize the compilation environment of Ubuntu PC and install the necessary software packages for the compilation process
- b. Download the source code for u-boot and Linux kernel (if cached, only update the code)
- c. Compile the u-boot source code and generate the deb package for u-boot
- d. Compile Linux source code and generate deb packages related to Linux
- e. Creating a deb package for Linux firmware
- f. Create a deb package for the orangepi config tool
- g. Create deb packages that support board level support
- h. If compiling the desktop version image, desktop related deb packages will also be created
- i. Check if rootfs have been cached. If not, create a new rootfs. If cached, decompress and use it directly
- j. Install the deb package generated earlier into rootfs



- k. Make specific settings for different development boards and types of images, such as pre installing additional software packages, modifying system configurations, etc
- l. Then create an image file and format the partition, with the default type being ext4
- m. Copy the configured rootfs to the partition of the image again
- n. Then update initramfs
- o. Finally, write the bin file of u-boot to the image using the dd command

10) After compiling the image, the following message will be prompted

- a. The storage path of the compiled image

```
[ o.k. ] Done building  
[ output/images/Orangepizero3w_x.x.x_debian_jammy_linux5.15.xx_xfce_desktop/  
Orangepizero3w_x.x.x_debian_jammy_linux5.15.xx_xfce_desktop.img ]
```

- b. Compilation time used

```
[ o.k. ] Runtime [ 19 min ]
```

- c. The command to repeatedly compile the image can be used to start compiling the image without selecting through the graphical interface

```
[ o.k. ] Repeat Build Options [ sudo ./build.sh BOARD=orangepizero3w  
BRANCH=current BUILD_OPT=image RELEASE=jammy BUILD_MINIMAL=no  
BUILD_DESKTOP=no KERNEL_CONFIGURE=yes ]
```



5. Instructions for using Android 13 system

5.1. Supported Android versions

Android Version	kernel version
Android 13	linux5.15

5.2. Adaptation of Android 13 Features

Function	Android 13
HDMI Video	OK
HDMI Audio	OK
TypeC USB3.0	OK
TypeC USB2.0	OK
TypeC DP display	OK
TF card startup	OK
eMMC	OK
UFS	OK
WIFI	OK
Bluetooth	OK
CAM1 interface	OK
CAM2 interface	OK
MIPI LCD	OK
FAN	OK
LED	OK
40 pin GPIO	OK
40 pin I2C	OK
40 pin SPI	OK
40 pin UART	OK
40 pin PWM	OK

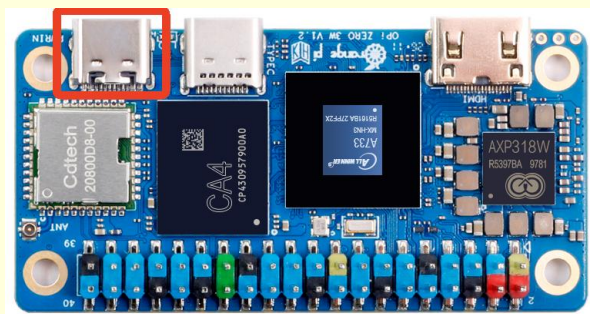


temperature sensor	OK
GPU	OK

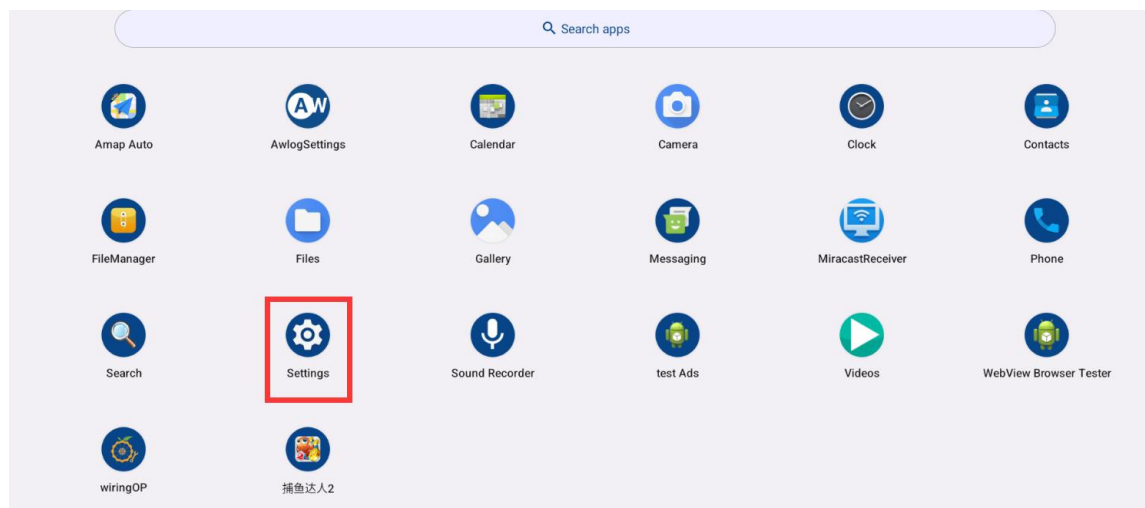
5. 3. Instructions for Using ADB

5. 3. 1. Method for switching USB OTG mode

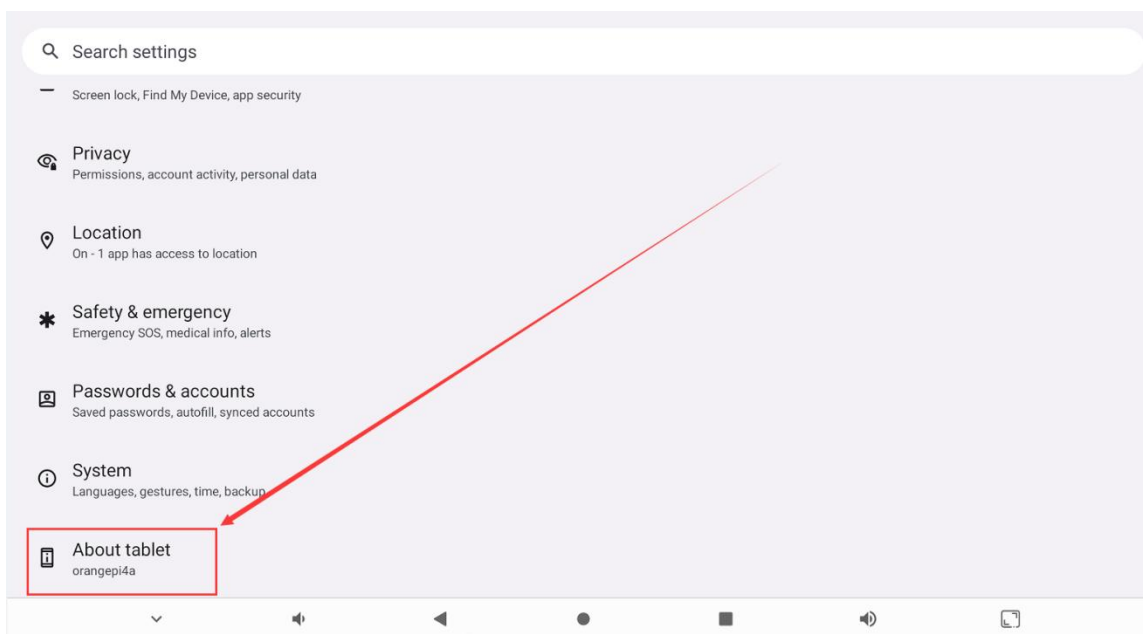
The development board has two TypeC interfaces, among which the TypeC USB 2.0 interface marked in red in the figure can support both Host mode and Device mode, while the other TypeC interface only supports Host mode.



1) First, open Settings



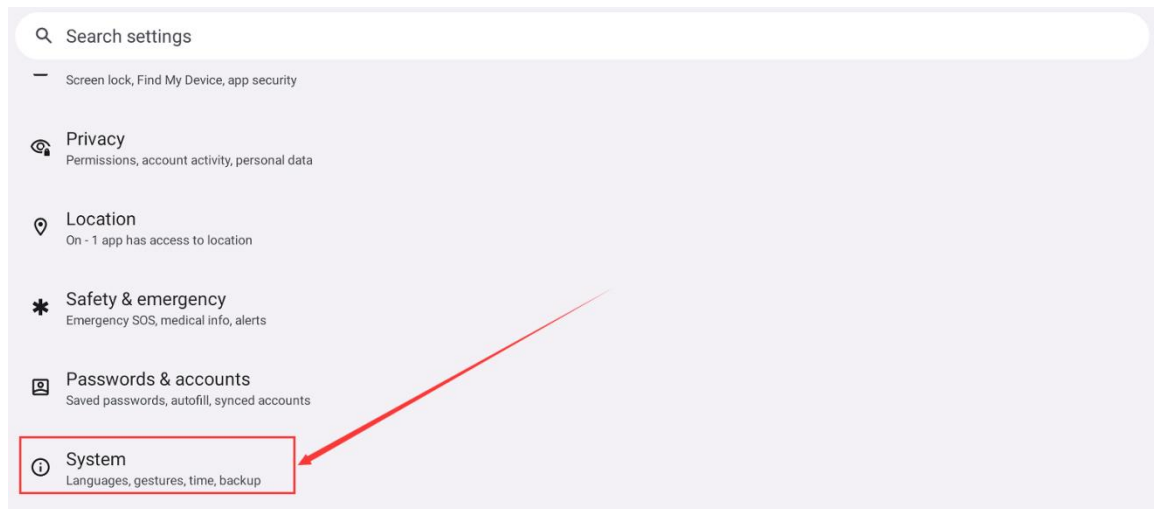
2) Then find **About tablet**



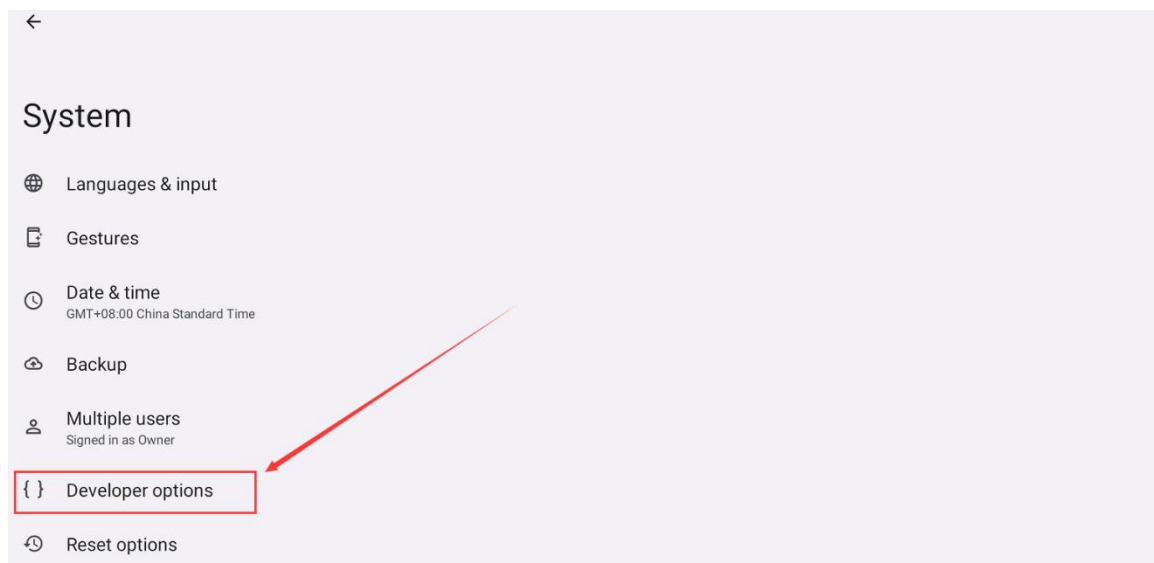
3) Then click the **Build number** option multiple times with the mouse until **You are now a developer!** the hint



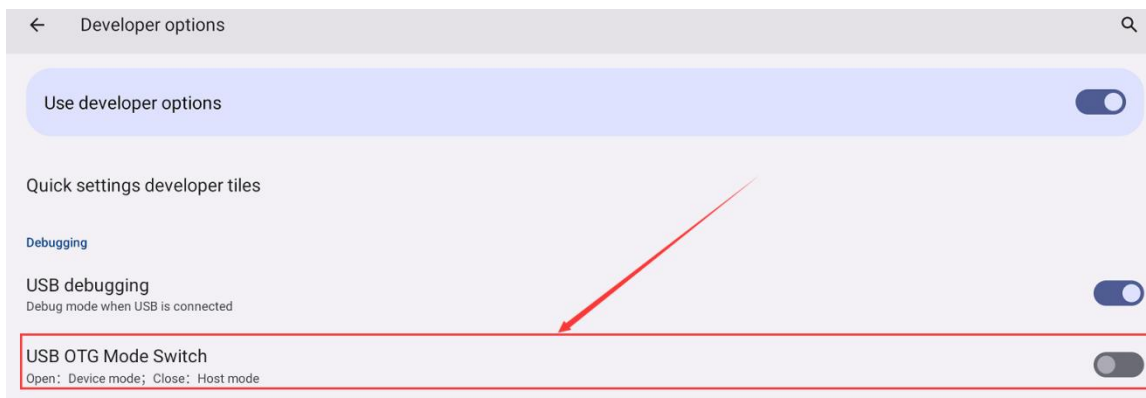
4) Then return to the previous menu and select **System**



5) Then select **Developer options**

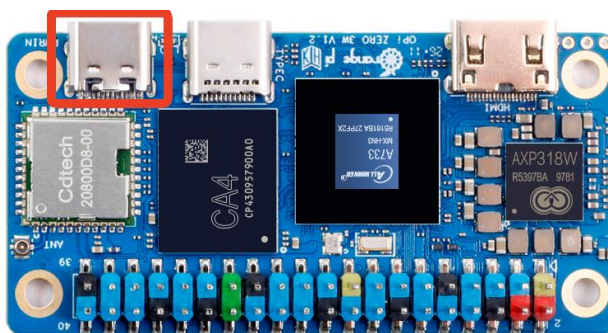


6) Finally, locate the **USB OTG Mode Switch** switch, **turn it on to switch to Device mode, and turn it off to switch to Host mode**



5. 3. 2. Connect adb debugging with data cable

1) Prepare a USB Type C interface data cable, insert one end of the USB interface into the USB interface of the computer, and insert one end of the USB Type C interface into the Type C interface of the development board (see the image on the right below for the location). In this case, the development board is powered by the computer's USB interface, so please ensure that the computer's USB interface can provide sufficient power to drive the development board.



2) Then switch to USB Device mode by following the method of switching to USB OTG mode.

3) Then install adb tools on Ubuntu PC

```
test@test:~$ sudo apt-get update
test@test:~$ sudo apt-get install -y adb
```

4) View recognized ADB devices

```
test@test:~$ adb devices
List of devices attached
```



```
4c00146473c28651dd0 device
```

5) Then you can log in to the Android system through adb shell on Ubuntu PC

```
test@test:~$ adb shell
a733-demo:/ #
```

5.3.3. Debugging with network connection adb

Using network adb does not require a Type-C data cable to connect the computer and development board, but communicates through the network. Therefore, first make sure that the wired or wireless network of the development board is connected, and then obtain the IP address of the development board, which will be used later.

1) Ensure that the **service.adb.tcp.port** of the Android system is set to port number 5555

```
console:/ # getprop | grep "adb.tcp"
[service.adb.tcp.port]: [5555]
```

2) If **service.adb.tcp.port** is not set, you can use the following command in the serial port to set the port number of the network adb

```
console:/ # setprop service.adb.tcp.port 5555

console:/ # stop adbd
console:/ # start adbd
```

3) Installing adb tools on Ubuntu PC

```
test@test:~$ sudo apt-get update
test@test:~$ sudo apt-get install -y adb
```

4) Then connect the network adb on Ubuntu PC

```
test@test:~$ adb connect 192.168.1.xxx:5555 (Need to modify to the IP address of
the development board)
* daemon not running; starting now at tcp:5037
* daemon started successfully
connected to 192.168.1.xxx:5555

test@test:~$ adb devices
```




List of devices attached

192.168.1.xxx:5555 device

5) Then you can log in to the Android system through adb shell on Ubuntu PC

```
test@test:~$ adb shell
```

```
a733-demo:/ #
```

5. 3. 4. **HDMI to VGA Display Test**

1) Firstly, the following accessories need to be prepared

- a. HDMI to VGA converter



- b. A VGA cable and a Mini HDMI adapter for HDMI female

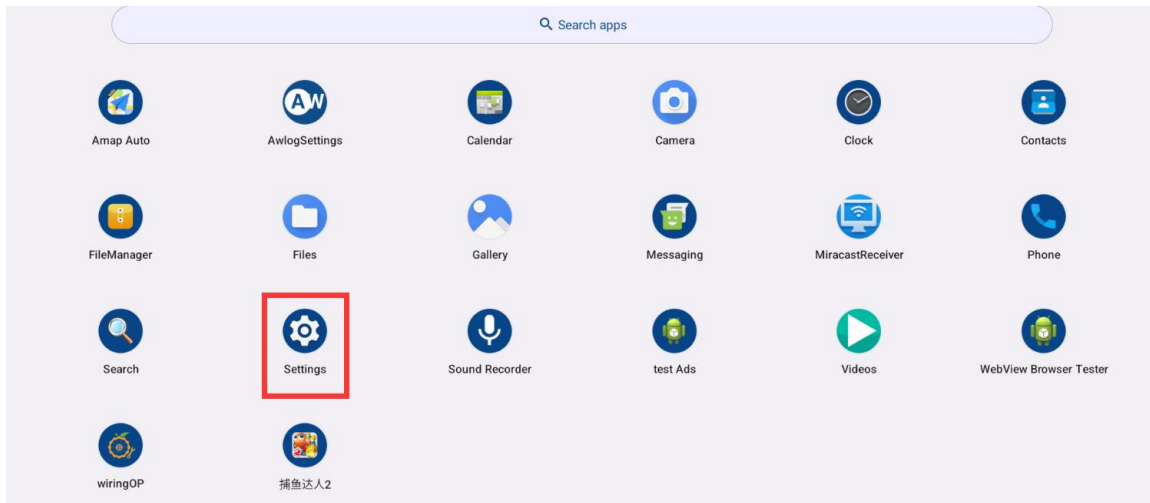


- c. A monitor or TV that supports VGA interface

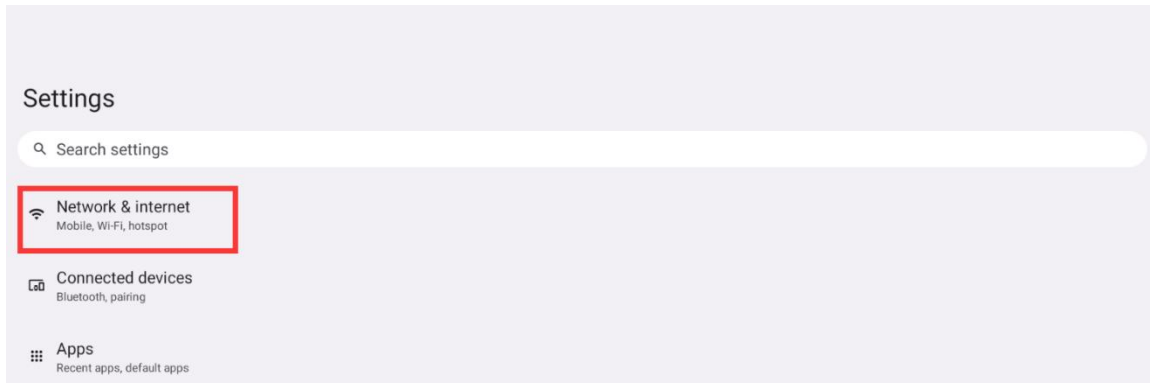
2) Then connect the cables properly, and HDMI to VGA will be able to display normally

5. 4. **Connection method of WI-FI**

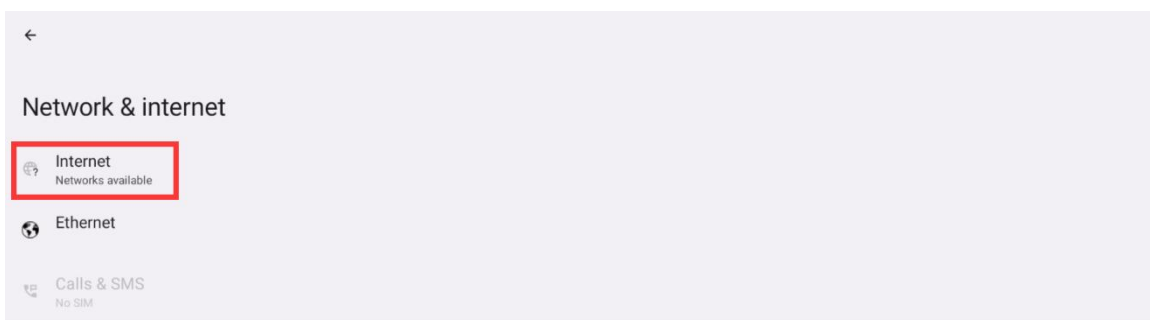
1) First select **Settings**



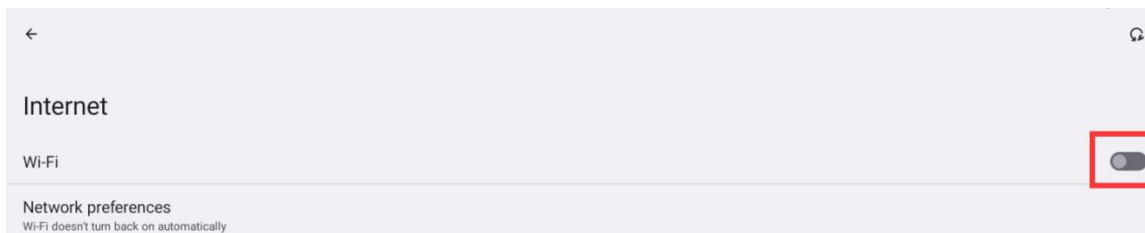
2) Then select **Network & Internet**



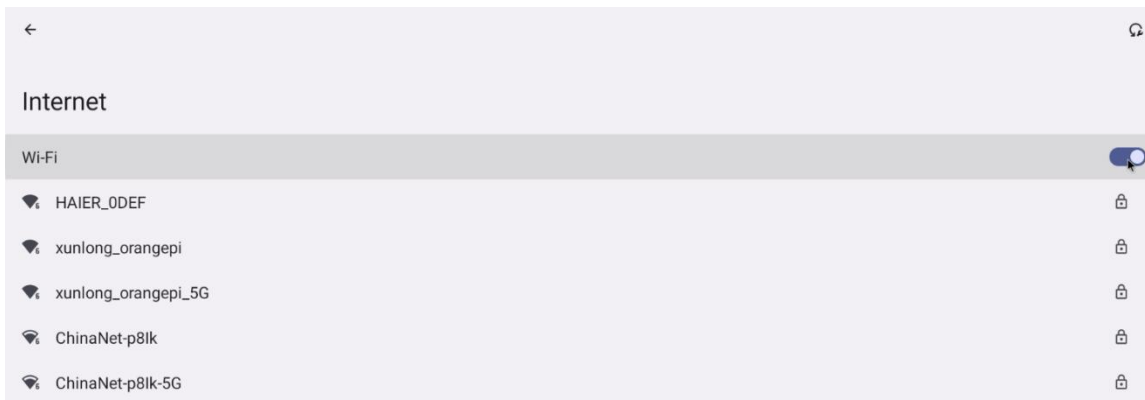
3) Then select **Internet**



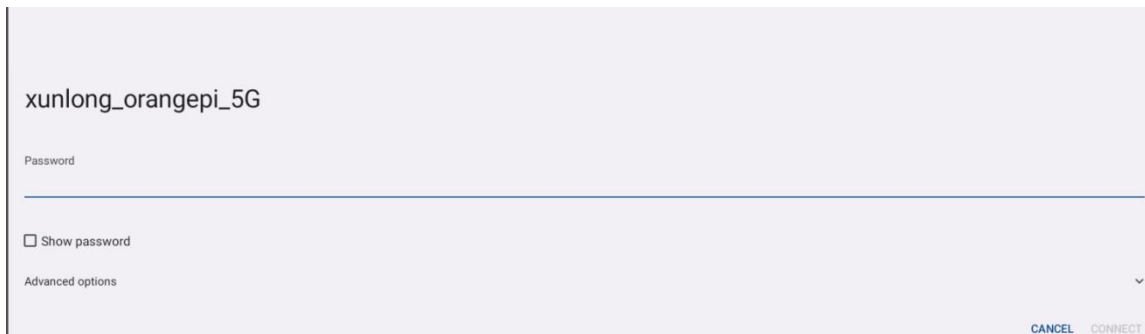
4) Then open WI-FI



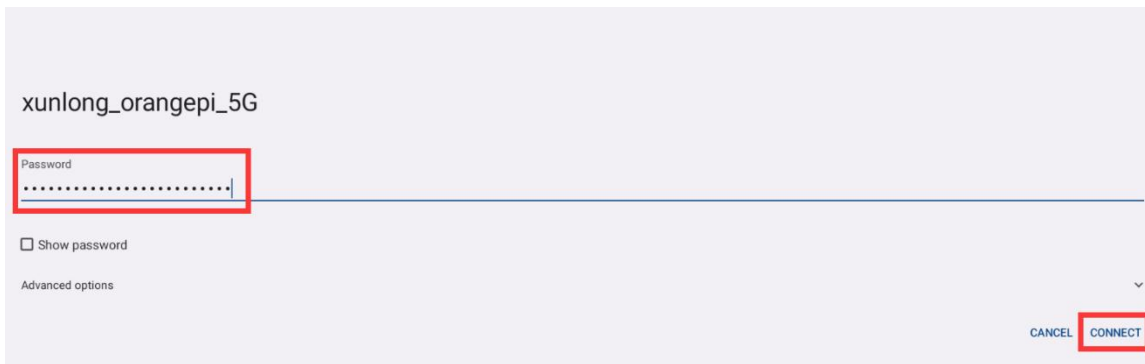
5) After opening WI-FI, you can see the searched signal under **Available networks**



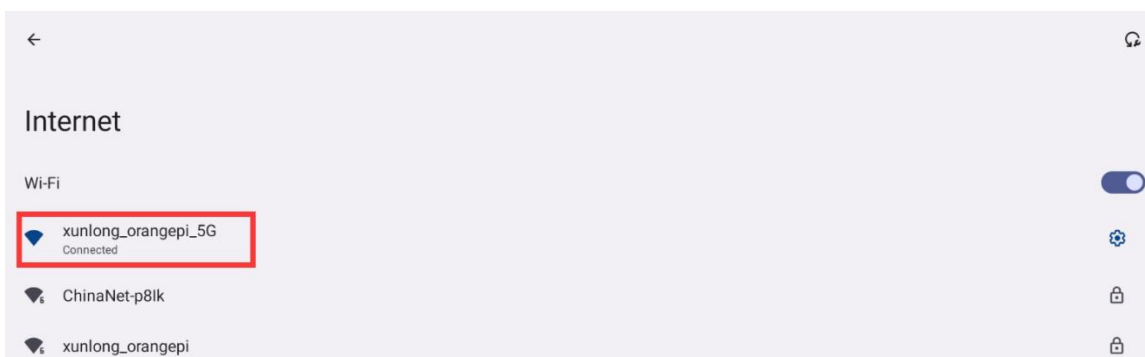
6) After selecting the desired WI-FI network, a password input interface will pop up as shown in the figure below



7) Then, use the keyboard to enter the corresponding WI-FI password, and click the Enter button on the virtual keyboard with the **mouse** to start connecting to WI-FI

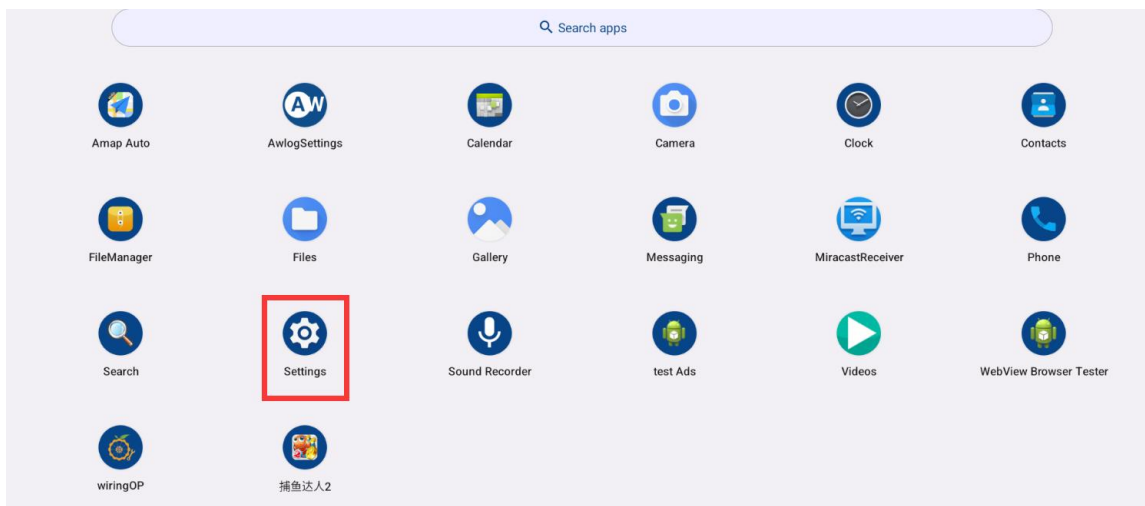


8) The display after successful WI-FI connection is shown in the following figure

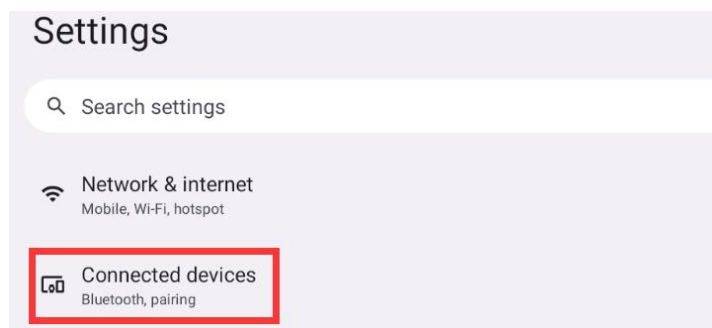


5. 5. Bluetooth connection method

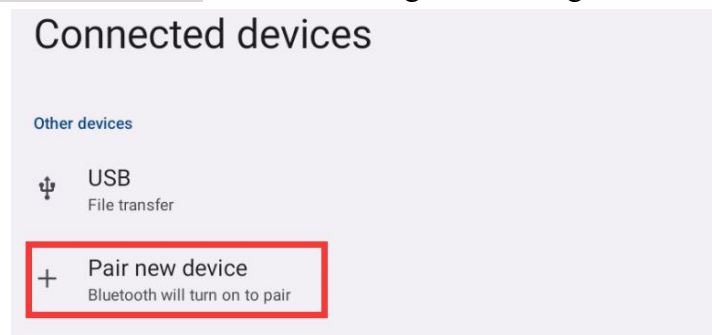
1) First select **Settings**



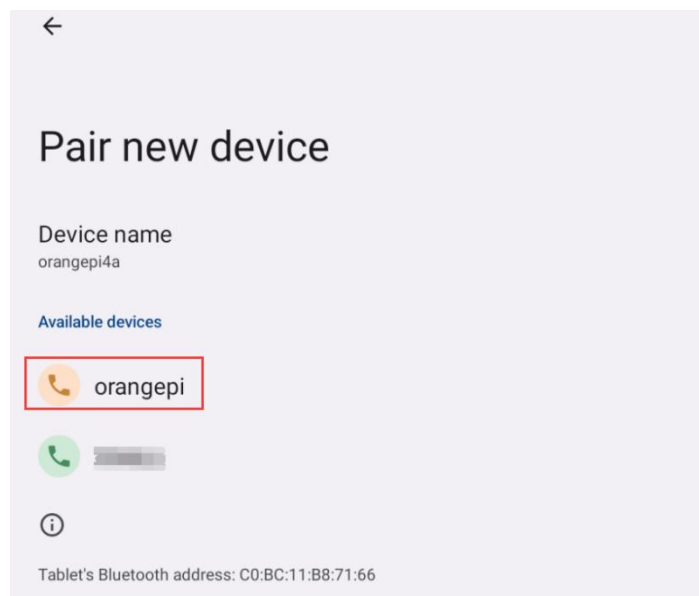
2) Then select **Connected devices**



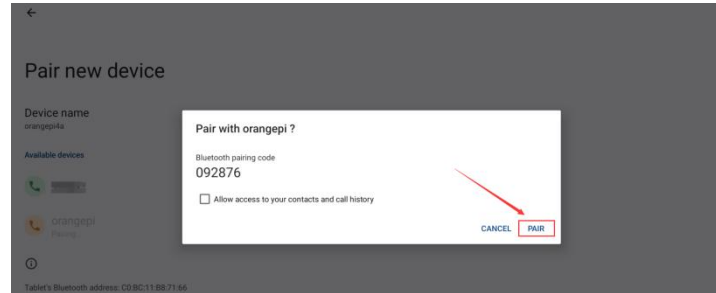
- 3) Then select **Pair new device** to start scanning surrounding Bluetooth devices



- 4) The Bluetooth devices found in the search will be displayed under **Available devices**

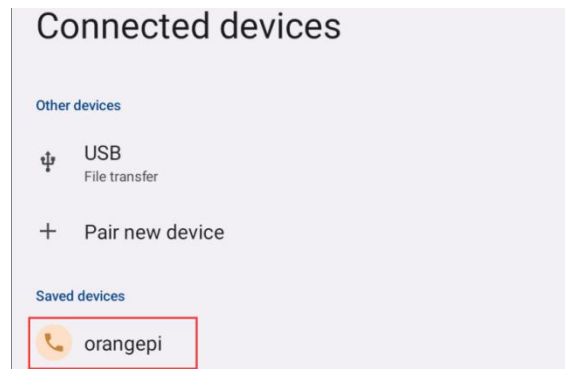


- 5) Then click on the Bluetooth device you want to connect to start pairing. When the interface below pops up, please use the mouse to select the **Pair** option

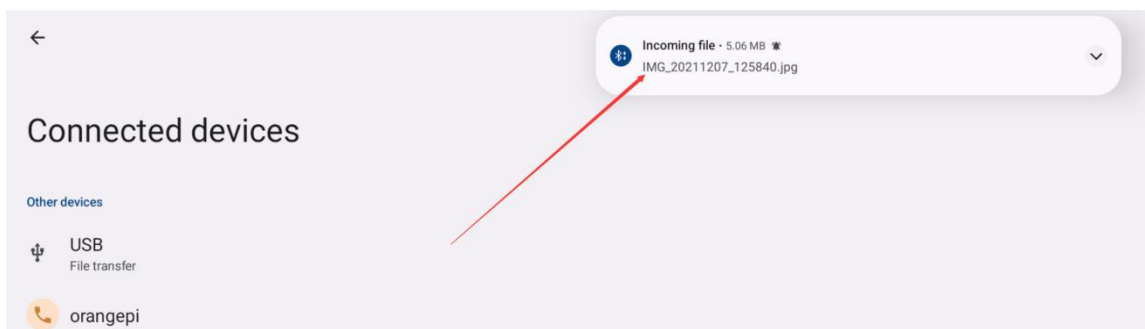


6) The test here is the configuration process of the development board and **Android phone** Bluetooth. At this time, a confirmation interface will pop up on the phone, and clicking the pairing button on the phone will start the pairing process

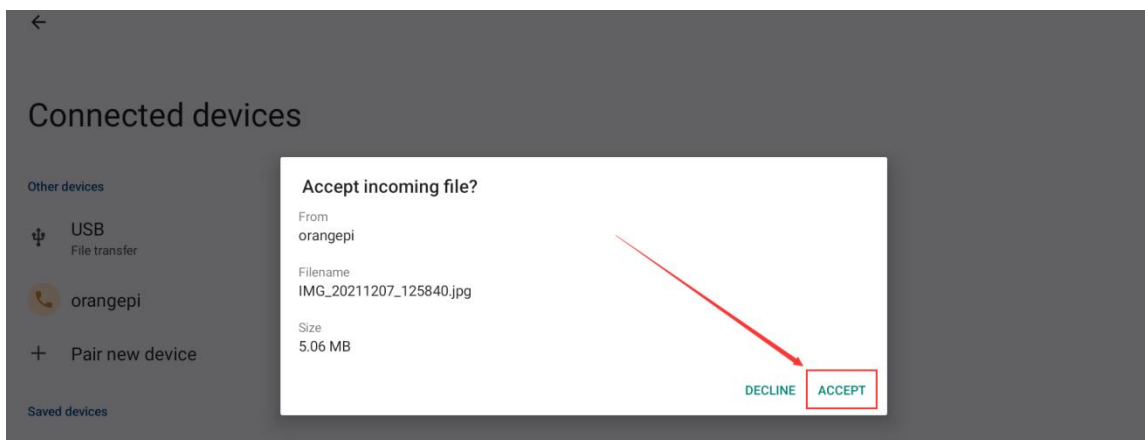
7) After pairing is complete, open **Paired devices** to see the paired Bluetooth devices



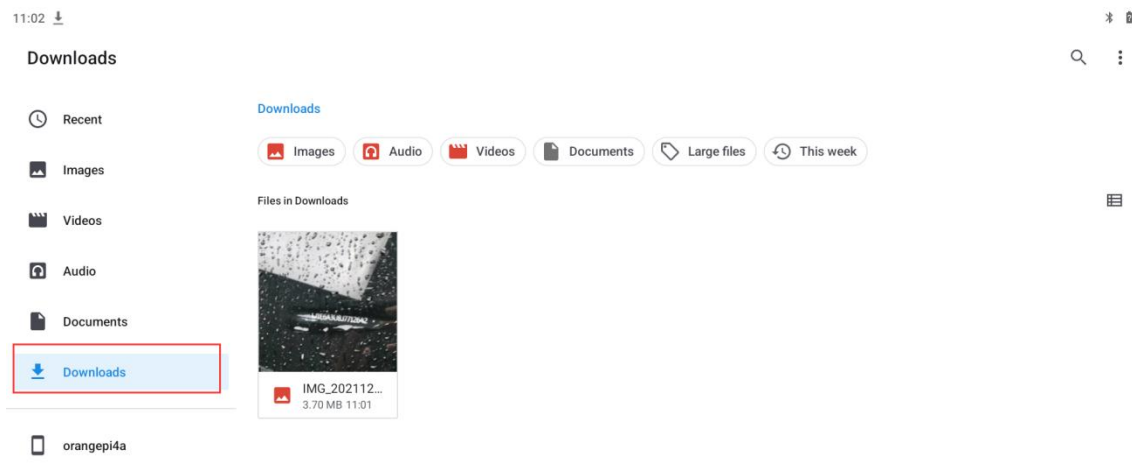
8) At this point, you can use your phone's Bluetooth to send an image to the development board. After sending, you can see the following prompt in the Android system of the development board, and then click on **Incoming file**



9) Then click **Accept** in the pop-up window to start receiving pictures sent by the phone

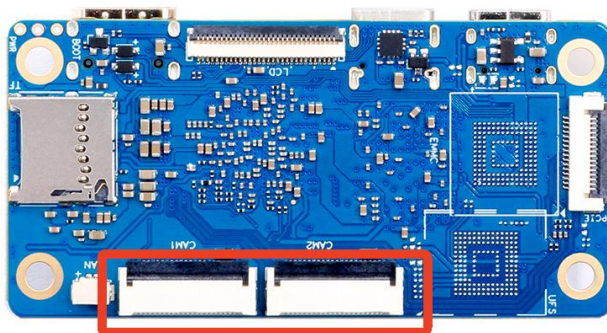


10) The images received by the Android system Bluetooth on the development board can be viewed by opening the Download directory in the file manager

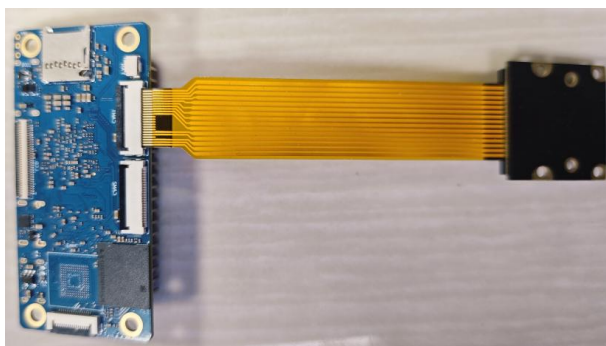
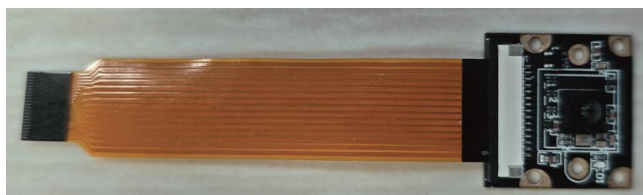


5. 6. Instructions for Using MIPI Camera

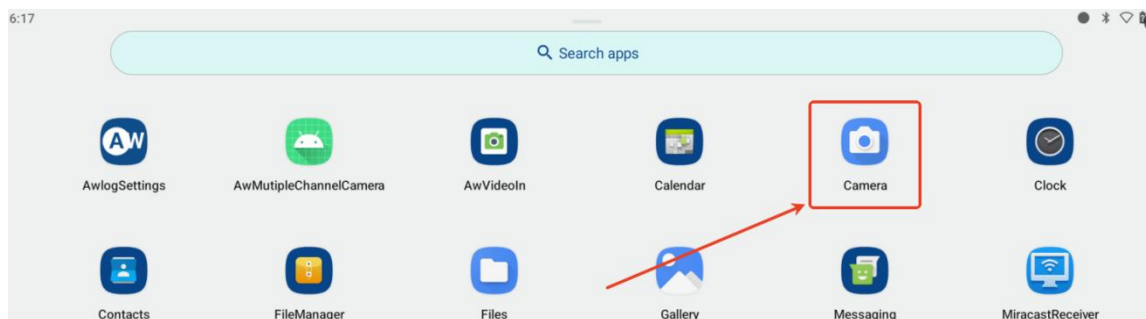
1) There are a total of 2 camera interfaces on the development board, currently only supporting the connection of IMX219 cameras. The positions of CAM1 and CAM2 are as follows:

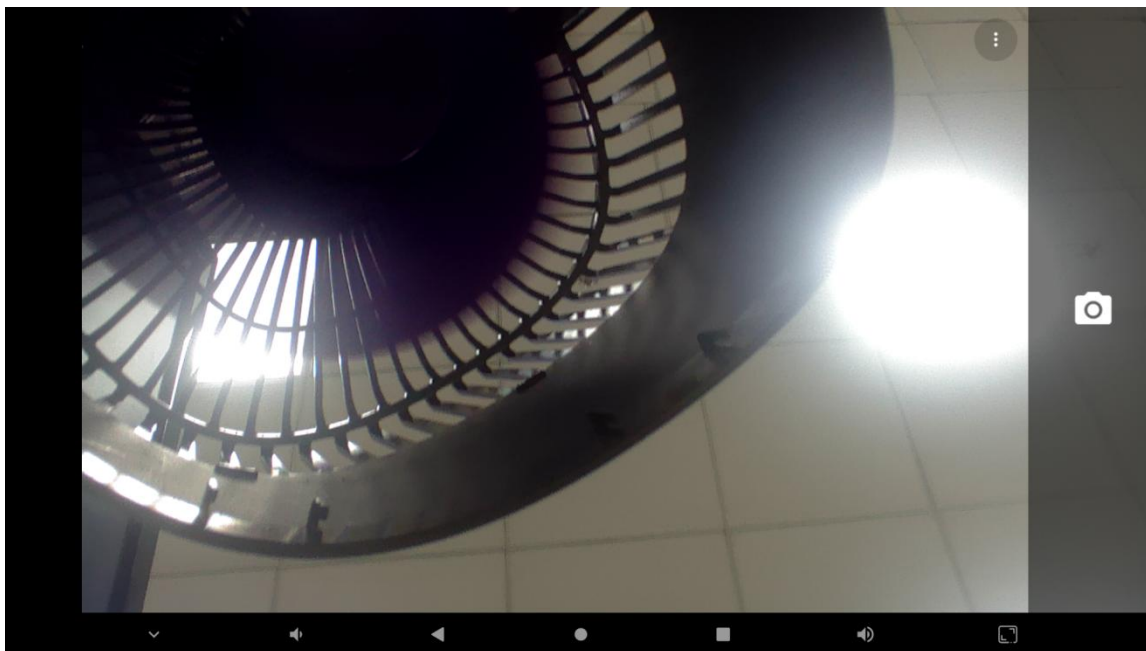


2) The camera interface specification is 22PIN 0.5mm spacing, and the IMX219 camera needs to be connected to the development board through a 22PIN to 15PIN ribbon cable. The CAM1 and CAM2 connections are the same



3) After connecting the camera, click on the camera icon on the desktop to preview the camera

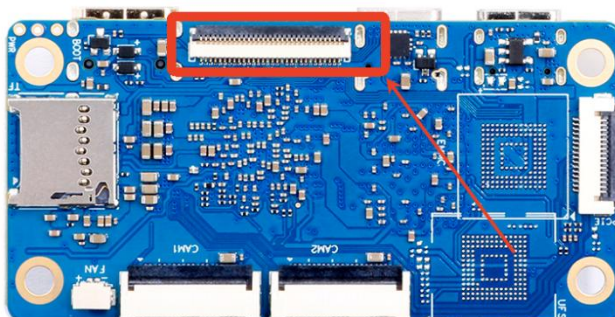




5.7. 10.1 inch MIPI screen usage method

Please ensure that the Android image used is of the following version:
OrangePiZero3W_A733_Android13_lcd_vX.X.img

- 1) Firstly, the screen needs to be assembled. Please refer to the assembly [method for a 10.1-inch MIPI screen](#)
- 2) The interface position of the mipi LCD screen on the development board is shown in the following figure:



- 3) Connect the assembled screen to the LCD interface of the development board, **pay attention to unplugging the HDMI interface**, connect the Type-C power supply to the



board, and power it on. After the system starts up, you can see the screen display as shown in the following figure (**default is vertical screen**)



5. 8. Instructions for Using USB Camera

1) Firstly, insert a USB (UVC protocol) camera into the USB interface of the development board

2) If the USB camera recognizes normally, it will generate the corresponding video device node in `/dev`

```
console:/ # ls /dev/video*  
/dev/video0
```

3) Then ensure that the adb connection between Ubuntu PC and the development board is normal. Please refer to the instructions in the "[ADB Usage](#)" section for instructions on how to use adb

4) Download the USB camera testing app from the **official tool** on the development board download page



Official Resources



User Manual

[Downloads](#)

Schematic

[Downloads](#)

Mechanical

[Downloads](#)

Official Tools

[Downloads](#)

官方工具

[保存到网盘](#)

🕒 2020-11-03 14:09 失效时间: 永久有效

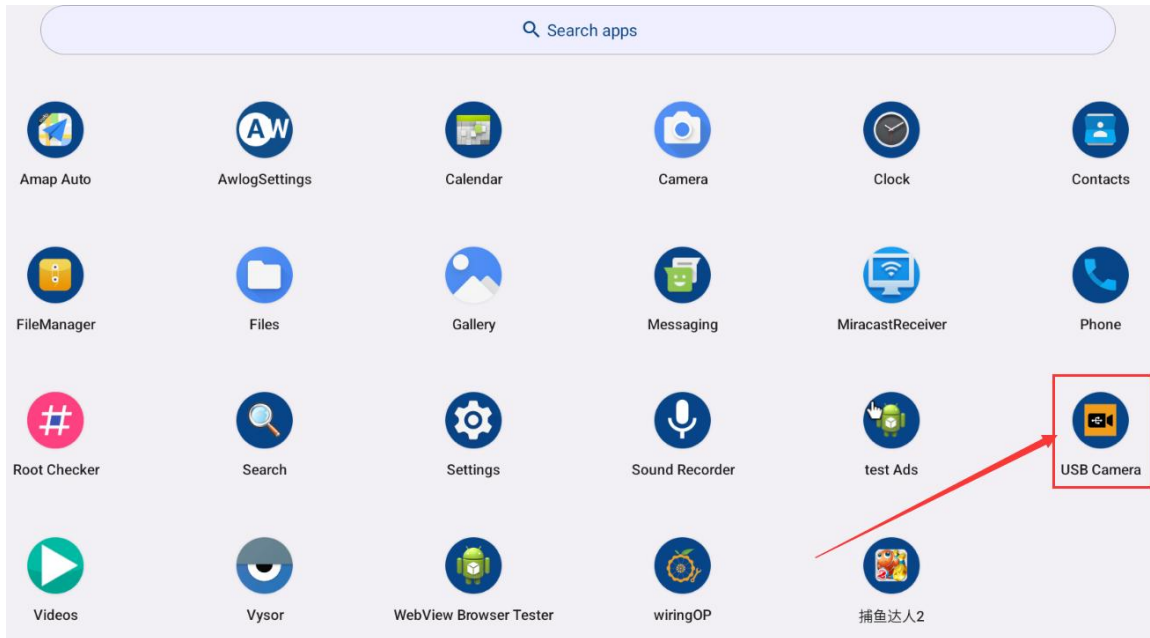
[返回上一级](#) | [全部文件](#) | [官方工具](#) | [Android测试APP](#)

☐ 文件名	大小	修改日期
☐ usbcamera.apk	20M	2020-11-04 13:56
☐ rootcheck.apk	2M	2020-11-04 13:48
☐ REFile.apk	4.4M	2020-11-04 13:48
☐ bledemo.apk	4.1M	2020-11-04 13:48

5) Then use the adb command to install the USB camera test app into the Android system, or you can also install it by copying it from a USB drive

```
test@test:~$ adb install usbcamera.apk
```

6) After installation, the USB camera startup icon can be seen on the Android app interface



7) Then double-click to open the USB camera app and you can see the output video of the USB camera

5.9. Android System ROOT Description

The Android system released by Orange Pi has been ROOT and can be tested using the following method.

1) Download **rootcheck.apk** from the **official tool** on the development board download page

川市迅龙软件有... Orange Pi Plus - O... 查流量来源[Sif K5物流系统 Gmail Outlook 百度翻译-您的超级... 263邮箱 Amazon.com 阿里巴巴1688.com...

Official Resources



User Manual

[Downloads](#)



Schematic

[Downloads](#)



Mechanical

[Downloads](#)



Official Tools

[Downloads](#)



官方工具 保存到网盘

2020-11-03 14:09 失效时间: 永久有效

[返回上一级](#) [全部文件](#) [官方工具](#) [Android测试APP](#)

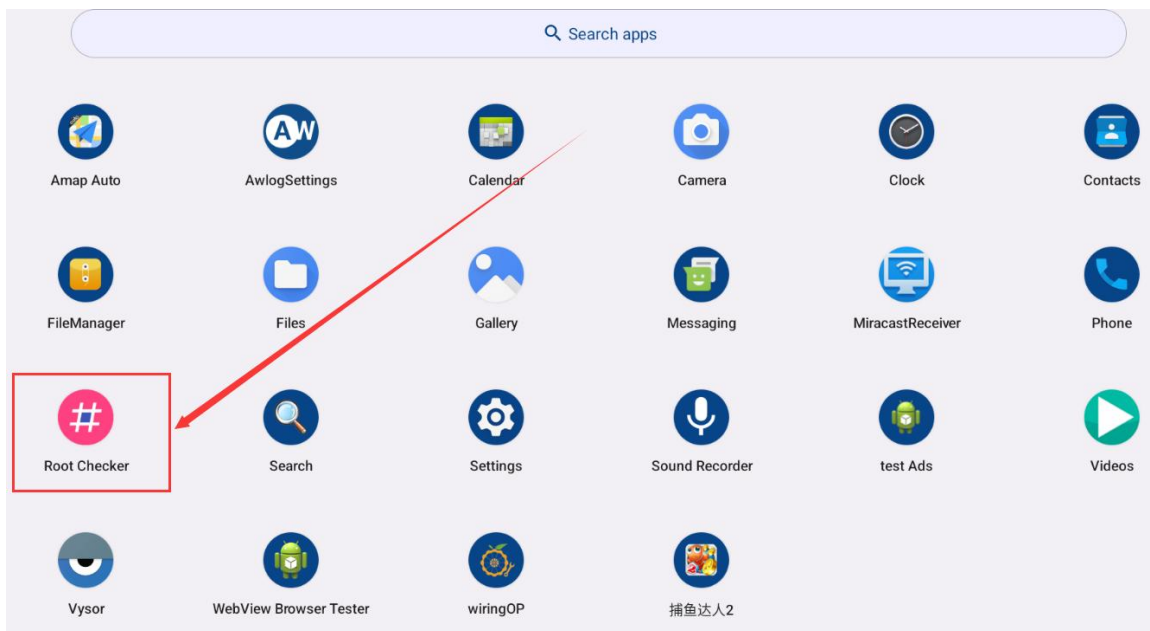
文件名	大小	修改日期
☐ usbcamera.apk	20M	2020-11-04 13:56
☐ rootcheck.apk	2M	2020-11-04 13:48
☐ REFile.apk	4.4M	2020-11-04 13:48

2) Then ensure that the adb connection between Ubuntu PC and the development board is normal. Please refer to the instructions in the "[ADB Usage](#)" section for instructions on how to use adb

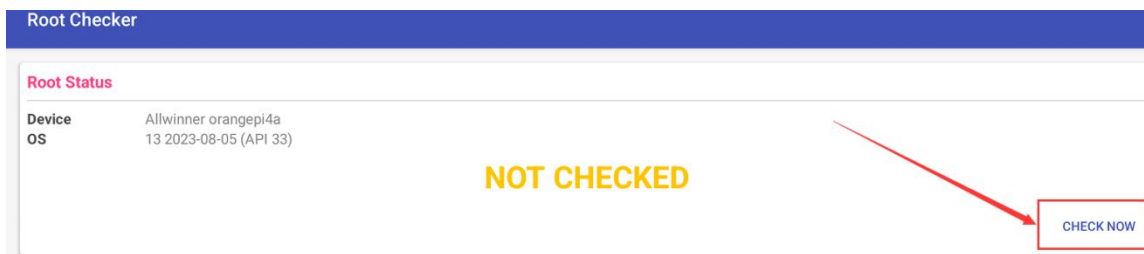
3) Then use the adb command to install rootcheck.apk into the Android system, or you can install it by copying it from a USB drive

```
test@test:~$ adb install rootcheck.apk
```

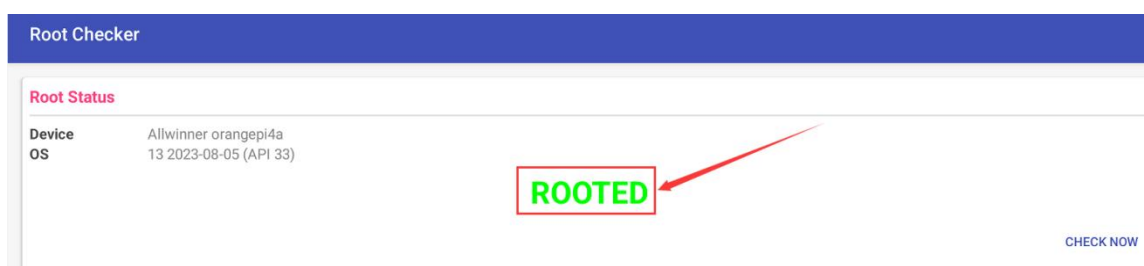
4) After installation, the ROOT testing tool's startup icon can be seen on the Android app interface



5) The display interface after opening the **ROOT testing tool** for the first time is shown in the following figure



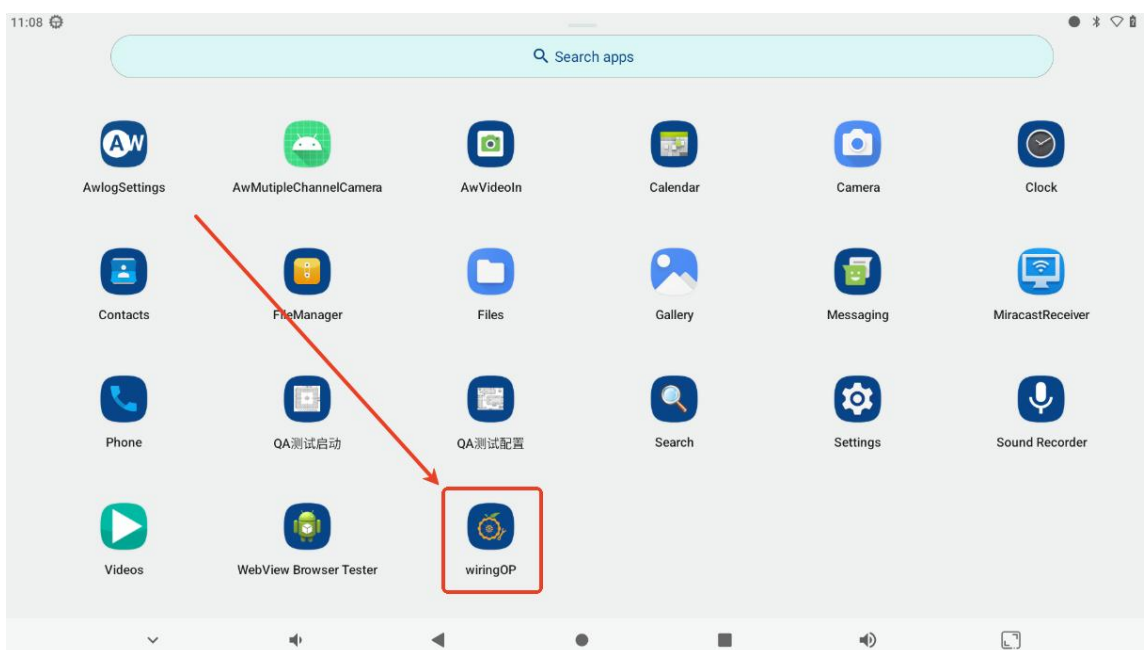
- a. Then you can click **CHECK NOW** to start checking the ROOT status of the Android system. After checking, the display will be as follows, indicating that the Android system has obtained ROOT permission



5. 10. 40 pin interface GPIO, UART, SPI testing

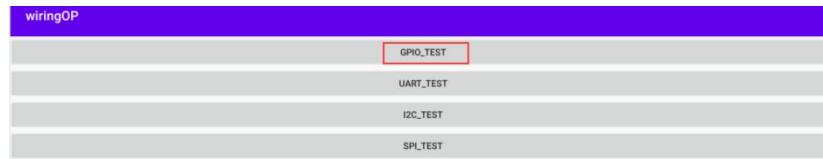
5. 10. 1. 40 Pin GPIO Port Testing Method

- 1) First, open the WiringOP app on the desktop

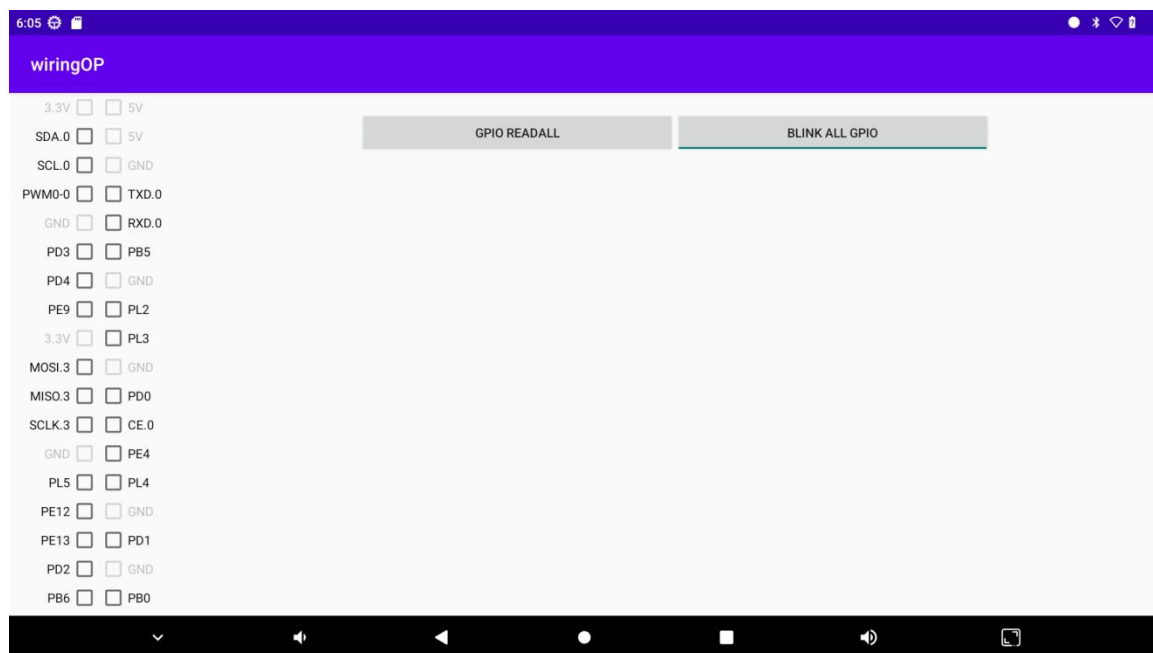




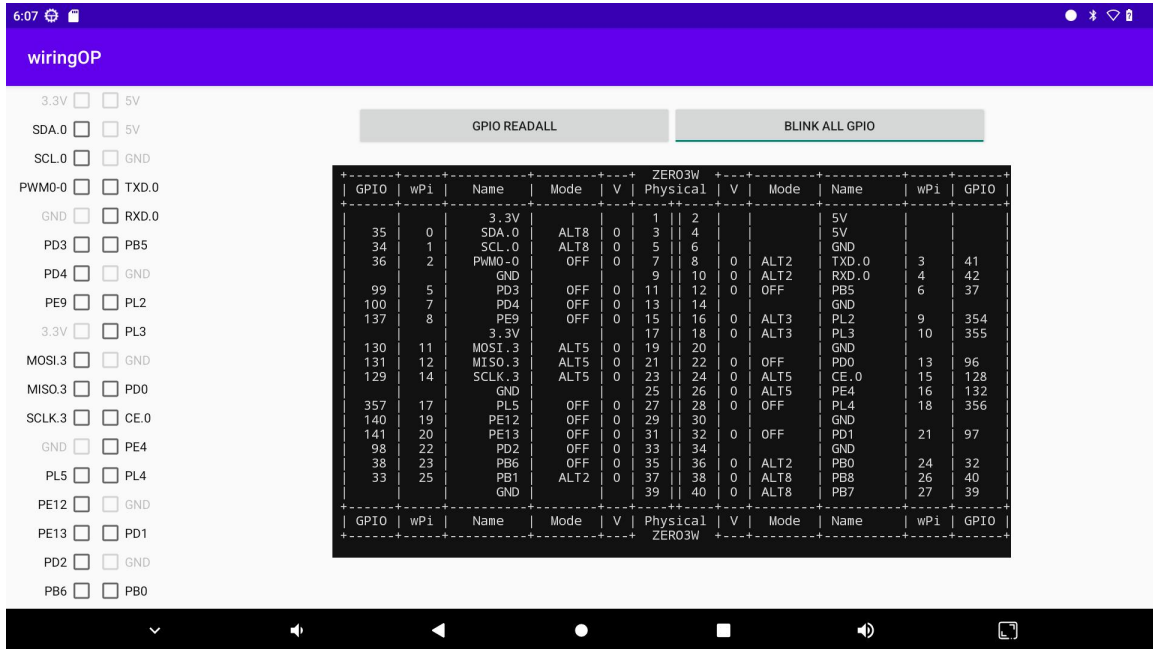
2) Then click the **GPIO_TEST** button to open the GPIO testing interface



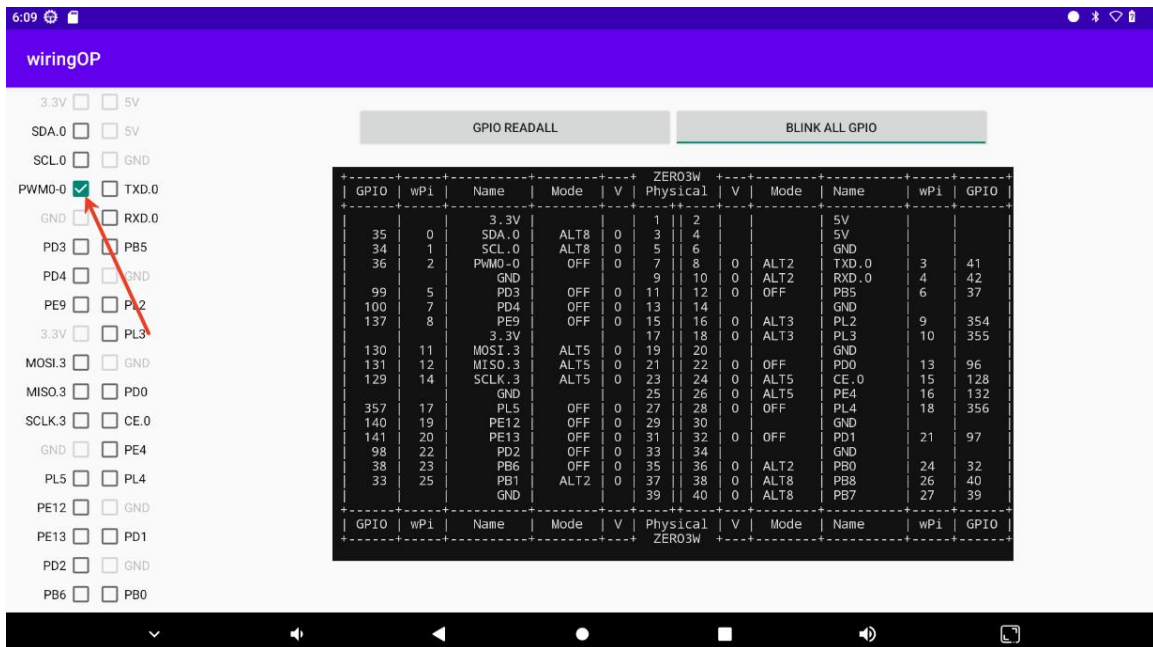
3) The GPIO testing interface is shown in the following figure, where the two rows of **CheckBox** buttons on the left correspond one-to-one with the 40 pin pin. When the **CheckBox** button is selected, the corresponding GPIO pin will be set to OUT mode and the pin level will be set to high level; When unchecked, the GPIO pin level is set to low level; When clicking the **GPIO READALL** button on the right, information such as wPi number, GPIO mode, pin level, etc. can be obtained. When clicking the **BLINK ALL GPIO** button, all pins will continuously switch between high and low levels



4) Then click the **GPIO READALL** button, and the output information is shown in the following figure:

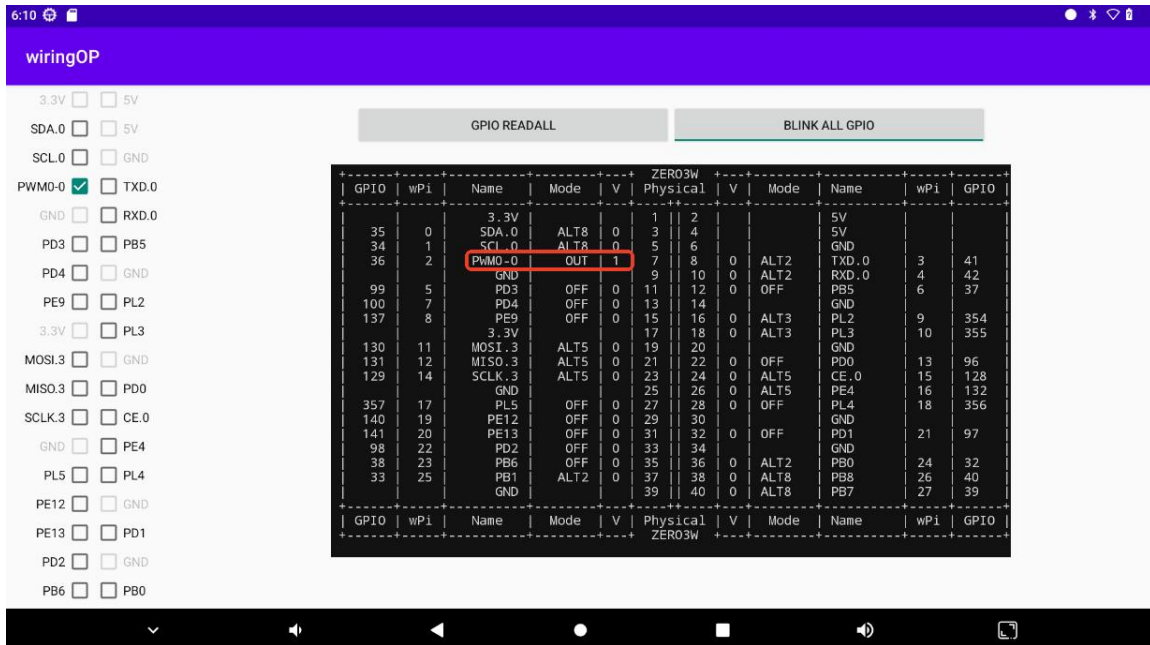


5) There are a total of 28 GPIO ports available in the 40 pin development board. Taking pin 7- corresponding to GPIO PB4- corresponding to wPi serial number 2- as an example, we will demonstrate how to set the high and low levels of GPIO ports. Firstly, click on the **CheckBox** button corresponding to pin 7. When the button is selected, pin 7 will be set to a high level. After setting, you can use a multimeter to measure the voltage value of the pin. If it is **3.3v**, it means that the high level has been successfully set

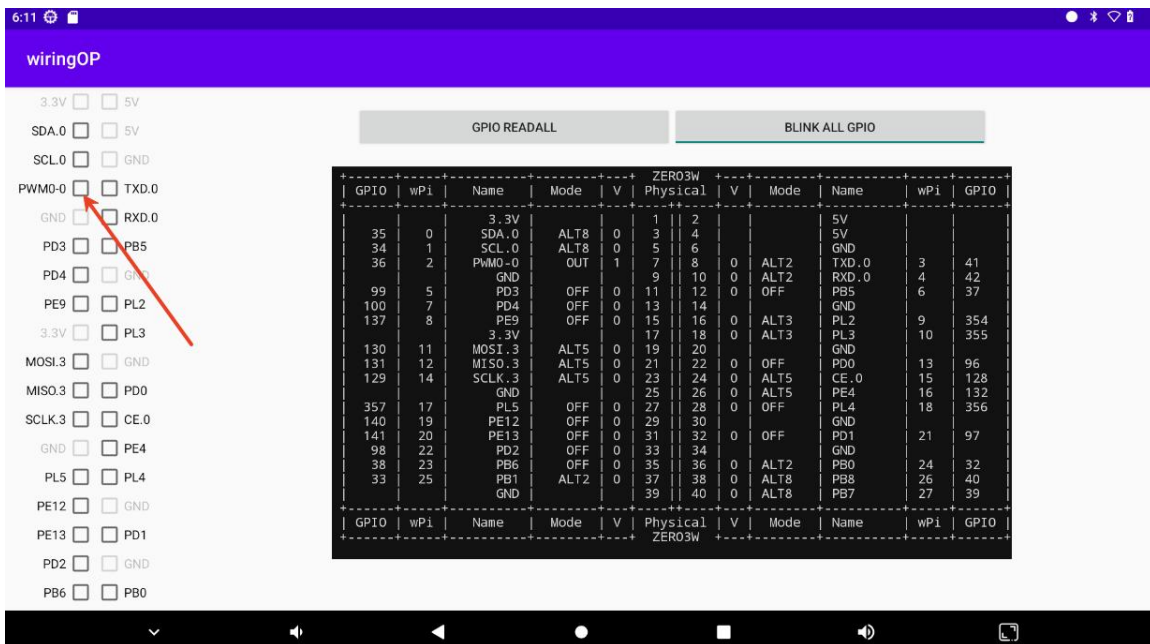




6) Then click the **GPIO READALL** button, and you can see that the current mode of pin 7 is **OUT** and the pin level is high



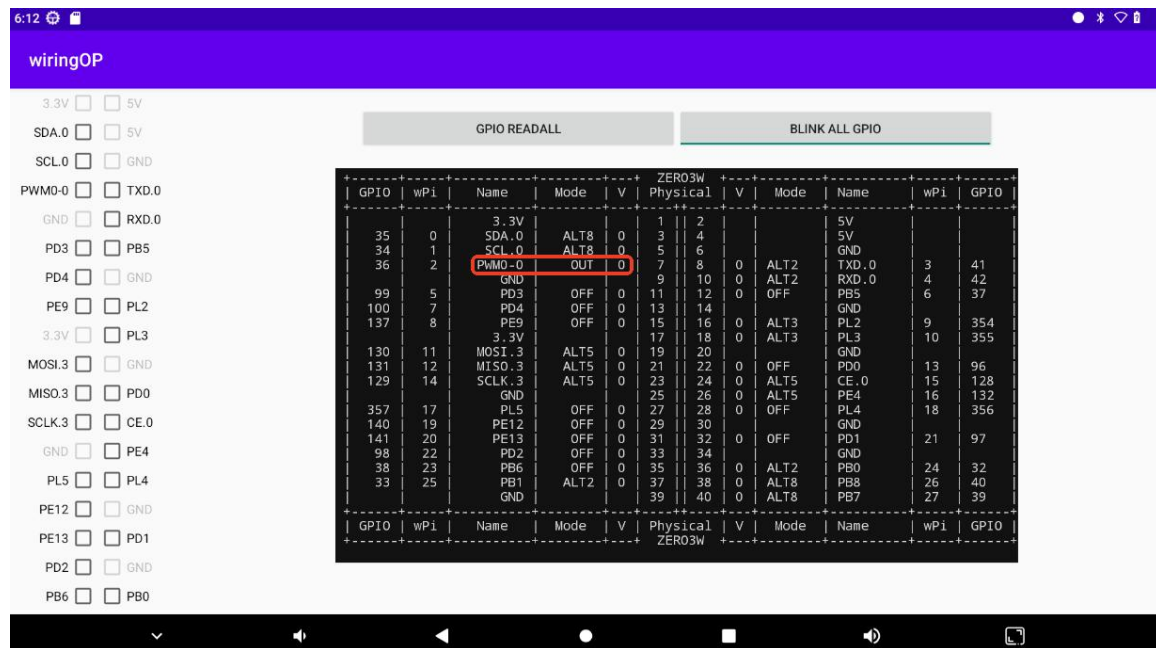
7) Click the **CheckBox** button in the following image again to uncheck the status. Pin 7 will be set to a low level. After setting, you can use a multimeter to measure the voltage value of the pin. If it is **0v**, it means that the low level has been successfully set



8) Then click the **GPIO READALL** button, and you can see that the current mode of



pin 7 is OUT and the pin level is low



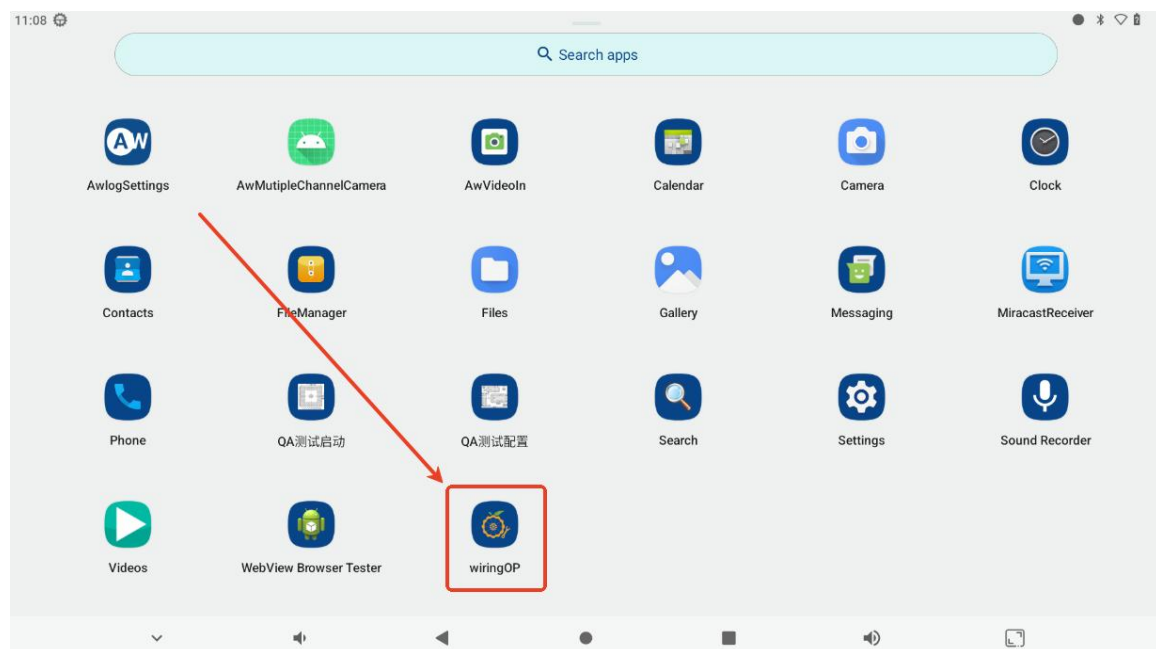
5. 10. 2. 40 pin UART testing method

1) By default, **UART2** and **UART7** are enabled in Android, corresponding to device nodes **/dev/ttyAS2** and **/dev/ttyAS7**

```
a733-demo:/ $ ls /dev/ttyAS*
```

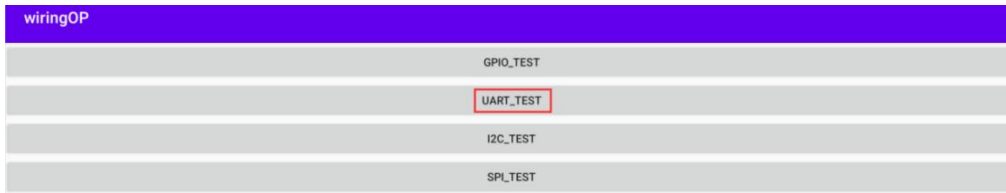
```
/dev/ttyAS0 /dev/ttyAS1 /dev/ttyAS2 /dev/ttyAS7
```

2) First, open the WiringOP app on the desktop

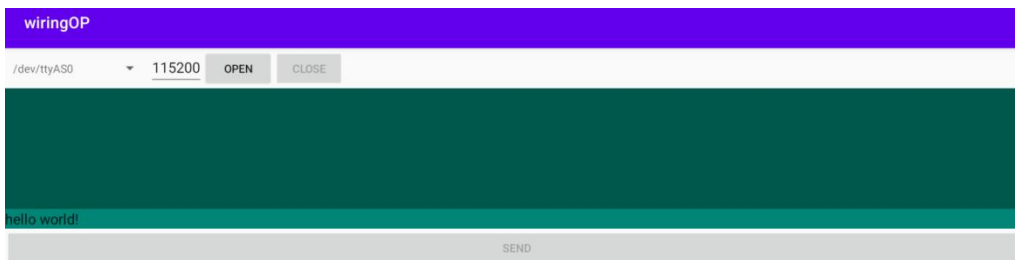




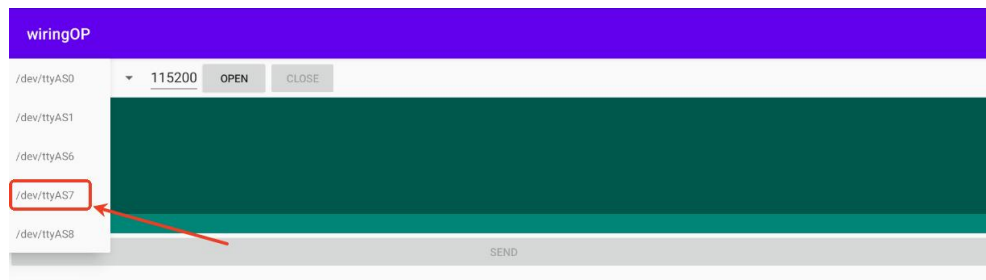
- 3) Then click the **UART_TEST** button to open the UART test interface



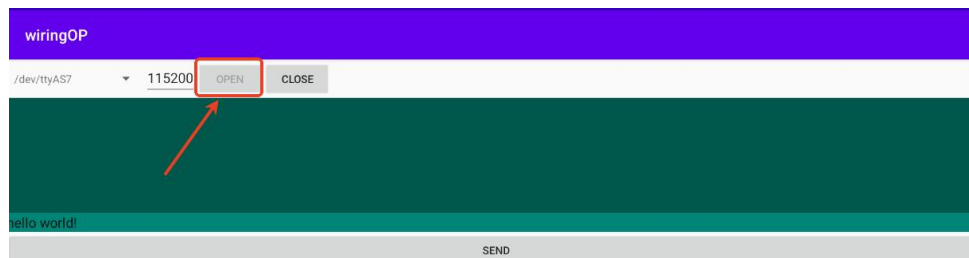
- 4) The serial port test interface of WiringOP is shown in the following figure



- 5) Taking testing UART7 as an example, select the **/dev/ttyAS7** node in the selection box



- 6) Enter the desired baud rate in the editing box, then click the **OPEN** button to open the **/dev/ttyAS7** node. After successful opening, the **OPEN** button becomes unselectable, and the **CLOSE** and **SEND** buttons become selectable



- 7) Then use DuPont wire to short-circuit the rx and tx pins of UART 7

	uart2	uart7
--	-------	-------



TX pin	Corresponding to pin 11 of pin 40	Corresponding to pin 16 of pin 40
Rx pin	Corresponding to pin 13 of pin 40	Corresponding to pin 18 of pin 40

8) Then you can enter a character in the send edit box below and click the **SEND** button to start sending

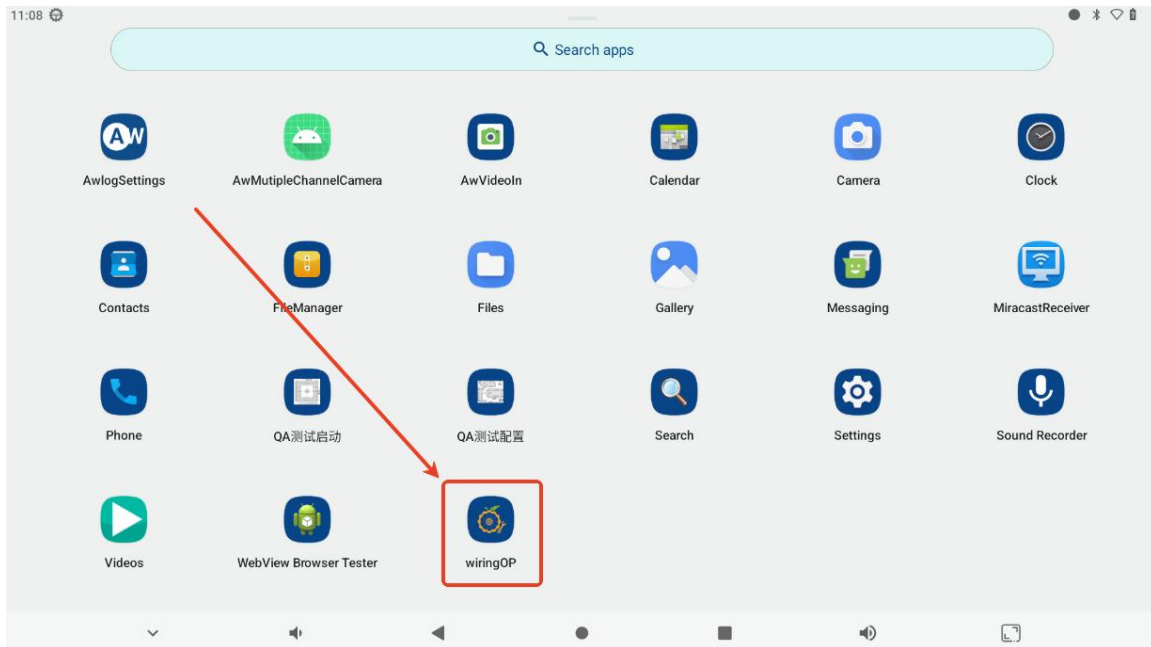


9) If everything is normal, the received string will be displayed in the receiving box



5. 10. 3. 40 pin SPI testing method

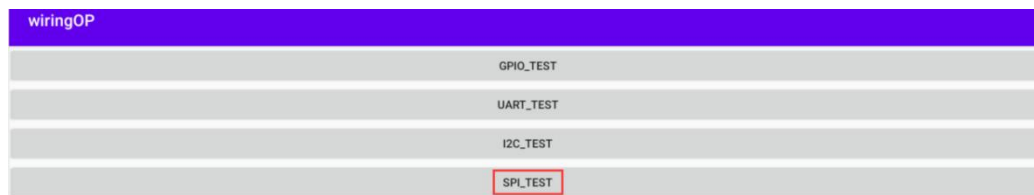
1) The SPI that can be used in 40 pins is SPI3, and the corresponding device node is **/dev/spidev3.0**



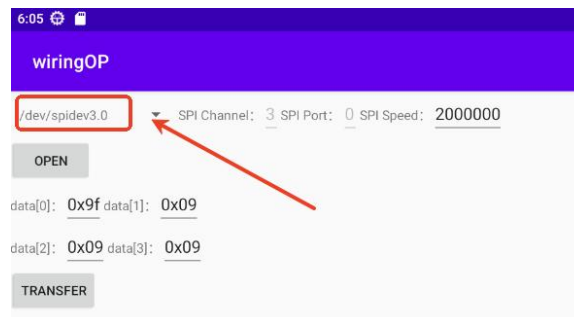
2) Here is a demonstration of testing the SP3 interface using the **w25q64** module. First, connect the w25q64 module to the SPI3 interface

3) Then open the WiringoP app on the desktop

4) Then click the **SPI_TEST** button to open the SPI testing interface



5) Then select the device node of SPI in the upper left corner, please choose **/dev/spidev3.0**





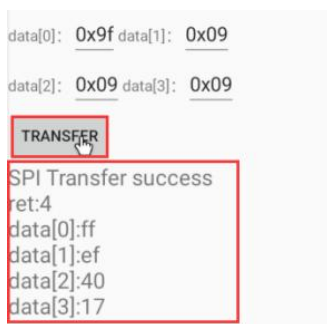
6) Then click the **OPEN** button to initialize SPI



7) Then fill in the bytes that need to be sent, such as reading the ID information of SPIFlash, filling in the address 0x9f in data [0], and then click the **TRANSFER** button



8) Finally, the app will display the read SPI Flash ID information



9) The MANUFACTURER ID of the w25q64 module is EFh, and the Device ID is 4017h, which corresponds to the values read above (h represents hexadecimal)



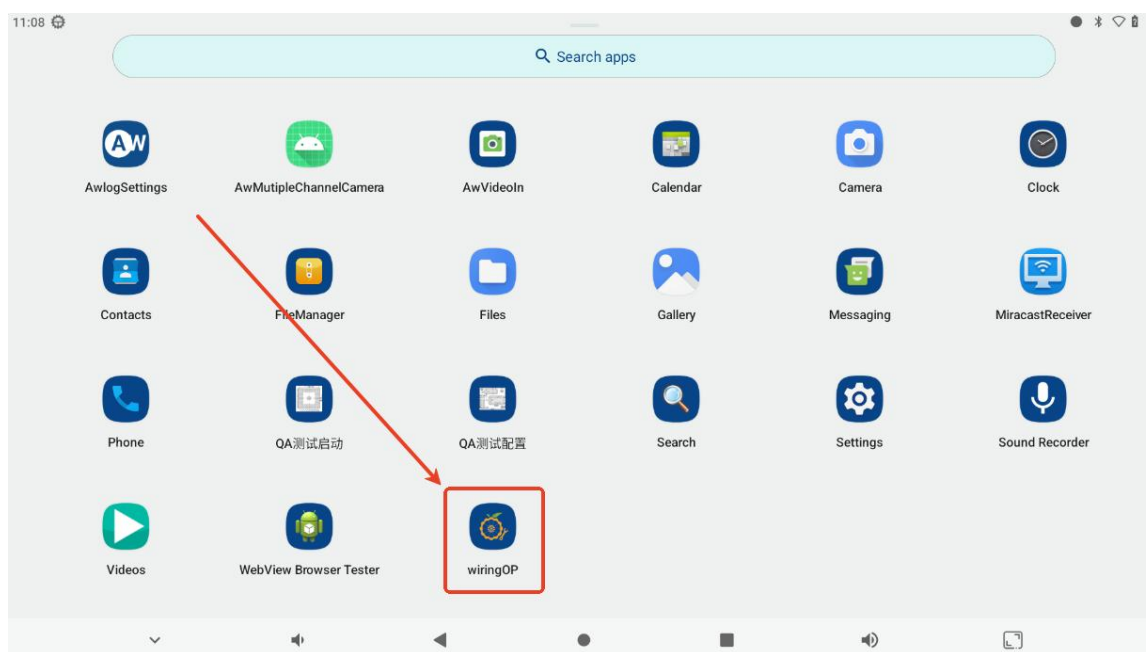
MANUFACTURER ID	(MF7 - MF0)	
Winbond Serial Flash	EFh	
Device ID	(ID7 - ID0)	(ID15 - ID0)
Instruction	ABh, 90h, 92h, 94h	9Fh
W25Q64FV (SPI)	16h	4017h
W25Q64FV (QPI)	16h	6017h

5. 10. 4. 40 pin I2C testing method

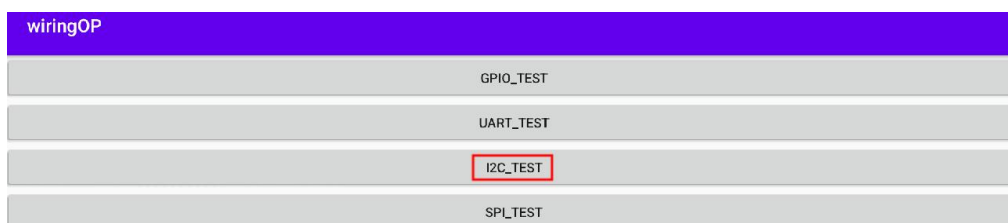
1) In Android, i2c0 and i2c1 in pin 40 are enabled by default, corresponding to device nodes `/dev/i2c-0` and `/dev/i2c-1`, respectively

```
console:/ # ls /dev/i2c-*
/dev/i2c-0  /dev/i2c-1
```

2) First, open the WiringOP app on the desktop



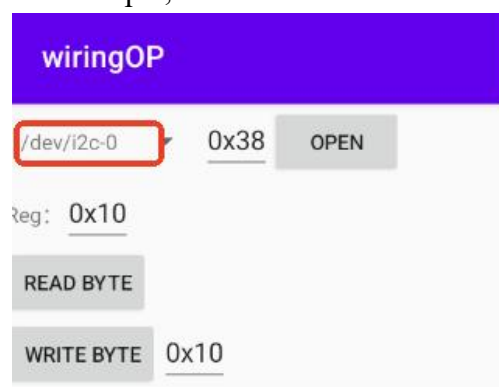
3) Then click the **I2C_TEST** button to open the testing interface of i2c



4) The i2c testing interface of WiringOP is shown in the following figure



5) Taking i2c0 testing as an example, the default selection box is the **/dev/i2c-0** node



6) Then connect an I2C device to the 40 pin I2C 0 pin, using the DS1307 RTC module as an example



Pin of RTC module	The pins corresponding to the 40 pins on the development board
5V	Pin 2
GND	Pin 6
SDA	Pin 3
SCL	Pin 5

7) The i2c address of the ds1307 RTC module is 0x68. After connecting the cable, we



can use the **i2cdetect -y -r 0** command in the serial port command line to check if we can scan the i2c address of the ds1307 RTC module. As shown in the figure below, if the address 0x68 can be seen, it indicates that the wiring of the ds1307 RTC module is correct.

```
console:/ # i2cdetect -y 0
```

	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
00:				--	--	--	--	--	--	--	--	--	--	--	--	--
10:	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--
20:	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--
30:	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--
40:	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--
50:	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--
60:	--	--	--	--	--	--	--	68	--	--	--	--	--	--	--	--
70:	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

8) Then set the address of i2c to 0x68 in WirgOP, and click the **OPEN** button to open i2c0

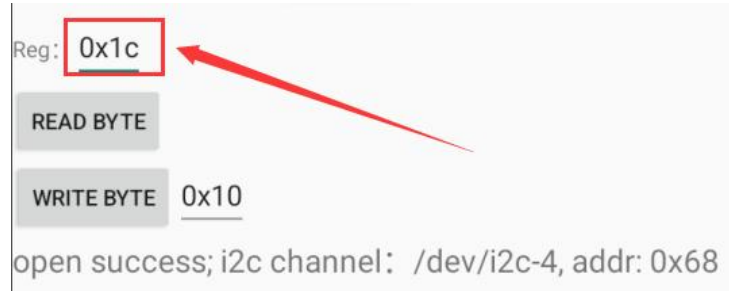


9) After clicking the **OPEN** button to open i2c0, the display is as follows:



10) Then we will test writing a value to the register of the RTC module, such as writing 0x55 to address 0x1c

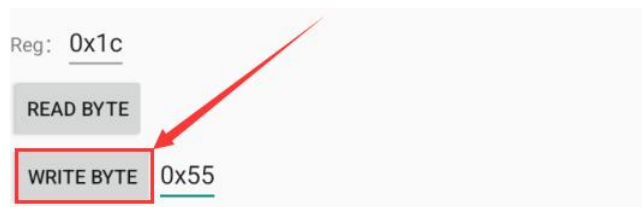
- a. We first set the address of the register that needs to be written to 0x1c



- b. Then set the value to be written as 0x55



- c. Then click the **WRITE BYTE** button to perform the write action



- 11) Then click the **READ BYTE** button to read the value of the 0x1c register. If it displays 0x55, it means that the i2c read-write test has passed



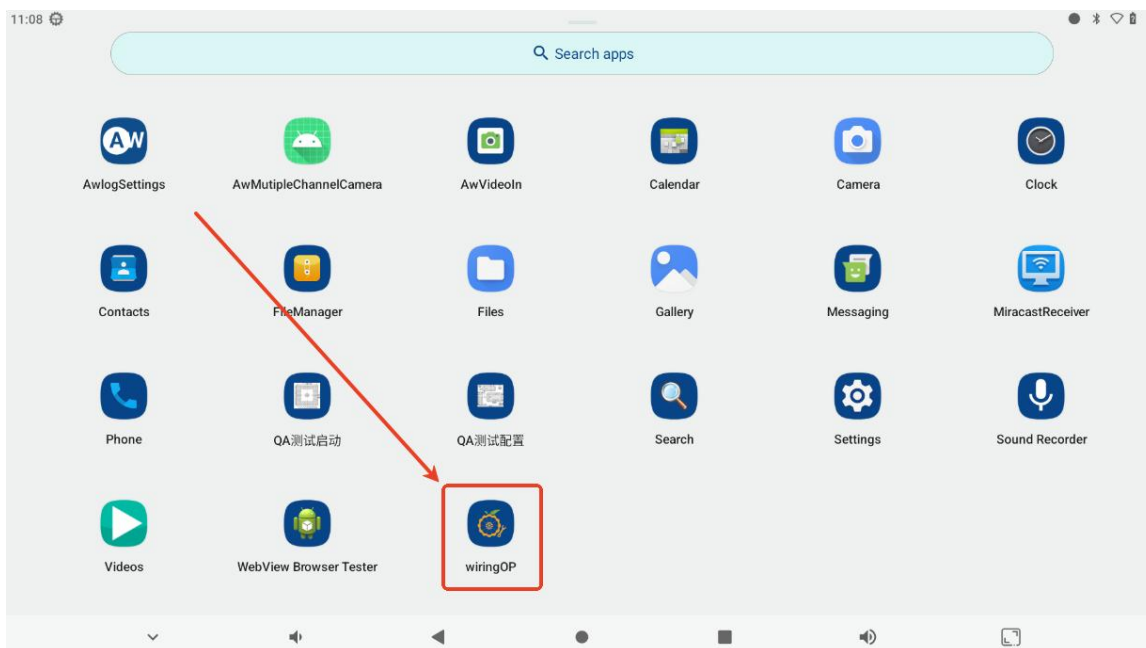
5. 10. 5. 40 pin PWM testing method

- 1) Android has enabled **PWM0_0**, **PWM0_1**, and **PWM0_2** by default, and the corresponding pins are located at the 40pin position as shown in the following figure

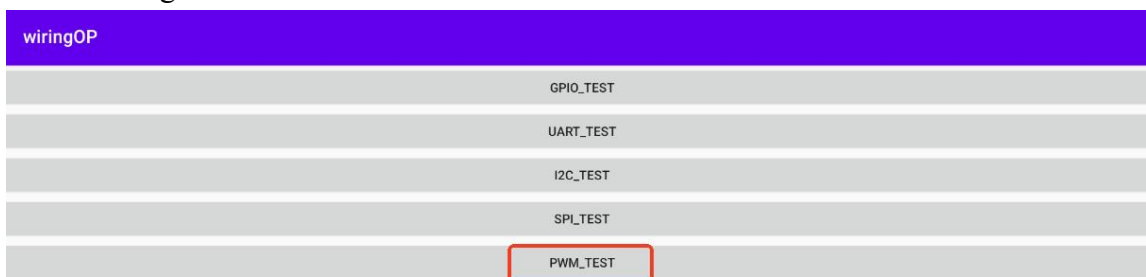


复用功能	复用功能	复用功能	GPIO	GPIO序号	引脚序号	引脚序号	GPIO序号	GPIO	复用功能	复用功能
			3.3V		1	2		5V		
		TWI0-SDA	PE3	35	3	4		5V		
		TWI0-SCK	PE2	34	5	6		GND		
		PWM0-0	PE4	36	7	8	41	PE9	UART0-TX	
			GND		9	10	42	PE10	UART0-RX	
		UART2-TX	PE0	32	11	12	37	PE5		
		UART2-RX	PE1	33	13	14		GND		
			PE9	137	15	16	354	PL2	UART7-TX	
			3.3V		17	18	355	PL3	UART7-RX	
								GND		
	TWI2-SDA	SPI3-MOSI	PE2	130	19	20		PDO		
	TWI3-SCK	SPI3-MISO	PE3	131	21	22	96	PE0	SPI3-CS0	UART6-TX
UART6-RX	TWI2-SCK	SPI3-CLK	PE1	129	23	24	128	PE4	SPI3-CS1	TWI3-SDA
			GND		25	26	132			
			PL5	357	27	28	356	PL4		
			PE12	140	29	30		GND		
			PE13	141	31	32	97	PD1	PWM0-1	
		PWM0-3	PD3	99	33	34		GND		
		PWM0-8	PE6	38	35	36	98	PD2	PWM0-2	
		PWM0-4	PD4	100	37	38	40	PE8	TWI1-SDA	
			GND		39	40	39	PE7	TWI1-SCL	PWM0-9

2) First, click on the WiringOP icon to open the WiringOP app



3) Then click the **PWM_TEST** button on the main interface of WiringOP to enter the PWM testing interface



4) The base address of **PWM0_0**, **PWM0_1**, and **PWM0_2** is **2527000**. Taking testing **PWM0_0** as an example, the base address corresponding to **PWM0_0** is **2527000**. When selecting pwmchip0 from the drop-down menu, the base address displayed on the right is



2527000.pwm. If the displayed base address is incorrect, please click the drop-down option to select another pwmchip until **2527000** is displayed on the right

wiringOP

Controller: pwmchip0 ▼ 2527000.pwm

Channel: 0

Period: 50000 ns

EXPORT

5) Since **PWM0_0** corresponds to the 0th channel of PWM0, the channel needs to be set to 0 and the PWM cycle needs to be confirmed. The default configuration is **50000ns**, and the converted PWM frequency is **20KHz**, which can be modified by clicking the EXPORT button to export **PWM0_0**

wiringOP

Controller: pwmchip0 ▼ 2527000.pwm

Channel: 0

Period: 50000 ns

EXPORT

☐ Enable pwmchip0 Duty

6) Then drag the drag bar below to change the PWM duty cycle, and select Enable to output the PWM waveform

wiringOP

Controller: pwmchip0 ▼ 2527000.pwm

Channel: 0

Period: 50000 ns

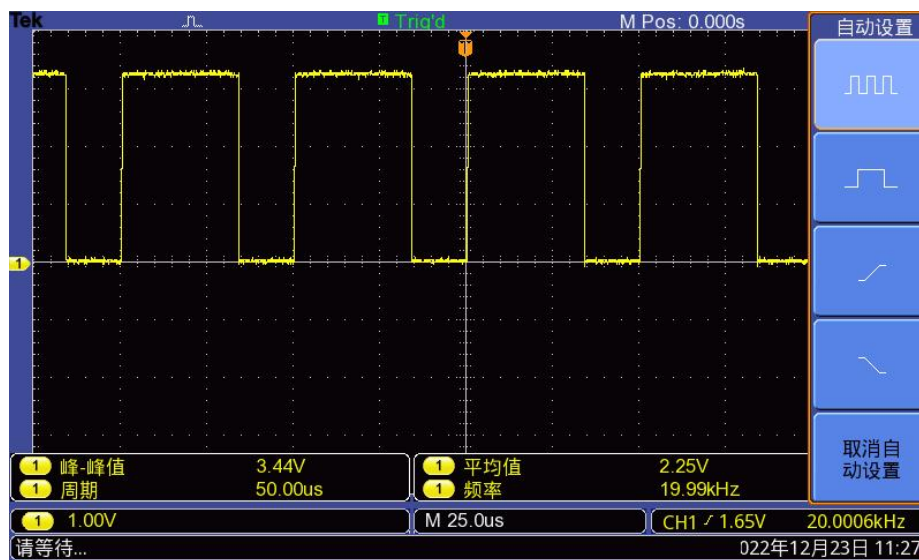
UNEXPORT

☒ Enable pwmchip0 Duty

7) Then use an oscilloscope to measure pin 7 of the 40 pins on the development board,



and you can see the waveform below



6. Compilation method of Android 13 source code

6.1. Download the source code of Android 13

1) Firstly, download the compressed Android 13 source code from Baidu or Google Drive

2) After downloading the compressed Android 13 source code, please check if the MD5 checksum is correct. If it is not correct, please download the source code again. The method for checking MD5 checksum is as follows:

```
test@test:~$ md5sum -c md5sum
a733_android13.tar.gz00: OK
a733_android13.tar.gz01: OK
.....
```

3) Execute the following command to extract the Android source code

```
test@test:~$ cat a733_android13.tar.gz0* | tar -xvzf -
```



6.2. Compile the source code for Android 13

The compilation of Android 13 is performed on an x86_64 computer with **Ubuntu 22.04** installed. There may be some differences in the dependencies of other versions of Ubuntu system packages. The download address for the Ubuntu 22.04 **amd64** version image is as follows:

<https://repo.huaweicloud.com/ubuntu-releases/22.04/ubuntu-22.04.2-desktop-amd64.iso>

The recommended hardware configuration for compiling Android 13 source code on an x86_64 computer is 16GB or more of memory, 200GB or more of hard disk space, and as many CPU cores as possible.

1) First, install and compile the software packages required for Android 13 source code

```
test@test:~$ sudo apt-get update
test@test:~$ sudo apt-get install -y git gnupg flex bison gperf build-essential \
zip curl zlib1g-dev gcc-multilib g++-multilib libc6-dev-i386 \
lib32ncurses5-dev x11proto-core-dev libx11-dev lib32z1-dev ccache \
libgl1-mesa-dev libxml2-utils xsltproc unzip u-boot-tools python-is-python3 \
libssl-dev libncurses5 clang gawk
```

2) Then compile the code in the **longan** folder, which mainly contains u-boot and Linux kernel

a. First run **./build.sh config** sets compilation options

```
test@test:~$ cd a733_android13/longan
test@test:~/a733_android13/longan$ ./build.sh config
=====ACTION List: mk_config ;=====
options :
All available platform:
    0. android
    1. linux
Choice [android]: 0
All available ic:
    0. a523
    1. a527
```



```
2. a733
3. t527
4. t736
Choice [a733]: 2
All available board:
0. QA
1. demo_aiot
2. demo_car
3. evb1
4. fpga
5. perf1
6. pro2
7. pro3
Choice [demo_aiot]: 1
All available flash:
0. default
1. nor
Choice [default]: 0
Setup BSP files
INFO: Prepare toolchain ...
```

b. Then run **./build.sh** script can start compiling now

```
test@test:~/a733_android13/longan$ ./build.sh
```

c. After compilation, you will see the following output

```
sun60iw2p1 compile all(Kernel+modules+boot.img) successful

INFO: build dts ...
INFO: Prepare toolchain ...
Setup BSP files
'/home/test/a733_android13/longan/out/a733/kernel/staging/sunxi.dtb' ->
'/home/test/a733_android13/longan/out/a733/demo/android/sunxi.dtb'
INFO: build rootfs ...
INFO: skip make rootfs for android
INFO: -----
INFO: build Tina OK.
```



INFO: -----

3) Then use the following command to compile the Android source code and generate the final Android image

```
test@test:~$ cd a733_android13
test@test:~/a733_android13$ source build/envsetup.sh
test@test:~/a733_android13$ lunch a733_demo_aiot_arm64-userdebug
test@test:~/a733_android13$ make -j8
test@test:~/a733_android13$ pack
```

4) The storage path of the compiled Android image is:

longan/out/a733_android13_demo_aiot_uart0.img

7. Appendix

7.1. User Manual Update History

Version	DATE	Update Notes
v1.0	2026-03-26	Initial version

7.2. Image update history

DATE	Update Notes
2026-03-26	OrangePi_Zero3W_A733_Android13_v1.0.tar.gz



	OrangePi_Zero3W_A733_Android13_lcd_v1.0.tar.gz Orangepizero3w_1.0.0_ubuntu_jammy_server_linux5.15.147.7z Orangepizero3w_1.0.0_debian_bullseye_server_linux5.15.147.7z Orangepizero3w_1.0.0_debian_bookworm_server_linux5.15.147.7z Orangepizero3w_1.0.0_ubuntu_jammy_desktop_xfce_linux5.15.147.7z Orangepizero3w_1.0.0_debian_bullseye_desktop_xfce_linux5.15.147.7z Orangepizero3w_1.0.0_debian_bookworm_desktop_xfce_linux5.15.147.7z Orangepizero3w_1.0.0_ubuntu_jammy_server_linux6.6.98.7z Orangepizero3w_1.0.0_debian_bullseye_server_linux6.6.98.7z Orangepizero3w_1.0.0_debian_bookworm_server_linux6.6.98.7z Orangepizero3w_1.0.0_ubuntu_jammy_desktop_xfce_linux6.6.98.7z Orangepizero3w_1.0.0_debian_bullseye_desktop_xfce_linux6.6.98.7z Orangepizero3w_1.0.0_debian_bookworm_desktop_xfce_linux6.6.98.7z * Initial version
--	--